

Igor Luppi de Oliveira

**Desenvolvimento e Modelagem do Sistema
Operacional
Laboratório de Sistemas Computacionais:
Sistemas Operacionais**

São José dos Campos - Brasil

Dezembro de 2018

Igor Luppi de Oliveira

Desenvolvimento e Modelagem do Sistema Operacional

Laboratório de Sistemas Computacionais: Sistemas Operacionais

Relatório apresentado à Universidade Federal de São Paulo como parte dos requisitos para aprovação na disciplina de Laboratório de Sistemas Computacionais: Sistemas Operacionais.

Docente: Prof. Dr. Tiago de Oliveira

Universidade Federal de São Paulo - UNIFESP

Instituto de Ciência e Tecnologia - Campus São José dos Campos

São José dos Campos - Brasil

Dezembro de 2018

Resumo

O projeto possui o objetivo geral de desenvolver um sistema operacional completo, com gerenciamento de processos, memória, sistemas de arquivos e dispositivos de entrada e saída. O S.O. será desenvolvido sobre um processador completo com memória e I/O, feito via linguagem de *hardware Verilog*. Nesta etapa foram finalizados o código do sistema operacional e dos algoritmos a serem executados, em c-menos, além de terminar o desenvolvimento em linguagem de *hardware Verilog* dos últimos componentes, como o interruptor, e adaptação de alguns blocos elaborados no PC3. Ao final, foram realizados os testes na FPGA para comprovar os resultados esperados e atestar o funcionamento do sistema operacional elaborado.

Palavras-chaves: sistema operacional. processador. verilog. gerenciamento.

Lista de ilustrações

Figura 1 – Expressões regulares adotadas para o Analisador Léxico.	19
Figura 2 – Saída do analisador léxico do Algoritmo 3.2 (GCD).	20
Figura 3 – Gramática adotada para o Compilador C-.	21
Figura 4 – Trecho da Gramática BNF no Bison.	22
Figura 5 – Árvore sintática gerada pelo analisador sintático do Algoritmo 3.2 (GCD).	22
Figura 6 – Tabela de símbolos gerada do analisador semântico do Algoritmo 3.2 (GCD).	24
Figura 7 – Trecho de código da operação relacional <i>Less Than</i>	25
Figura 8 – Função <code>cGen(TreeNode * tree)</code>	26
Figura 9 – Código <i>Assembly</i> gerado do Algoritmo 3.2 (GCD).	27
Figura 10 – Código executável gerado do Algoritmo 3.2 (GCD).	28
Figura 11 – Diagrama de estados dos processos.	30
Figura 12 – Gerenciamento de memória por mapa de <i>bits</i>	32
Figura 13 – Estrutura de arquivos. (a) Sequência de <i>bytes</i> , (b) Sequência de registros e (c) Árvore.	33
Figura 14 – Tipos de arquivos. (a) Executáveis e (b) Texto.	34
Figura 15 – Diretórios. (a) Nível único, (b) Nível duplo e (c) Hierárquico.	35
Figura 16 – Esquemático do DMA.	36
Figura 17 – <i>Datapath</i> do Processador	37
Figura 18 – Plataforma de <i>hardware</i> do Sistema.	38
Figura 19 – Memória de Dados e Pilha.	39
Figura 20 – BIOS.	44
Figura 21 – HD simulado.	46
Figura 22 – <i>Timer</i>	48
Figura 23 – Unidades de <i>Output</i> do S.O.	50
Figura 24 – <i>Datapath</i> do Processador com as alterações realizadas.	52
Figura 25 – Diagrama de blocos do CPU.	53
Figura 26 – Diagrama de requisitos do CPU.	54
Figura 27 – Diagrama de atividades da inicialização do SO.	55
Figura 28 – Estrutura da memória de dados.	60
Figura 29 – Estrutura da pilha de dados.	61
Figura 30 – Estrutura da memória de instruções.	62
Figura 31 – Estrutura do disco rígido.	65
Figura 32 – (1) BIOS iniciando, (2) BIOS carregando SO e (3) BIOS fazendo checagem de I/O e memória.	74
Figura 33 – Menu Completo do Sistema Operacional.	75

Figura 34 – Preparação e Execução de Programas.	76
Figura 35 – Estado de Troca de Contexto.	77
Figura 36 – Finalização da execução de programas.	77
Figura 37 – Saídas dos Programas (6), (7) e (8).	78
Figura 38 – Criação de um Programa novo.	79
Figura 39 – Execução do Novo Programa Criado.	80
Figura 40 – Renomear Programa.	81
Figura 41 – Deletar Programa.	82
Figura 42 – Listar Programas.	83
Figura 43 – Desligando e Reiniciando o Sistema Operacional.	84

Lista de tabelas

Tabela 1	–	Formato das instruções tipo R	12
Tabela 2	–	Formato das instruções tipo I	12
Tabela 3	–	Formato das instruções tipo J	13
Tabela 4	–	Instruções especiais para controle	13
Tabela 5	–	Tabela de Convenções Léxicas	20
Tabela 6	–	Descrição do Conjunto de Instruções	90

Sumário

1	INTRODUÇÃO	9
2	OBJETIVOS	10
2.1	Geral	10
2.2	Específico	10
3	FUNDAMENTAÇÃO TEÓRICA	11
3.1	Arquitetura Alvo	11
3.1.1	Conjunto de Instruções	11
3.1.1.1	Categorias de Instrução	13
3.1.2	Arquitetura MIPS	13
3.1.2.1	<i>Program Counter</i>	14
3.1.2.2	Memória de Instruções	14
3.1.2.3	Banco de Registradores	14
3.1.2.4	Extensor de <i>bits</i>	14
3.1.2.5	Unidade Lógica e Aritmética	15
3.1.2.6	Controle ULA	15
3.1.2.7	<i>Branch Control</i>	15
3.1.2.8	Memória de dados	15
3.1.2.9	Pilha de dados	15
3.1.2.10	Módulo de Entrada e Saída	16
3.1.2.11	Entrada de Dados	16
3.1.2.12	Saída de Dados	16
3.1.2.13	Temporizador	16
3.1.2.14	Multiplexadores	16
3.1.2.15	Unidade de Controle	16
3.1.3	<i>Softwares</i>	17
3.1.3.1	Verilog HDL	18
3.1.4	<i>Hardwares</i>	18
3.2	Compilador c-menos	18
3.2.1	Análise Léxica	18
3.2.2	Análise Sintática	21
3.2.3	Análise Semântica	22
3.2.3.1	Tabela de Símbolos	23
3.2.4	Geração do código <i>Assembly</i>	24
3.2.5	Geração do código executável	27

3.2.6	Gerenciamento da memória	28
3.3	Gerenciamento de Recursos do Sistema	29
3.3.1	<i>Basic Input/Output System</i> (BIOS)	29
3.3.2	Gerenciamento de Processos	30
3.3.2.1	Escalonador de processos	30
3.3.3	Gerenciamento de Memória	31
3.3.3.1	Alocação e divisão da memória principal	31
3.3.4	Gerenciamento do Sistemas de Arquivos	32
3.3.4.1	Estrutura dos arquivos	32
3.3.4.2	Tipos de arquivos	33
3.3.4.3	Diretórios	34
3.3.4.4	Acesso e alocação de arquivos	35
3.3.5	Gerenciamento de Dispositivos de E/S	36
4	DESENVOLVIMENTO	37
4.1	Arquitetura Alvo	37
4.2	Modificações e Alterações na Arquitetura	37
4.2.1	Organização da Memória de Dados e Pilha	38
4.2.2	BIOS e Gerenciamento do PC	39
4.2.3	Disco Rígido	45
4.2.3.1	Atraso do Disco Rígido Simulado	47
4.2.4	Relógio e Interrupção	48
4.2.5	Terminal de Informações	50
4.2.6	<i>Dathpath</i> com Alterações na Arquitetura	52
4.2.7	Diagrama de Blocos do Sistema	53
4.2.8	Diagrama de Requisitos do Sistema	54
4.2.9	Diagramas de Atividades do Sistema	55
4.3	Gerenciamento de Processos	55
4.4	Gerenciamento de Memória	59
4.5	Gerenciamento do Sistema de Arquivos	63
4.6	Gerenciamento de Dispositivos E/S	69
4.6.1	<i>Syscalls</i>	69
4.6.2	<i>Display</i> LCD	70
4.6.3	Menu <i>bash</i>	71
5	RESULTADOS	73
5.1	BIOS	73
5.2	Sistema Operacional	75
5.2.1	Executar Programas	76
5.2.2	Escrever Programa	79

5.2.3	Renomear Programa	81
5.2.4	Deletar programa	82
5.2.5	Listar Programas	82
5.2.6	Desligar e Reiniciar SO	83
6	CONSIDERAÇÕES FINAIS	85
6.1	Dificuldades Encontradas	85
	 REFERÊNCIAS	 86
	 APÊNDICES	 88
	APÊNDICE A – CONJUNTO DE INSTRUÇÕES	89
	APÊNDICE B – CÓDIGO FINAL DE MÁQUINA DO SO	91

1 Introdução

Ao passo que a tecnologia evoluiu e novas arquiteturas foram aparecendo, a necessidade de utilizar máquinas de propósito geral foram surgindo. Nos primórdios, as arquiteturas suportavam somente sistemas em lotes com execuções sequenciais e sem interrupções, da forma como conhecemos hoje, além de serem controlados por um único usuário.

Devido a isso, novas técnicas de gerenciamento dos recursos da máquina foram surgindo, e com esses o sistema operacional viria a evoluir com o tempo. Assim, poderia prover para o usuário uma maneira melhor, e mais otimizada, de usar os recursos, principalmente no que diz respeito a programas orientados a CPU ou I/O.

Como é citado por Tanenbaum (1), o primeiro computador digital foi projetado entre 1792-1871 pelo inglês Charles Babbage. Ele percebeu que sua máquina analítica precisava de um *software* para controlar seu funcionamento, assim contratou a Ada Lovelace, conhecida como a primeira programadora do mundo, para tal serviço. Este evento histórico pode ser considerado como um dos primeiros passos para a criação do sistema operacional, mesmo que muito diferente do que conhecemos hoje em dia, foi o que deu início a este processo histórico cheio de avanços e evoluções.

2 Objetivos

2.1 Geral

O projeto visa desenvolver um sistema operacional, com um processador alvo com memória, HD e outros dispositivos de E/S. Esse sistema operacional deve ser capaz de gerenciar processos, memória, sistema de arquivos e dispositivos de entrada e saída.

2.2 Específico

Implementar o sistema operacional projetado e os algoritmos a serem executados pelo S.O. em c-menos, além de realizar os testes na FPGA e comprovar os resultados esperados.

3 Fundamentação Teórica

3.1 Arquitetura Alvo

Segundo Floyd (2), “A CPU é o “cérebro” do computador; ela supervisiona tudo que o computador faz. A CPU é um microprocessador com circuitos associados que controlam a execução dos programas (softwares). Basicamente, a CPU obtém (busca) cada instrução do programa a partir da memória e realiza (executa) essa instrução”. A *Central Processing Unit* tem o importante papel no gerenciamento do fluxo de dados. De maneira simples, o funcionamento do processador consiste em, buscar os dados, os processar e retornar um resultado.

3.1.1 Conjunto de Instruções

Conjunto de instruções (3) são as operações básicas, em nível de *hardware*, que o CPU pode executar. Define como o mesmo irá funcionar e é uma das principais características de um processador, pois essa arquitetura é exclusiva de cada. Com o passar dos anos, os processadores, cada vez mais, aumentavam seu *instruction set*, como, por exemplo, o Intel Pentium® (4). Os modelos de ISA RISC (*Reduced Instruction Set Computer*) (3), em geral, se caracterizam por possuírem menos instruções em sua arquitetura, poucas que realizam uma determinada execução, e acesso a memória principal restrito a algumas instruções, e, em sua maioria, são mais básicas, além de manipular poucos operandos. Por outro lado, os modelos CISC (*Complex Instruction Set Computer*), possuem mais instruções, e mais complexas, que podem ter várias com mesma execução, mas que buscam os dados de diversas maneiras.

O conjunto de instruções (3) segue uma arquitetura mais clássica RISC (3), muito baseada no MIPS. Possui 38 instruções com tamanho fixo de 32 *bits*, e pode possuir até três operandos por instrução, que podem ser endereçados por três modos de endereçamento (5):

- Imediato: O operando está na própria instrução.
- Registrador Direto: O operando está no registrador que o endereço descreve.
- Deslocamento: O operando está na memória, descrita pela soma do valor de um registrador com um imediato.

Foram definidos três formatos diferentes de instruções, cada qual com suas particularidades. O primeiro formato é o tipo **R**, que possui os seguintes campos:

- *Opcode*: Define o código da instrução. No tipo *R*, em particular, esse campo é preenchido com 000000.
- *RS*: Registrador fonte. Armazena o dado a ser processado.
- *RT*: Registrador fonte secundário. Atua como o *RS*, mas não é usado em instruções **R** com dois operandos.
- *RD*: Registrador destino. Armazena o dado pós-processado.
- *Shamt*: Valor em binário que corresponde quanto deve ser o *shift* do conteúdo de *RS*. Apenas usado em instruções da categoria deslocamento.
- *Function*: Define efetivamente a instrução a ser executada nas instruções tipo **R**.

Como se pode perceber na [Tabela 1](#), que também mostra a quantidade de bits definida para cada campo.

Tabela 1 – Formato das instruções tipo R

Descrição	<i>Opcode</i>	RS	RT	RD	<i>Shamt</i>	<i>Function</i>
<i>bits</i>	[31:26]	[25:21]	[20:16]	[15:11]	[10:6]	[5:0]

Fonte: O Autor

O segundo formato é o tipo **I**, que possui os seguintes campos:

- *Opcode*: Define o código da instrução a ser executada.
- *RS*: Registrador fonte. Armazena o dado a ser processado.
- *RT*: Registrador secundário. Pode atuar como fonte ou destino, depende da instrução a ser executada.
- *Immediate*: Define um valor imediato a ser usado na execução da instrução.

A [Tabela 2](#) mostra como está distribuído os *bits* por campo.

Tabela 2 – Formato das instruções tipo I

Descrição	<i>Opcode</i>	RS	RT	<i>Immediate</i>
<i>bits</i>	[31:26]	[25:21]	[20:16]	[15:0]

Fonte: O Autor

O terceiro formato é o tipo **J**, que possui os seguintes campos:

- *Opcode*: Define o código da instrução.

- *Address*: Define um endereço de memória de instrução.

Como mostra exatamente a [Tabela 3](#).

Tabela 3 – Formato das instruções tipo J

Descrição	Opcode	Address
<i>bits</i>	[31:26]	[25:0]

Fonte: O Autor

As instruções especiais de controle estão na [Tabela 4](#) abaixo, a lista completa das instruções está na [Tabela 6](#), no [Apêndice A](#).

Tabela 4 – Instruções especiais para controle

Instrução	Descrição	Representação	Categoria
NOOP	<i>No Operation</i>	00000000000000000000000000000000	Controle
HALT	Finaliza a execução	11111111111111111111111111111111	Controle

Fonte: O Autor

3.1.1.1 Categorias de Instrução

As instruções podem ser agrupadas por sua funcionalidade ou objetivo ([3](#)):

- Aritméticas: Realizam cálculos matemáticos.
- Lógicos: Realizam operações booleanas.
- Deslocamento: Manipulam os *bits* do operando.
- Saltos: Alteram o curso da execução, ou seja, podem criar desvios no fluxo das instruções a serem executadas.
- Transferência: Realizam transferência de dados entre registradores/memória.
- Controle: Funções para o gerenciamento da execução.
- I/O: Instruções específicas para entrada e saída de dados.

3.1.2 Arquitetura MIPS

MIPS (*Microprocessor without Interlocked Pipeline Stages*) ([6](#))([3](#)) é uma arquitetura clássica de processadores, baseada em modelo RISC. É uma arquitetura simples, simplificada, altamente escalável e de fácil adaptação. Possui instruções de mesmo tamanho e definidas em três tipos diferentes. É muito bem planejada para ser executada em pipeline, que pode executar cinco instruções ao mesmo tempo, tornando mais eficiente o

processamento dos dados, mas seu desenvolvimento sem o mesmo é bem válido, tanto em monociclo como em multiciclo. Possui um conjunto de instruções dividido em três categorias, **R**, **I** e **J**, e dependendo da versão, contém inúmeras instruções.

3.1.2.1 Program Counter

O módulo *program counter* é responsável por armazenar o endereço da instrução atual a ser executada. É um componente síncrono, pois trabalha na subida do *clock*. Seu funcionamento consiste em receber o endereço da instrução atual e calcular o subsequente, para no próximo ciclo, poder recebê-lo, se não houver desvio no fluxo do programa.

Possui como entrada, além do *clock* e do PC, o *reset*, que tem objetivo de zerar o PC atual, fazendo o programa recomeçar. Como saída, devolve o valor atual do PC e o subsequente.

3.1.2.2 Memória de Instruções

É a parte da memória de armazena as instruções a serem executadas pelo processador. Toda vez que a entrada *reset* é acionada, na subida do *clock*, as instruções são armazenadas. Além disso, possui um endereço como entrada, de modo que a saída seja a instrução alocada nessa posição da memória.

3.1.2.3 Banco de Registradores

É onde estão os registradores para uso do programa, possuindo um total de 32, de modo que, 31 são de uso geral e 1 para uso específico, que no caso, é o registrador \$ra, que armazena o PC atual quando é executado instruções de *jump and link*. Eles podem armazenar dados de 32 *bits*, e a escrita dos mesmos está atuando na subida do *clock*.

Possui como entrada o *clock*, RS, RT, RD, esses que representam os registradores fonte e destino, como mostra os tipos de instruções, localizado na [subseção 3.1.1](#), e algumas podem não usar todos, além do dado a ser atribuído no registrador de destino.

3.1.2.4 Extensor de *bits*

Esse módulo tem como objetivo estender uma entrada para uma saída de tamanho maior, no caso, está estendendo de 16 *bits* para 32 *bits*. Pode estender para ambos lados, para esquerda é o convencional, pois não altera o valor da entrada, já para direita são para instruções que carregam na parte mais significativa dos *bits*, como a LUI, conforme está descrito na [Tabela 6](#). Também considera se o dado é *unsigned* ou *signed*, pois se for o último, a extensão de bits pode ser diferente, por conta dos números negativos, que possuem *bits* 1 à esquerda, ao invés de 0.

3.1.2.5 Unidade Lógica e Aritmética

A ULA é responsável por executar cálculos e operações lógicas no sistema. Operando dados com 32 *bits*, que correspondem as entradas *a* e *b*, já a saída desse resultado da operação é armazenado em *rs*, e o *shift*, que é o valor de *Shamt* das operações tipo **R**, como descreve a [Tabela 1](#), onde informa o deslocamento da entrada para instruções dessa categoria. Além disso, tem um *bit* condicional de saída quando algum salto condicional tem resultado positivo na sua verificação. Uma ressalva na operação MULT que é interessante notar, sua operação calcula apenas os 16 *bits* menos significativos, para a saída não ocorrer *overflow*, por ter apenas 32 *bits*.

3.1.2.6 Controle ULA

Módulo responsável e exclusivo para gerenciar a ULA, onde por meio da entrada *opcode* e *function*, calcula a operação a ser realizada na ALU. Se faz necessário pois certas instruções possuem o mesmo *opcode*, como é o caso das instruções tipo **R**, que são diferenciadas pelo *function* delas, como é apresentado na [subseção 3.1.1](#).

3.1.2.7 Branch Control

Unidade responsável por controlar a ação do salto condicional. Quando a ULA realiza o cálculo da condição, ela retorna a essa unidade um *bit* de controle, que indica se a saída deve ser o PC subsequente ou a soma do PC corrente com o valor imediato que é dado como entrada, esse novo valor é calculado nesse mesmo bloco.

3.1.2.8 Memória de dados

É a parte da memória (7) responsável por armazenar dados gerais a respeito do programa. Foi projetada para armazenar valores de 32 *bits*, como os registradores, e opera na subida do *clock* para escrita de dados.

Como entrada, além do *clock*, possui o endereço da RAM a ser lido/escrito e o dado escrito na memória. Já para saída, possui o dado lido. Por fim, possui *bits* de controle que indicam se a escrita e leitura estão habilitadas no *clock* atual.

3.1.2.9 Pilha de dados

É uma parte da memória separada da memória principal, que é uma pilha de dados do processador. Conforme a [Tabela 6](#), as instruções de PUSH e POP utilizam dessa pilha implementada em *hardware*, permitindo o usuário inserir ou retirar do topo da pilha por meio dessas instruções específicas.

3.1.2.10 Módulo de Entrada e Saída

Componentes responsáveis para interagir ao mundo externo, podendo receber dados variáveis fora do programa e mostrar resultados. No desenvolvimento, foram implementados de forma separada.

3.1.2.11 Entrada de Dados

Recebe um valor *signed* de 16 *bits* dos *switches* de entrada, além de um *switch* que indica se vai escrever essa entrada na parte mais ou menos significativa do registrador e *button_in*, representando um acionamento, ou seja, enquanto não for pressionado, indica que o usuário ainda não configurou a posição das alavancas externas.

3.1.2.12 Saída de Dados

Esse módulo é responsável por converter a entrada de 32 *bits*, para *display* de sete segmentos. Serão usados 7 *displays*, onde será mostrado um valor inteiro em decimal. A saída tem um controle para indicar se deve ser ligado ou desligado os LEDs deles, assim evitando uma poluição visual para o usuário quando estiver realizando operações que não são *I/O Bound*.

Vale notar que esse componente não opera sozinho, possui mais três módulos associados, o conversor de complemento de dois, para saber se é positivo ou negativo o valor, o conversor BCD, para deixar a entrada binária fácil de ser manipulada pelos *displays* e por fim o próprio *display*.

3.1.2.13 Temporizador

Esse módulo é responsável por diminuir a frequência de entrada nativa da FPGA (8) alvo, de 50mHz para 5kHz, que é a frequência atual usada pelo processador elaborado. Utiliza o princípio de máquina de estados finita para dividir o *clock* de entrada.

3.1.2.14 Multiplexadores

São as unidades combinacionais muito utilizadas no desenvolvimento, pois realizam a seleção do fluxo do dados. No total são 5 MUXs, em todos, pois meio de *bits* seletos é escolhido a saída dessas unidades.

3.1.2.15 Unidade de Controle

Esse módulo tem o objetivo de modificar os valores das *flags*, dependendo das entradas e instruções a serem executadas. Elaborado como um circuito combinacional, possui como entrada *opcode* e *funct*, como decodificadores para a saída das *flags*, e algumas

entradas do usuário, que influenciam nesse processo, utilizadas nas instruções de entrada e saída.

O *button_in*, é um botão de confirmação de entrada, que faz o processador ficar na instrução de entrada, até o usuário entrar com os dados e o pressionar, o *button_out*, é um botão de confirmação de saída, que faz o processador ficar na instrução de saída, até o usuário visualizar os dados e o pressionar, por fim, *switchSide*, que indica se a entrada vai para a parte mais/menos significativa do registrador, já que há uma limitação de 16 *bits* de entrada, contudo o mesmo possui 32 bits, dessa forma é possível aproveitar ao máximo seu potencial.

Como saída da UC (Unidade de Controle), possui as seguintes *flags*:

- *regDest*: Indica qual o registrador destino, para instruções que escrevem nos mesmos.
- *regWrite*: Habilita a escrita no banco de registradores.
- *extSide*: Indica para qual lado estender o imediato no extensor de *bits*.
- *twoComplement*: Indica se o dado operado está em complemento de dois.
- *regAlu*: Indica se o segundo operando da ALU é registrador ou imediato.
- *memWrite*: Habilita a escrita na memória de dados.
- *memRead*: Habilita a leitura na memória de dados.
- *jumpOp*: Indica se há saltos incondicionais.
- *PCDest*: Indica de onde provém o valor do próximo PC.
- *outputControl*: Indica se a instrução deve utilizar o módulo de saída.
- *resultDest*: Indica de onde provém o valor a ser armazenado no registrador destino.

3.1.3 Softwares

O QUARTUS II © (8) da *Altera Corporation*, um *software* de projetos para dispositivos programáveis, que permite o desenvolvimento por modelos esquemáticos ou por uma HDL (*Hardware Description Language*), como VHDL (*Very High Speed Integrated Circuits* HDL) e Verilog HDL. *ModelSIM Simulation* © da *Mentor Graphics*, para verificar a saída dos circuitos por meio de forma de onda.

3.1.3.1 Verilog HDL

É uma linguagem de descrição de *hardware* (9) do padrão IEEE (2), que é muito usado para modelar os sistemas eletrônicos. O uso de tal linguagem torna o desenvolvimento mais ágil, uma vez que é mais rápido a implementação do que modelagem via esquemáticos, torna mais visível o que está acontecendo do ponto de vista de alto nível, o que facilita para procurar inconsistências e também quanto portabilidade do mesmo, que torna a linguagem independente do *software* utilizado.

3.1.4 Hardwares

A FPGA (*Field Programmable Gate Array*) (10) é um circuito integrado que pode ser reconfigurado, podendo projetar qualquer circuito dentro de suas limitações físicas. Tornando possível verificar os modelos teóricos e testar os tempos de resposta de entrada/saída.

3.2 Compilador c-menos

Ao passo que a tecnologia evoluiu e novas arquiteturas foram surgindo, a necessidade de utilizar linguagens de alto nível para criação de *softwares* foi essencial para tornar possível tudo o que temos hoje, visto a dificuldade de se compreender e programar em linguagens de baixo nível. Por conta disso, os compiladores surgiram como uma necessidade de identificar os símbolos de uma linguagem, compreende-los e traduzi-los para uma linguagem de uma arquitetura alvo (11).

Surgiram também algumas linguagens intermediárias, as de montagem, conhecidas como *Assembly*, que trazia um nível de compreensão superior das linguagens binárias das máquinas, tornando a criação de compiladores intermediários (nível *Assembly*) e compiladores objeto, também conhecido como montadores *Assembler*, que traduz *Assembly* para código executável.

3.2.1 Análise Léxica

A primeira etapa da fase de análise é a léxica (11), conhecida também como varredura, pois é nesse primeiro momento que será lido o arquivo fonte como uma cadeia de caracteres e será separado em marcas ou *tokens*. Esses *tokens* são constituídos de uma sequência de caracteres e representam unidades lógicas do nosso arquivo fonte, que posteriormente serão usados nas próximas fases de análise.

No que diz respeito a um programa, os *tokens* representam palavras reservadas, identificadores, números, símbolos especiais, etc. Desse modo, identificam os padrões do

código e para que isso seja possível, utiliza-se expressões regulares para esse reconhecimento de padrões.

O [Algoritmo 3.1](#) abaixo representa um trecho de código de um algoritmo em linguagem C-.

```
1 int valor;  
2 valor = 0;
```

Algoritmo 3.1 – Trecho de código em linguagem C- (c-menos).

Após uma varredura, um resultado de saída para essa análise é obtida como mostra os itens abaixo. É destacado cada lexema obtido, a linha em que ocorre e sua categoria.

```
1 palavra reservada: int  
1 identificador: valor  
1 símbolo especial: ;  
2 identificador: valor  
2 símbolo especial: =  
2 número: 0  
2 símbolo especial: ;
```

Para elaborar o analisador léxico C-, foi utilizado o gerador de varreduras *Flex* em C, que por meio de expressões regulares pode-se facilmente definir os *tokens*. Possui funções próprias que facilitam no desenvolvimento, como a **yylex**, que atua como rotina de varredura, entre outras. Gera como resultado o **lex.yy.c**, que posteriormente será compilado para ser executado sobre um arquivo de entrada de interesse.

Algumas expressões regulares foram adotadas para serem utilizadas no *Flex*, como mostra a [Figura 1](#). Eventualmente, pode haver mais de uma expressão que satisfaz a cadeia de caracteres atual, dessa forma o *Flex* utiliza a regra da cadeia mais longa para resolver essa ambiguidade.

Figura 1 – Expressões regulares adotadas para o Analisador Léxico.

digit	[0-9]
number	{digit}+
letter	[a-zA-Z]
alphanumeric	[a-zA-Z0-9]
identifier	{letter}+{alphanumeric}*
newline	[\n]
whitespace	[\t\r]+

Fonte: O Autor

Para gerar os *tokens*, foram adotadas as seguintes convenções da linguagem C-, como mostra a [Tabela 5](#), que representa a tabela de convenções léxicas, que será usada para fazer a identificação desses *tokens*. Além disso, quando algum caractere não é identificado como *token*, é retornado um erro.

Tabela 5 – Tabela de Convenções Léxicas

Convenções Léxicas C-	
Palavras Reservadas	if, else, int, return, void, while
Símbolos Especiais	+ - * / «= »= == != = ; , () [] { }
Identificadores	Inicia com letra, seguido de letras ou números
Número	Inteiro sem sinal
Espaço em Branco	Espaços, tabulações e novas linhas
Comentários	/* comentários */

Fonte: O Autor

O [Algoritmo 3.2](#), um exemplo de código em C-, foi submetido na rotina de varredura da análise léxica, sua saída é apresentada na [Figura 2](#).

```

1 int gcd (int u, int v){
2     if (v == 0) return u;
3     else return gcd(v,u-u/v*v); /* u-u/v*v == u mod v */
4 }
5 void main(void){
6     int x; int y;
7     x = input(); y = input();
8     output(gcd(x,y));
9 }

```

Algoritmo 3.2 – Algoritmo GCD em C-.

Figura 2 – Saída do analisador léxico do [Algoritmo 3.2](#) (GCD).

```

CMINUS COMPILATION: tst/teste.cn
1: reserved word: int      4: ID, name = v          11: {
1: ID, name = gcd         4: )                    11: ID, name = x
1: (                      4: ;                    11: ,
1: reserved word: int     6: }                    11: ID, name = y
1: ID, name = u           8: reserved word: void   11: )
1: ,                      8: ID, name = main      11: )
1: reserved word: int     8: (                    11: ;
1: ID, name = v           8: reserved word: void   12: )
1: )                      8: )                    12: }
2: {                      9: {
3: reserved word: if      9: reserved word: int
3: (                      9: ID, name = x
3: ID, name = v           9: ;
3: ==                    9: reserved word: int
3: NUM, val = 0          9: ID, name = y
3: )                      9: ;
3: reserved word: return 10: ID, name = x
3: ID, name = u          10: =
3: ;                     10: ID, name = input
4: reserved word: else    10: (
4: reserved word: return  10: )
4: ID, name = gcd         10: ;
4: (                      10: ID, name = y
4: ID, name = v           10: =
4: ,                     10: ID, name = input
4: ID, name = u           10: (
4: -                      10: )
4: ID, name = u           10: ;
4: /                      11: ID, name = output
4: ID, name = v           11: (
4: *                      11: ID, name = gcd

```

Fonte: O Autor

3.2.2 Análise Sintática

A análise sintática, conhecida como *parser* (11), tem o papel de determinar a estrutura sintática de um programa à partir da varredura, ou seja, por meio dos *tokens* obtidos. Para isso, utiliza de uma gramática livre de contexto que ira ditar as regras de como será essa estrutura. Nessa fase é requisitado, por meio da função **getToken** os *tokens* gerado na análise léxica, e caso não tenha erros, é gerado a árvore de análise sintática.

É utilizado uma gramática para definir as regras pois a mesma permite a recursão em suas derivações. Para programar uma gramática livre de contexto, é feito em uma **gramática BNF** (Forma de Backus-Naur) (11), que é comumente usada na programação.

Para fazer a análise sintática fora usado o gerador *parser* **Bison** em C, que é um **YACC** (*Yet Another Compiler-Compiler*) disponibilizado pelo GNU. Ele gera uma rotina **yyparse** que é responsável pela análise sintática do código fonte. O Bison elabora, à partir das regras, um *parser* **LALR(1)** (*Look-Ahead Left Right 1*), que faz a verificação à frente de um único símbolo para se obter o próximo *token*, de forma que a entrada processa da esquerda para à direita com derivação à direita. Além disso, consegue operar em conjunto com o *Flex*, o que os torna uma combinação muito interessante para o desenvolvimento do analisador.

A gramática utilizada para o Compilador C- está apresentada na [Figura 3](#).

Figura 3 – Gramática adotada para o Compilador C-.

```

programa → declaração-lista
declaração-lista → declaração-lista declaração | declaração
declaração → var-declaração | fun-declaração
var-declaração → tipo-especificador ID ; | tipo-especificador ID [ NUM ] ;
tipo-especificador → int | void
fun-declaração → tipo-especificador ID ( params ) composto-decl
params → param-lista | void
param-lista → param-lista, param | param
param → tipo-especificador ID | tipo-especificador ID [ ]
composto-decl → { local-declarações statement-lista } | { local-declarações } | { statement-lista } | { }
local-declarações → local-declarações var-declaração | var-declaração
statement-lista → statement-lista statement | statement
statement → expressão-decl | composto-decl | seleção-decl | iteração-decl | retorno-decl
expressão-decl → expressão ; ;
seleção-decl → if ( expressão ) statement | if ( expressão ) statement else statement
iteração-decl → while ( expressão ) statement
retorno-decl → return ; | return expressão;
expressão → var = expressão | simples-expressão
var → ID | ID [ expressão ]
simples-expressão → soma-expressão relacional soma-expressão | soma-expressão
relacional → <= | < | > | >= | == | !=
soma-expressão → soma-expressão soma termo | termo
soma → + | -
termo → termo mult fator | fator
mult → * | /
fator → ( expressão ) | var | ativação | NUM
ativação → ID ( arg-lista ) | ID ( )
arg-lista → arg-lista , expressão | expressão

```

Fonte: O Autor

A [Figura 4](#) é um trecho de código retirado do arquivo elaborado para Bison, que segue as regras apresentadas na [Figura 3](#).

Figura 4 – Trecho da Gramática BNF no Bison.

```
tipo_identif:      INT {
                    $$ = newStmtNode(IntegerK);
                    $$->type = Integer;
                    $$->op = INT;
                    }
                    |
                    VOID {
                    $$ = newStmtNode(VoidK);
                    $$->type = Void;
                    $$->op = VOID;
                    }
                    ;
```

Fonte: O Autor

Seguindo o exemplo apresentado na [subseção 3.2.1](#), o [Algoritmo 3.2](#) após a análise sintática tem como saída a árvore sintática apresentada na [Figura 5](#).

Figura 5 – Árvore sintática gerada pelo analisador sintático do [Algoritmo 3.2](#) (GCD).

```
Syntax tree:
int
  Function: gcd
    Params
      int
        ID: u
      int
        ID: v
    Scope
      If
        Relational Op: ==
          ID: v
          Literal: 0
        Return
          ID: u
        Return
          Call: gcd
            ID: v
            Arith Op: -
              ID: u
            Arith Op: *
              Arith Op: /
                ID: u
                ID: v
                ID: v
void
  Function: main
    Params
      void
    Scope
      int
        ID: x
      int
        ID: y
      Atrib: =
        ID: x
        Call: input
      Atrib: =
        ID: y
        Call: input
      Call: output
        Call: gcd
          ID: x
          ID: y
```

Fonte: O Autor

3.2.3 Análise Semântica

A última etapa é a fase da análise semântica ([11](#)), onde é verificadas certas regras que não foram possíveis, ou seriam muito complexas de serem atestadas na fase da análise

sintática, por conta disso certas regras da linguagem que se está compilando devem ser verificadas nessa etapa. Além disso, nessa parte é elaborado a tabela de símbolos, onde é armazenado informações muito importantes que serão utilizadas na etapa de síntese.

A linguagem C- possui algumas regras semânticas, e 7 delas foram verificadas e atestadas nessa etapa:

- Variável utilizada não foi declarada.
- Declaração inválida de variável: Void não é tipo válido.
- Declaração inválida de variável: Variável já declarada.
- Chamada de função não declarada.
- Função **main** não declarada.
- Declaração inválida: O identificador já fora usado como nome de função.
- Atribuição inválida: Void não pode ser atribuído ou comparado em expressão lógica.

Outras regras fora essas também foram implementadas, como a verificação se o usuário tentou declarar funções com identificadores reservados para funções primitivas da linguagem. As funções **int input(void)** e **void output(int valor)** não podem ser sobrescritas pelo usuário, por serem primitivas.

Para realizar a análise semântica, foi elaborado uma heurística que avalia os nós de identificador e aplica as regras propostas, enquanto elabora a tabela de símbolos, que também é utilizada na verificação, por meio de uma tabela *hash*, realizado pela função **buildSymtab**. Essa avaliação é feita utilizando apenas 2 passadas, ou seja, é executado o percurso na árvore sintática duas vezes, a primeira, descrita pela função **insertNode**, verifica as primeiras 6 regras semânticas e a segunda, descrita pela função **checkNode**, para a última, que diz respeito a checagem da semântica dos tipos.

3.2.3.1 Tabela de Símbolos

Conforme é feito o percurso na árvore, é inserido as variáveis e funções na tabela de símbolos, além de outras informações, como o tipo de dado, tipo de identificador, posição relativa em memória, tamanho em memória, parâmetros e ocorrências. A tabela de símbolos, nesse modelo implementado, é dividida em várias tabelas, nas quais cada uma representa um escopo diferente.

A [Figura 6](#) representa a tabela de símbolos gerada pelo exemplo do [Algoritmo 3.2](#) (GCD).

Figura 6 – Tabela de símbolos gerada do analisador semântico do Algoritmo 3.2 (GCD).

```
Building Symbol Table...
```

Tabelas de símbolos:

< __GLOBAL__ > : size(0)

Variable Name	Origem	Type ID	Type Data	Memloc	Size	Params	Line Numbers
main	Global	Function	void	-1	2	0	8
gcd	Global	Function	int	-1	2	2	1

< gcd >

Variable Name	Origem	Type ID	Type Data	Memloc	Size	Line Numbers
u	Parameter	Var	int	0	1	1 3 4
v	Parameter	Var	int	1	1	1 3 4
gcd	Local	Call	int	-1	0	4

< main >

Variable Name	Origem	Type ID	Type Data	Memloc	Size	Line Numbers
x	Local	Var	int	0	1	9 10 11
y	Local	Var	int	1	1	9 10 11
gcd	Local	Call	int	-1	0	11

Checking Types...

Type Checking Finished

Fonte: O Autor

3.2.4 Geração do código *Assembly*

A geração do código *Assembly* (11), elaborado em linguagem C, foi baseado no modelo de código de três endereços, conduzido por realizar a tradução diretamente para a linguagem de montagem, foi necessário também realizar o gerenciamento de registradores e de memória, controle da referência de vetores como parâmetro de função e recursividade. Para gerenciar os registradores, foi elaborado uma técnica **UFA** (*Use First Available*), onde foi definido um conjunto de registradores para uso geral e o algoritmo sempre usa o primeiro disponível, e para isso foi criada uma pilha em C (em tempo de compilação) que coordenada os registradores.

Para isso, duas funções principais são utilizadas:

- ***push_temp()***: Empilha o uso do registrador, ou seja, indica que o resultado de alguma operação será atribuída nesse registrador, que vai pra pilha.
- ***pop_temp()***: Desempilha o uso do registrador, ou seja, algum registrador que foi empilhado será usado para descarregar seu valor na instrução atual.

Valor ressaltar que essa pilha só gerencia qual registrador irá aparecer na instrução, o papel de executar as instruções é feito quando aplicado no processador alvo. Para exemplificar, a Figura 7 mostra o trecho do nó de operador relacional *less than*, nele podemos ver o uso das funções descritas:

- **cGen(f1)**: Empilha um registrador da execução da sub-árvore esquerda.
- **cGen(f2)**: Empilha um registrador da execução da sub-árvore direita.
- **op3 = pop_temp()**: Desempilha o registrador gerado por cGen(f2).
- **op2 = pop_temp()**: Desempilha o registrador gerado por cGen(f1).
- **op1 = push_temp()**: Empilha um registrador dessa instrução, que será usado por alguma outra instrução que requisitou a operação relacional.

Apesar de não termos o controle de saber para qual registrador está sendo atribuído na instrução, o fluxo do programa garante que todo registrador empilhado será desempilhado e usado, assim assegurando o funcionamento do código gerado.

Figura 7 – Trecho de código da operação relacional *Less Than*.

```
case RelK:
    cGen(f1);
    cGen(f2);
    switch (tree->op){
        case LT:
            op3 = pop_temp();
            op2 = pop_temp();
            op1 = push_temp();
            formatThreeArg(SLT, op1, op2, op3, NULL);
            break;
```

Fonte: O Autor

Por conta dessa técnica, também foi menos complexo a implementação da recursividade, pois como será melhor explicado na [subseção 3.2.6](#), quando uma função chama outra em seu escopo, duas funções são executadas:

- **push_context()**: Que empilha os registradores em uso na pilha do processador, por meio da instrução PUSH. Dessa forma salvando o contexto da execução daquela função.
- **pop_context()**: Que resgata os registradores que estavam sendo usados antes da chamada, desempilhando-os da pilha do processador, por meio da instrução POP. Dessa forma resgatando o contexto da execução daquela função, após retornar da chamada feita.

A função *cGen(TreeNode * tree)* é a que inicia o processo, que é realizado de forma recursiva, percorrendo a árvore que fora gerada na execução da fase de análise. Nela é chamada todas funções de cada tipo de nó: *Stmt*, *Exp*, *Decl* e *VarCall*. A [Figura 8](#) mostra esse procedimento.

Figura 8 – Função `cGen(TreeNode * tree)`.

```

void cGen(TreeNode * tree){
    if (tree != NULL){
        switch(tree->nodekind){
            case StmtK:
                cgenStmt(tree);
                break;

            case ExpK:
                cgenExp(tree);
                break;

            case VarK:
                if(tree->kind.var.mode == DeclK){
                    cgenVarDecl(tree);
                } else { // tree->kind.var.mode == VarCallK
                    cgenVarCall(tree);
                }
                break;

            default: // Nothing
                printf("error at cGen()\n");
                break;
        }
        cGen(tree->sibling);
    }
}

```

Fonte: O Autor

Cada tipo apresentado tem sua função específica:

- *Stmt*: Gerencia os nós de tomada de decisão (*If*, *While*), Retorno e de tipagens (*INT*, *VOID*).
- *Exp*: Gerencia os nós de atribuição, relacional e aritmético.
- *Decl*: Gerencia os nós de declaração de função e variável.
- *VarCall*: Gerencia os nós de chamada, seja de função, uso de variável, vetor ou valor numérico.

Outras funções também são usadas, como a *formatThreeArg()*, apresentada na Figura 7, que formata a saída da instrução de 3 operandos e escreve em um arquivo texto de saída. A Figura 9 mostra o resultado da execução do exemplo GCD, do Algoritmo 3.2.

Vale ressaltar que a primeira linha gerada no *Assembly*, no caso a 'RAM_INST', não é traduzida para código binário, apenas indica o rótulo da memória usada no processador alvo. Para os saltos condicionais e chamadas de função, também são atribuído rótulos, como no exemplo temos 'gcd' para indicar onde começa a função *gcd()* e '*_IfBegin0_*' indica o início da tomada de decisão do primeiro *If*. Esses rótulos terão seu papel na etapa de síntese do código executável, onde serão convertidos para valores binários.

Figura 9 – Código *Assembly* gerado do Algoritmo 3.2 (GCD).

```

RAM_INST      sw 2 $fp 0
li $fp 0      jal gcd
jump main     subi $fp $fp 2
noop / gcd    move 2 $rv
push $ra      move $rv 2
lw 2 $fp 1    pop $ra
li 3 0        jr $ra
set 2 2 3     noop / _IfExit0_
beq 2 $zero _ElseBegin0_ pop $ra
lw 2 $fp 0    jr $ra
move $rv 2    noop / main
pop $ra       in 2
jr $ra        sw 2 $fp 0
jump _IfExit0_ in 2
noop / _ElseBegin0_ sw 2 $fp 1
lw 2 $fp 1    lw 2 $fp 0
lw 3 $fp 0    lw 3 $fp 1
lw 4 $fp 0    addi $fp $fp 2
lw 5 $fp 1    sw 3 $fp 1
div 4 4 5     sw 2 $fp 0
lw 5 $fp 1    jal gcd
mult 4 4 5    subi $fp $fp 2
sub 3 3 4     move 2 $rv
addi $fp $fp 2 out 2
sw 3 $fp 1    halt / _halt_

```

Fonte: O Autor

3.2.5 Geração do código executável

Na última fase, é onde se elabora o código executável, em binário, seguindo o formato do conjunto de instruções da arquitetura alvo. Para essa etapa, se utiliza o arquivo texto gerado pela fase de síntese do *Assembly* e o realiza sua conversão em um novo arquivo texto. Devido a isso, nessa etapa foi feito um programa na linguagem JAVA, pelo fato de dar um suporte melhor a manipulação de *Strings* e arquivos, do que a linguagem C que fora utilizada em todo projeto.

Para realiza a conversão, todo arquivo é lido e salvo em uma *String*, e por meio da classe **StringTokenizer**, é feito a quebra e manipulação das instruções. Um passo importante que fora tratado aqui é a substituição dos rótulos feitos em cadeiras de caracteres na fase em número binário, por meio da classe **HashMap**, que faz um mapeamento das *labels* nas suas respectivas posições nas linhas, pois foi adotado que cada rótulo fica associado a uma instrução, e também é usado um **HashMap** para os registradores, convertendo os registradores nomeados, como por exemplo \$RA no valor 31, que é o número específico do registrador *Return Address* da arquitetura alvo.

Ao passo que lia cada linha do *Assembly*, é associado uma linha na saída resultante, e é identificado a instrução pelo seu *opcode*, e eventualmente o *funct*, em uma rotina de *switch case*. Algumas funções auxiliares foram elaboradas, como a **String toBinary(int**

num) e algumas expressões regulares, para verificar se o rótulo já está no formato numérico e não em cadeia de caracteres.

A Figura 10 mostra o resultado final do exemplo do GCD, descrito pelo Algoritmo 3.2. Pode-se verificar que já temos a saída em binário e também já é associado a memória do processador alvo, descrito pelo 'RAM_INST', e como comentário (à direita) o *Assembly* original.

Figura 10 – Código executável gerado do Algoritmo 3.2 (GCD).

```
RAM_INST[0] = 32'b000101_00000_11101_000000000000000; // li $fp 0
RAM_INST[1] = 32'b001011_0000000000000000000100010; // jump main
RAM_INST[2] = 32'b0000000000000000000000000000000; // noop / gcd
RAM_INST[3] = 32'b001111_11110_11111_000000000000000; // push $ra
RAM_INST[4] = 32'b000001_11101_00010_000000000000001; // lw 2 $fp 1
RAM_INST[5] = 32'b000101_00000_00011_000000000000000; // li 3 0
RAM_INST[6] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
RAM_INST[7] = 32'b000111_00010_00000_00000000000110; // beq 2 $zero _ElseBegin0_
RAM_INST[8] = 32'b000001_11101_00010_000000000000000; // lw 2 $fp 0
RAM_INST[9] = 32'b000000_00010_11100_00000_00000_010000; // move $rv 2
RAM_INST[10] = 32'b010000_11110_11111_000000000000000; // pop $ra
RAM_INST[11] = 32'b000000_11111_00000_00000_00000_010100; // jr $ra
RAM_INST[12] = 32'b001011_00000000000000000000011111; // jump _IfExit0_
RAM_INST[13] = 32'b0000000000000000000000000000000; // noop / _ElseBegin0_
RAM_INST[14] = 32'b000001_11101_00010_000000000000001; // lw 2 $fp 1
RAM_INST[15] = 32'b000001_11101_00011_000000000000000; // lw 3 $fp 0
RAM_INST[16] = 32'b000001_11101_00100_000000000000000; // lw 4 $fp 0
RAM_INST[17] = 32'b000001_11101_00101_000000000000001; // lw 5 $fp 1
RAM_INST[18] = 32'b000000_00100_00101_00100_00000_001000; // div 4 4 5
RAM_INST[19] = 32'b000001_11101_00101_000000000000001; // lw 5 $fp 1
RAM_INST[20] = 32'b000000_00100_00101_00100_00000_000111; // mult 4 4 5
RAM_INST[21] = 32'b000000_00011_00100_00011_00000_000110; // sub 3 3 4
RAM_INST[22] = 32'b000011_11101_11101_000000000000010; // addi $fp $fp 2
RAM_INST[23] = 32'b000010_11101_00011_000000000000001; // sw 3 $fp 1
RAM_INST[24] = 32'b000010_11101_00010_000000000000000; // sw 2 $fp 0
RAM_INST[25] = 32'b001100_000000000000000000000010; // jal gcd
RAM_INST[26] = 32'b000100_11101_11101_000000000000010; // subi $fp $fp 2
RAM_INST[27] = 32'b000000_11100_00010_00000_00000_010000; // move 2 $rv
RAM_INST[28] = 32'b000000_00010_11100_00000_00000_010000; // move $rv 2
RAM_INST[29] = 32'b010000_11110_11111_000000000000000; // pop $ra
RAM_INST[30] = 32'b000000_11111_00000_00000_00000_010100; // jr $ra
RAM_INST[31] = 32'b0000000000000000000000000000000; // noop / _IfExit0_
RAM_INST[32] = 32'b010000_11110_11111_000000000000000; // pop $ra
RAM_INST[33] = 32'b000000_11111_00000_00000_00000_010100; // jr $ra
RAM_INST[34] = 32'b0000000000000000000000000000000; // noop / main
RAM_INST[35] = 32'b001001_00000_00010_000000000000000; // in 2
RAM_INST[36] = 32'b000010_11101_00010_000000000000000; // sw 2 $fp 0
RAM_INST[37] = 32'b001001_00000_00010_000000000000000; // in 2
RAM_INST[38] = 32'b000010_11101_00010_000000000000001; // sw 2 $fp 1
RAM_INST[39] = 32'b000001_11101_00010_000000000000000; // lw 2 $fp 0
RAM_INST[40] = 32'b000001_11101_00011_000000000000001; // lw 3 $fp 1
RAM_INST[41] = 32'b000011_11101_11101_000000000000010; // addi $fp $fp 2
RAM_INST[42] = 32'b000010_11101_00011_000000000000001; // sw 3 $fp 1
RAM_INST[43] = 32'b000010_11101_00010_000000000000000; // sw 2 $fp 0
RAM_INST[44] = 32'b001100_000000000000000000000010; // jal gcd
RAM_INST[45] = 32'b000100_11101_11101_000000000000010; // subi $fp $fp 2
RAM_INST[46] = 32'b000000_11100_00010_00000_00000_010000; // move 2 $rv
RAM_INST[47] = 32'b001010_00010_00000_000000000000000; // out 2
RAM_INST[48] = 32'b1111111111111111111111111111111; // halt / _halt_
```

Fonte: O Autor

3.2.6 Gerenciamento da memória

O gerenciamento da memória foi feito utilizando o princípio de pilha, alocando memória a alguma função de forma dinâmica, ao passo que se tinha a chamada daquela função. Para isso foi usado uma pilha explícita da arquitetura, que tinha o papel de salvar

o contexto registradores em uso (como por exemplo o \$RA), por meio de PUSH e POP, e também um pilha implícita na memória principal, que teve o papel de controlar as chamadas de funções e alocar de forma dinâmica o uso da memória.

Devido a isso foi possível realizar:

- Chamadas encadeadas: Uma função pode chamar outras funções, pois seu contexto será salvo na pilha e na memória.
- Recursividade: Por salvar o contexto dos registradores e alocar memória de forma dinâmica, a recursividade foi possível, pois cada chamadas tem seu próprio espaço de memória separado.
- Retorno de Função: O retorno pelo \$RA em tempo de execução é possível, pois seu valor era sempre empilhado na pilha antes de uma nova chamada de função, e recuperado após a mesma.

O que tornou mais fácil o gerenciamento da memória foi o ponto da tabela de símbolos, gerada na fase de análise semântica, estar completa e com todas informações pertinentes a respeito de memória relativa e quantidade de espaço necessário por escopo, permitindo na etapa de síntese um controle mais intuitivo da memória.

3.3 Gerenciamento de Recursos do Sistema

O sistema operacional, como é apresentado por Tanenbaum (1), é "[...] *software*, cujo trabalho é gerenciar esses componentes e fornecer aos programas do usuário uma interface com o *hardware* mais simples". O SO pode ser compreendido como uma máquina estendida ou máquina virtual, cuja função é facilitar a programação, do ponto de vista do usuário, evitando o contato direto com os mecanismos de *hardware* presentes no sistema, sendo portanto um provedor de uma interface a seus usuários, considerada uma visão *top-down*. Por outro lado, pode ser visto também como um gerenciador de recursos, sendo uma visão *bottom-up*, onde tem o trabalho de gerenciar as partes de um sistema complexo, como cita Tanenbaum (1) "[...] fornecer uma alocação ordenada e controlada de processadores, memórias e dispositivos de E/S entre vários programas que competem por eles".

3.3.1 Basic Input/Output System (BIOS)

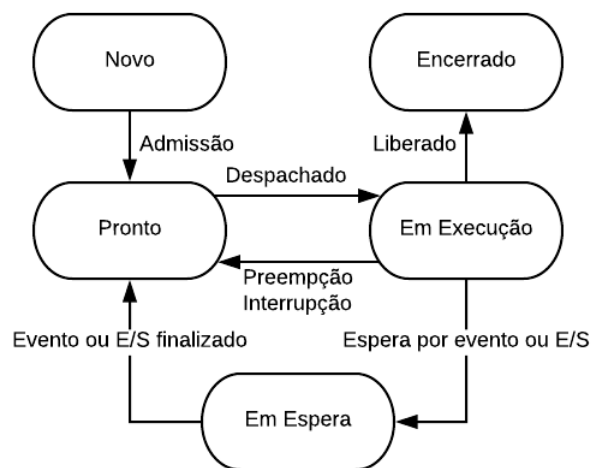
A BIOS é o componente intrínseco dos sistemas computacionais, muitas vezes pré-designado nas *motherboards*, cujo objetivo compete a ativar o *hardware* e inicializar o sistema como um todo, carregando o SO na memória, iniciando os dispositivos de E/S, checagem de básica de erros, etc. Após isso, transfere o trabalho de gerenciar os recursos para o sistema operacional carregado.

3.3.2 Gerenciamento de Processos

Um sistema operacional moderno opera sobre múltiplos processos, ou seja, consegue executar vários processos de forma concorrente, escalonando o uso dos recursos entre eles respeitando a política imposta e a justiça de divisão dos recursos. Por esse motivo, é necessário um *Scheduler*, que irá realizar o papel de escalonar os processos que estão executando, bloquear processos e liberar processos.

Como é apresentado na Figura 11, o escalonador precisa admitir novos processos, executar os processos da fila de pronto, e liberar, bloquear ou preemptar aqueles em execução, além de recolocar na fila de pronto aqueles processos bloqueados.

Figura 11 – Diagrama de estados dos processos.



Fonte: O Autor

3.3.2.1 Escalonador de processos

Como é apresentado na literatura (1), tem-se algoritmos clássicos para escalonamento em vários ambientes, como os sistemas: **Lote**, **Interativo** e **Tempo real**. Desses, será tomada uma atenção especial para os sistemas em lote e interativos.

Para sistemas em lote tem-se:

- *First Come, first served*: Há uma fila única de processos que devem ser executados. Quando um processo em execução é bloqueado, o primeiro da fila de pronto será executado, e quando o bloqueado fica pronto, é colocado no final da fila.
- *Shortest job first*: Supõe que é previamente conhecido o tempo de execução dos processos, então é feita uma fila ordenada, cujo critério seguido será os menores processos executam primeiro.

Para sistemas interativos tem-se:

- Escalonamento por alternância circular (*Round-Robin*): Cada processo é atribuído um intervalo de tempo, denominado *quantum*, no qual ele é permitido executar (1). Ao final dessa fatia de tempo, se o processo atual ainda estiver executando, será feito a preempção e assim o próximo processo irá tomar a CPU, e esse processo que saiu irá retornar para o final da fila. Dessa forma, o escalonador só precisa manter uma fila dos processos executáveis.
- Escalonamento por prioridades: A cada processo é atribuído uma prioridade, e ao processo executável com maior prioridade é permitido executar. Parecido com o *Round-Robin*, contudo o escalonador deve manter nesse várias filas de prioridade, ao invés de uma única.
- Próximo processo mais curto (*Shortest process next*): Tenta adaptar o *Shortest job first* para sistemas interativos, assim ele utiliza de uma estimativa de tempo de execuções passadas de um mesmo processo, atribuindo pesos para essa estimativa conforme acontece novas execuções, essa técnica é muitas vezes conhecida como *aging*.
- Escalonamento por loteria: É distribuído uma quantidade de bilhetes para um processo, e por meio de um sorteio pelo pela CPU, o processo a qual compete aquele bilhete, toma posse dos recursos do sistema. Dessa forma, é como um escalonamento por prioridades, pois os processos com mais bilhetes tem mais chance de tomar posse dos recursos.

3.3.3 Gerenciamento de Memória

Programas tendem a se expandir a fim de tomar toda memória disponível, como cita Tanenbaum (1). Dessa forma, o papel do gerenciador de memória é manter o controle dos espaços de memória em uso e liberados, alocando memória para os processos que os requisitam e liberando para aqueles que terminam. Além disso pode realizar *swapping*, que é utilizar partes do HD como uma memória estendida, uma vez que esse é um recurso mais abundante do que a memória principal, contudo não é necessariamente necessário.

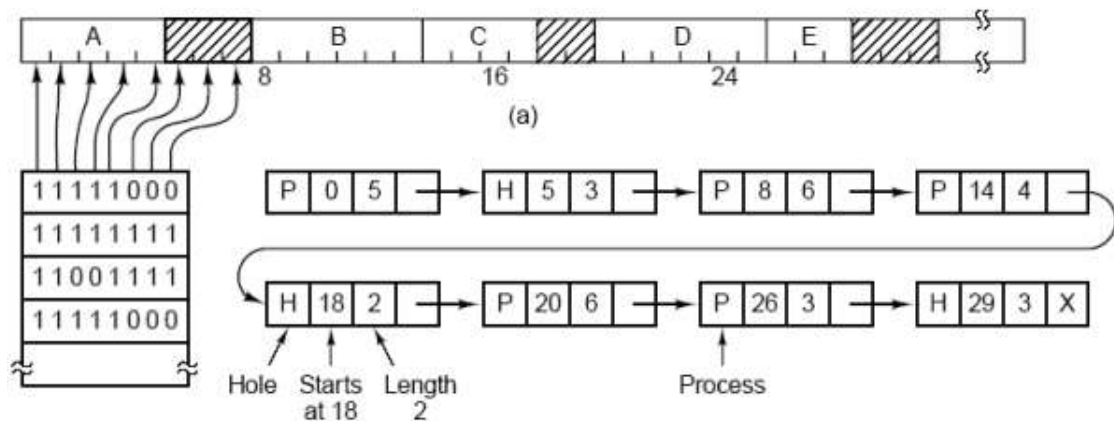
3.3.3.1 Alocação e divisão da memória principal

Algumas técnicas conhecidas para divisão dos recursos de memória, tais quais:

- Partições fixas: Dividi-se a memória em n partições, possivelmente com tamanhos distintos. Assim quando necessário, é alocado o espaço de memória para o processo que o requisita.

- Segmentação: A memória pode ser realocada com tamanho requisitado, atribuindo de forma sequencial. Acaba criando espaço segmentados, por não conseguir realocar os espaço atualmente vigentes na memória.
- Mapa de *bits*: Usada para alocações dinâmicas, onde guarda o início e fim do espaço de memória atualmente utilizado pelos processos. Como é apresentado na Figura 12.
- Paginação: É feito uma divisão em páginas, assim pode-se estender a memória original em uma memória virtual, utilizando partes do HD para salvar partes da memória principal (*swapping*), para buscar uma página que não fora encontrada, um *page miss*. Para realizar o *swapping*, existem algumas técnicas como: Primeiro a entrar, primeiro a sair, Não usado recentemente, Segunda chance, Relógio, etc.

Figura 12 – Gerenciamento de memória por mapa de *bits*.



Fonte: Sistemas Operacionais Modernos - Tanenbaum (1).

3.3.4 Gerenciamento do Sistemas de Arquivos

Arquivos são formas de salvar os dados de forma persistente, ou seja, podem ser recuperados futuramente, diferentemente de dados salvos na memória principal. Como Tanenbaum diz (1), arquivos são gerenciados pelo sistema operacional, pois ele irá atuar no modo como são estruturados, nomeados, acessados, usados, protegidos e implementados.

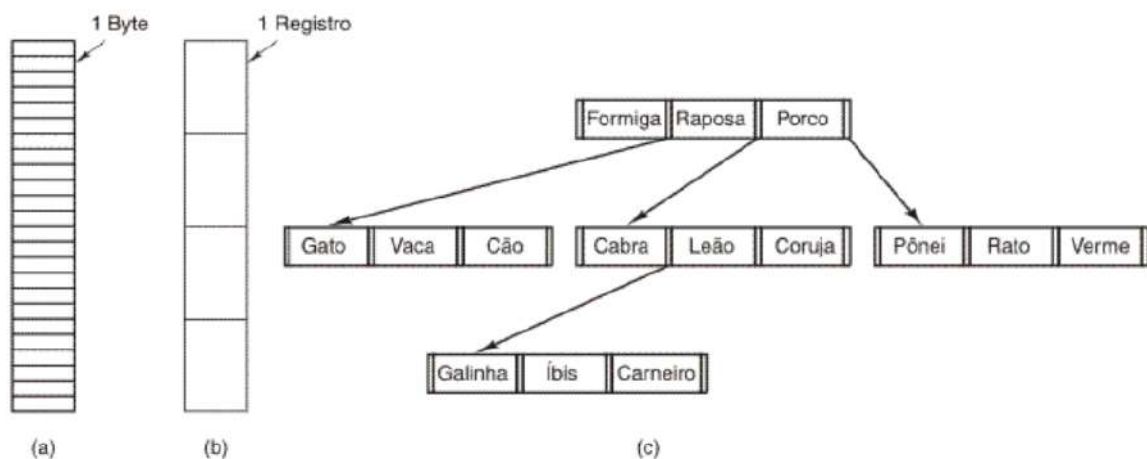
3.3.4.1 Estrutura dos arquivos

No que diz respeito à estrutura dos arquivos, usa-se três metodologias mais comuns para sua estruturação (1), como mostra a Figura 13, que são:

- Sequência de *bytes*. O arquivo é visto como uma coleção de *bytes*, não importando o conteúdo.

- Sequência de registros. O arquivo é uma sequência de registros e tamanho fixo, cada uma com uma estrutura interna.
- Árvores. O arquivo é considerado uma árvore de registros, não necessariamente todos do mesmo tamanho e todos contendo um campo-chave em uma posição fixa no registro.

Figura 13 – Estrutura de arquivos. (a) Sequência de *bytes*, (b) Sequência de registros e (c) Árvore.

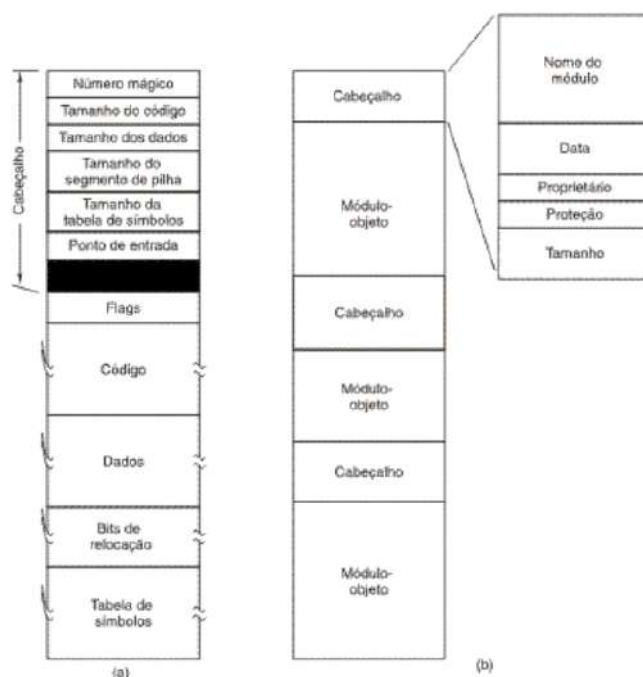


Fonte: Sistemas Operacionais Modernos - Tanenbaum (1).

3.3.4.2 Tipos de arquivos

No que compõe as características dos arquivos, podemos dividi-los em arquivos executáveis e arquivos texto. O primeiro irá descrever um programa que poderá ser executado pelo sistema operacional sobre a arquitetura alvo, o segundo apenas contém informações para serem recuperadas futuramente. A [Figura 14](#) descreve como é essa divisão.

Figura 14 – Tipos de arquivos. (a) Executáveis e (b) Texto.



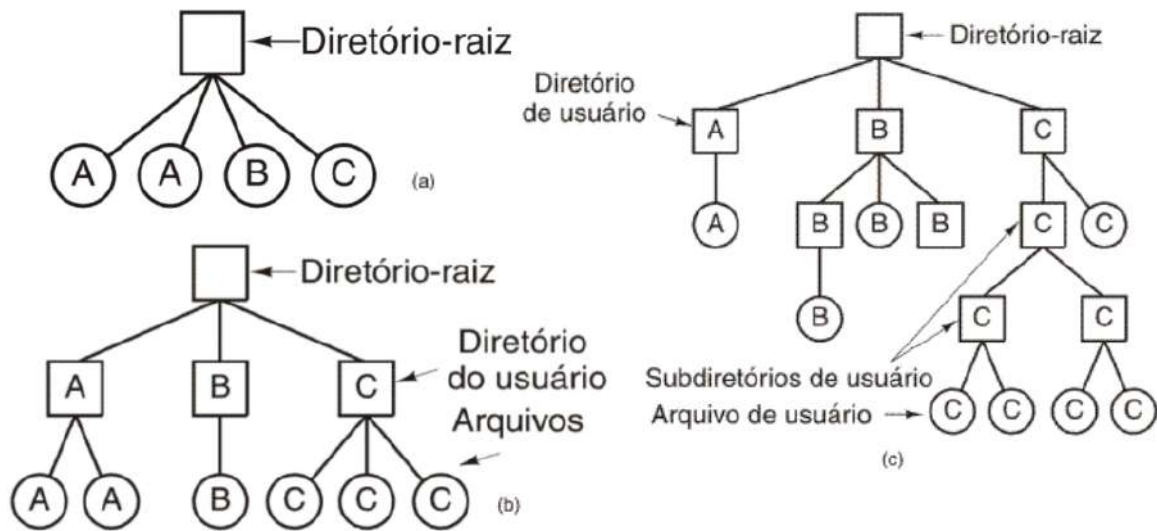
Fonte: Sistemas Operacionais Modernos - Tanenbaum (1).

3.3.4.3 Diretórios

Para gerenciar os arquivos utiliza-se diretórios, que controla os arquivos e pastas por nomes, em geral de forma hierárquica. As maneiras mais comuns de elaborar um diretório são três, como é apresentado na [Figura 15](#).

- **Nível único:** Organiza todos os arquivos na raiz, não opera sobre pastas.
- **Nível duplo:** Organiza os arquivos nas pastas de usuários, tornando possível cada usuário separar seus próprios arquivos.
- **Hierárquico:** Organiza os arquivos de forma hierárquica, tornando possível a criação de pastas e subpastas, além de permitir a montagem de novos dispositivos de E/S em qualquer ponto do diretório.

Figura 15 – Diretórios. (a) Nível único, (b) Nível duplo e (c) Hierárquico.



Fonte: Sistemas Operacionais Modernos - Tanenbaum (1).

3.3.4.4 Acesso e alocação de arquivos

Para realizar o acesso aos arquivos, pode-se ter:

- Acesso sequencial: Lê os *bytes* desde o início, não podendo saltar para a posição desejada.
- Acesso aleatório: Lê os *bytes* ou registros em qualquer ordem, pode saltar para posição desejada, como a trilha e setor de um disco rígido.

Para criar um novo arquivos é necessário alocar espaço em disco para isso, e portanto são necessárias técnicas para manter esse registro do espaço utilizados. As metodologias mais utilizadas são (1):

- Alocação contígua: É armazenado os arquivos em blocos contíguos de tamanho fixo do disco. É interessante para armazenar onde se encontra o arquivo, além da leitura ser inteira, uma vez que armazena todo arquivo junto. Contudo, possuem a desvantagem de deixar o disco segmentado ao realizar operações de remoção de arquivos.
- Alocação por lista encadeada: É armazenado os arquivos em blocos encadeados por lista, utilizando melhor o disco e não permitindo segmentações, porém torna ineficiente uma leitura aleatória, percorrendo muitos blocos para se encontrar a posição desejada.

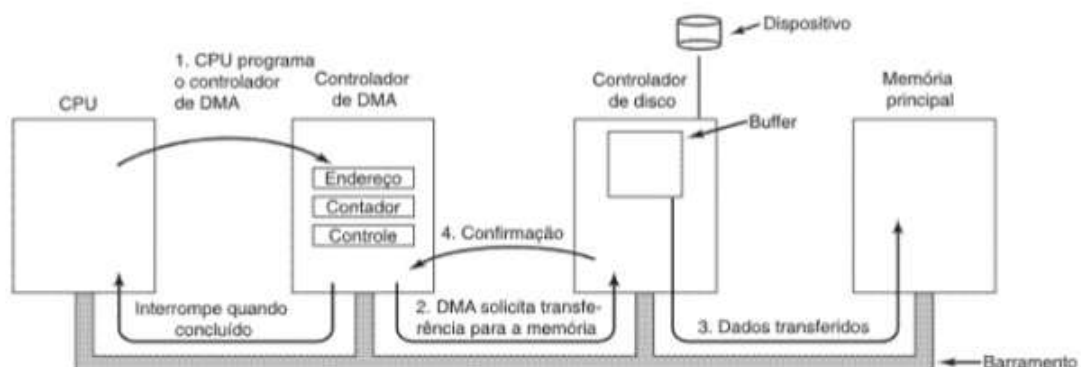
- Alocação por lista encadeada usando uma tabela na memória: Semelhante ao item anterior, mas implementa uma tabela em memória para salvar os índices dos arquivos, eliminando o problema de busca, contudo é necessário um grande espaço em memória.
- *i-node*: associa arquivos a estrutura de dados, para controlar os blocos a qual cada arquivo pertence. Dado um *i-node* pode-se encontrar todos os blocos daquele arquivo. E diferentemente do método anterior, é necessário salvar os índices apenas no momento que o arquivo está aberto (em uso).

3.3.5 Gerenciamento de Dispositivos de E/S

Fornece maneiras de controlar os dispositivos e realizar o *buffer* dos dados a serem lidos/escritos, por meio de interrupções. Como o sistema pode ter inúmeros dispositivos de E/S é necessário ter um controle de cada dispositivo. Muitas vezes se associa o componente eletrônico de controlador do dispositivo ou adaptador. Para realizar o gerenciamento desses dispositivos, algumas técnicas que são empregadas:

- E/S mapeada em memória: Cada controlador tem registradores usados para efetuar a comunicação com a CPU. Assim, por meio da escrita sobre esses registradores, pode-se comandar o dispositivo para receber ou enviar dados. O mapeamento pode ser feito de forma separada do espaço da memória de dados ou junto.
- *Direct memory access* (DMA): O controlador da DMA tem acesso direto ao barramento do sistema, de forma independente da CPU. Dessa forma, ao finalizar o *buffering* dos dados, faz uma interrupção à CPU, indicando que os dados estão prontos na memória principal. A Figura 16 mostra o esquemático do funcionamento da DMA com a CPU.

Figura 16 – Esquemático do DMA.



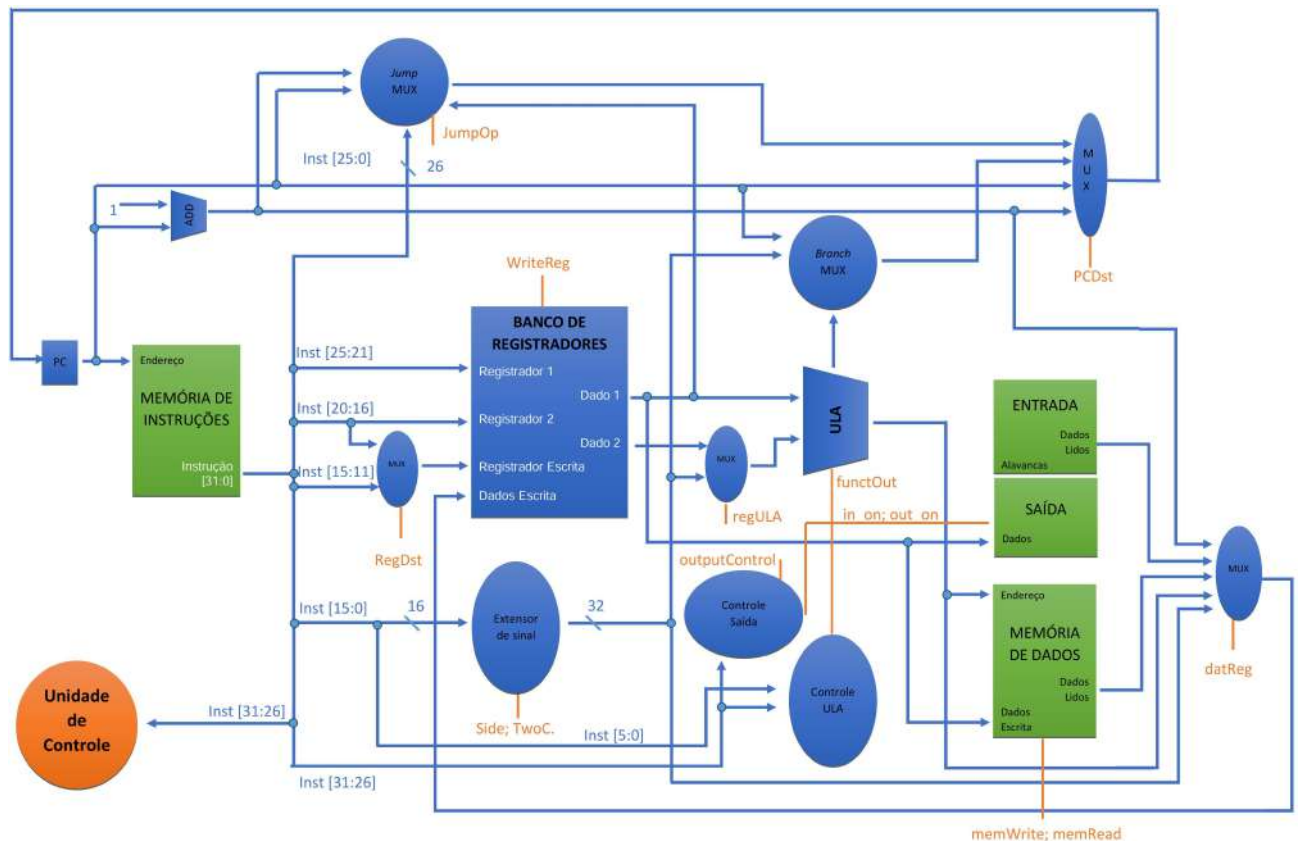
Fonte: Sistemas Operacionais Modernos - Tanenbaum (1).

4 Desenvolvimento

4.1 Arquitetura Alvo

O processador desenvolvido, baseado na arquitetura MIPS (6), que trabalha em *Single Cycle*, realiza operações em dados na forma de complemento de dois. Opera com uma frequência de $5kHz$, que garante uma execução de algoritmos de forma extremamente rápida. A Figura 17 mostra o *datapath* da arquitetura alvo e a Tabela 6 representa o conjunto de instruções elaborado, em Apêndice A.

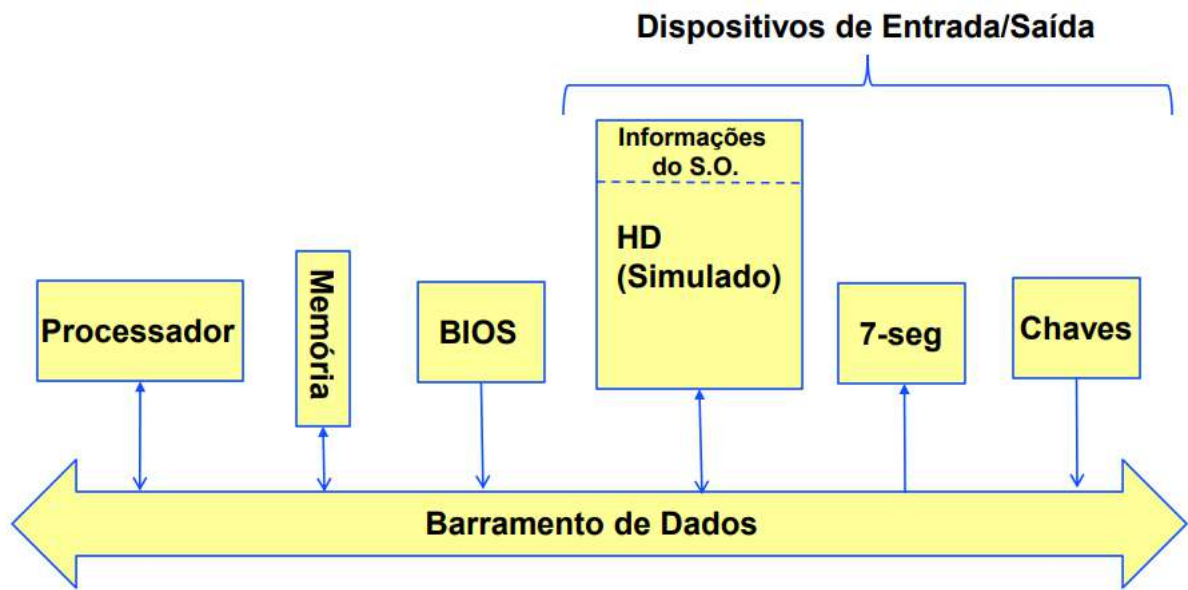
Figura 17 – *Datapath* do Processador



Fonte: O Autor

4.2 Modificações e Alterações na Arquitetura

O modelo apresentado na Figura 18 mostra os componentes do sistema a serem elaborados. Para isso é necessário alterar componentes da CPU e incluir a unidade de BIOS e HD, assim o sistema operacional irá operar sobre essa nova arquitetura, previamente adequada para seu funcionamento.

Figura 18 – Plataforma de *hardware* do Sistema.

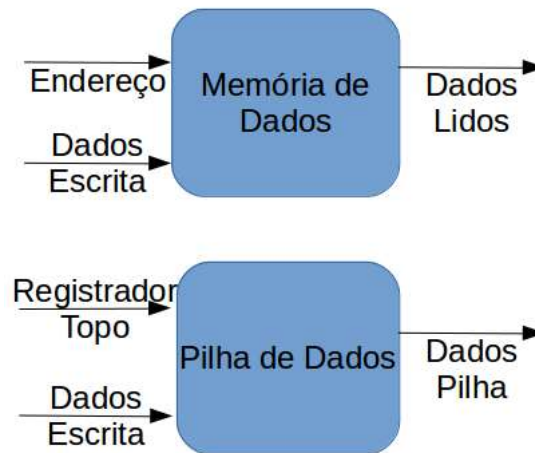
Fonte: Material de Apoio da UC (12).

4.2.1 Organização da Memória de Dados e Pilha

A memória de dados (7) é responsável por armazenar dados gerais a respeito do programa, com exceção das instruções que são armazenadas em uma memória à parte, seguindo o conceito de uma arquitetura *Harvard* (13). Foi projetada para armazenar valores de 32 *bits*, opera na subida do *clock* para escrita de dados, e atualmente realiza a leitura dos dados na subida do *clock_board*, que é de 50Mhz, simulando uma leitura direta, dessa forma o *software Quartus* (8) consegue associar a uma memória nativa da placa FPGA (10).

A pilha de dados é uma parte da memória separada da memória principal, que é uma pilha do processador. Como mostra na Tabela 6 em Apêndice A, as instruções de PUSH e POP utilizam essa pilha implementada em *hardware*, permitindo o usuário inserir ou retirar do topo da pilha por meio dessas instruções específicas. Para ter o controle do topo da pilha, usa-se um registrador \$sp que guarda o topo da pilha do processo vigente, então o endereçamento é feito de forma implícita, o usuário não consegue alterar diretamente esse valor, somente o *kernel* do S.O. pode modificá-lo. A Figura 19 mostra como ficou a divisão da memória principal com a pilha de dados.

Figura 19 – Memória de Dados e Pilha.



Fonte: O Autor.

O Algoritmo 4.1 apresenta a codificação em Verilog da pilha de dados elaborada, essa que tem como finalidade ser utilizada pelo compilador para realizar chamadas de funções em cascata e recursividade.

```

1 module stack_data(write_clock, read_clock, stackPointer, data, dataOut, stackOp);
2     parameter DATA_WIDTH = 32;
3     parameter STACK_BITS = 12;
4     localparam STACK_SIZE = 2**STACK_BITS;
5     input [DATA_WIDTH-1:0] data;
6     input [DATA_WIDTH-1:0] stackPointer;
7     input [1:0] stackOp;
8     input write_clock, read_clock;
9     output [DATA_WIDTH-1:0] dataOut;
10    reg [DATA_WIDTH-1:0] dataRead;
11    reg [DATA_WIDTH-1:0] STACK [STACK_SIZE:0];
12    always @ (posedge write_clock)
13    begin
14        if (stackOp == 2'b01) begin
15            STACK[stackPointer] <= data;
16        end
17    end
18    always @ (posedge read_clock)
19    begin
20        dataRead <= STACK[stackPointer-1];
21    end
22    assign dataOut = (stackOp != 2'b10 || stackPointer == 0) ? 0 : dataRead;
23 endmodule

```

Algoritmo 4.1 – Pilha de Dados desenvolvida em Verilog.

4.2.2 BIOS e Gerenciamento do PC

A BIOS é um componente ROM do sistema, cujas instruções de execução são pré-fixadas em código *Verilog* (9), de forma que deve realizar os seguintes procedimentos antes da execução do S.O.:

- Teste de partida: Verificação e diagnóstico dos componentes do sistema.
- Identificação do dispositivo de armazenamento principal. Neste caso, o disco rígido.
- Carregar o S.O. para a memória principal e inicializar a execução da mesma.
- Ativar a CPU para a execução do S.O.

O Algoritmo 4.2 apresenta a BIOS elaborada em Verilog.

```

1 module bios(write_clock, read_clock, reset, biosAddress, out);
2     parameter DATA_WIDTH = 32;
3     parameter BIOS_BITS = 9;
4     localparam BIOS_SIZE = 2**BIOS_BITS;
5
6     input write_clock, read_clock, reset;
7     input [DATA_WIDTH-1:0] biosAddress;
8     output reg [DATA_WIDTH-1:0] out;
9
10    reg [DATA_WIDTH-1:0] BIOS [BIOS_SIZE-1:0];
11
12    localparam N0 = 8'h30, N1 = 8'h31, N2 = 8'h32, N3 = 8'h33, N4 = 8'h34, N5 = 8'h35,
13    N6 = 8'h36, N7 = 8'h37, N8 = 8'h38, N9 = 8'h39,
14    AA = 8'h41, BB = 8'h42, CC = 8'h43, DD = 8'h44, EE = 8'h45, FF = 8'h46, GG = 8'h47,
15    HH = 8'h48, II = 8'h49, JJ = 8'h4A, KK = 8'h4B, LL = 8'h4C, MM = 8'h4D, NN = 8'h4E,
16    OO = 8'h4F, PP = 8'h50, QQ = 8'h51, RR = 8'h52, SS = 8'h53, TT = 8'h54, UU = 8'h55,
17    VV = 8'h56, WW = 8'h57, XX = 8'h58, YY = 8'h59, ZZ = 8'h5A,
18    A = 8'h61, B = 8'h62, C = 8'h63, D = 8'h64, E = 8'h65, F = 8'h66, G = 8'h67,
19    H = 8'h68, I = 8'h69, J = 8'h6A, K = 8'h6B, L = 8'h6C, M = 8'h6D, N = 8'h6E,
20    O = 8'h6F, P = 8'h70, Q = 8'h71, R = 8'h72, S = 8'h73, T = 8'h74, U = 8'h75,
21    V = 8'h76, W = 8'h77, X = 8'h78, Y = 8'h79, Z = 8'h7A,
22    BLANK = 8'h20, PIPE = 8'h7C, EQUAL = 8'h3D, LTHAN = 8'h3C, GTHAN = 8'h3E,
23    TIMES = 8'h2A, PLUS = 8'h2B, MINUS = 8'h2D, DIV = 8'h2F,
24    QMARK = 8'h3F;
25
26    always @ (posedge read_clock)
27    begin
28        out <= BIOS[biosAddress];
29    end
30
31    always @ (*)
32    begin
33        BIOS[0] <= 32'b000101_00000_00001_0000000000001010; // li 1 10
34        BIOS[1] <= 32'b000101_00000_00010_0110000110101000; // li 2 25000 2 deslocado
35        BIOS[2] <= 32'b000101_00000_00011_0110000110101000; // li 3 25000
36        BIOS[3] <= 32'b000101_00000_00100_0000000000000000; // li 4 0 / so_begin
37        BIOS[4] <= 32'b000101_00000_00101_0000101110110011; // li 5 2995 / so_end
38        BIOS[5] <= 32'b000000_00010_00010_00000_00010_000000; // sll 2 2 2
39        BIOS[6] <= 32'b000000_00011_00011_00000_00001_000000; // sll 3 3 1
40        BIOS[7] <= 32'b00000000000000000000000000000000; //noop
41        BIOS[8] <= 32'b00000000000000000000000000000000; //noop
42        BIOS[9] <= 32'b00000000000000000000000000000000; //noop
43        BIOS[10] <= 32'b00000000000000000000000000000000; //noop
44        BIOS[11] <= 32'b00000000000000000000000000000000; //noop
45        BIOS[12] <= 32'b00000000000000000000000000000000; //noop
46        BIOS[13] <= 32'b00000000000000000000000000000000; //noop
47        BIOS[14] <= 32'b00000000000000000000000000000000; //noop
48        BIOS[15] <= 32'b00000000000000000000000000000000; //noop

```

```

49     BIOS[16] <= {24'b100111_00001_00000_00000_000, BB}; // noop / _BIOS__STARTING_
50     BIOS[17] <= {24'b100111_00010_00000_00000_000, II}; // noop
51     BIOS[18] <= {24'b100111_00011_00000_00000_000, OO}; // noop
52     BIOS[19] <= {24'b100111_00100_00000_00000_000, SS}; // noop
53     BIOS[20] <= {24'b100111_00111_00000_00000_000, SS}; // noop
54     BIOS[21] <= {24'b100111_01000_00000_00000_000, TT}; // noop
55     BIOS[22] <= {24'b100111_01001_00000_00000_000, AA}; // noop
56     BIOS[23] <= {24'b100111_01010_00000_00000_000, RR}; // noop
57     BIOS[24] <= {24'b100111_01011_00000_00000_000, TT}; // noop
58     BIOS[25] <= {24'b100111_01100_00000_00000_000, II}; // noop
59     BIOS[26] <= {24'b100111_01101_00000_00000_000, NN}; // noop
60     BIOS[27] <= {24'b100111_01110_00000_00000_000, GG}; // noop
61     BIOS[28] <= 32'b000000_00010_01010_00000_00000_010000; // move 10 2
62     BIOS[29] <= 32'b000000_01010_01010_00000_00001_000001; // srl 10 10 1
63     BIOS[30] <= 32'b000101_00000_01011_0000000000000000; // li 11 0
64     BIOS[31] <= 32'b000011_01011_01011_0000000000000001; // inc 11 / sleep1
65     BIOS[32] <= 32'b001000_01011_01010_1111111111111111; // bne 11 10 sleep1
66     BIOS[33] <= 32'b100110_000000000000000000000000; // clear
67     BIOS[34] <= {24'b100111_00001_00000_00000_000, SS}; // noop / _System_Loading_
68     BIOS[35] <= {24'b100111_00010_00000_00000_000, Y}; // noop
69     BIOS[36] <= {24'b100111_00011_00000_00000_000, S}; // noop
70     BIOS[37] <= {24'b100111_00100_00000_00000_000, T}; // noop
71     BIOS[38] <= {24'b100111_00101_00000_00000_000, E}; // noop
72     BIOS[39] <= {24'b100111_00110_00000_00000_000, M}; // noop
73     BIOS[40] <= {24'b100111_01000_00000_00000_000, LL}; // noop
74     BIOS[41] <= {24'b100111_01001_00000_00000_000, O}; // noop
75     BIOS[42] <= {24'b100111_01010_00000_00000_000, A}; // noop
76     BIOS[43] <= {24'b100111_01011_00000_00000_000, D}; // noop
77     BIOS[44] <= {24'b100111_01100_00000_00000_000, I}; // noop
78     BIOS[45] <= {24'b100111_01101_00000_00000_000, N}; // noop
79     BIOS[46] <= {24'b100111_01110_00000_00000_000, G}; // noop
80     BIOS[47] <= 32'b000000_00011_01010_00000_00000_010000; // move 10 3
81     BIOS[48] <= 32'b000000_01010_01010_00000_00001_000001; // srl 10 10 1
82     BIOS[49] <= 32'b000101_00000_01011_0000000000000000; // li 11 0
83     BIOS[50] <= 32'b000000_01011_01011_00000_00000_001001; // inc 11 / sleep2
84     BIOS[51] <= 32'b001000_01011_01010_1111111111111111; // bne 11 10 sleep2
85     BIOS[52] <= {24'b100111_10000_00000_00000_000, GTHAN}; // noop
86     BIOS[53] <= {24'b100111_11111_00000_00000_000, LTHAN}; // noop
87     BIOS[54] <= 32'b000000_00011_01010_00000_00000_010000; // move 10 3
88     BIOS[55] <= 32'b000000_01010_01010_00000_00001_000001; // srl 10 10 1
89     BIOS[56] <= 32'b000101_00000_01011_0000000000000000; // li 11 0
90     BIOS[57] <= 32'b000000_01011_01011_00000_00000_001001; // inc 11 / sleep3
91     BIOS[58] <= 32'b001000_01011_01010_1111111111111111; // bne 11 10 sleep3
92     BIOS[59] <= {24'b100111_10010_00000_00000_000, GTHAN}; // noop
93     BIOS[60] <= {24'b100111_11101_00000_00000_000, LTHAN}; // noop
94     BIOS[61] <= 32'b000000_00011_01010_00000_00000_010000; // move 10 3
95     BIOS[62] <= 32'b000000_01010_01010_00000_00001_000001; // srl 10 10 1
96     BIOS[63] <= 32'b000101_00000_01011_0000000000000000; // li 11 0
97     BIOS[64] <= 32'b000000_01011_01011_00000_00000_001001; // inc 11 / sleep4
98     BIOS[65] <= 32'b001000_01011_01010_1111111111111111; // bne 11 10 sleep4
99     BIOS[66] <= {24'b100111_10100_00000_00000_000, GTHAN}; // noop
100    BIOS[67] <= {24'b100111_11011_00000_00000_000, LTHAN}; // noop
101    BIOS[68] <= 32'b000000_00011_01010_00000_00000_010000; // move 10 3
102    BIOS[69] <= 32'b000000_01010_01010_00000_00001_000001; // srl 10 10 1
103    BIOS[70] <= 32'b000101_00000_01011_0000000000000000; // li 11 0
104    BIOS[71] <= 32'b000000_01011_01011_00000_00000_001001; // inc 11 / sleep5
105    BIOS[72] <= 32'b001000_01011_01010_1111111111111111; // bne 11 10 sleep5
106    BIOS[73] <= {24'b100111_10110_00000_00000_000, GTHAN}; // noop
107    BIOS[74] <= {24'b100111_11001_00000_00000_000, LTHAN}; // noop
108    BIOS[75] <= 32'b000000_00011_01010_00000_00000_010000; // move 10 3

```

```

109     BIOS[76] <= 32'b000000_01010_01010_00000_00001_000001; // srl 10 10 1
110     BIOS[77] <= 32'b000101_00000_01011_0000000000000000; // li 11 0
111     BIOS[78] <= 32'b000000_01011_01011_00000_00000_001001; // inc 11 / sleep6
112     BIOS[79] <= 32'b001000_01011_01010_1111111111111111; // bne 11 10 sleep6
113     BIOS[80] <= 32'b000000_00100_01010_00000_00000_010000; // move 10 4
114     BIOS[81] <= 32'b000000_00101_01011_00000_00000_010000; // move 11 5
115     BIOS[82] <= 32'b100011_00000_01010_01010_00000_000000; // hdtoint / load_so
116     BIOS[83] <= 32'b000000_01010_01010_00000_00000_001001; // inc 10
117     BIOS[84] <= 32'b001000_01010_01011_1111111111111110; // bne 10 11 load_so
118     BIOS[85] <= 32'b100110_00000000000000000000000000; // noop / clear1
119     BIOS[86] <= {24'b100111_00001_00000_00000_000, SS}; // noop / _System__loaded_
120     BIOS[87] <= {24'b100111_00010_00000_00000_000, Y}; // noop
121     BIOS[88] <= {24'b100111_00011_00000_00000_000, S}; // noop
122     BIOS[89] <= {24'b100111_00100_00000_00000_000, T}; // noop
123     BIOS[90] <= {24'b100111_00101_00000_00000_000, E}; // noop
124     BIOS[91] <= {24'b100111_00110_00000_00000_000, M}; // noop
125     BIOS[92] <= {24'b100111_01001_00000_00000_000, LL}; // noop
126     BIOS[93] <= {24'b100111_01010_00000_00000_000, O}; // noop
127     BIOS[94] <= {24'b100111_01011_00000_00000_000, A}; // noop
128     BIOS[95] <= {24'b100111_01100_00000_00000_000, D}; // noop
129     BIOS[96] <= {24'b100111_01101_00000_00000_000, E}; // noop
130     BIOS[97] <= {24'b100111_01110_00000_00000_000, D}; // noop
131     BIOS[98] <= {24'b100111_10000_00000_00000_000, E}; // noop / enter_zero_input
132     BIOS[99] <= {24'b100111_10001_00000_00000_000, N}; // noop
133     BIOS[100] <= {24'b100111_10010_00000_00000_000, T}; // noop
134     BIOS[101] <= {24'b100111_10011_00000_00000_000, E}; // noop
135     BIOS[102] <= {24'b100111_10100_00000_00000_000, R}; // noop
136     BIOS[103] <= {24'b100111_10110_00000_00000_000, Z}; // noop
137     BIOS[104] <= {24'b100111_10111_00000_00000_000, E}; // noop
138     BIOS[105] <= {24'b100111_11000_00000_00000_000, R}; // noop
139     BIOS[106] <= {24'b100111_11001_00000_00000_000, O}; // noop
140     BIOS[107] <= {24'b100111_11011_00000_00000_000, I}; // noop
141     BIOS[108] <= {24'b100111_11100_00000_00000_000, N}; // noop
142     BIOS[109] <= {24'b100111_11101_00000_00000_000, P}; // noop
143     BIOS[110] <= {24'b100111_11110_00000_00000_000, U}; // noop
144     BIOS[111] <= {24'b100111_11111_00000_00000_000, T}; // noop
145     BIOS[112] <= 32'b001001_00000_01100_0000000000000000; // in 12 / zero_input
146     BIOS[113] <= 32'b000010_00000_01100_0000000000000000; // sw 12 $zero 0
147     BIOS[114] <= 32'b000001_00000_01101_0000000000000000; // lw 13 $zero 0
148     BIOS[115] <= 32'b001000_01101_00000_1111111111111101; // bne 13 $zero zero_input
149     BIOS[116] <= 32'b00000000000000000000000000000000; //noop
150     BIOS[117] <= 32'b00000000000000000000000000000000; //noop
151     BIOS[118] <= 32'b00000000000000000000000000000000; //noop
152     BIOS[119] <= 32'b00000000000000000000000000000000; //noop
153     BIOS[120] <= 32'b00000000000000000000000000000000; //noop
154     BIOS[121] <= 32'b00000000000000000000000000000000; //noop
155     BIOS[122] <= 32'b00000000000000000000000000000000; //noop
156     BIOS[123] <= 32'b00000000000000000000000000000000; //noop
157     BIOS[124] <= 32'b100110_00000000000000000000000000; // noop / clear2
158     BIOS[125] <= {24'b100111_00011_00000_00000_000, WW}; // noop / ___Welcome_to___
159     BIOS[126] <= {24'b100111_00100_00000_00000_000, E}; // noop
160     BIOS[127] <= {24'b100111_00101_00000_00000_000, L}; // noop
161     BIOS[128] <= {24'b100111_00110_00000_00000_000, C}; // noop
162     BIOS[129] <= {24'b100111_00111_00000_00000_000, O}; // noop
163     BIOS[130] <= {24'b100111_01000_00000_00000_000, M}; // noop
164     BIOS[131] <= {24'b100111_01001_00000_00000_000, E}; // noop
165     BIOS[132] <= {24'b100111_01011_00000_00000_000, T}; // noop
166     BIOS[133] <= {24'b100111_01100_00000_00000_000, O}; // noop
167     BIOS[134] <= {24'b100111_10001_00000_00000_000, I}; // noop / _iloop32_system_
168     BIOS[135] <= {24'b100111_10010_00000_00000_000, L}; // noop

```

```

169     BIOS[136] <= {24'b100111_10011_00000_00000_000, 0}; // noop
170     BIOS[137] <= {24'b100111_10100_00000_00000_000, 0}; // noop
171     BIOS[138] <= {24'b100111_10101_00000_00000_000, P}; // noop
172     BIOS[139] <= {24'b100111_10110_00000_00000_000, N3}; // noop
173     BIOS[140] <= {24'b100111_10111_00000_00000_000, N2}; // noop
174     BIOS[141] <= {24'b100111_11001_00000_00000_000, S}; // noop
175     BIOS[142] <= {24'b100111_11010_00000_00000_000, Y}; // noop
176     BIOS[143] <= {24'b100111_11011_00000_00000_000, S}; // noop
177     BIOS[144] <= {24'b100111_11100_00000_00000_000, T}; // noop
178     BIOS[145] <= {24'b100111_11101_00000_00000_000, E}; // noop
179     BIOS[146] <= {24'b100111_11110_00000_00000_000, M}; // noop
180     BIOS[147] <= 32'b000000_00010_01010_00000_00000_010000; // move 10 2
181     BIOS[148] <= 32'b000000_01010_01010_00000_00001_000001; // srl 10 10 1
182     BIOS[149] <= 32'b000101_00000_01011_000000000000000000; // li 11 0
183     BIOS[150] <= 32'b000000_01011_01011_00000_00000_001001; // inc 11 / sleep7
184     BIOS[151] <= 32'b001000_01011_01010_1111111111111111; // bne 11 10 sleep7
185     BIOS[152] <= 32'b000101_00000_00001_000000000000000000; // li 1 0
186     BIOS[153] <= 32'b000101_00000_00010_000000000000000000; // li 2 0
187     BIOS[154] <= 32'b000101_00000_00011_000000000000000000; // li 3 0
188     BIOS[155] <= 32'b000101_00000_00100_000000000000000000; // li 4 0
189     BIOS[156] <= 32'b000101_00000_00101_000000000000000000; // li 5 0
190     BIOS[157] <= 32'b000101_00000_00110_000000000000000000; // li 6 0
191     BIOS[158] <= 32'b000101_00000_00111_000000000000000000; // li 7 0
192     BIOS[159] <= 32'b000101_00000_01000_000000000000000000; // li 8 0
193     BIOS[160] <= 32'b000101_00000_01001_000000000000000000; // li 9 0
194     BIOS[161] <= 32'b000101_00000_01010_000000000000000000; // li 10 0
195     BIOS[162] <= 32'b000101_00000_01011_000000000000000000; // li 11 0
196     BIOS[163] <= 32'b000101_00000_01100_000000000000000000; // li 12 0
197     BIOS[164] <= 32'b000101_00000_01101_000000000000000000; // li 13 0
198     BIOS[165] <= 32'b000101_00000_01110_000000000000000000; // li 14 0
199     BIOS[166] <= 32'b000101_00000_01111_000000000000000000; // li 15 0
200     BIOS[167] <= 32'b000101_00000_10000_000000000000000000; // li 16 0
201     BIOS[168] <= 32'b000101_00000_10001_000000000000000000; // li 17 0
202     BIOS[169] <= 32'b000101_00000_10010_000000000000000000; // li 18 0
203     BIOS[170] <= 32'b000101_00000_10011_000000000000000000; // li 19 0
204     BIOS[171] <= 32'b000101_00000_10100_000000000000000000; // li 20 0
205     BIOS[172] <= 32'b000101_00000_10101_000000000000000000; // li 21 0
206     BIOS[173] <= 32'b000101_00000_10110_000000000000000000; // li 22 0
207     BIOS[174] <= 32'b000101_00000_10111_000000000000000000; // li 23 0
208     BIOS[175] <= 32'b000101_00000_11000_000000000000000000; // li 24 0
209     BIOS[176] <= 32'b000101_00000_11001_000000000000000000; // li 25 0
210     BIOS[177] <= 32'b000101_00000_11010_000000000000000000; // li 26 0
211     BIOS[178] <= 32'b000101_00000_11011_000000000000000000; // li 27 0
212     BIOS[179] <= 32'b000101_00000_11100_000000000000000000; // li 28 0
213     BIOS[180] <= 32'b000101_00000_11101_000000000000000000; // li 29 0
214     BIOS[181] <= 32'b000101_00000_11110_000000000000000000; // li 30 0
215     BIOS[182] <= 32'b000101_00000_11111_000000000000000000; // li 31 0
216     BIOS[183] <= 32'b100010_0000000000000000000000000000; // upmem
217     end
218
219 endmodule

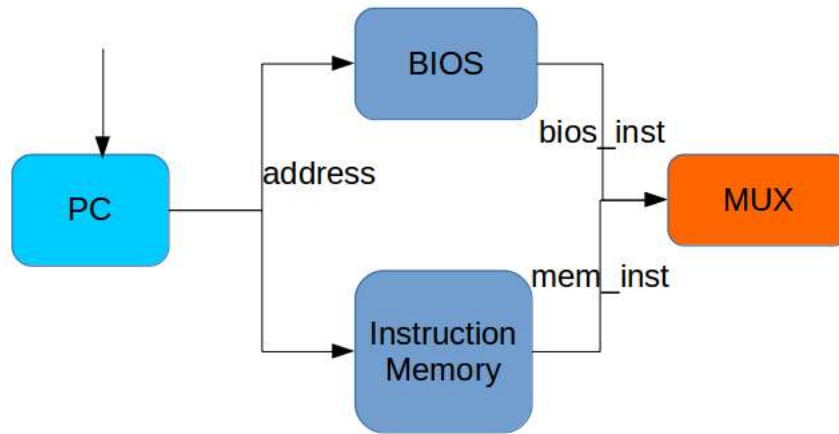
```

Algoritmo 4.2 – Modulo da BIOS em Verilog.

Como é apresentado na [Figura 20](#), o *program counter* irá tomar uma instrução ou da BIOS (inicialização) ou da memória de instruções (execução), e um componente multiplexador irá tomar a instrução a ser executada. Para dar a posse de execução do PC entre a BIOS e a memória de instruções, duas instruções específicas são utilizadas:

UPMEM, da a posse para a memória e **UPBIOS**, da a posse para a BIOS, ambas realizam o *reset* do PC.

Figura 20 – BIOS.



Fonte: O Autor.

O [Algoritmo 4.3](#) realiza a multiplexação entre BIOS e memória de instruções. O estado inicial ao ligar o sistema é da BIOS, que irá realizar os procedimentos especificados acima, feito em *assembly*, e ao final irá executar a instrução **UPMEM**, dando a posse de execução a memória de instruções, naturalmente o sistema operacional, visto que o mesmo estará carregado.

```

1 module bios_control(clock, reset, resetPC, biosInst, memInst, out);
2     parameter DATA_WIDTH = 32;
3     localparam BIOS = 1'b0, MEMORY = 1'b1;
4     localparam UPBIOS = 6'b100001, UPMEM = 6'b100010;
5     input clock, reset;
6     input [DATA_WIDTH-1:0] biosInst, memInst;
7     output [DATA_WIDTH-1:0] out;
8     output reg resetPC;
9     reg state;
10    always @ (posedge clock) begin
11        if(reset) begin
12            state <= BIOS;
13            resetPC <= 1'b1;
14        end
15        else if (state == BIOS) begin
16            case(biosInst[31:26])
17                UPMEM: begin
18                    state <= MEMORY;
19                    resetPC <= 1'b1;
20                end
21                UPBIOS: begin
22                    state <= BIOS;
23                    resetPC <= 1'b1;
24                end
25                default: begin
26                    resetPC <= 1'b0;
27                end

```

```

28         endcase
29     end else begin
30         case(memInst[31:26])
31             UPMEM: begin
32                 state <= MEMORY;
33                 resetPC <= 1'b1;
34             end
35             UPBIOS: begin
36                 state <= BIOS;
37                 resetPC <= 1'b1;
38             end
39             default: begin
40                 resetPC <= 1'b0;
41             end
42         endcase
43     end
44 end
45 assign out = (state == BIOS) ? biosInst : memInst;
46 endmodule

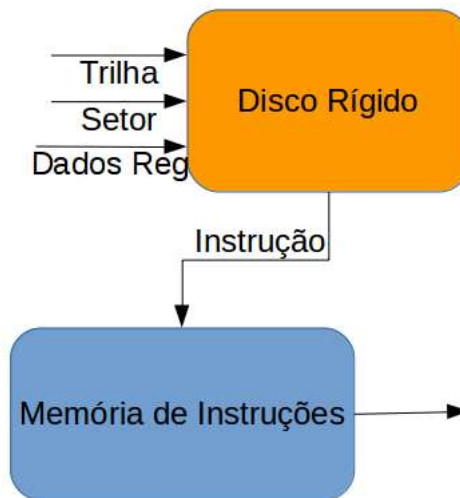
```

Algoritmo 4.3 – Gerenciamento do uso da BIOS e Memória de Instruções.

4.2.3 Disco Rígido

O disco rígido é simulado em *hardware*, por meio de *verilog*. A partir dos modelos de memória convencionais, como é feito por exemplo com a memória RAM. Contudo, são utilizados parâmetros, como trilha e setor, para simular esta memória como um disco rígido. E como é apresentado na [Figura 21](#), por meio de *syscalls*, é feito o acesso ao HD e é carregado esses dados lidos na memória de instruções, como por exemplo o próprio sistema operacional, quando a arquitetura foi inicializada ou resetada. Essas *syscalls* fazem com que o *kernel* use novas instruções, que são **HDTOREG**, carregar dados do HD em registrador, **REGTOHD**, carregar dados de registradores no HD e **HDTOINST**, carregar do HD para memória de instruções.

Figura 21 – HD simulado.



Fonte: O Autor.

O Algoritmo 4.4 é o HD desenvolvido em *verilog*, que utiliza parâmetros de trilha e setor para simular o acesso a algum conteúdo salvo nesta unidade. No bloco *initial* é salvo o sistema operacional, os programas iniciais e demais informações necessárias para inicialização.

```

1 module hard_disk(write_clock, read_clock, reset, trilha, setor, data, dataOut, hdWrite,
  hdRead);
2     parameter DATA_WIDTH = 32; // X bits
3     parameter HD_SETORES_BITS = 10;
4     localparam HD_TRILHAS_SIZE = 32;
5     localparam HD_SETORES_SIZE = 2**HD_SETORES_BITS;
6     localparam HD_SIZE = HD_TRILHAS_SIZE*HD_SETORES_SIZE;
7     input [DATA_WIDTH-1:0] data;
8     input [DATA_WIDTH-1:0] trilha;
9     input [DATA_WIDTH-1:0] setor;
10    input reset, hdWrite, hdRead, write_clock, read_clock;
11    output [DATA_WIDTH-1:0] dataOut;
12    reg [DATA_WIDTH-1:0] dataRead;
13    reg [DATA_WIDTH-1:0] HD [HD_SIZE-1:0]; // HD_SIZE words of 32-bit memory
14    initial
15    begin : INIT
16        //conteudo
17    end
18    always @ (posedge write_clock)
19    begin
20        if (hdWrite == 1'b1) begin
21            HD[trilha*HD_SETORES_SIZE+setor] <= data;
22        end
23    end
24    always @ (posedge read_clock)
25    begin
26        dataRead <= HD[trilha*HD_SETORES_SIZE+setor];
27    end
28    assign dataOut = (hdRead == 1'b1) ? dataRead : 0;

```

29 `endmodule`

Algoritmo 4.4 – Disco Rígido em Verilog.

4.2.3.1 Atraso do Disco Rígido Simulado

Para deixar mais próximo de uma simulação real, foi necessário desenvolver um componente que representasse o atraso do acesso ao HD, assim ele deve tomar mais tempo do CPU para realizar uma instrução básica, tanto de leitura ou escrita, desse modo este componente atrasa uma operação do HD em 2^N *clocks* do CPU, de modo que N é um parâmetro em *hardware*.

O Algoritmo 4.5 é o componente em *hardware* que realiza o atraso do HD. Por padrão $N = 2$, deixando assim as operações do HD com um custo de 4 *clocks*, mas pode ser facilmente alterado para qualquer valor desejado.

```

1 module hd_timer(isAct, opcode, clk, hd_ready, isHDOP);
2     parameter DIVISOR_BITS = 2;
3     localparam DIVISOR = 2**DIVISOR_BITS;
4     localparam HDT0INST = 6'b100011, HDT0REG = 6'b100100, REGT0HD = 6'b100101;
5     input clk, isAct;
6     input [5:0] opcode;
7     output reg hd_ready;
8     output isHDOP;
9     reg [DIVISOR_BITS-1:0] estado = 0;
10    always@(posedge clk)
11    begin
12        if (isAct) begin
13            if(estado == DIVISOR - 1) begin
14                hd_ready = 1'b1;
15            end
16            else begin
17                hd_ready = 1'b0;
18            end
19            if (isHDOP) begin
20                estado = estado + 1;
21            end
22            else begin
23                estado = 0;
24            end
25        end
26        else begin
27            hd_ready <= 1'b1;
28            estado <= 0;
29        end
30    end
31    assign isHDOP = (opcode == HDT0INST || opcode == HDT0REG || opcode == REGT0HD);
32 endmodule

```

Algoritmo 4.5 – Atraso do Disco Rígido.

Para essa configuração, como o *clock* do CPU é 50Khz, o *clock* do HD será de 12,5Khz, logo pode carregar até 12500 registros por segundo, como cada registro do HD é de 4 *bytes*, podemos considerar que o HD projetado tem uma velocidade de 50KBps.


```

9      input clk, reset;
10     input [1:0] PCDest;
11     input [ADDRESS_SIZE-1:0] PCout, inst_mem;
12     input isHDOP;
13     input [1:0] isIO;
14     output [ADDRESS_SIZE-1:0] inst_out;
15     output reg intr;
16     reg [QUANTUM_COUNT-1:0] count;
17     reg state;
18     always@(posedge clk)
19     begin
20         if(reset || intr) begin
21             state <= OFF;
22         end
23         else begin
24             case(inst_mem[31:26])
25                 INTR_ON: begin
26                     state <= ON;
27                 end
28                 INTR_OFF: begin
29                     state <= OFF;
30                 end
31                 HALT: begin
32                     state <= OFF;
33                 end
34                 RETURN: begin
35                     state <= OFF;
36                 end
37                 SYSCALL: begin
38                     if(state == ON) begin
39                         if(count[QUANTUM_COUNT-1:2] == 6'b000000) begin
40                             count = 0;
41                         end
42                         else begin
43                             count = count - 8'b100;
44                         end
45                     end
46                 end
47             endcase
48         end
49         if (reset || state == OFF) begin
50             count <= 0;
51             intr <= OFF;
52         end
53         else begin
54             if(count == QUANTUM - 1) begin
55                 intr = ON;
56             end
57             else begin
58                 intr = OFF;
59             end
60             if (PCDest != 2'b01 && isHDOP == 1'b0 && isIO == 2'b00)
61                 begin
62                     count = count + {{QUANTUM_COUNT_M1{1'b0}},1'b1};
63                 end
64             end
65         assign inst_out = (intr == OFF) ? inst_mem : PREEMP_INST;

```

66 `endmodule`

Algoritmo 4.6 – Interrupção de Relógio.

4.2.5 Terminal de Informações

Os valores numéricos, resultantes dos cálculos e algoritmos a serem executados, serão apresentados no *display* de 7 segmentos. Além disso, o *display* LCD irá apresentar os dados do sistema operacional, no que diz respeito ao estado do programa atual e também atuará como um menu de informações e seleção. A [Figura 23](#) mostra os blocos apresentados. No que cabe ao menu apresentado, podem ser realizadas as seguintes operações pelo usuário:

- Executar programas.
 - Execução com preempção.
 - Execução serial.
- Escrever programas.
- Renomear programas.
- Deletar programas.
- Listar programas.
- Reiniciar SO.
- Desligar SO.
- Tela de *help*.

Figura 23 – Unidades de *Output* do S.O.

Fonte: O Autor.

O *display* LCD criado utiliza a biblioteca disponibilizada por Loomis (14), que compõe o básico necessário para utilizar este componente. Além disso, foi elaborado uma memória de LCD, que guarda um *char* em binário para cada posição do LCD. O [Algoritmo 4.7](#) descreve esse módulo.

```

1 module lcd_ram(write_clock, read_clock, reset, lcdPos, lcdPosRead, data, lcdmemWrite,
  dataRead);
2     parameter DATA_WIDTH = 8;
3     parameter BITS_SIZE = 5;
4     localparam LCD_RAM_SIZE = 2**BITS_SIZE;
5     input [DATA_WIDTH-1:0] data;
6     input [BITS_SIZE-1:0] lcdPos, lcdPosRead;
7     input write_clock, read_clock, reset;
8     input [1:0] lcdmemWrite;
9     output reg [DATA_WIDTH-1:0] dataRead;
10    reg [DATA_WIDTH-1:0] LCD_RAM [LCD_RAM_SIZE-1:0];
11    integer i;
12    always @ (posedge write_clock)
13    begin
14        if (lcdmemWrite == 2'b10 || reset == 1'b1) begin
15            for(i = 0; i < 32; i = i + 1) begin
16                LCD_RAM[i] <= 8'h20; //blank
17            end
18        end
19        else if (lcdmemWrite == 2'b01) begin
20            LCD_RAM[lcdPos] <= data;
21        end
22    end
23    always @ (posedge read_clock)
24    begin
25        dataRead <= LCD_RAM[lcdPosRead];
26    end
27 endmodule

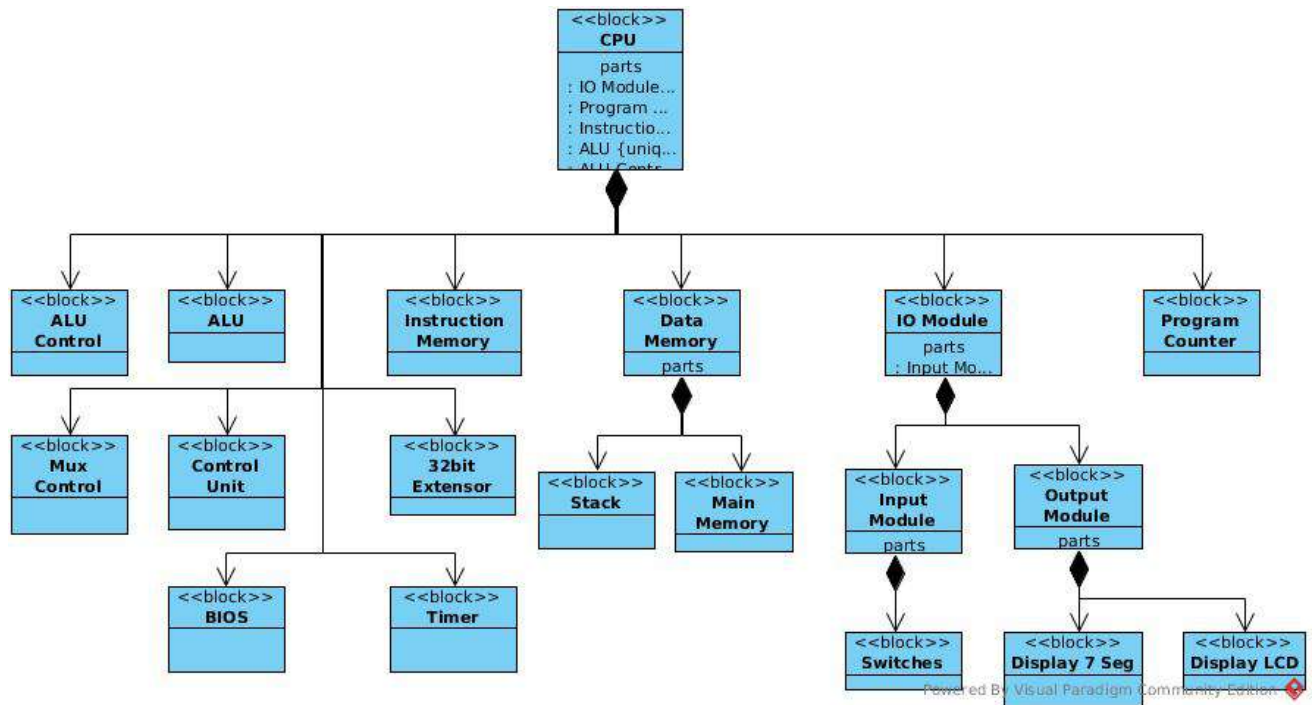
```

Algoritmo 4.7 – Memória do LCD.

4.2.7 Diagrama de Blocos do Sistema

A Figura 25 representa o diagrama de blocos da arquitetura desenvolvida, incluindo as modificações propostas para atender os requisitos do sistema operacional.

Figura 25 – Diagrama de blocos do CPU.

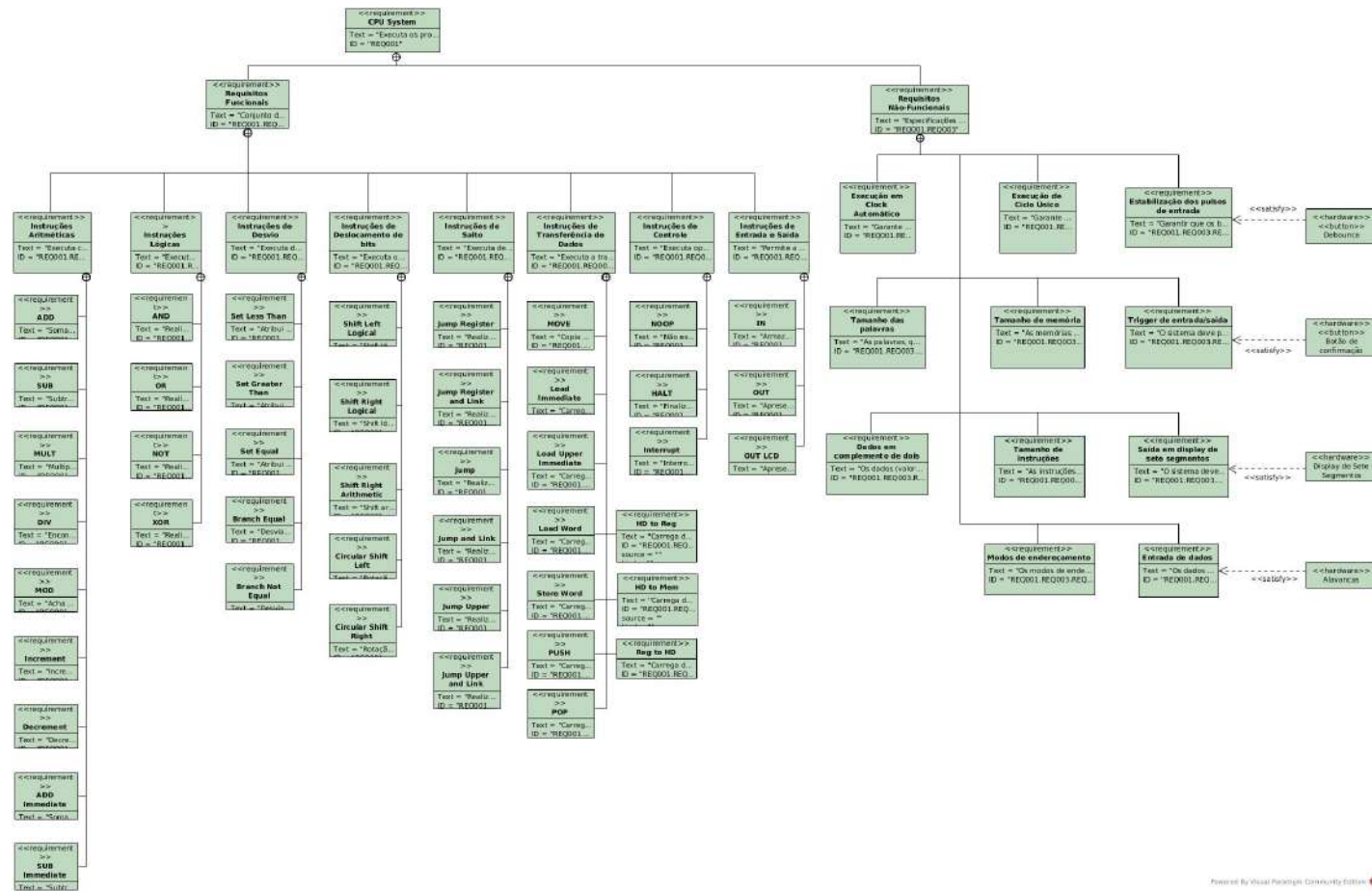


Fonte: O Autor

4.2.8 Diagrama de Requisitos do Sistema

A Figura 26 representa os requisitos da arquitetura desenvolvida, incluindo as modificações, para atender as necessidades do S.O.

Figura 26 – Diagrama de requisitos do CPU.

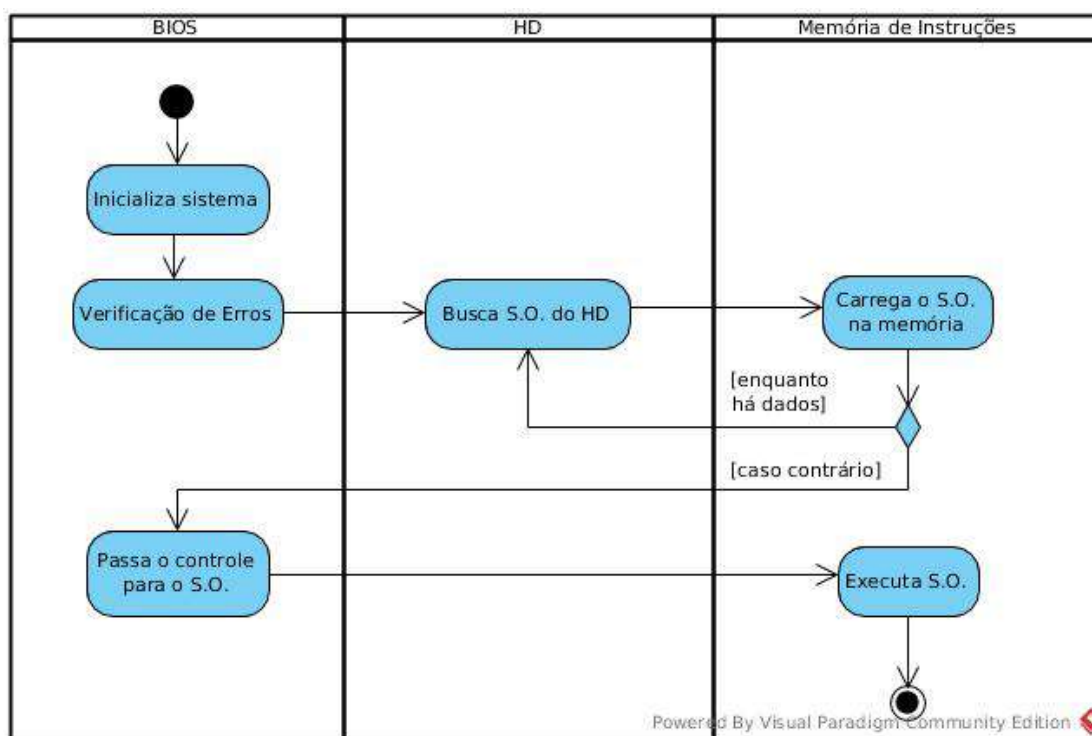


Fonte: O Autor

4.2.9 Diagramas de Atividades do Sistema

A Figura 27 representa o diagrama de atividades da inicialização do sistema, esse que inclui a BIOS e o HD no processo de inicializar o S.O.

Figura 27 – Diagrama de atividades da inicialização do SO.



Fonte: O Autor

4.3 Gerenciamento de Processos

Para realizar o gerenciamento de processos, foi utilizado o algoritmo de *Round Robin*, na qual um vetor estático irá simular uma lista circular da lista de processos prontos, de forma que o processo no início da fila toma os recursos da arquitetura.

Para limitar o uso dos recursos para um único processo e realizar a troca de contexto, é utilizado um *quantum* fixo, com base no número de instruções executadas, em *hardware*, com exceções de instruções que tomam mais de um *clock*, como de entrada e saída. Desse modo, um novo módulo, chamado de *Interruptor*, estará presente na arquitetura, de modo que irá armazenar contadores, e ao passo que é executado os algoritmos, ele envia um sinal para o *Controller*, indicando que é necessário realizar a preempção do programa atual, assim efetivamente o *Interruptor* toma o controle e realiza saltos condicionais para uma região da memória de instruções específica, de modo que o SO retoma o controle e realiza a troca de contexto de forma efetiva, descrito pelo Algoritmo 4.8, onde pode-se notar funções de sistema que o compilador interpreta, como: **label**, define um ponto de

retorno, **preemptive_mode**, liga ou desliga o modo preempção, e **jump_label**, devolve o controle para algum ponto de retorno, com essas funções e variáveis de controle, o SO consegue retomar de onde parou e passar a vez para outro processo tomar posse dos recursos. O componente *Interruptor* está descrito na [subseção 4.2.4](#). Além disso, o programa pode se preemptar, chamando o *return* para indicar que finalizou e o sistema remove-lo da fila de processos ou realizar uma *syscall* de entrada e saída, dando a prioridade para o sistema mais uma vez, que efetivamente faz o *input/output*.

```

1 void retoma_execucao_so(){
2     label(0);          /* system function - it comes after return */
3     HALT_PROG = 1;     /* program has finished */
4     label(3);          /* system function - it comes after preemp */
5     preemptive_mode_off(); /* stop timer */
6     if (RETOMA_EXECUCAO == 1){ /* if is not preemptive */
7         jump_label(1);      /* system function */
8     } else if (RETOMA_EXECUCAO == 2){ /* if is preemptive */
9         jump_label(2);      /* system function */
10    }
11 }

```

Algoritmo 4.8 – Procedimento para retomar o contexto.

Para a realização da troca de contexto são usadas as funções primárias:

- **store_inst**: Armazena o conteúdo de um registrador específico em uma posição de memória base com deslocamento. **load_inst**: Resgata o conteúdo de uma memória base com deslocamento em um registrador específico. **jump_inst**: Salta para uma posição específica, para conseguir retomar do ponto do onde o programa parou. **reg_read**: Lê de um registrador. **reg_write**: Escreve em um registrador.

Dessa forma, o [Algoritmo 4.9](#) mostra o procedimento da troca de contexto, onde são salvos/resgatados os registradores que podem ser utilizados pelos programas, que são do 10 a 19, para uso geral, e 25 a 31, esses últimos são respectivamente: **\$IO**, guarda I/O feito pelo SO para o processo, **\$PC**, guarda o PC atual do processo quando tem preempção, **\$global**, guarda o início do processo na RAM, **\$rv**, para retorno de função, **\$fp**, para controle da memória RAM em chamada de função, **\$sp**, para controle da pilha em chamada de função, e **\$ra**, para retomar à função anterior. O sistema operacional utiliza outros registradores específicos, dessa forma o sistema não precisa guardar o próprio contexto, ele sempre está ativo, precisando realizar a troca de contexto entre os programas. Esses registradores ficam em uma região na memória RAM, que será abordado na [subseção 3.2.6](#).

```

1 void load_registradores(int processoIndex){
2     int memIndex;
3
4     memIndex = processoIndex * BLOCO_PROGRAMA_MEM + BLOCO_SO_EXTRA_MEM;
5
6     load_inst(10, memIndex, 10);
7     /* ... */

```

```

8      /* ate o 19 */
9      /* ... */
10     load_inst(19, memIndex, 19);
11
12     load_inst(25, memIndex, 25);
13     /* ... */
14     /* ate o 31 */
15     /* ... */
16     load_inst(31, memIndex, 31);
17 }
18
19 void store_registradores(int processoIndex){
20     int memIndex;
21
22     memIndex = processoIndex * BLOCO_PROGRAMA_MEM + BLOCO_SO_EXTRA_MEM;
23
24     store_inst(10, memIndex, 10);
25     /* ... */
26     /* ate o 19 */
27     /* ... */
28     store_inst(19, memIndex, 19);
29
30     store_inst(25, memIndex, 25);
31     /* ... */
32     /* ate o 31 */
33     /* ... */
34     store_inst(31, memIndex, 31);
35 }

```

Algoritmo 4.9 – Procedimentos de troca de contexto.

No que diz respeito a troca de contexto, como fora mencionado, um vetor estático irá armazenar os processos, identificados por inteiros, simulando uma fila circular, que será controlado pelo *Scheduler*, ao passo que é necessário realizar a troca de contexto, todos os registradores serão salvos na memória principal, em uma área reservada, descrito na [seção 4.4](#), de modo que se possa voltar em sua execução. Caso o programa atual tenha dados salvos, tanto em memória de dados como em pilha, o programa conseguirá retomar a execução dos mesmos, uma vez que registradores de finalidade específica armazenam o início da memória do programa, que é finita e previamente definida para cada processo, e também o topo do pilha, que também é previamente alocado estaticamente para cada processo.

O [Algoritmo 4.10](#) apresenta os procedimentos criados para o gerenciamento da fila. Esses que são:

- **organiza_fila_pronto:** Realiza o *shift* total ou parcial da fila, quando é necessário remover alguém ou passar a vez.
- **fila_existe_processo:** Verifica se um programa está alocado na fila de processos, apenas para impedir múltiplas execuções do mesmo programa (apenas critério de projeto).

- **remove_processo_fila_pronto**: Caso exista, o processo é removido da fila de pronto. É utilizado quando o processo indica que finalizou para o SO.
- **insere_processo_fila_pronto**: Caso tenha espaço na fila, insere um processo nela. É utilizado na tela de seleção de programas a serem executados.
- **fila_proximo**: Remove o primeiro processo e passa a vez para o próximo processo da fila. É utilizado quando uma preempção de finalização do programa ocorre.
- **fila_passa_vez**: Passa a vez para o próximo processo da fila, e o atual é inserido no final da fila. É utilizado quando uma preempção de relógio ocorre.

```
1 void organiza_fila_pronto(int inicial) { /*shift da fila*/
2     if(inicial > 0){
3         while (inicial < PROCESSOS) {
4             FILA_PRONTO[inicial-1] = FILA_PRONTO[inicial];
5             inicial = inicial + 1;
6         }
7         if(inicial <= MAX_PROCESSOS){
8             FILA_PRONTO[inicial-1] = PROCESSO_DESALOCADO;
9         }
10    }
11 }
12
13 int fila_existe_processo(int processo){
14     int i; i = 0;
15     while (i < PROCESSOS) {
16         if (FILA_PRONTO[i] == processo) {
17             return i;
18         }
19         i = i + 1;
20     }
21     return PROCESS_NOT_FOUND;
22 }
23
24 void remove_processo_fila_pronto(int processo) {
25     int i; i = 0;
26     while (i < PROCESSOS) {
27         if (FILA_PRONTO[i] == processo) {
28             organiza_fila_pronto(i + 1);
29             PROCESSOS = PROCESSOS - 1;
30         }
31         i = i + 1;
32     }
33 }
34
35 void insere_processo_fila_pronto(int processo) {
36     int i; i = 0;
37     if(PROCESSOS < MAX_PROCESSOS){
38         if (fila_existe_processo(processo) == PROCESS_NOT_FOUND){
39             FILA_PRONTO[PROCESSOS] = processo;
40             PROCESSOS = PROCESSOS + 1;
41         }
42     }
43 }
44
```

```
45 int fila_proximo(){
46     int processo;
47     if (FILA_PRONTO[0] != PROCESSO_DESALOCADO) {
48         processo = FILA_PRONTO[0];
49         organiza_fila_pronto(1);
50         PROCESSOS = PROCESSOS - 1;
51         return processo;
52     }
53     return LISTA_VAZIA;
54 }
55
56 void fila_passa_vez(){
57     int processo;
58     if (FILA_PRONTO[0] != PROCESSO_DESALOCADO) {
59         processo = FILA_PRONTO[0];
60         organiza_fila_pronto(1);
61         FILA_PRONTO[PROCESSOS-1] = processo;
62     }
63 }
```

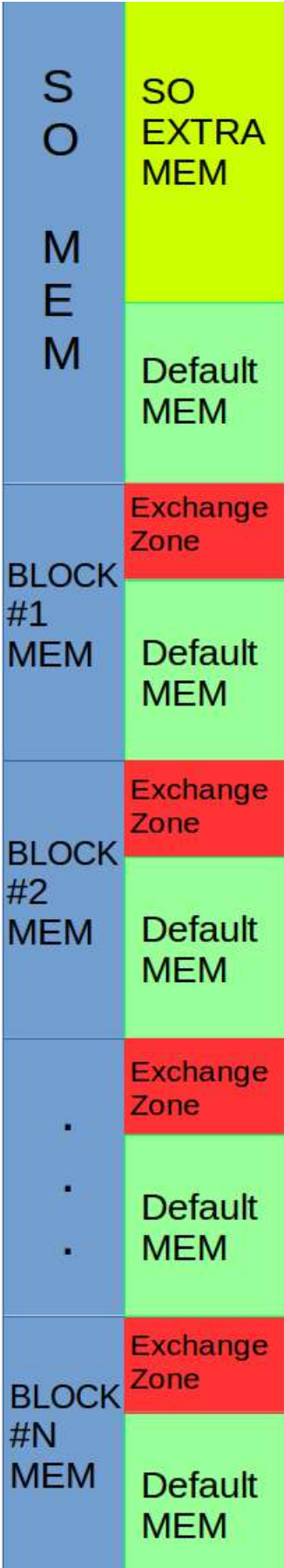
Algoritmo 4.10 – Procedimentos necessários para o SO organizar a Fila de Processos.

4.4 Gerenciamento de Memória

O gerenciamento de memória é realizado da seguinte forma:

- Gerenciamento por Alocação em Blocos: Cada processo terá um espaço de memória reservado, tanto da memória de dados como de pilha, de modo que o espaço será finito e fixo, contudo devido ao espaço de memória da arquitetura atual ser abundante, todos os processos ativos que estarão presentes na arquitetura terão memória suficiente para serem executados, logo se forem ser executados N programas, o sistema operacional reserva N blocos de memória para esses processos.
- Controle dos Blocos de Processos: Cada processo tem registradores de propósito específico, cuja finalidade é localizar os espaços de memória, para pilha e dados, e assim guardar a informação dos dados do processo presentes nos mesmos. Como já foi apresentado, cada processo tem seus próprios registradores, assim o escalonador irá cuidar para fazer a troca de contexto do mesmo, para sempre o programa em execução ter ciência de onde se encontra seus dados.
- Memória de Instrução: Os dados acerca de instruções são inseridos nessa memória de forma dinâmica, assim ao iniciar a arquitetura, a BIOS irá carregar o sistema operacional, salvo no HD, na memória de instruções, e devido a sua capacidade, todo S.O. será carregado para a mesma. Além disso, os programas também serão carregados para esta memória, ao passo que tal processo se torne o protagonista, o mesmo toma o espaço de memória de instruções para execução de programas.

Figura 28 – Estrutura da memória de dados.



Fonte: O Autor

A Figura 28 apresenta a separação da memória de dados, onde todo processo possui uma **Default MEM**, que consiste em um espaço de memória para qualquer processo a ser executado, pois o compilador o utiliza quando faz uso de variáveis. Devido ao tamanho do sistema operacional, foi introduzido uma **SO Extra MEM**, que adiciona mais espaço de RAM para o mesmo, apenas para garantir que não tenha falta de memória. Para os processos, também é reservado mais um espaço de memória de 32 *bits* chamado de **exchange zone**, onde poderá ser utilizado para salvar o contexto dos registradores do processo na eventualidade do mesmo ser preemptado. Assim cada bloco possui um espaço de troca de contexto e de memória padrão, e cabe ao sistema operacional alocar o uso de N blocos ao passo que se executar N programas, de forma dinâmica, realizando esse controle pelo registrador $\$global$ de cada processo, que indica onde será o espaço de memória. Nesse caso temos que:

$$\$global = block_{num} * blockSize + SO_EXTRA + ExZoneSize \quad (4.1)$$

A alocação ocorre de forma dinâmica e contínua, do ponto de vista da fila. Por exemplo, se alocado 3 programas com identificadores 5, 2 e 7, respectivamente irão usar os blocos 1, 2 e 3 da memória de dados.

Figura 29 – Estrutura da pilha de dados.



Fonte: O Autor

A Figura 29 apresenta a separação da pilha de dados, onde todo processo, incluindo o próprio sistema operacional, possui um **block**, que consiste em um espaço de pilha padrão para qualquer processo ser executado, usado pelo compilador para chamadas de funções, recursividade, etc. Como não é necessário muito espaço de pilha, o SO usa somente um bloco igual todos processos. Semelhante à memória RAM, cabe ao sistema operacional alocar os blocos de pilhas aos processos, alocando N blocos para N processos em execução, realizando esse controle pelo registrador $\$SP$, que diz onde começa o bloco na pilha, da seguinte forma:

$$\$sp = block_{num} * blockSize \quad (4.2)$$

Figura 30 – Estrutura da memória de instruções.



Fonte: O Autor

A Figura 30 mostra como é dividida a memória de instruções, que basicamente é parte para o SO e outra parte para o processo atual em execução. Essa alocação é feita de forma fixa, tomando um valor constante do bloco de SO e início do bloco de processo protagonista. Por padrão, o processo atual começa no índice 3000, este que foi escolhido por conta do tamanho do SO ser menor que tal valor. Quando é necessário realizar a troca de contexto ou executar qualquer outro processo, o SO é encarregado de carregar do HD para a memória de instruções o programa daquele processo que será executado.

No que diz respeito ao SO, antes de começar a execução do menu *bash*, é chamado o Algoritmo 4.11, que inicia as variáveis globais do sistema operacional na memória de dados, que serão utilizadas ao longo de toda execução do sistema operacional.

```

1 void init_global(){
2     int i; i = 0;
3
4     MAX_PROGRAMAS = 15; /*15*/
5     PROGRAMA_INST_BEGIN = 3000; /*3000*/
6     BLOCO_PROGRAMA_MEM = 256; /*256*/
7     BLOCO_SO_EXTRA_MEM = 2048; /*2048*/
8     BLOCO_PROGRAMA_STACK = 256; /*70*/
9     BLOCO_CONTEXTO = 32; /*32*/
10    RETOMA_EXECUCAO = 0; /*0*/
11    MAX_PROCESSOS = 10; /*10*/
12    PROCESSOS = 0; /*0*/
13    while(i < 10){
14        FILA_PRONTO[i] = 0; /*0*/
15        i = i + 1;
16    }
17    HALT_PROG = 0; /*0*/
18    /* Constantes*/
19    PROCESSO_DESALOCADO = 0; /*0*/
20    LISTA_VAZIA = 0-1; /*-1*/
21    PROCESS_NOT_FOUND = 0-1; /*-1*/
22    FILE_SYSTEM_BEGIN = 3; /*3*/
23    FILE_NOT_ACTIVIED = 0; /*0*/
24    FILE_NOT_FOUND = 0-1; /*-1*/
25    FILE_SYSTEM_FULL = 0-2; /*-2*/
26    FILE_ALREADY_EXISTS = 0-3; /*-3*/
27    FILE_HEADER_SIZE = 3; /*3*/
28    FILE_HEADER_PROG_ACTIVIED = 0; /*0*/
29    FILE_HEADER_PROG_ID = 1; /*1*/
30    FILE_HEADER_PROG_SIZE = 2; /*2*/
31    FILE_HEADER_PROG_BEGIN = 3; /*3*/
32 }

```

Algoritmo 4.11 – Procedimento de inicialização da variáveis do SO na memória.

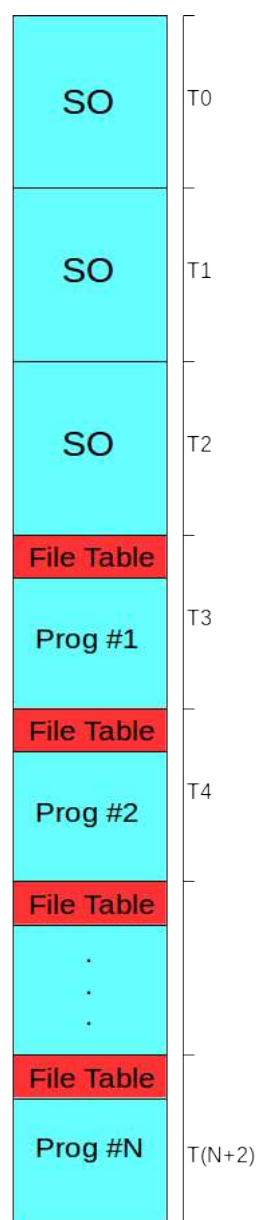
4.5 Gerenciamento do Sistema de Arquivos

O sistema operacional, no que diz respeito à manipulação de arquivos, atende os seguintes métodos: criar, mover, renomear e remover arquivos. Além disso pode listar os arquivos, para facilitar na verificação dos métodos anteriores. Para isso o gerenciamento de arquivos é feito da seguinte forma:

- *Header*: Irá identificar metadados dos arquivos, como seu nome, que é representado por identificadores numéricos, tamanho e arquivo desalocado, que indica caso o arquivo seja excluído.
- *Estrutura dos Arquivos*: O arquivo é identificado como uma coleção de registros de 4 *Bytes*, cujo tamanho está especificado em seu *header*.

- **Diretórios:** Foi feito um diretório de nível único, ou seja, todos os arquivos estão presentes no mesmo diretório, o raiz, e além disso, não há divisão por usuário, todos que utilizarem a arquitetura, estão sujeitos a compartilhar o mesmo diretório e os mesmos arquivos.
- **Acesso e Alocação de Arquivos:** O acesso é feito de maneira aleatória, ou seja, por meio da simulação do HD, acessando a posição desejada por meio dos parâmetros de trilha e setor, a serem especificados pelo usuário ao solicitar uma *syscall* desse busca do HD, assim não é necessário percorrer de forma sequencial o HD. Além disso, a alocação é contígua, todo o arquivo é salvo de forma única, sem separação. Mesmo que tenha segmentação no disco, não é um problema, devido a quantidade abundante de memória que é utilizada para simular o HD.
- **Tabela de Arquivos Distribuída:** A tabela de arquivos, que contém informações básicas dos arquivos, é distribuída, ou seja, está em várias trilhas.
- **Hard Disk (HD):** Será o dispositivo que irá armazenar os dados dos arquivos, será um dispositivo simulado em *Verilog*. Mais detalhes nas modificações estão na [subseção 4.2.3](#).

Figura 31 – Estrutura do disco rígido.



Fonte: O Autor

A Figura 31 apresenta a estrutura do HD, dividido por trilhas e setores, cada trilha, representada pela segmentação desses blocos, contém 1024 setores de registros de 4 *Bytes*. O sistema operacional por ser muito grande, ocupa um total de 3 trilhas. Os programas ocupam as trilhas começando à partir da número 3, com um limite atual de 32 trilhas, podendo atender em *hardware* 29 programas. A trilhas de programa contém no começo da trilha a tabela de arquivo distribuída, que traz informações importantes para o gerenciamento do arquivo, como já abordado acima sobre o *header file*.

O Algoritmo 4.12 contém os procedimentos usados para gerenciar arquivos, que são:

- **posicao_programa**: Retorna a trilha que o programa está, ou caso contrário *FILE_NOT_FOUND*.
- **posicao_livre_programa**: Retorna uma trilha livre para alocar programa, ou caso contrário *FILE_SYSTEM_FULL*.
- **tamanho_programa**: Retorna o tamanho do programa com certo identificador, ou caso contrário *FILE_NOT_FOUND*.
- **carrega_programa_memoria_inst**: Carrega o programa na região de programa da memória de instruções com base no identificador.
- **cria_programa**: Cria um novo programa em uma trilha livre.
- **deleta_programa**: Deleta um programa se ele existir.
- **renomear_programa**: Renomeia um programa se ele existir.
- **trilha_possui_programa**: Retorna o identificador do programa se ele existir, com base na trilha.
- **existe_programa_posicao**: Retorna o identificador do programa se ele existir, com base na posição.

Além disso, algumas funções primitivas o compilador que faz o tratamento:

- **hd_read**: Lê de uma posição do HD com base em trilha e setor, e retorna seu valor.
- **hd_load**: Lê de uma posição do HD com base em trilha e setor, e carrega em uma posição específica da memória de instruções.
- **hd_write**: Escreve o conteúdo de uma variável em uma posição do HD com base em trilha e setor.

```

1  int posicao_programa(int idPrograma){
2      int i; int trilha; int setor; int idProgramaHD; int posicao;
3      trilha = FILE_SYSTEM_BEGIN;
4      setor = FILE_HEADER_PROG_ID;
5      i = 0;
6      while (i < MAX_PROGRAMAS) {
7          posicao = trilha + i;
8          if(hd_read(posicao,FILE_HEADER_PROG_ACTIVED) != FILE_NOT_ACTIVED){
9              idProgramaHD = hd_read(posicao, setor); /* system function - return value */
10             if(idProgramaHD == idPrograma){
11                 return posicao;
12             }
13         }
14         i = i + 1;
15     }
16     return FILE_NOT_FOUND;

```

```
17 }
18
19 int posicao_livre_programa(){
20     int i; int trilha; int setor; int posicao;
21     trilha = FILE_SYSTEM_BEGIN;
22     setor = FILE_HEADER_PROG_ACTIVATED;
23     i = 0;
24     while (i < MAX_PROGRAMAS) {
25         posicao = trilha + i;
26         if(hd_read(posicao,setor) == FILE_NOT_ACTIVATED){
27             return posicao;
28         }
29         i = i + 1;
30     }
31     return FILE_SYSTEM_FULL;
32 }
33
34 int tamanho_programa(int idPrograma){
35     int trilha; int setor;
36     trilha = posicao_programa(idPrograma);
37     if(trilha == FILE_NOT_FOUND){
38         return 0-1;
39     }
40     setor = FILE_HEADER_PROG_BEGIN;
41     return hd_read(trilha, FILE_HEADER_PROG_SIZE);
42 }
43
44
45 int carrega_programa_memoria_inst(int idPrograma){
46     int i; int trilha; int setor; int tamPrograma;
47
48     i = 0;
49     trilha = posicao_programa(idPrograma);
50     if(trilha == FILE_NOT_FOUND){
51         return 0-1; /*TODO: error value*/
52     }
53     setor = FILE_HEADER_PROG_BEGIN;
54     tamPrograma = hd_read(trilha, FILE_HEADER_PROG_SIZE);
55     while (i < tamPrograma){
56         hd_load(trilha, setor + i, PROGRAMA_INST_BEGIN + i); /* system function - attr
57             value to mem */
58         i = i + 1;
59     }
60     return tamPrograma;
61 }
62
63 int cria_programa(int idPrograma, int tamPrograma){
64     int trilha; int setor; int scan1; int scan2; int i;
65     trilha = posicao_programa(idPrograma);
66     if(trilha == FILE_NOT_FOUND){
67         trilha = posicao_livre_programa();
68
69         hd_write(trilha, FILE_HEADER_PROG_ACTIVATED, 1);
70         hd_write(trilha, FILE_HEADER_PROG_ID, idPrograma);
71         hd_write(trilha, FILE_HEADER_PROG_SIZE, tamPrograma);
72
73         setor = FILE_HEADER_SIZE;
74         tamPrograma = tamPrograma + FILE_HEADER_SIZE;
75         i = 0;
76         while(setor < tamPrograma){
```

```

76         print_value(i, 0, 15);
77         scan1 = input();
78         scan2 = input();
79         scan1 = scan1 + scan2;
80         hd_write(trilha, setor, scan1);
81         setor = setor + 1;
82         i = i + 1;
83     }
84     return 1;
85 }
86 return FILE_ALREADY_EXISTS;
87 }
88
89 int deleta_programa(int idPrograma){
90     int trilha;
91     trilha = posicao_programa(idPrograma);
92     if(trilha != FILE_NOT_FOUND){
93         hd_write(trilha, FILE_HEADER_PROG_ACTIVED, 0); /* system function - attr value to
94             hd */
95         return 1;
96     }
97     return FILE_NOT_FOUND;
98 }
99
100 int renomear_programa(int idPrograma, int idProgramaNew){
101     int trilha;
102     if(posicao_programa(idProgramaNew) == FILE_NOT_FOUND){
103         trilha = posicao_programa(idPrograma);
104         if(trilha != FILE_NOT_FOUND){
105             hd_write(trilha, FILE_HEADER_PROG_ID, idProgramaNew);
106             return 1;
107         } else { /*TODO: file does not exist error*/
108             return FILE_NOT_FOUND;
109         }
110     } else { /*TODO: file already exists error*/
111         return FILE_ALREADY_EXISTS;
112     }
113 }
114
115 int trilha_possui_programa(int trilha){
116     if(trilha >= FILE_SYSTEM_BEGIN){
117         if(hd_read(trilha, FILE_HEADER_PROG_ACTIVED) != FILE_NOT_ACTIVED){
118             return hd_read(trilha, FILE_HEADER_PROG_ID); /* system function - return
119                 value */
120         }
121     }
122     return FILE_NOT_FOUND;
123 }
124
125 int existe_programa_posicao(int posicao){ /* posicao starts at 0*/
126     return trilha_possui_programa(posicao + FILE_SYSTEM_BEGIN);
127 }

```

Algoritmo 4.12 – Procedimentos necessários para o SO gerenciar o Sistema de Arquivos.

4.6 Gerenciamento de Dispositivos E/S

Como é apresentado na [Figura 18](#), os dispositivos de E/S são HD, *display* 7-Segmentos, *switches*, *buttons* e também *display* LCD. O sistema operacional terá *syscalls*, cuja responsabilidade será realizar interrupções no programa para fazer I/O de *switch* e *display* 7-Seg. Desse modo, o usuário não irá acessar diretamente as informações e requisições de acesso a esses dispositivos. O usuário porém, não pode usar o LCD para fazer saídas, somente o SO pode alterar as mensagem do LCD, o usuário pode requisitar saída do *display* 7-Seg. Tais dispositivos atuam de forma modular, e serão definidos da seguinte forma:

- **HD:** Simulado por *hardware* em *verilog*, por uma memória da placa FPGA. Seu acesso será feito pelo S.O. por meio das informações de trilha e setor, que será necessário para realizar a *syscall* e localizar os dados desejados. Mais detalhes na [subseção 4.2.3](#).
- **Display 7-seg:** Irá apresentar os dados inteiros, requisitados via *syscall*. É uma execução bloqueante, ou seja, a execução dos processos é interrompida e somente volta mediante uma sinalização via *buttons* na FPGA.
- **Switches/Buttons:** Fazer a entrada de dados. Para entradas via *Switches*, é feita de forma bloqueante, ou seja, a execução dos processos é interrompida e somente volta mediante uma sinalização via *buttons*.
- **Display LCD:** Realiza saída de informações acerca do estado do sistema operacional, também é onde fica o *prompt*. As mensagens serão configuradas via *software*, pelo SO, que atualiza as mensagem do LCD conforme o estado atual do sistema.

4.6.1 Syscalls

No que diz respeito a *syscall*, o usuário faz uma requisição ao SO, informando se é entrada (0) ou saída (1). Então o SO toma posse dos recursos para realizar para o usuário o procedimento do [Algoritmo 4.13](#), e faz o seguinte:

- **syscall 0:** O sistema faz a entrada e devolve para o registrador **\$IO**, que passa para um registrador temporário do programa.
- **syscall 1:** O sistema faz a saída do registrador **\$IO**, que previamente recebe de um registrador temporário do programa.

```
1 /* Tratamento de IO */
2 void syscall_io(){
3     label(4);           /* INPUT */
4     reg_write(25, input()); /* system does input operation */
5     jump_reg(26);       /* system gives it back */
```

```

6   label(5);           /* OUTPUT */
7   output(reg_read(25)); /* system does output operation */
8   jump_reg(26);       /* system gives it back */
9 }

```

Algoritmo 4.13 – Procedimentos necessários para organizar a Fila de Processos.

4.6.2 Display LCD

O *Display* LCD é de uso exclusivo do sistema operacional, que pode atualizar as mensagens no mesmo via *software*, com as seguintes funções de sistema:

- **lcdclean**: Limpa o conteúdo do LCD.
- **lcdwrite0**: Escreve na primeira linha do LCD.
- **lcdwrite1**: Escreve na segunda linha do LCD.
- **lcdwrite_at**: Escreve um único valor em alguma posição do LCD.

Dessa forma, com base na tabela ASCII, é salvo em uma memória do LCD o valor inteiro do caractere que deseja representar. O [Algoritmo 4.14](#) contém algumas funções importantes para atualizar o LCD, a primeira serve para imprimir um valor numérico no LCD em alguma posição, e a segunda imprime mensagens *default*.

```

1 void print_value(int prog, int linha, int posicaoUltimo){
2     int value;
3
4     value = mod(prog, 10) + 48;
5     lcdwrite_at(linha * 16 + posicaoUltimo, value);
6     if(prog >= 10){
7         value = mod(prog/10, 10) + 48;
8     } else {
9         value = 32;
10    }
11    lcdwrite_at(linha * 16 + (posicaoUltimo - 1), value);
12    if(prog >= 100){
13        value = mod(prog/100, 10) + 48;
14    } else {
15        value = 32;
16    }
17    lcdwrite_at(linha * 16 + (posicaoUltimo - 2), value);
18 }
19
20 void print_default_msg(int v){
21     if(v == 1){
22         /* 'ERRO: BAD VALUE!' */
23         lcdwrite0(69, 82, 82, 79, 58, 32, 66, 65, 68, 32, 86, 65, 76, 85, 69, 33);
24     } else if(v == 2){
25         /* 'Selecione o prog' */
26         lcdwrite0(83, 101, 108, 101, 99, 105, 111, 110, 101, 32, 111, 32, 112, 114, 111,
27                 103);
28     } else if(v == 3){

```

```

28      /* 'Erro: Prog.      ' */
29      lcdwrite0(69, 114, 114, 111, 58, 32, 80, 114, 111, 103, 46, 32, 32, 32, 32);
30  } else if(v == 4){
31      /* ' nao encontrado ' */
32      lcdwrite1(32, 110, 97, 111, 32, 101, 110, 99, 111, 110, 116, 114, 97, 100, 111,
33                32);
34  } else if(v == 5){
35      /* ' Programa      ' */
36      lcdwrite0(32, 80, 114, 111, 103, 114, 97, 109, 97, 32, 32, 32, 32, 32, 32, 32);
37  } else if(v == 6){
38      /* 'ID deve ser > 0!' */
39      lcdwrite1(73, 68, 32, 100, 101, 118, 101, 32, 115, 101, 114, 32, 62, 32, 48, 33);
40  } else if(v == 7){
41      /* ' ja usa esse ID ' */
42      lcdwrite1(32, 106, 97, 32, 117, 115, 97, 32, 101, 115, 115, 101, 32, 73, 68, 32);
43  } else if(v == 8){
44      /* ' sendo executado' */
45      lcdwrite1(32, 115, 101, 110, 100, 111, 32, 101, 120, 101, 99, 117, 116, 97, 100,
46                111);
47  } else if(v == 9){
48      /* ' foi finalizado ' */
49      lcdwrite1(32, 102, 111, 105, 32, 102, 105, 110, 97, 108, 105, 122, 97, 100, 111,
50                32);
51  }
52 }

```

Algoritmo 4.14 – Procedimentos básicos para atualizar o LCD.

4.6.3 Menu *bash*

O menu *bash*, como já comentado na [subseção 4.2.5](#), foi elaborado para apresentar as informações no LCD. O [Algoritmo 4.15](#) apresenta os procedimentos de atualização do menu, da qual o usuário pode mudar de tela inserindo 1 ou 2, ou escolhendo acessar o conteúdo da tela atual inserindo 3, como entrada. Então com base na tela atual ele escolhe o que deseja fazer. Funções auxiliares de avançar/retroceder tela foram criadas para deixar circular a transição das telas.

```

1  int tela_prox(int tela){
2      int min; int max;
3      min = 0; max = 7;
4
5      if (tela == max) return min;
6      else return tela + 1;
7  }
8
9  int tela_ant(int tela){
10     int min; int max;
11     min = 0; max = 7;
12
13     if (tela == min) return max;
14     else return tela - 1;
15 }
16
17 void bash_OS(){
18     int tela; int command;
19

```



```
20     tela = 0;
21     while(1){
22         print_menu(tela);
23         command = input();
24         if(command == 1){
25             tela = tela_prox(tela);
26         } else if(command == 2){
27             tela = tela_ant(tela);
28         } else if(command == 3){
29             if(tela == 0){
30                 menu_opt0_executar_prog();
31             } else if(tela == 1){
32                 menu_opt1_escrever_prog();
33             } else if(tela == 2){
34                 menu_opt2_renomear_prog();
35             } else if(tela == 3){
36                 menu_opt3_deletar_prog();
37             } else if(tela == 4){
38                 menu_opt4_listar_prog();
39             } else if(tela == 5){
40                 menu_opt5_reiniciar();
41             } else if(tela == 6){
42                 menu_opt6_desligar();
43                 return; /* shutdown */
44             } else { /* tela == 7 */
45                 menu_opt7_help();
46             }
47         }
48     }
49 }
```

Algoritmo 4.15 – Procedimentos de atualização do menu.

5 Resultados

Os resultados obtidos serão apresentados e discutidos nessa seção. Os algoritmos testados em bancada e apresentados foram:

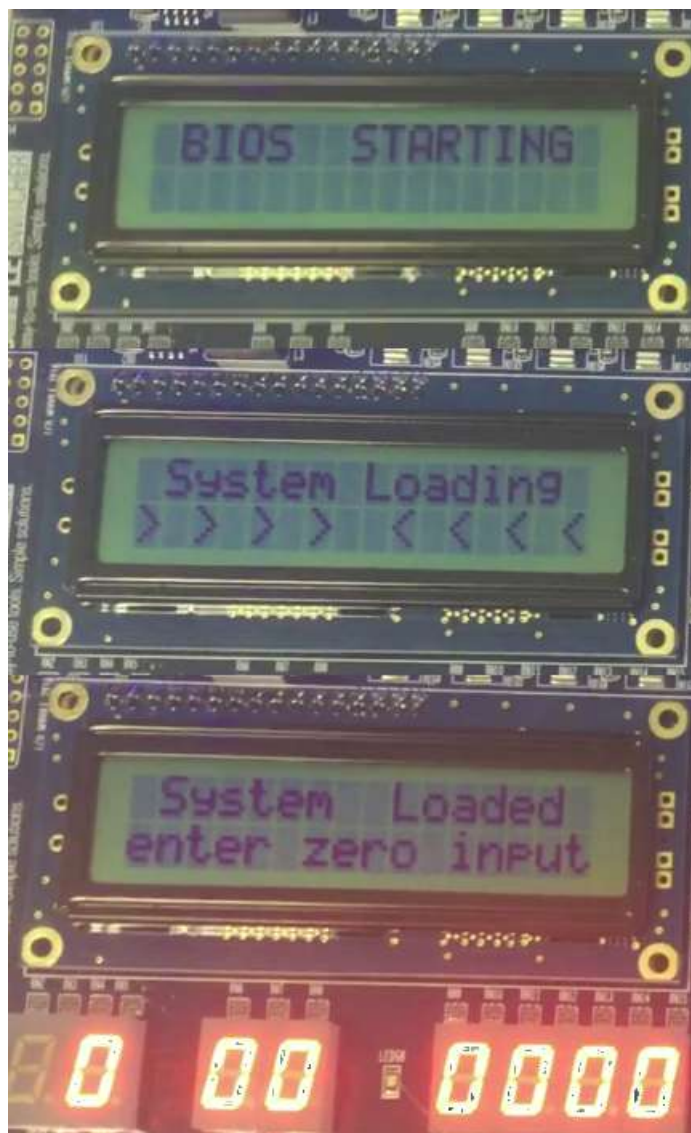
1. *Selection Sort*: ✓
2. Busca Binária Recursiva: ✓
3. Média de um Vetor: ✓
4. Variância de um Vetor: ✓
5. Extremos Vetor: ✓
6. *Greatest common divisor* (GCD): ✓
7. *Fibonacci*: ✓
8. Fatorial: ✓
9. Calculadora Básica: ✓
10. Potenciação: ✓

Para apreciar mais dos resultados, foi elaborado uma breve **demonstração em vídeo** (15) com a execução do Sistema Operacional. Nesse Capítulo serão apresentados alguns desses resultados como *screenshots*. O código final em *Assembly* e binário do SO está apresentado no [Algoritmo B.1](#) no [Apêndice B](#).

5.1 BIOS

A BIOS elaborada funcionou conforme previsto, iniciou corretamente, fez o carregando do SO salvo no HD e a passou no teste verificação de I/O e memória. A [Figura 32](#) mostra a sequência de telas da BIOS.

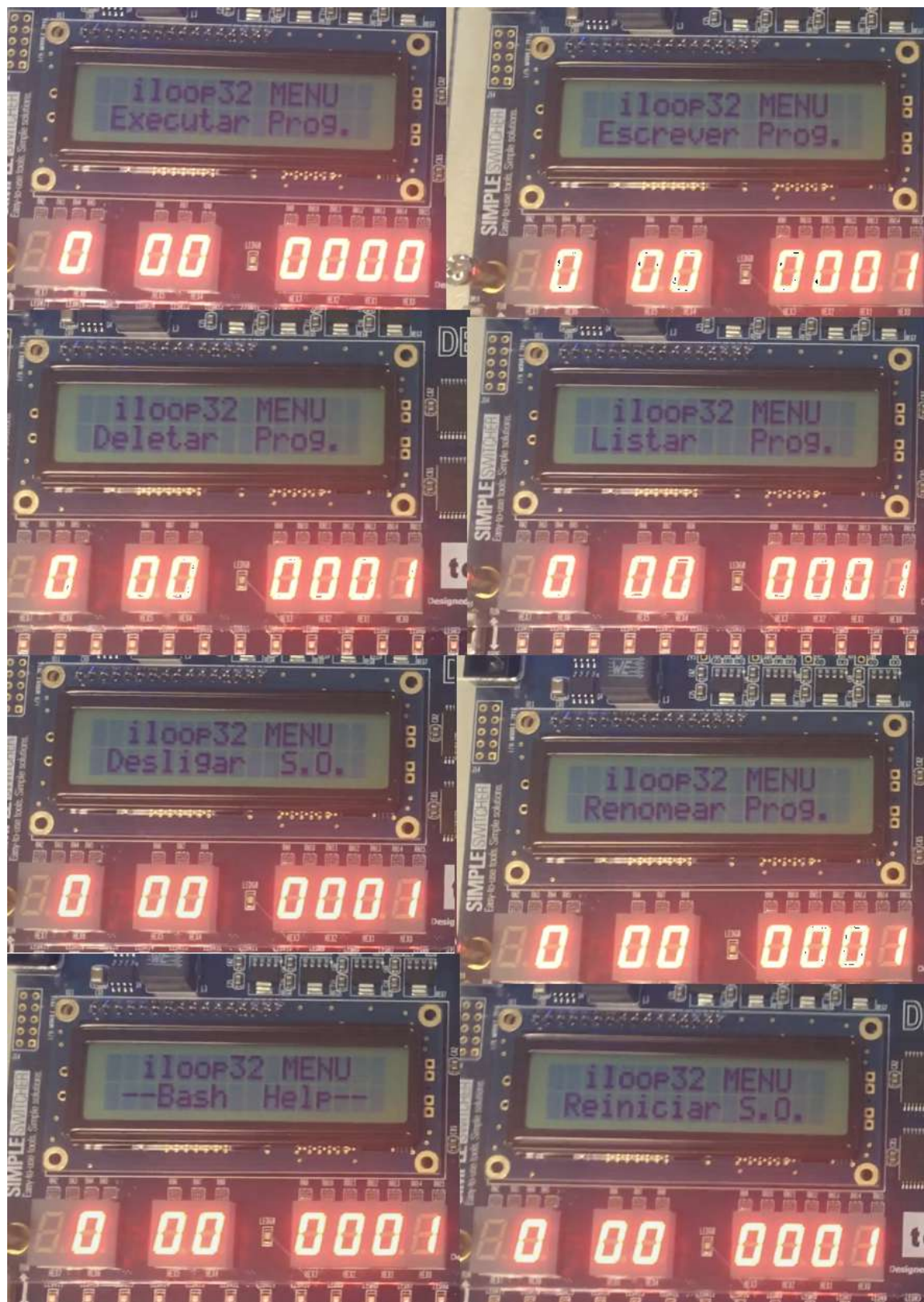
Figura 32 – (1) BIOS iniciando, (2) BIOS carregando SO e (3) BIOS fazendo checagem de I/O e memória.



Fonte: O Autor

5.2 Sistema Operacional

Figura 33 – Menu Completo do Sistema Operacional.



Fonte: O Autor

A Figura 33 mostra todas as telas do menu elaborado. O menu é imediatamente acesso após a checagem de I/O da BIOS, e inicia por padrão na tela de executar programas.

5.2.1 Executar Programas

A Figura 34 mostra a preparação e o início da execução de programas:

- Escolhe se será modo com preempção ou serial.
- Escolhe a quantidade de programas.
- Escolhe o conjunto de programas por ID a serem executados.
 - A cada inserção é emitido um aviso indicando que o programa X entrou na fila.
- Inicia-se a execução dos programas.
- Executa o primeiro da fila.

Figura 34 – Preparação e Execução de Programas.



Fonte: O Autor

No caso da execução preemptiva, ao passo que o temporizador atinge a quantidade de instruções previamente definido em *hardware*, é iniciado a preempção do processo, como mostra a Figura 35, então o próximo processo da fila toma posse dos recursos do sistema.

Figura 35 – Estado de Troca de Contexto.



Fonte: O Autor

A Figura 36 mostra a finalização dos processos, a primeira tela é emitida pelo sistema operacional quando o processos atinge o final da sua execução, na instrução de *return*. Após todos processos serem finalizados, com ou sem preempção, a tela de processos finalizados é emitida pelo SO.

Figura 36 – Finalização da execução de programas.

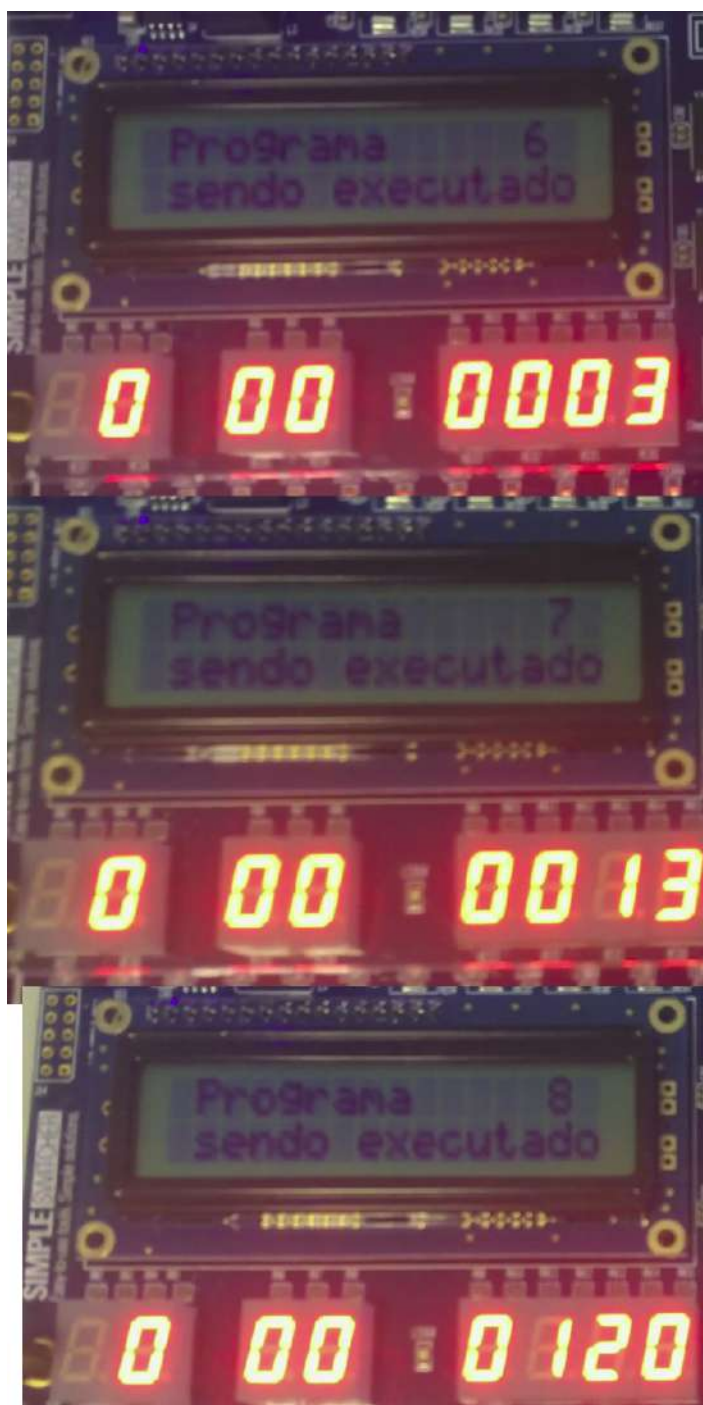


Fonte: O Autor

A Figura 37 apresenta a saída dos programas 6, 7 e 8, na seguinte configuração:

- $\text{GCD}(15,12)$: 3 ✓
- *Fibonacci*(7): 13 ✓
- Fatorial(5): 120 ✓

Figura 37 – Saídas dos Programas (6), (7) e (8).



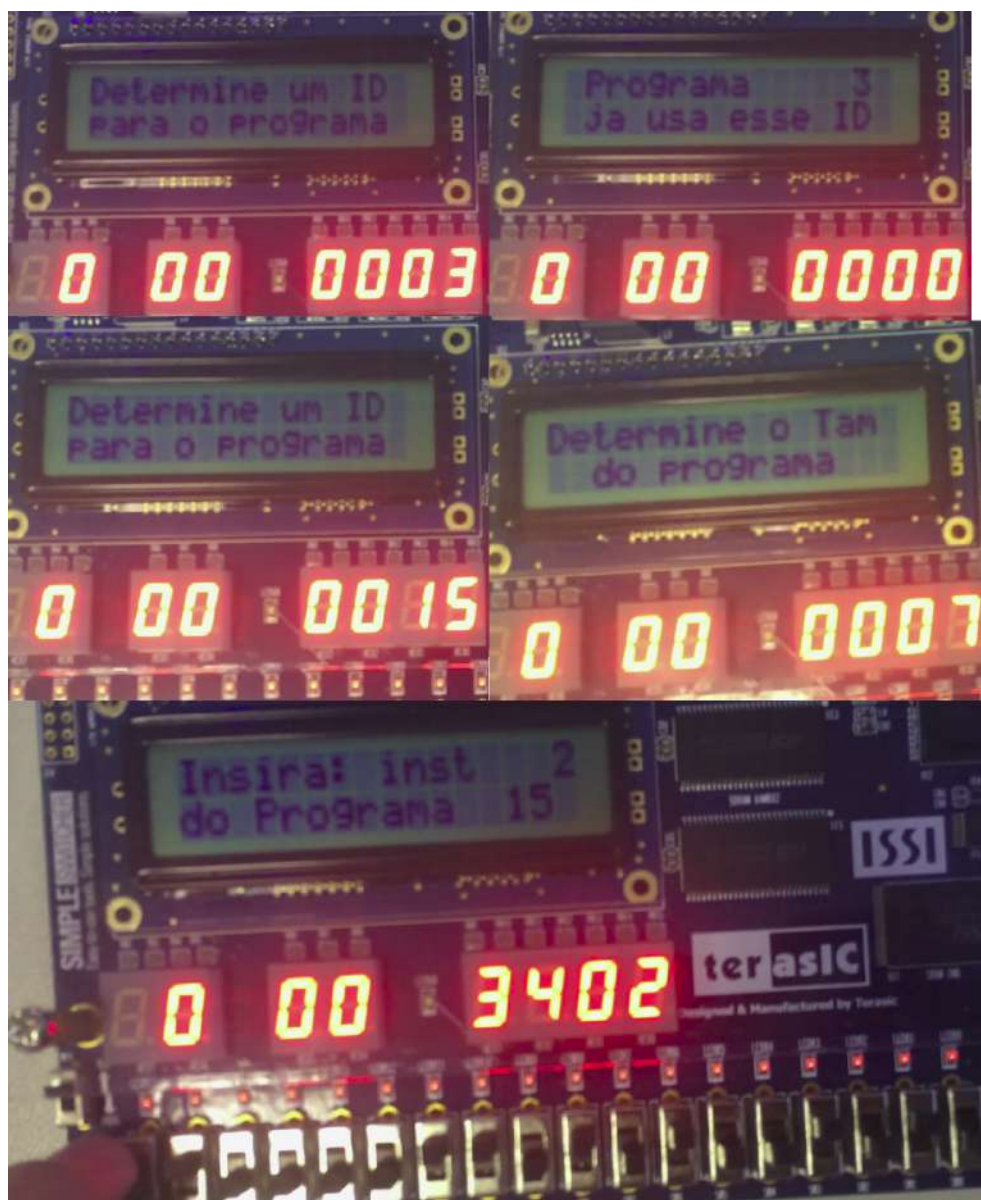
Fonte: O Autor

5.2.2 Escrever Programa

A Figura 38 mostra o passo-a-passo para criação de um novo programa:

- É escolhido um identificador para o novo programa.
 - Caso o identificador já esteja em uso, um alerta será emitido pelo SO.
- Após a escolha de um ID disponível, é informado o tamanho do programa.
- Então escreve-se o programa, linha a linha, inserindo sempre a parte menos e mais significativa de uma instrução, logo se N linhas devem ser escritas, serão necessários $2N$ inputs.

Figura 38 – Criação de um Programa novo.



Fonte: O Autor

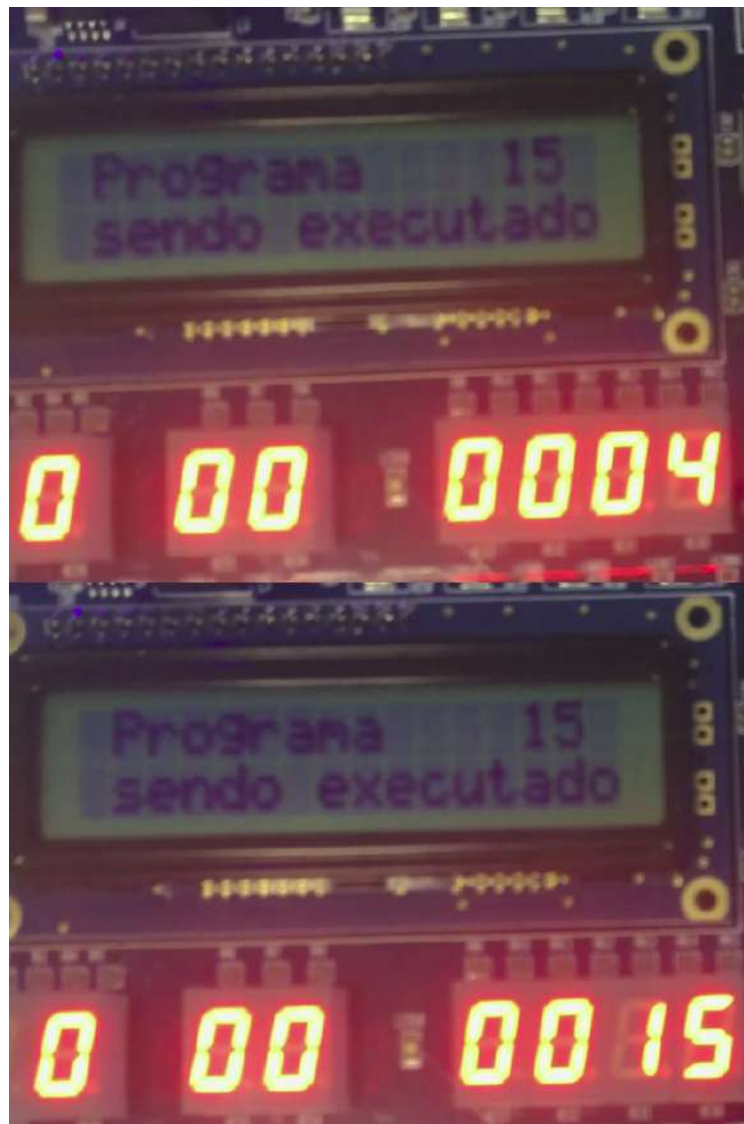
O Algoritmo 5.1 foi criado como teste simples para criação de novo arquivo, onde é realizado um *input* e tem como *output* o valor de entrada incrementado de 11. A Figura 39 mostra a execução desse novo programa (ID = 15), após a criação do mesmo.

```
1 syscall 0      /* Faz input no $R25 */
2 move $R10 $R25 /* Move o input para um temporario */
3 addi $R10 $R10 11 /* Incrementa em 11 o valor do Registrador */
4 move $R25 $R10 /* Move o temporario para o Registrador de saida */
5 syscall 1      /* Faz output do $R25 */
6 return        /* Finaliza o programa */
```

Algoritmo 5.1 – Programa Básico para Teste.

- **Input:** 4; **Output:** 15. ✓

Figura 39 – Execução do Novo Programa Criado.

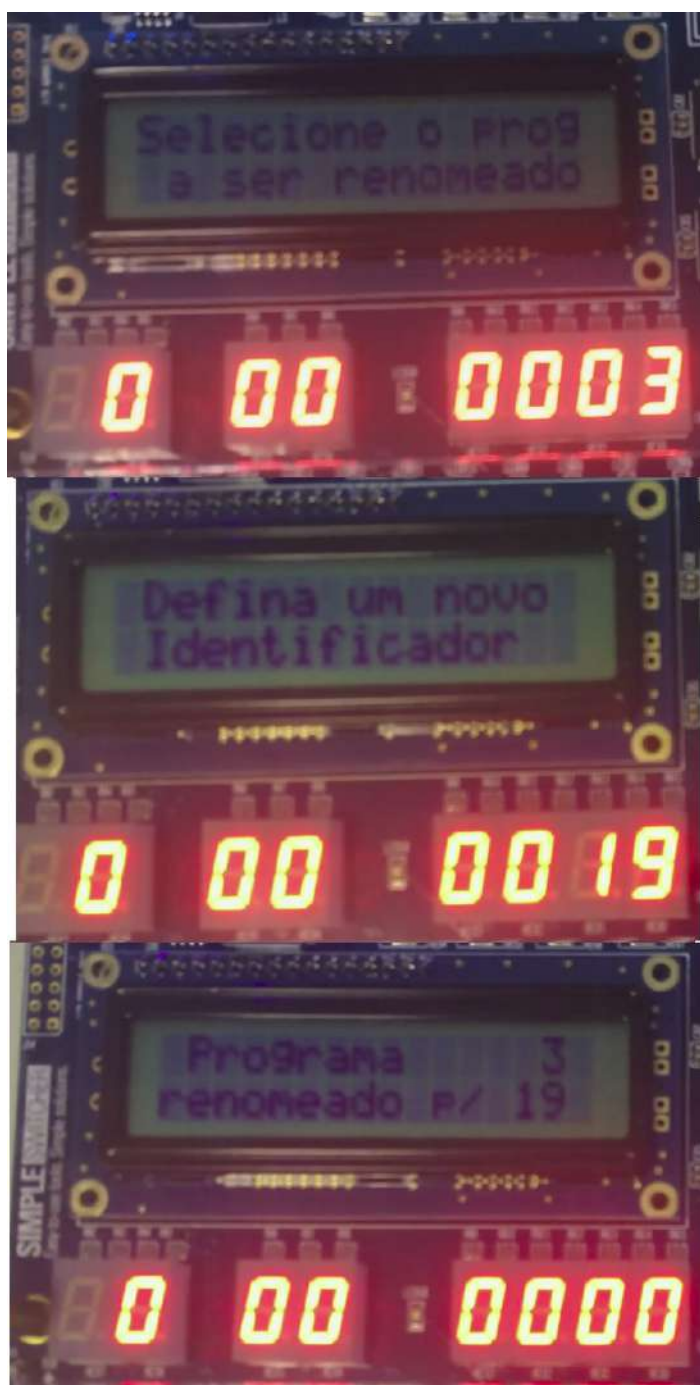


Fonte: O Autor

5.2.3 Renomear Programa

Para renomear um programa, basta selecionar um programa válido e um novo identificador disponível. Caso alguma das condições seja violada, o SO emite um alerta e impede a alteração. A Figura 40 mostra o processo de renomear um programa.

Figura 40 – Renomear Programa.



Fonte: O Autor

5.2.4 Deletar programa

Para deletar um programa, basta selecionar um programa válido. Caso não seja um identificador válido, o SO emite um alerta e impede a remoção. A [Figura 41](#) mostra o processo de deletar um programa.

Figura 41 – Deletar Programa.



Fonte: O Autor

5.2.5 Listar Programas

O listar programas foi uma *feature* bônus elaborada para ajudar na verificação dos outros métodos, pois o mesmo é capaz de:

- Mostra a quantidade de programa no HD.
- Mostra a posição e o identificador do programa no HD.

Com isso, é mais fácil fazer a verificação dos métodos de criação, remoção e renomeação de programas. A [Figura 42](#) mostra uma tela onde está sendo executado a listagem, podemos ver que o HD possui 9 programas e o terceiro é o de ID 19.

Figura 42 – Listar Programas.



Fonte: O Autor

5.2.6 Desligar e Reiniciar SO

Foi implementado a opção de desligar e reiniciar o SO, com finalidade de torná-lo mais próximo de uma experiência real. Quando o sistema é desligado, ele fica em estado *halt* e os *displays* são desligados, para religar basta apertar o botão de *reset*. Quando o sistema é reiniciado, é como se o botão de *reset* fosse apertado, na prática o SO passa a posse dos recursos para a BIOS, que irá carregar novamente o sistema e recomeçar todo o processo. Quando o sistema é desligado/reiniciado, as alterações nos programas do HD ainda são mantidas. A [Figura 43](#) mostra a tela de "desligando sistema" e "reiniciando sistema".

Figura 43 – Desligando e Reiniciando o Sistema Operacional.



Fonte: O Autor

6 Considerações Finais

O estudo e elaboração, tanto da modelagem quanto do próprio do sistema operacional, além das modificações que foram realizadas arquitetura alvo, foram bem desafiadoras e necessitaram de muito afinho e dedicação. Foi possível aplicar e muito o conhecimento teórico obtido na disciplina de Sistema Operacionais, adaptando da melhor forma para o contexto prático, a partir da definição de metodologias e abordagens diferentes. Devido a isso, a elaboração do sistema operacional em c-menos não foi nada fácil, porém bem factível, para tal muito tempo de planejamento e discussões foram requisitadas. Apesar dessa demanda, foi ótimo poder contribuir e receber ideias incríveis com amigos e colegas da disciplina. Pessoalmente falando, é uma ótima experiência o laboratório, pois contribui para elaboração de grandes ideias e mais dinâmicas com o time de trabalho.

Para as próximas etapas, será necessário retomar o sistema desenvolvido, afim de realizar comunicações com outros sistemas, via redes, e assim finalizar o sistema computacional que estamos desenvolvendo.

6.1 Dificuldades Encontradas

Foi necessário demandar muito tempo, tanto no planejamento quanto na prática, pois as soluções não são triviais, e muitas vezes existem inúmeras abordagens a se seguir. Acredito que compartilhar esse tempo com outras disciplinas complexas complicaram um pouco o desenvolvimento, que ocasionou a entrega do produto nos momentos finais da disciplina.

Referências

- 1 TANENBAUM, A. *Sistemas operacionais modernos*. Prentice-Hall do Brasil, 2003. ISBN 9788587918574. Disponível em: <<https://books.google.com.br/books?id=meCAGQAACAAJ>>. Citado 9 vezes nas páginas 9, 29, 30, 31, 32, 33, 34, 35 e 36.
- 2 FLOYD, T. L. *Sistemas Digitais: Fundamentos e Aplicações*. 9ª edição. ed. [S.l.]: Artmed Editora S.A., 2007. Citado 2 vezes nas páginas 11 e 18.
- 3 PATTERSON, D. A.; HENNESY, J. L. *Computer Organization and Design*. 5th edition. ed. Waltham/MA, EUA: Morgan Kaufmann, 2007. Citado 2 vezes nas páginas 11 e 13.
- 4 INTEL Pentium® Processor G4600 (3M Cache, 3.60 GHz). Acessado em 04 de maio de 2017. Disponível em: <https://ark.intel.com/pt-br/products/97453/Intel-Pentium-Processor-G4600-3M-Cache-3_60-GHz>. Citado na página 11.
- 5 CAPPABIANCO, F. A. M. *Circuitos Digitais, Aula 16: Flip-Flops*. [S.l.]: UNIFESP, 2016. Citado na página 11.
- 6 OREGON State University, Introduction to the MIPS Architecture. Acessado em 05 de maio de 2017. Disponível em: <<http://web.engr.oregonstate.edu/~walkiner/cs271-wi13/slides/02-MIPSArchitecture.pdf>>. Citado 2 vezes nas páginas 13 e 37.
- 7 STALLINGS, W. *Computer Organization and Architecture: Designing for Performance*. Upper Saddle River, NJ, USA: Pearson Education, Inc., 2010. Citado 2 vezes nas páginas 15 e 38.
- 8 ALTERA®. *Intel FPGA - Quartus II/modelSIM softwares*. Acessado em 07 de maio de 2017. Disponível em: <<https://www.altera.com/>>. Citado 3 vezes nas páginas 16, 17 e 38.
- 9 VERILOG. *Verilog Resources*. Acessado em 06 de maio de 2017. Disponível em: <<http://www.verilog.com/>>. Citado 2 vezes nas páginas 18 e 39.
- 10 ALTERA®. *DE2-115 USER MANUAL*. [S.l.], 2003–2010. Disponível em: <<https://www.altera.com/>>. Citado 2 vezes nas páginas 18 e 38.
- 11 LOUDEN, K. *Compiladores - Princípios e Práticas*. Pioneira Thomson Learning, 2004. ISBN 9788522104222. Disponível em: <https://books.google.com.br/books?id=a4tuyCBcM_MC>. Citado 4 vezes nas páginas 18, 21, 22 e 24.
- 12 OLIVEIRA, T. *Laboratório de Sistemas Computacionais: Sistemas Operacionais, Material de Apoio da Disciplina*. [S.l.]: UNIFESP, 2018. Citado na página 38.
- 13 MARCON, C. A. M.; MORENO, E. I. *Organização de Arquitetura de Computadores, Arquiteturas Von Neumann e Harvard*. Acessado em 07 de maio de 2017. Disponível em: <http://www.inf.pucrs.br/emoreno/undergraduate/CC/orgarqi/class_files/Aula17/Aula17b.pdf>. Citado na página 38.

- 14 LOOMIS, J. S. *Digital Labs using the Altera DE2 Board*. Acessado em 16 de novembro de 2018. Disponível em: <<http://www.johnloomis.org/digitallab/>>. Citado na página 50.
- 15 LUPPI, I. *Apresentação do Sistema Operacional*. Acessado em 28 de dezembro de 2018. Disponível em: <<https://www.youtube.com/watch?v=eYMAzot8pWw>>. Citado na página 73.

Apêndices

APÊNDICE A – Conjunto de Instruções

Segue a descrição do conjunto de instruções na [Tabela 6](#) abaixo.

Tabela 6 – Descrição do Conjunto de Instruções

Instrução	Opcode/Func	Representação	Tipo/Categoria
SLL	000000	$RT \leftarrow RS \ll \text{shift}$	R/Deslocamento
SRL	000001	$RT \leftarrow RS \gg \text{shift}$	R/Deslocamento
SRA	000010	$RT \leftarrow RS \ggg \text{shift}$	R/Deslocamento
CSL	000011	$RT \leftarrow \text{rotação_esquerda}(RS)$	R/Deslocamento
CSR	000100	$RT \leftarrow \text{rotação_direita}(RS)$	R/Deslocamento
ADD	000101	$RD \leftarrow RS + RT$	R/Aritmético
SUB	000110	$RD \leftarrow RS - RT$	R/Aritmético
MULT	000111	$RD \leftarrow RS * RT$	R/Aritmético
DIV	001000	$RD \leftarrow RS / RT$	R/Aritmético
INC	001001	$RT \leftarrow RS + 1$	R/Aritmético
DEC	001010	$RT \leftarrow RS - 1$	R/Aritmético
MOD	001011	$RD \leftarrow RS \% RT$	R/Aritmético
AND	001100	$RD \leftarrow RS \& RT$	R/Lógico
OR	001101	$RD \leftarrow RS RT$	R/Lógico
NOT	001110	$RT \leftarrow \sim RS$	R/Lógico
XOR	001111	$RD \leftarrow RS \wedge RT$	R/Lógico
MOVE	010000	$RT \leftarrow RS$	R/Transferência
SLT	010001	$RD \leftarrow (RS < RT) ? 1 : 0$	R/Saltos
SGT	010010	$RD \leftarrow (RS > RT) ? 1 : 0$	R/Saltos
SET	010011	$RD \leftarrow (RS == RT) ? 1 : 0$	R/Saltos
JR	010100	$PC \leftarrow RS$	R/Saltos
JALR	010101	$\$RA \leftarrow PC; PC \leftarrow RS$	R/Saltos
LW	000001	$RT \leftarrow \text{Mem}(RS+IM)$	I/Transferência
SW	000010	$\text{Mem}(RS+IM) \leftarrow RT$	I/Transferência
ADDI	000011	$RT \leftarrow RS + IM$	I/Aritmético
SUBI	000100	$RT \leftarrow RS + IM$	I/Aritmético
LI	000101	$RT \leftarrow IM$	I/Transferência
LUI	000110	$RT \leftarrow (IM \ll 16)$	I/Transferência
BEQ	000111	$PC \leftarrow (RS == RT) ? PC + IM : PC + 1$	I/Saltos
BNE	001000	$PC \leftarrow (RS \neq RT) ? PC + IM : PC + 1$	I/Saltos
IN	001001	$RT \leftarrow \text{INPUT}$	I/(E/S)
OUT	001010	$\text{OUTPUT} \leftarrow RS$	I/(E/S)
J	001011	$PC \leftarrow \{PC[31:26], \text{Address}\}$	J/Saltos
JAL	001100	$\$RA \leftarrow PC; PC \leftarrow \{PC[31:26], \text{Address}\}$	J/Saltos
JU	001101	$PC \leftarrow (\text{Address} \ll 6)$	J/Saltos
JUAL	001110	$\$RA \leftarrow PC; PC \leftarrow (\text{Address} \ll 6)$	J/Saltos
PUSH	001111	$\text{Stack} \leftarrow RT; \$SP \leftarrow \$SP + 1$	I/Transferência
POP	010000	$RT \leftarrow \text{Stack}; \$SP \leftarrow \$SP - 1$	I/Transferência
NOOP	000000	No operation	C/Control
HALT	111111	Stop system	C/Control
UPBIOS	100001	BIOS gets system's control	C/Control
UPMEM	100010	MEM gets system's control	C/Control
HDTOINST	100011	$\text{MEM}(RD) \leftarrow \text{HD}(RS, RT)$	C/Control
HDTOREG	100100	$RD \leftarrow \text{HD}(RS, RT)$	C/Control
REGTOHD	100101	$\text{HD}(RS, RT) \leftarrow RD$	C/Control
LDCLEAN	100110	Clean LCD	C/(E/S)
LCDWRITE	100111	Write LCD	C/(E/S)
JALOS	101000	$\$RA_{OS} \leftarrow PC; PC \leftarrow \{PC[31:26], \text{Address}\}$	C/Saltos
PREEMP	101001	It does a preemptive jump	C/Saltos
INTR_ON	101010	Start quantum's counter	C/Control
INTR_OFF	101011	Stop quantum's counter	C/Control
RETURN	101100	Finalize a program, return to master.	C/Saltos

Fonte: O Autor

APÊNDICE B – Código Final de Máquina do SO

```

1 HD[0] = 32'b000101_00000_00000_0000000000000000; // li $global 0
2 HD[1] = 32'b000011_00000_10101_0000000000100001; // addi $fp $global 33
3 HD[2] = 32'b001011_00000000000000101101101010; // jump main
4 HD[3] = 32'b001111_10110_10111_0000000000000000; // push $ra / syscall_io
5 HD[4] = 32'b00000000000000000000000000000000; // noop / _label4_
6 HD[5] = 32'b001001_00000_00010_0000000000000000; // in 2
7 HD[6] = 32'b000000_00010_11001_00000_00000_010000; // move 25 2
8 HD[7] = 32'b000000_11010_00000_00000_00000_010100; // jr 26
9 HD[8] = 32'b00000000000000000000000000000000; // noop / _label5_
10 HD[9] = 32'b000000_11001_00010_00000_00000_010000; // move 2 25
11 HD[10] = 32'b001010_00010_00000_0000000000000000; // out 2
12 HD[11] = 32'b000000_11010_00000_00000_00000_010100; // jr 26
13 HD[12] = 32'b010000_10110_10111_0000000000000000; // pop $ra
14 HD[13] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
15 HD[14] = 32'b001111_10110_10111_0000000000000000; // push $ra / retoma_execucao_so
16 HD[15] = 32'b00000000000000000000000000000000; // noop / _label0_
17 HD[16] = 32'b000101_00000_00010_0000000000000001; // li 2 1
18 HD[17] = 32'b000010_00000_00010_00000000000010011; // sw 2 $global 19
19 HD[18] = 32'b00000000000000000000000000000000; // noop / _label3_
20 HD[19] = 32'b101011_0000000000000000000000000000; // intr_off
21 HD[20] = 32'b000001_00000_00010_00000000000000110; // lw 2 $global 6
22 HD[21] = 32'b000101_00000_00011_0000000000000001; // li 3 1
23 HD[22] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
24 HD[23] = 32'b000111_00010_00000_0000000000000011; // beq 2 $zero _ElseBegin0_
25 HD[24] = 32'b001011_00000000000000000000111100111; // jump _label1_
26 HD[25] = 32'b001011_00000000000000000000000011111; // jump _IfExit0_
27 HD[26] = 32'b000001_00000_00010_00000000000000110; // lw 2 $global 6 / _ElseBegin0_
28 HD[27] = 32'b000101_00000_00011_00000000000000010; // li 3 2
29 HD[28] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
30 HD[29] = 32'b000111_00010_00000_00000000000000010; // beq 2 $zero _IfExit1_
31 HD[30] = 32'b001011_00000000000000000010010110010; // jump _label2_
32 HD[31] = 32'b010000_10110_10111_0000000000000000; // pop $ra / _IfExit1_ _IfExit0_
33 HD[32] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
34 HD[33] = 32'b001111_10110_10111_0000000000000000; // push $ra / sleep
35 HD[34] = 32'b000101_00000_00010_0000000000000000; // li 2 0
36 HD[35] = 32'b000010_10101_00010_00000000000000010; // sw 2 $fp 2
37 HD[36] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
38 HD[37] = 32'b000101_00000_00011_0000000000000000; // li 3 0
39 HD[38] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
40 HD[39] = 32'b000111_00010_00000_00000000000000110; // beq 2 $zero _ElseBegin2_
41 HD[40] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
42 HD[41] = 32'b000101_00000_00011_0001100001101010; // li 3 6250
43 HD[42] = 32'b000000_00010_00011_00010_00000_000111; // mult 2 2 3
44 HD[43] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
45 HD[44] = 32'b001011_0000000000000000000000110011; // jump _IfExit2_
46 HD[45] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0 / _ElseBegin2_
47 HD[46] = 32'b000101_00000_00011_00000000000011001; // li 3 25
48 HD[47] = 32'b000000_00010_00011_00010_00000_000111; // mult 2 2 3
49 HD[48] = 32'b000101_00000_00011_00000000000000100; // li 3 4
50 HD[49] = 32'b000000_00010_00011_00010_00000_001000; // div 2 2 3
51 HD[50] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0

```

```

52 HD[51] = 32'b000001_10101_00010_0000000000000010; // lw 2 $fp 2 / _IfExit2_ _WhileBegin0_
53 HD[52] = 32'b000001_10101_00011_0000000000000000; // lw 3 $fp 0
54 HD[53] = 32'b000000_00010_00011_00010_00000_010001; // slt 2 2 3
55 HD[54] = 32'b000111_00010_00000_00000000000000101; // beq 2 $zero _WhileExit0_
56 HD[55] = 32'b000001_10101_00010_0000000000000010; // lw 2 $fp 2
57 HD[56] = 32'b000011_00010_00010_0000000000000001; // addi 2 2 1
58 HD[57] = 32'b000010_10101_00010_0000000000000010; // sw 2 $fp 2
59 HD[58] = 32'b001011_00000000000000000000110011; // jump _WhileBegin0_
60 HD[59] = 32'b010000_10110_10111_0000000000000000; // pop $ra / _WhileExit0_
61 HD[60] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
62 HD[61] = 32'b001111_10110_10111_0000000000000000; // push $ra / vetor_existe_valor
63 HD[62] = 32'b000101_00000_00010_0000000000000000; // li 2 0
64 HD[63] = 32'b000010_10101_00010_00000000000000011; // sw 2 $fp 3
65 HD[64] = 32'b000001_10101_00010_00000000000000011; // lw 2 $fp 3 / _WhileBegin1_
66 HD[65] = 32'b000001_10101_00011_0000000000000001; // lw 3 $fp 1
67 HD[66] = 32'b000000_00010_00011_00010_00000_010001; // slt 2 2 3
68 HD[67] = 32'b000111_00010_00000_0000000000010000; // beq 2 $zero _WhileExit1_
69 HD[68] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
70 HD[69] = 32'b000001_10101_00011_0000000000000011; // lw 3 $fp 3
71 HD[70] = 32'b000000_00010_00011_00010_00000_000101; // add 2 2 3
72 HD[71] = 32'b000001_00010_00010_0000000000000000; // lw 2 2 0
73 HD[72] = 32'b000001_10101_00011_0000000000000010; // lw3 $fp 2
74 HD[73] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
75 HD[74] = 32'b000111_00010_00000_00000000000000101; // beq 2 $zero _IfExit3_
76 HD[75] = 32'b000001_10101_00010_0000000000000011; // lw 2 $fp 3
77 HD[76] = 32'b000000_00010_10100_00000_00000_010000; // move $rv 2
78 HD[77] = 32'b010000_10110_10111_0000000000000000; // pop $ra
79 HD[78] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
80 HD[79] = 32'b000001_10101_00010_0000000000000011; // lw 2 $fp 3 / _IfExit3_
81 HD[80] = 32'b000011_00010_00010_0000000000000001; // addi 2 2 1
82 HD[81] = 32'b000010_10101_00010_0000000000000011; // sw 2 $fp 3
83 HD[82] = 32'b001011_00000000000000000000100000; // jump _WhileBegin1_
84 HD[83] = 32'b000101_00000_00010_0000000000000000; // li 2 0 / _WhileExit1_
85 HD[84] = 32'b000100_00010_00010_0000000000000001; // subi 2 2 1
86 HD[85] = 32'b000000_00010_10100_00000_00000_010000; // move $rv 2
87 HD[86] = 32'b010000_10110_10111_0000000000000000; // pop $ra
88 HD[87] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
89 HD[88] = 32'b010000_10110_10111_0000000000000000; // pop $ra
90 HD[89] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
91 HD[90] = 32'b001111_10110_10111_0000000000000000; // push $ra / mod
92 HD[91] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
93 HD[92] = 32'b000001_10101_00011_0000000000000000; // lw 3 $fp 0
94 HD[93] = 32'b000001_10101_00100_0000000000000001; // lw 4 $fp 1
95 HD[94] = 32'b000000_00011_00100_00011_00000_001000; // div 3 3 4
96 HD[95] = 32'b000001_10101_00100_0000000000000001; // lw 4 $fp 1
97 HD[96] = 32'b000000_00011_00100_00011_00000_000111; // mult 3 3 4
98 HD[97] = 32'b000000_00010_00011_00010_00000_000110; // sub 2 2 3
99 HD[98] = 32'b000000_00010_10100_00000_00000_010000; // move $rv 2
100 HD[99] = 32'b010000_10110_10111_0000000000000000; // pop $ra
101 HD[100] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
102 HD[101] = 32'b010000_10110_10111_0000000000000000; // pop $ra
103 HD[102] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
104 HD[103] = 32'b001111_10110_10111_0000000000000000; // push $ra / print_value
105 HD[104] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
106 HD[105] = 32'b000101_00000_00011_0000000000001010; // li 3 10
107 HD[106] = 32'b000011_10101_10101_0000000000000100; // addi $fp $fp 4
108 HD[107] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
109 HD[108] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
110 HD[109] = 32'b101000_000000000000000000001011010; // jal mod
111 HD[110] = 32'b000100_10101_10101_0000000000000100; // subi $fp $fp 4

```

```

112 HD[111] = 32'b000000_10100_00010_00000_00000_010000; // move 2 $rv
113 HD[112] = 32'b000011_00010_00010_0000000000110000; // addi 2 2 48
114 HD[113] = 32'b000010_10101_00010_0000000000000011; // sw 2 $fp 3
115 HD[114] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
116 HD[115] = 32'b000101_00000_00011_0000000000010000; // li 3 16
117 HD[116] = 32'b000000_00010_00011_00010_00000_000111; // mult 2 2 3
118 HD[117] = 32'b000001_10101_00011_0000000000000010; // lw 3 $fp 2
119 HD[118] = 32'b000000_00010_00011_00010_00000_000101; // add 2 2 3
120 HD[119] = 32'b000001_10101_00011_0000000000000011; // lw 3 $fp 3
121 HD[120] = 32'b100111_00010_00011_00011_000000000000; // lcdwrite 2 3 3
122 HD[121] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
123 HD[122] = 32'b000101_00000_00011_0000000000001010; // li 3 10
124 HD[123] = 32'b000000_00010_00011_00010_00000_010001; // slt 2 2 3
125 HD[124] = 32'b000000_00010_00000_00010_00000_010011; // set 2 2 $zero
126 HD[125] = 32'b000111_00010_00000_0000000000001110; // beq 2 $zero _ElseBegin4_
127 HD[126] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
128 HD[127] = 32'b000101_00000_00011_0000000000001010; // li 3 10
129 HD[128] = 32'b000000_00010_00011_00010_00000_001000; // div 2 2 3
130 HD[129] = 32'b000101_00000_00011_0000000000001010; // li 3 10
131 HD[130] = 32'b000011_10101_10101_0000000000000100; // addi $fp $fp 4
132 HD[131] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
133 HD[132] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
134 HD[133] = 32'b101000_0000000000000000001011010; // jal mod
135 HD[134] = 32'b000100_10101_10101_0000000000000100; // subi $fp $fp 4
136 HD[135] = 32'b000000_10100_00010_00000_00000_010000; // move 2 $rv
137 HD[136] = 32'b000011_00010_00010_0000000000110000; // addi 2 2 48
138 HD[137] = 32'b000010_10101_00010_0000000000000011; // sw 2 $fp 3
139 HD[138] = 32'b001011_00000000000000000010001101; // jump _IfExit4_
140 HD[139] = 32'b000101_00000_00010_0000000000100000; // li 2 32 / _ElseBegin4_
141 HD[140] = 32'b000010_10101_00010_0000000000000011; // sw 2 $fp 3
142 HD[141] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1 / _IfExit4_
143 HD[142] = 32'b000101_00000_00011_0000000000010000; // li 3 16
144 HD[143] = 32'b000000_00010_00011_00010_00000_000111; // mult 2 2 3
145 HD[144] = 32'b000001_10101_00011_0000000000000010; // lw 3 $fp 2
146 HD[145] = 32'b000100_00011_00011_0000000000000001; // subi 3 3 1
147 HD[146] = 32'b000000_00010_00011_00010_00000_000101; // add 2 2 3
148 HD[147] = 32'b000001_10101_00011_0000000000000011; // lw 3 $fp 3
149 HD[148] = 32'b100111_00010_00011_00011_000000000000; // lcdwrite 2 3 3
150 HD[149] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
151 HD[150] = 32'b000101_00000_00011_0000000001100100; // li 3 100
152 HD[151] = 32'b000000_00010_00011_00010_00000_010001; // slt 2 2 3
153 HD[152] = 32'b000000_00010_00000_00010_00000_010011; // set 2 2 $zero
154 HD[153] = 32'b000111_00010_00000_0000000000001110; // beq 2 $zero _ElseBegin5_
155 HD[154] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
156 HD[155] = 32'b000101_00000_00011_0000000001100100; // li 3 100
157 HD[156] = 32'b000000_00010_00011_00010_00000_001000; // div 2 2 3
158 HD[157] = 32'b000101_00000_00011_0000000000001010; // li 3 10
159 HD[158] = 32'b000011_10101_10101_0000000000000100; // addi $fp $fp 4
160 HD[159] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
161 HD[160] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
162 HD[161] = 32'b101000_0000000000000000001011010; // jal mod
163 HD[162] = 32'b000100_10101_10101_0000000000000100; // subi $fp $fp 4
164 HD[163] = 32'b000000_10100_00010_00000_00000_010000; // move 2 $rv
165 HD[164] = 32'b000011_00010_00010_0000000000110000; // addi 2 2 48
166 HD[165] = 32'b000010_10101_00010_0000000000000011; // sw 2 $fp 3
167 HD[166] = 32'b001011_00000000000000000010101001; // jump _IfExit5_
168 HD[167] = 32'b000101_00000_00010_0000000000100000; // li 2 32 / _ElseBegin5_
169 HD[168] = 32'b000010_10101_00010_0000000000000011; // sw 2 $fp 3
170 HD[169] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1 / _IfExit5_
171 HD[170] = 32'b000101_00000_00011_0000000000010000; // li 3 16

```

```

172 HD[171] = 32'b000000_00010_00011_00010_00000_000111; // mult 2 2 3
173 HD[172] = 32'b000001_10101_00011_0000000000000010; // lw 3 $fp 2
174 HD[173] = 32'b000100_00011_00011_0000000000000010; // subi 3 3 2
175 HD[174] = 32'b000000_00010_00011_00010_00000_000101; // add 2 2 3
176 HD[175] = 32'b000001_10101_00011_0000000000000011; // lw 3 $fp 3
177 HD[176] = 32'b100111_00010_00011_00011_000000000000; // lcdwrite 2 3 3
178 HD[177] = 32'b010000_10110_10111_0000000000000000; // pop $ra
179 HD[178] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
180 HD[179] = 32'b001111_10110_10111_0000000000000000; // push $ra / print_default_msg
181 HD[180] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
182 HD[181] = 32'b000101_00000_00011_0000000000000001; // li 3 1
183 HD[182] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
184 HD[183] = 32'b000111_00010_00000_0000000000010010; // beq 2 $zero _ElseBegin6_
185 HD[184] = 32'b100111_00000_00000_00000_00001000101; // lcdwrite 0 69
186 HD[185] = 32'b100111_00001_00000_00000_00001010010; // lcdwrite 1 82
187 HD[186] = 32'b100111_00010_00000_00000_00001010010; // lcdwrite 2 82
188 HD[187] = 32'b100111_00011_00000_00000_00001001111; // lcdwrite 3 79
189 HD[188] = 32'b100111_00100_00000_00000_00000111010; // lcdwrite 4 58
190 HD[189] = 32'b100111_00101_00000_00000_00000100000; // lcdwrite 5 32
191 HD[190] = 32'b100111_00110_00000_00000_00001000010; // lcdwrite 6 66
192 HD[191] = 32'b100111_00111_00000_00000_00001000001; // lcdwrite 7 65
193 HD[192] = 32'b100111_01000_00000_00000_00001000100; // lcdwrite 8 68
194 HD[193] = 32'b100111_01001_00000_00000_00000100000; // lcdwrite 9 32
195 HD[194] = 32'b100111_01010_00000_00000_00001010110; // lcdwrite 10 86
196 HD[195] = 32'b100111_01011_00000_00000_00001000001; // lcdwrite 11 65
197 HD[196] = 32'b100111_01100_00000_00000_00001001100; // lcdwrite 12 76
198 HD[197] = 32'b100111_01101_00000_00000_00001010101; // lcdwrite 13 85
199 HD[198] = 32'b100111_01110_00000_00000_00001000101; // lcdwrite 14 69
200 HD[199] = 32'b100111_01111_00000_00000_00000100001; // lcdwrite 15 33
201 HD[200] = 32'b001011_0000000000000000000101110000; // jump _IfExit6_
202 HD[201] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0 / _ElseBegin6_
203 HD[202] = 32'b000101_00000_00011_0000000000000010; // li 3 2
204 HD[203] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
205 HD[204] = 32'b000111_00010_00000_0000000000010010; // beq 2 $zero _ElseBegin7_
206 HD[205] = 32'b100111_00000_00000_00000_00001010011; // lcdwrite 0 83
207 HD[206] = 32'b100111_00001_00000_00000_00001100101; // lcdwrite 1 101
208 HD[207] = 32'b100111_00010_00000_00000_00001101100; // lcdwrite 2 108
209 HD[208] = 32'b100111_00011_00000_00000_00001100101; // lcdwrite 3 101
210 HD[209] = 32'b100111_00100_00000_00000_00001100011; // lcdwrite 4 99
211 HD[210] = 32'b100111_00101_00000_00000_00001101001; // lcdwrite 5 105
212 HD[211] = 32'b100111_00110_00000_00000_00001101111; // lcdwrite 6 111
213 HD[212] = 32'b100111_00111_00000_00000_00001101110; // lcdwrite 7 110
214 HD[213] = 32'b100111_01000_00000_00000_00001100101; // lcdwrite 8 101
215 HD[214] = 32'b100111_01001_00000_00000_00000100000; // lcdwrite 9 32
216 HD[215] = 32'b100111_01010_00000_00000_00001101111; // lcdwrite 10 111
217 HD[216] = 32'b100111_01011_00000_00000_00000100000; // lcdwrite 11 32
218 HD[217] = 32'b100111_01100_00000_00000_00001110000; // lcdwrite 12 112
219 HD[218] = 32'b100111_01101_00000_00000_00001110010; // lcdwrite 13 114
220 HD[219] = 32'b100111_01110_00000_00000_00001101111; // lcdwrite 14 111
221 HD[220] = 32'b100111_01111_00000_00000_00001100111; // lcdwrite 15 103
222 HD[221] = 32'b001011_0000000000000000000101110000; // jump _IfExit7_
223 HD[222] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0 / _ElseBegin7_
224 HD[223] = 32'b000101_00000_00011_0000000000000011; // li 3 3
225 HD[224] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
226 HD[225] = 32'b000111_00010_00000_0000000000010010; // beq 2 $zero _ElseBegin8_
227 HD[226] = 32'b100111_00000_00000_00000_00001000101; // lcdwrite 0 69
228 HD[227] = 32'b100111_00001_00000_00000_00001110010; // lcdwrite 1 114
229 HD[228] = 32'b100111_00010_00000_00000_00001110010; // lcdwrite 2 114
230 HD[229] = 32'b100111_00011_00000_00000_00001101111; // lcdwrite 3 111
231 HD[230] = 32'b100111_00100_00000_00000_00000111010; // lcdwrite 4 58

```

```

232 HD[231] = 32'b100111_00101_00000_00000_00000100000; // lcdwrite 5 32
233 HD[232] = 32'b100111_00110_00000_00000_00001010000; // lcdwrite 6 80
234 HD[233] = 32'b100111_00111_00000_00000_00001110010; // lcdwrite 7 114
235 HD[234] = 32'b100111_01000_00000_00000_00001101111; // lcdwrite 8 111
236 HD[235] = 32'b100111_01001_00000_00000_00001100111; // lcdwrite 9 103
237 HD[236] = 32'b100111_01010_00000_00000_00000101110; // lcdwrite 10 46
238 HD[237] = 32'b100111_01011_00000_00000_00000100000; // lcdwrite 11 32
239 HD[238] = 32'b100111_01100_00000_00000_00000100000; // lcdwrite 12 32
240 HD[239] = 32'b100111_01101_00000_00000_00000100000; // lcdwrite 13 32
241 HD[240] = 32'b100111_01110_00000_00000_00000100000; // lcdwrite 14 32
242 HD[241] = 32'b100111_01111_00000_00000_00000100000; // lcdwrite 15 32
243 HD[242] = 32'b001011_000000000000000000101110000; // jump _IfExit8_
244 HD[243] = 32'b000001_10101_00010_00000000000000000; // lw 2 $fp 0 / _ElseBegin8_
245 HD[244] = 32'b000101_00000_00011_00000000000000100; // li 3 4
246 HD[245] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
247 HD[246] = 32'b000111_00010_00000_00000000000010010; // beq 2 $zero _ElseBegin9_
248 HD[247] = 32'b100111_10000_00000_00000_00000100000; // lcdwrite 16 32
249 HD[248] = 32'b100111_10001_00000_00000_00001101110; // lcdwrite 17 110
250 HD[249] = 32'b100111_10010_00000_00000_00001100001; // lcdwrite 18 97
251 HD[250] = 32'b100111_10011_00000_00000_00001101111; // lcdwrite 19 111
252 HD[251] = 32'b100111_10100_00000_00000_00000100000; // lcdwrite 20 32
253 HD[252] = 32'b100111_10101_00000_00000_00001100101; // lcdwrite 21 101
254 HD[253] = 32'b100111_10110_00000_00000_00001101110; // lcdwrite 22 110
255 HD[254] = 32'b100111_10111_00000_00000_00001100011; // lcdwrite 23 99
256 HD[255] = 32'b100111_11000_00000_00000_00001101111; // lcdwrite 24 111
257 HD[256] = 32'b100111_11001_00000_00000_00001101110; // lcdwrite 25 110
258 HD[257] = 32'b100111_11010_00000_00000_00001110100; // lcdwrite 26 116
259 HD[258] = 32'b100111_11011_00000_00000_00001110010; // lcdwrite 27 114
260 HD[259] = 32'b100111_11100_00000_00000_00001100001; // lcdwrite 28 97
261 HD[260] = 32'b100111_11101_00000_00000_00001100100; // lcdwrite 29 100
262 HD[261] = 32'b100111_11110_00000_00000_00001101111; // lcdwrite 30 111
263 HD[262] = 32'b100111_11111_00000_00000_00000100000; // lcdwrite 31 32
264 HD[263] = 32'b001011_000000000000000000101110000; // jump _IfExit9_
265 HD[264] = 32'b000001_10101_00010_00000000000000000; // lw 2 $fp 0 / _ElseBegin9_
266 HD[265] = 32'b000101_00000_00011_00000000000000101; // li 3 5
267 HD[266] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
268 HD[267] = 32'b000111_00010_00000_00000000000010010; // beq 2 $zero _ElseBegin10_
269 HD[268] = 32'b100111_00000_00000_00000_00000100000; // lcdwrite 0 32
270 HD[269] = 32'b100111_00001_00000_00000_00001010000; // lcdwrite 1 80
271 HD[270] = 32'b100111_00010_00000_00000_00001110010; // lcdwrite 2 114
272 HD[271] = 32'b100111_00011_00000_00000_00001101111; // lcdwrite 3 111
273 HD[272] = 32'b100111_00100_00000_00000_00001100111; // lcdwrite 4 103
274 HD[273] = 32'b100111_00101_00000_00000_00001110010; // lcdwrite 5 114
275 HD[274] = 32'b100111_00110_00000_00000_00001100001; // lcdwrite 6 97
276 HD[275] = 32'b100111_00111_00000_00000_00001101101; // lcdwrite 7 109
277 HD[276] = 32'b100111_01000_00000_00000_00001100001; // lcdwrite 8 97
278 HD[277] = 32'b100111_01001_00000_00000_00000100000; // lcdwrite 9 32
279 HD[278] = 32'b100111_01010_00000_00000_00000100000; // lcdwrite 10 32
280 HD[279] = 32'b100111_01011_00000_00000_00000100000; // lcdwrite 11 32
281 HD[280] = 32'b100111_01100_00000_00000_00000100000; // lcdwrite 12 32
282 HD[281] = 32'b100111_01101_00000_00000_00000100000; // lcdwrite 13 32
283 HD[282] = 32'b100111_01110_00000_00000_00000100000; // lcdwrite 14 32
284 HD[283] = 32'b100111_01111_00000_00000_00000100000; // lcdwrite 15 32
285 HD[284] = 32'b001011_000000000000000000101110000; // jump _IfExit10_
286 HD[285] = 32'b000001_10101_00010_00000000000000000; // lw 2 $fp 0 / _ElseBegin10_
287 HD[286] = 32'b000101_00000_00011_00000000000000110; // li 3 6
288 HD[287] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
289 HD[288] = 32'b000111_00010_00000_00000000000010010; // beq 2 $zero _ElseBegin11_
290 HD[289] = 32'b100111_10000_00000_00000_00001001001; // lcdwrite 16 73
291 HD[290] = 32'b100111_10001_00000_00000_00001000100; // lcdwrite 17 68

```



```

292 HD[291] = 32'b100111_10010_00000_00000_00000100000; // lcdwrite 18 32
293 HD[292] = 32'b100111_10011_00000_00000_00001100100; // lcdwrite 19 100
294 HD[293] = 32'b100111_10100_00000_00000_00001100101; // lcdwrite 20 101
295 HD[294] = 32'b100111_10101_00000_00000_00001110110; // lcdwrite 21 118
296 HD[295] = 32'b100111_10110_00000_00000_00001100101; // lcdwrite 22 101
297 HD[296] = 32'b100111_10111_00000_00000_00000100000; // lcdwrite 23 32
298 HD[297] = 32'b100111_11000_00000_00000_00001110011; // lcdwrite 24 115
299 HD[298] = 32'b100111_11001_00000_00000_00001100101; // lcdwrite 25 101
300 HD[299] = 32'b100111_11010_00000_00000_00001110010; // lcdwrite 26 114
301 HD[300] = 32'b100111_11011_00000_00000_00000100000; // lcdwrite 27 32
302 HD[301] = 32'b100111_11100_00000_00000_00000111110; // lcdwrite 28 62
303 HD[302] = 32'b100111_11101_00000_00000_00000100000; // lcdwrite 29 32
304 HD[303] = 32'b100111_11110_00000_00000_00000110000; // lcdwrite 30 48
305 HD[304] = 32'b100111_11111_00000_00000_00000100001; // lcdwrite 31 33
306 HD[305] = 32'b001011_000000000000000000101110000; // jump _IfExit11_
307 HD[306] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0 / _ElseBegin11_
308 HD[307] = 32'b000101_00000_00011_0000000000000111; // li 3 7
309 HD[308] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
310 HD[309] = 32'b000111_00010_00000_00000000000010010; // beq 2 $zero _ElseBegin12_
311 HD[310] = 32'b100111_10000_00000_00000_00000100000; // lcdwrite 16 32
312 HD[311] = 32'b100111_10001_00000_00000_00001101010; // lcdwrite 17 106
313 HD[312] = 32'b100111_10010_00000_00000_00001100001; // lcdwrite 18 97
314 HD[313] = 32'b100111_10011_00000_00000_00000100000; // lcdwrite 19 32
315 HD[314] = 32'b100111_10100_00000_00000_00001110101; // lcdwrite 20 117
316 HD[315] = 32'b100111_10101_00000_00000_00001110011; // lcdwrite 21 115
317 HD[316] = 32'b100111_10110_00000_00000_00001100001; // lcdwrite 22 97
318 HD[317] = 32'b100111_10111_00000_00000_00000100000; // lcdwrite 23 32
319 HD[318] = 32'b100111_11000_00000_00000_00001100101; // lcdwrite 24 101
320 HD[319] = 32'b100111_11001_00000_00000_00001110011; // lcdwrite 25 115
321 HD[320] = 32'b100111_11010_00000_00000_00001110011; // lcdwrite 26 115
322 HD[321] = 32'b100111_11011_00000_00000_00001100101; // lcdwrite 27 101
323 HD[322] = 32'b100111_11100_00000_00000_00000100000; // lcdwrite 28 32
324 HD[323] = 32'b100111_11101_00000_00000_00001001001; // lcdwrite 29 73
325 HD[324] = 32'b100111_11110_00000_00000_00001000100; // lcdwrite 30 68
326 HD[325] = 32'b100111_11111_00000_00000_00000100000; // lcdwrite 31 32
327 HD[326] = 32'b001011_000000000000000000101110000; // jump _IfExit12_
328 HD[327] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0 / _ElseBegin12_
329 HD[328] = 32'b000101_00000_00011_0000000000001000; // li 3 8
330 HD[329] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
331 HD[330] = 32'b000111_00010_00000_00000000000010010; // beq 2 $zero _ElseBegin13_
332 HD[331] = 32'b100111_10000_00000_00000_00000100000; // lcdwrite 16 32
333 HD[332] = 32'b100111_10001_00000_00000_00001110011; // lcdwrite 17 115
334 HD[333] = 32'b100111_10010_00000_00000_00001100101; // lcdwrite 18 101
335 HD[334] = 32'b100111_10011_00000_00000_00001101110; // lcdwrite 19 110
336 HD[335] = 32'b100111_10100_00000_00000_00001100100; // lcdwrite 20 100
337 HD[336] = 32'b100111_10101_00000_00000_00001101111; // lcdwrite 21 111
338 HD[337] = 32'b100111_10110_00000_00000_00000100000; // lcdwrite 22 32
339 HD[338] = 32'b100111_10111_00000_00000_00001100101; // lcdwrite 23 101
340 HD[339] = 32'b100111_11000_00000_00000_00001111000; // lcdwrite 24 120
341 HD[340] = 32'b100111_11001_00000_00000_00001100101; // lcdwrite 25 101
342 HD[341] = 32'b100111_11010_00000_00000_00001100011; // lcdwrite 26 99
343 HD[342] = 32'b100111_11011_00000_00000_00001110101; // lcdwrite 27 117
344 HD[343] = 32'b100111_11100_00000_00000_00001110100; // lcdwrite 28 116
345 HD[344] = 32'b100111_11101_00000_00000_00001100001; // lcdwrite 29 97
346 HD[345] = 32'b100111_11110_00000_00000_00001100100; // lcdwrite 30 100
347 HD[346] = 32'b100111_11111_00000_00000_00001101111; // lcdwrite 31 111
348 HD[347] = 32'b001011_000000000000000000101110000; // jump _IfExit13_
349 HD[348] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0 / _ElseBegin13_
350 HD[349] = 32'b000101_00000_00011_0000000000001001; // li 3 9
351 HD[350] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3

```

```

352 HD[351] = 32'b000111_00010_00000_00000000000010001; // beq 2 $zero _IfExit14_
353 HD[352] = 32'b100111_10000_00000_00000_00000100000; // lcdwrite 16 32
354 HD[353] = 32'b100111_10001_00000_00000_00001100110; // lcdwrite 17 102
355 HD[354] = 32'b100111_10010_00000_00000_00001101111; // lcdwrite 18 111
356 HD[355] = 32'b100111_10011_00000_00000_00001101001; // lcdwrite 19 105
357 HD[356] = 32'b100111_10100_00000_00000_00000100000; // lcdwrite 20 32
358 HD[357] = 32'b100111_10101_00000_00000_00001100110; // lcdwrite 21 102
359 HD[358] = 32'b100111_10110_00000_00000_00001101001; // lcdwrite 22 105
360 HD[359] = 32'b100111_10111_00000_00000_00001101110; // lcdwrite 23 110
361 HD[360] = 32'b100111_11000_00000_00000_00001100001; // lcdwrite 24 97
362 HD[361] = 32'b100111_11001_00000_00000_00001101100; // lcdwrite 25 108
363 HD[362] = 32'b100111_11010_00000_00000_00001101001; // lcdwrite 26 105
364 HD[363] = 32'b100111_11011_00000_00000_00001111010; // lcdwrite 27 122
365 HD[364] = 32'b100111_11100_00000_00000_00001100001; // lcdwrite 28 97
366 HD[365] = 32'b100111_11101_00000_00000_00001100100; // lcdwrite 29 100
367 HD[366] = 32'b100111_11110_00000_00000_00001101111; // lcdwrite 30 111
368 HD[367] = 32'b100111_11111_00000_00000_00000100000; // lcdwrite 31 32
369 HD[368] = 32'b010000_10110_10111_0000000000000000; // pop $ra / _IfExit14_ _IfExit13_
    _IfExit12_ _IfExit11_ _IfExit10_ _IfExit9_ _IfExit8_ _IfExit7_ _IfExit6_
370 HD[369] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
371 HD[370] = 32'b001111_10110_10111_0000000000000000; // push $ra / load_registradores
372 HD[371] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
373 HD[372] = 32'b000001_00000_00011_00000000000000010; // lw 3 $global 2
374 HD[373] = 32'b000000_00010_00011_00010_00000_000111; // mult 2 2 3
375 HD[374] = 32'b000001_00000_00011_00000000000000011; // lw 3 $global 3
376 HD[375] = 32'b000000_00010_00011_00010_00000_000101; // add 2 2 3
377 HD[376] = 32'b000010_10101_00010_0000000000000001; // sw 2 $fp 1
378 HD[377] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
379 HD[378] = 32'b000001_00010_01010_00000000000001010; // lw 10 2 10
380 HD[379] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
381 HD[380] = 32'b000001_00010_01011_00000000000001011; // lw 11 2 11
382 HD[381] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
383 HD[382] = 32'b000001_00010_01100_00000000000001100; // lw 12 2 12
384 HD[383] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
385 HD[384] = 32'b000001_00010_01101_00000000000001101; // lw 13 2 13
386 HD[385] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
387 HD[386] = 32'b000001_00010_01110_00000000000001110; // lw 14 2 14
388 HD[387] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
389 HD[388] = 32'b000001_00010_01111_00000000000001111; // lw 15 2 15
390 HD[389] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
391 HD[390] = 32'b000001_00010_10000_0000000000010000; // lw 16 2 16
392 HD[391] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
393 HD[392] = 32'b000001_00010_10001_0000000000010001; // lw 17 2 17
394 HD[393] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
395 HD[394] = 32'b000001_00010_10010_0000000000010010; // lw 18 2 18
396 HD[395] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
397 HD[396] = 32'b000001_00010_10011_0000000000010011; // lw 19 2 19
398 HD[397] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
399 HD[398] = 32'b000001_00010_11001_0000000000011001; // lw 25 2 25
400 HD[399] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
401 HD[400] = 32'b000001_00010_11010_0000000000011010; // lw 26 2 26
402 HD[401] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
403 HD[402] = 32'b000001_00010_11011_0000000000011011; // lw 27 2 27
404 HD[403] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
405 HD[404] = 32'b000001_00010_11100_0000000000011100; // lw 28 2 28
406 HD[405] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
407 HD[406] = 32'b000001_00010_11101_0000000000011101; // lw 29 2 29
408 HD[407] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
409 HD[408] = 32'b000001_00010_11110_0000000000011110; // lw 30 2 30
410 HD[409] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1

```

```

411 HD[410] = 32'b000001_00010_11111_00000000000011111; // lw 31 2 31
412 HD[411] = 32'b010000_10110_10111_0000000000000000; // pop $ra
413 HD[412] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
414 HD[413] = 32'b001111_10110_10111_0000000000000000; // push $ra / store_registradores
415 HD[414] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
416 HD[415] = 32'b000001_00000_00011_0000000000000010; // lw 3 $global 2
417 HD[416] = 32'b000000_00010_00011_00010_00000_000111; // mult 2 2 3
418 HD[417] = 32'b000001_00000_00011_0000000000000011; // lw 3 $global 3
419 HD[418] = 32'b000000_00010_00011_00010_00000_000101; // add 2 2 3
420 HD[419] = 32'b000010_10101_00010_0000000000000001; // sw 2 $fp 1
421 HD[420] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
422 HD[421] = 32'b000010_00010_01010_0000000000001010; // sw 10 2 10
423 HD[422] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
424 HD[423] = 32'b000010_00010_01011_0000000000001011; // sw 11 2 11
425 HD[424] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
426 HD[425] = 32'b000010_00010_01100_0000000000001100; // sw 12 2 12
427 HD[426] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
428 HD[427] = 32'b000010_00010_01101_0000000000001101; // sw 13 2 13
429 HD[428] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
430 HD[429] = 32'b000010_00010_01110_0000000000001110; // sw 14 2 14
431 HD[430] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
432 HD[431] = 32'b000010_00010_01111_0000000000001111; // sw 15 2 15
433 HD[432] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
434 HD[433] = 32'b000010_00010_10000_0000000000010000; // sw 16 2 16
435 HD[434] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
436 HD[435] = 32'b000010_00010_10001_0000000000010001; // sw 17 2 17
437 HD[436] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
438 HD[437] = 32'b000010_00010_10010_0000000000010010; // sw 18 2 18
439 HD[438] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
440 HD[439] = 32'b000010_00010_10011_0000000000010011; // sw 19 2 19
441 HD[440] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
442 HD[441] = 32'b000010_00010_11001_0000000000011001; // sw 25 2 25
443 HD[442] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
444 HD[443] = 32'b000010_00010_11010_0000000000011010; // sw 26 2 26
445 HD[444] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
446 HD[445] = 32'b000010_00010_11011_0000000000011011; // sw 27 2 27
447 HD[446] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
448 HD[447] = 32'b000010_00010_11100_0000000000011100; // sw 28 2 28
449 HD[448] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
450 HD[449] = 32'b000010_00010_11101_0000000000011101; // sw 29 2 29
451 HD[450] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
452 HD[451] = 32'b000010_00010_11110_0000000000011110; // sw 30 2 30
453 HD[452] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
454 HD[453] = 32'b000010_00010_11111_0000000000011111; // sw 31 2 31
455 HD[454] = 32'b010000_10110_10111_0000000000000000; // pop $ra
456 HD[455] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
457 HD[456] = 32'b001111_10110_10111_0000000000000000; // push $ra / organiza_fila_pronto
458 HD[457] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
459 HD[458] = 32'b000101_00000_00011_0000000000000000; // li 3 0
460 HD[459] = 32'b000000_00010_00011_00010_00000_010010; // sgt 2 2 3
461 HD[460] = 32'b000111_00010_00000_0000000000011010; // beq 2 $zero _IfExit15_
462 HD[461] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0 / _WhileBegin2_
463 HD[462] = 32'b000001_00000_00011_0000000000001000; // lw 3 $global 8
464 HD[463] = 32'b000000_00010_00011_00010_00000_010001; // slt 2 2 3
465 HD[464] = 32'b000111_00010_00000_0000000000011100; // beq 2 $zero _WhileExit2_
466 HD[465] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
467 HD[466] = 32'b000000_00010_00000_00010_00000_000101; // add 2 2 $global
468 HD[467] = 32'b000001_00010_00010_000000000001001; // lw 2 2 9
469 HD[468] = 32'b000001_10101_00011_0000000000000000; // lw 3 $fp 0
470 HD[469] = 32'b000100_00011_00011_0000000000000001; // subi 3 3 1

```

```

471 HD[470] = 32'b000000_00011_00000_00011_00000_000101; // add 3 3 $global
472 HD[471] = 32'b000010_00011_00010_00000000000001001; // sw 2 3 9
473 HD[472] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
474 HD[473] = 32'b000011_00010_00010_0000000000000001; // addi 2 2 1
475 HD[474] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
476 HD[475] = 32'b001011_00000000000000000111001101; // jump _WhileBegin2_
477 HD[476] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0 / _WhileExit2_
478 HD[477] = 32'b000001_00000_00011_00000000000000111; // lw 3 $global 7
479 HD[478] = 32'b000000_00010_00011_00010_00000_010010; // sgt 2 2 3
480 HD[479] = 32'b000000_00010_00000_00010_00000_010011; // set 2 2 $zero
481 HD[480] = 32'b000111_00010_00000_00000000000000110; // beq 2 $zero _IfExit16_
482 HD[481] = 32'b000001_00000_00010_00000000000010100; // lw 2 $global 20
483 HD[482] = 32'b000001_10101_00011_0000000000000000; // lw 3 $fp 0
484 HD[483] = 32'b000100_00011_00011_0000000000000001; // subi 3 3 1
485 HD[484] = 32'b000000_00011_00000_00011_00000_000101; // add 3 3 $global
486 HD[485] = 32'b000010_00011_00010_0000000000001001; // sw 2 3 9
487 HD[486] = 32'b010000_10110_10111_0000000000000000; // pop $ra / _IfExit16_ _IfExit15_
488 HD[487] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
489 HD[488] = 32'b001111_10110_10111_0000000000000000; // push $ra / fila_existe_processo
490 HD[489] = 32'b000101_00000_00010_0000000000000000; // li 2 0
491 HD[490] = 32'b000010_10101_00010_0000000000000001; // sw 2 $fp 1
492 HD[491] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1 / _WhileBegin3_
493 HD[492] = 32'b000001_00000_00011_0000000000001000; // lw 3 $global 8
494 HD[493] = 32'b000000_00010_00011_00010_00000_010001; // slt 2 2 3
495 HD[494] = 32'b000111_00010_00000_0000000000001111; // beq 2 $zero _WhileExit3_
496 HD[495] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
497 HD[496] = 32'b000000_00010_00000_00010_00000_000101; // add 2 2 $global
498 HD[497] = 32'b000001_00010_00010_0000000000001001; // lw 2 2 9
499 HD[498] = 32'b000001_10101_00011_0000000000000000; // lw 3 $fp 0
500 HD[499] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
501 HD[500] = 32'b000111_00010_00000_00000000000000101; // beq 2 $zero _IfExit17_
502 HD[501] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
503 HD[502] = 32'b000000_00010_10100_00000_00000_010000; // move $rv 2
504 HD[503] = 32'b010000_10110_10111_0000000000000000; // pop $ra
505 HD[504] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
506 HD[505] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1 / _IfExit17_
507 HD[506] = 32'b000011_00010_00010_0000000000000001; // addi 2 2 1
508 HD[507] = 32'b000010_10101_00010_0000000000000001; // sw 2 $fp 1
509 HD[508] = 32'b001011_00000000000000000111101011; // jump _WhileBegin3_
510 HD[509] = 32'b000001_00000_00010_0000000000010110; // lw 2 $global 22 / _WhileExit3_
511 HD[510] = 32'b000000_00010_10100_00000_00000_010000; // move $rv 2
512 HD[511] = 32'b010000_10110_10111_0000000000000000; // pop $ra
513 HD[512] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
514 HD[513] = 32'b010000_10110_10111_0000000000000000; // pop $ra
515 HD[514] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
516 HD[515] = 32'b001111_10110_10111_0000000000000000; // push $ra /
    remove_processo_fila_pronto
517 HD[516] = 32'b000101_00000_00010_0000000000000000; // li 2 0
518 HD[517] = 32'b000010_10101_00010_0000000000000001; // sw 2 $fp 1
519 HD[518] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1 / _WhileBegin4_
520 HD[519] = 32'b000001_00000_00011_0000000000001000; // lw 3 $global 8
521 HD[520] = 32'b000000_00010_00011_00010_00000_010001; // slt 2 2 3
522 HD[521] = 32'b000111_00010_00000_0000000000010100; // beq 2 $zero _WhileExit4_
523 HD[522] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
524 HD[523] = 32'b000000_00010_00000_00010_00000_000101; // add 2 2 $global
525 HD[524] = 32'b000001_00010_00010_0000000000001001; // lw 2 2 9
526 HD[525] = 32'b000001_10101_00011_0000000000000000; // lw 3 $fp 0
527 HD[526] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
528 HD[527] = 32'b000111_00010_00000_0000000000001010; // beq 2 $zero _IfExit18_
529 HD[528] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1

```

```

530 HD[529] = 32'b000011_00010_00010_0000000000000001; // addi 2 2 1
531 HD[530] = 32'b000011_10101_10101_00000000000000010; // addi $fp $fp 2
532 HD[531] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
533 HD[532] = 32'b101000_00000000000000000111001000; // jal organiza_fila_pronto
534 HD[533] = 32'b000100_10101_10101_00000000000000010; // subi $fp $fp 2
535 HD[534] = 32'b000001_00000_00010_00000000000001000; // lw 2 $global 8
536 HD[535] = 32'b000100_00010_00010_0000000000000001; // subi 2 2 1
537 HD[536] = 32'b000010_00000_00010_00000000000001000; // sw 2 $global 8
538 HD[537] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1 / _IfExit18_
539 HD[538] = 32'b000011_00010_00010_0000000000000001; // addi 2 2 1
540 HD[539] = 32'b000010_10101_00010_0000000000000001; // sw 2 $fp 1
541 HD[540] = 32'b001011_00000000000000000100000110; // jump _WhileBegin4_
542 HD[541] = 32'b010000_10110_10111_0000000000000000; // pop $ra / _WhileExit4_
543 HD[542] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
544 HD[543] = 32'b001111_10110_10111_0000000000000000; // push $ra /
    insere_processo_fila_pronto
545 HD[544] = 32'b000101_00000_00010_0000000000000000; // li 2 0
546 HD[545] = 32'b000010_10101_00010_0000000000000001; // sw 2 $fp 1
547 HD[546] = 32'b000001_00000_00010_00000000000001000; // lw 2 $global 8
548 HD[547] = 32'b000001_00000_00011_00000000000000111; // lw 3 $global 7
549 HD[548] = 32'b000000_00010_00011_00010_00000_010001; // slt 2 2 3
550 HD[549] = 32'b000111_00010_00000_00000000000001001; // beq 2 $zero _IfExit19_
551 HD[550] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
552 HD[551] = 32'b000011_10101_10101_00000000000000010; // addi $fp $fp 2
553 HD[552] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
554 HD[553] = 32'b101000_00000000000000000111101000; // jal fila_existe_processo
555 HD[554] = 32'b000100_10101_10101_00000000000000010; // subi $fp $fp 2
556 HD[555] = 32'b000000_10100_00010_00000_00000_010000; // move 2 $rv
557 HD[556] = 32'b000001_00000_00011_000000000000010110; // lw 3 $global 22
558 HD[557] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
559 HD[558] = 32'b000111_00010_00000_00000000000001000; // beq 2 $zero _IfExit20_
560 HD[559] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
561 HD[560] = 32'b000001_00000_00011_00000000000001000; // lw 3 $global 8
562 HD[561] = 32'b000000_00011_00000_00011_00000_000101; // add 3 3 $global
563 HD[562] = 32'b000010_00011_00010_00000000000001001; // sw 2 3 9
564 HD[563] = 32'b000001_00000_00010_00000000000001000; // lw 2 $global 8
565 HD[564] = 32'b000011_00010_00010_0000000000000001; // addi 2 2 1
566 HD[565] = 32'b000010_00000_00010_00000000000001000; // sw 2 $global 8
567 HD[566] = 32'b010000_10110_10111_0000000000000000; // pop $ra / _IfExit20_ _IfExit19_
568 HD[567] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
569 HD[568] = 32'b001111_10110_10111_0000000000000000; // push $ra / fila_proximo
570 HD[569] = 32'b000001_00000_00010_00000000000001001; // lw 2 $global 9
571 HD[570] = 32'b000001_00000_00011_00000000000001000; // lw 3 $global 20
572 HD[571] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
573 HD[572] = 32'b000000_00010_00000_00010_00000_010011; // set 2 2 $zero
574 HD[573] = 32'b000111_00010_00000_00000000000001111; // beq 2 $zero _IfExit21_
575 HD[574] = 32'b000001_00000_00010_00000000000001001; // lw 2 $global 9
576 HD[575] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
577 HD[576] = 32'b000101_00000_00010_0000000000000001; // li 2 1
578 HD[577] = 32'b000011_10101_10101_0000000000000001; // addi $fp $fp 1
579 HD[578] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
580 HD[579] = 32'b101000_00000000000000000111001000; // jal organiza_fila_pronto
581 HD[580] = 32'b000100_10101_10101_0000000000000001; // subi $fp $fp 1
582 HD[581] = 32'b000001_00000_00010_00000000000001000; // lw 2 $global 8
583 HD[582] = 32'b000100_00010_00010_0000000000000001; // subi 2 2 1
584 HD[583] = 32'b000010_00000_00010_00000000000001000; // sw 2 $global 8
585 HD[584] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
586 HD[585] = 32'b000000_00010_10100_00000_00000_010000; // move $rv 2
587 HD[586] = 32'b010000_10110_10111_0000000000000000; // pop $ra
588 HD[587] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra

```

```

589 HD[588] = 32'b000001_00000_00010_0000000000010101; // lw 2 $global 21 / _IfExit21_
590 HD[589] = 32'b000000_00010_10100_00000_00000_010000; // move $rv 2
591 HD[590] = 32'b010000_10110_10111_0000000000000000; // pop $ra
592 HD[591] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
593 HD[592] = 32'b010000_10110_10111_0000000000000000; // pop $ra
594 HD[593] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
595 HD[594] = 32'b001111_10110_10111_0000000000000000; // push $ra / fila_passa_vez
596 HD[595] = 32'b000001_00000_00010_00000000000001001; // lw 2 $global 9
597 HD[596] = 32'b000001_00000_00011_00000000000010100; // lw 3 $global 20
598 HD[597] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
599 HD[598] = 32'b000000_00010_00000_00010_00000_010011; // set 2 2 $zero
600 HD[599] = 32'b000111_00010_00000_0000000000001101; // beq 2 $zero _IfExit22_
601 HD[600] = 32'b000001_00000_00010_00000000000001001; // lw 2 $global 9
602 HD[601] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
603 HD[602] = 32'b000101_00000_00010_00000000000000001; // li 2 1
604 HD[603] = 32'b000011_10101_10101_00000000000000001; // addi $fp $fp 1
605 HD[604] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
606 HD[605] = 32'b101000_00000000000000000111001000; // jal organiza_fila_pronto
607 HD[606] = 32'b000100_10101_10101_00000000000000001; // subi $fp $fp 1
608 HD[607] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
609 HD[608] = 32'b000001_00000_00011_00000000000001000; // lw 3 $global 8
610 HD[609] = 32'b000100_00011_00011_00000000000000001; // subi 3 3 1
611 HD[610] = 32'b000000_00011_00000_00011_00000_000101; // add 3 3 $global
612 HD[611] = 32'b000010_00011_00010_00000000000001001; // sw 2 3 9
613 HD[612] = 32'b010000_10110_10111_0000000000000000; // pop $ra / _IfExit22_
614 HD[613] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
615 HD[614] = 32'b001111_10110_10111_0000000000000000; // push $ra / posicao_programa
616 HD[615] = 32'b000001_00000_00010_00000000000001011; // lw 2 $global 23
617 HD[616] = 32'b000010_10101_00010_000000000000000010; // sw 2 $fp 2
618 HD[617] = 32'b000001_00000_00010_00000000000001110; // lw 2 $global 30
619 HD[618] = 32'b000010_10101_00010_000000000000000011; // sw 2 $fp 3
620 HD[619] = 32'b000101_00000_00010_0000000000000000; // li 2 0
621 HD[620] = 32'b000010_10101_00010_00000000000000001; // sw 2 $fp 1
622 HD[621] = 32'b000001_10101_00010_00000000000000001; // lw 2 $fp 1 / _WhileBegin5_
623 HD[622] = 32'b000001_00000_00011_0000000000000000; // lw 3 $global 0
624 HD[623] = 32'b000000_00010_00011_00010_00000_010001; // slt 2 2 3
625 HD[624] = 32'b000111_00010_00000_00000000000011100; // beq 2 $zero _WhileExit5_
626 HD[625] = 32'b000001_10101_00010_000000000000000010; // lw 2 $fp 2
627 HD[626] = 32'b000001_10101_00011_00000000000000001; // lw 3 $fp 1
628 HD[627] = 32'b000000_00010_00011_00010_00000_000101; // add 2 2 3
629 HD[628] = 32'b000010_10101_00010_00000000000000101; // sw 2 $fp 5
630 HD[629] = 32'b000001_10101_00010_00000000000000101; // lw 2 $fp 5
631 HD[630] = 32'b000001_00000_00011_00000000000011101; // lw 3 $global 29
632 HD[631] = 32'b100100_00010_00011_00010_00000_000000; // hdtoreg 2 2 3
633 HD[632] = 32'b000001_00000_00011_00000000000011000; // lw 3 $global 24
634 HD[633] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
635 HD[634] = 32'b000000_00010_00000_00010_00000_010011; // set 2 2 $zero
636 HD[635] = 32'b000111_00010_00000_0000000000001101; // beq 2 $zero _IfExit23_
637 HD[636] = 32'b000001_10101_00010_00000000000000101; // lw 2 $fp 5
638 HD[637] = 32'b000001_10101_00011_000000000000000011; // lw 3 $fp 3
639 HD[638] = 32'b100100_00010_00011_00010_00000_000000; // hdtoreg 2 2 3
640 HD[639] = 32'b000010_10101_00010_00000000000000100; // sw 2 $fp 4
641 HD[640] = 32'b000001_10101_00010_00000000000000100; // lw 2 $fp 4
642 HD[641] = 32'b000001_10101_00011_0000000000000000; // lw 3 $fp 0
643 HD[642] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
644 HD[643] = 32'b000111_00010_00000_00000000000000101; // beq 2 $zero _IfExit24_
645 HD[644] = 32'b000001_10101_00010_00000000000000101; // lw 2 $fp 5
646 HD[645] = 32'b000000_00010_10100_00000_00000_010000; // move $rv 2
647 HD[646] = 32'b010000_10110_10111_0000000000000000; // pop $ra
648 HD[647] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra

```



```

649 HD[648] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1 / _IfExit24_ _IfExit23_
650 HD[649] = 32'b000011_00010_00010_0000000000000001; // addi 2 2 1
651 HD[650] = 32'b000010_10101_00010_0000000000000001; // sw 2 $fp 1
652 HD[651] = 32'b001011_00000000000000001001101101; // jump _WhileBegin5_
653 HD[652] = 32'b000001_00000_00010_00000000000011001; // lw 2 $global 25 / _WhileExit5_
654 HD[653] = 32'b000000_00010_10100_00000_00000_010000; // move $rv 2
655 HD[654] = 32'b010000_10110_10111_0000000000000000; // pop $ra
656 HD[655] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
657 HD[656] = 32'b010000_10110_10111_0000000000000000; // pop $ra
658 HD[657] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
659 HD[658] = 32'b001111_10110_10111_0000000000000000; // push $ra / posicao_livre_programa
660 HD[659] = 32'b000001_00000_00010_0000000000001011; // lw 2 $global 23
661 HD[660] = 32'b000010_10101_00010_0000000000000001; // sw 2 $fp 1
662 HD[661] = 32'b000001_00000_00010_00000000000011101; // lw 2 $global 29
663 HD[662] = 32'b000010_10101_00010_00000000000000010; // sw 2 $fp 2
664 HD[663] = 32'b000101_00000_00010_0000000000000000; // li 2 0
665 HD[664] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
666 HD[665] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0 / _WhileBegin6_
667 HD[666] = 32'b000001_00000_00011_0000000000000000; // lw 3 $global 0
668 HD[667] = 32'b000000_00010_00011_00010_00000_010001; // slt 2 2 3
669 HD[668] = 32'b000111_00010_00000_00000000000010011; // beq 2 $zero _WhileExit6_
670 HD[669] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
671 HD[670] = 32'b000001_10101_00011_0000000000000000; // lw 3 $fp 0
672 HD[671] = 32'b000000_00010_00011_00010_00000_000101; // add 2 2 3
673 HD[672] = 32'b000010_10101_00010_00000000000000011; // sw 2 $fp 3
674 HD[673] = 32'b000001_10101_00010_00000000000000011; // lw 2 $fp 3
675 HD[674] = 32'b000001_10101_00011_00000000000000010; // lw 3 $fp 2
676 HD[675] = 32'b100100_00010_00011_00010_00000_000000; // hdtoreg 2 2 3
677 HD[676] = 32'b000001_00000_00011_00000000000011000; // lw 3 $global 24
678 HD[677] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
679 HD[678] = 32'b000111_00010_00000_000000000000000101; // beq 2 $zero _IfExit25_
680 HD[679] = 32'b000001_10101_00010_00000000000000011; // lw 2 $fp 3
681 HD[680] = 32'b000000_00010_10100_00000_00000_010000; // move $rv 2
682 HD[681] = 32'b010000_10110_10111_0000000000000000; // pop $ra
683 HD[682] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
684 HD[683] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0 / _IfExit25_
685 HD[684] = 32'b000011_00010_00010_0000000000000001; // addi 2 2 1
686 HD[685] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
687 HD[686] = 32'b001011_000000000000000010011001; // jump _WhileBegin6_
688 HD[687] = 32'b000001_00000_00010_00000000000011010; // lw 2 $global 26 / _WhileExit6_
689 HD[688] = 32'b000000_00010_10100_00000_00000_010000; // move $rv 2
690 HD[689] = 32'b010000_10110_10111_0000000000000000; // pop $ra
691 HD[690] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
692 HD[691] = 32'b010000_10110_10111_0000000000000000; // pop $ra
693 HD[692] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
694 HD[693] = 32'b001111_10110_10111_0000000000000000; // push $ra / tamanho_programa
695 HD[694] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
696 HD[695] = 32'b000011_10101_10101_00000000000000011; // addi $fp $fp 3
697 HD[696] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
698 HD[697] = 32'b101000_00000000000000001001100110; // jal posicao_programa
699 HD[698] = 32'b000100_10101_10101_00000000000000011; // subi $fp $fp 3
700 HD[699] = 32'b000000_10100_00010_00000_00000_010000; // move 2 $rv
701 HD[700] = 32'b000010_10101_00010_0000000000000001; // sw 2 $fp 1
702 HD[701] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
703 HD[702] = 32'b000001_00000_00011_00000000000011001; // lw 3 $global 25
704 HD[703] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
705 HD[704] = 32'b000111_00010_00000_000000000000000110; // beq 2 $zero _IfExit26_
706 HD[705] = 32'b000101_00000_00010_0000000000000000; // li 2 0
707 HD[706] = 32'b000100_00010_00010_0000000000000001; // subi 2 2 1
708 HD[707] = 32'b000000_00010_10100_00000_00000_010000; // move $rv 2

```

```

709 HD[708] = 32'b010000_10110_10111_0000000000000000; // pop $ra
710 HD[709] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
711 HD[710] = 32'b000001_00000_00010_0000000000100000; // lw 2 $global 32 / _IfExit26_
712 HD[711] = 32'b000010_10101_00010_0000000000000010; // sw 2 $fp 2
713 HD[712] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
714 HD[713] = 32'b000001_00000_00011_0000000000011111; // lw 3 $global 31
715 HD[714] = 32'b100100_00010_00011_00010_00000_000000; // hdtoreg 2 2 3
716 HD[715] = 32'b000000_00010_10100_00000_00000_010000; // move $rv 2
717 HD[716] = 32'b010000_10110_10111_0000000000000000; // pop $ra
718 HD[717] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
719 HD[718] = 32'b010000_10110_10111_0000000000000000; // pop $ra
720 HD[719] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
721 HD[720] = 32'b001111_10110_10111_0000000000000000; // push $ra /
    carrega_programa_memoria_inst
722 HD[721] = 32'b000101_00000_00010_0000000000000000; // li 2 0
723 HD[722] = 32'b000010_10101_00010_0000000000000001; // sw 2 $fp 1
724 HD[723] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
725 HD[724] = 32'b000011_10101_10101_0000000000000101; // addi $fp $fp 5
726 HD[725] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
727 HD[726] = 32'b101000_00000000000000001001100110; // jal posicao_programa
728 HD[727] = 32'b000100_10101_10101_0000000000000101; // subi $fp $fp 5
729 HD[728] = 32'b000000_10100_00010_00000_00000_010000; // move 2 $rv
730 HD[729] = 32'b000010_10101_00010_0000000000000010; // sw 2 $fp 2
731 HD[730] = 32'b000001_10101_00010_0000000000000010; // lw 2 $fp 2
732 HD[731] = 32'b000001_00000_00011_0000000000011001; // lw 3 $global 25
733 HD[732] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
734 HD[733] = 32'b000111_00010_00000_0000000000000110; // beq 2 $zero _IfExit27_
735 HD[734] = 32'b000101_00000_00010_0000000000000000; // li 2 0
736 HD[735] = 32'b000100_00010_00010_0000000000000001; // subi 2 2 1
737 HD[736] = 32'b000000_00010_10100_00000_00000_010000; // move $rv 2
738 HD[737] = 32'b010000_10110_10111_0000000000000000; // pop $ra
739 HD[738] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
740 HD[739] = 32'b000001_00000_00010_0000000000100000; // lw 2 $global 32 / _IfExit27_
741 HD[740] = 32'b000010_10101_00010_0000000000000011; // sw 2 $fp 3
742 HD[741] = 32'b000001_10101_00010_0000000000000010; // lw 2 $fp 2
743 HD[742] = 32'b000001_00000_00011_0000000000011111; // lw 3 $global 31
744 HD[743] = 32'b100100_00010_00011_00010_00000_000000; // hdtoreg 2 2 3
745 HD[744] = 32'b000010_10101_00010_0000000000000100; // sw 2 $fp 4
746 HD[745] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1 / _WhileBegin7_
747 HD[746] = 32'b000001_10101_00011_0000000000000100; // lw 3 $fp 4
748 HD[747] = 32'b000000_00010_00011_00010_00000_010001; // slt 2 2 3
749 HD[748] = 32'b000111_00010_00000_0000000000001101; // beq 2 $zero _WhileExit7_
750 HD[749] = 32'b000001_10101_00010_0000000000000010; // lw 2 $fp 2
751 HD[750] = 32'b000001_10101_00011_0000000000000011; // lw 3 $fp 3
752 HD[751] = 32'b000001_10101_00100_0000000000000001; // lw 4 $fp 1
753 HD[752] = 32'b000000_00011_00100_00011_00000_000101; // add 3 3 4
754 HD[753] = 32'b000001_00000_00100_0000000000000001; // lw 4 $global 1
755 HD[754] = 32'b000001_10101_00101_0000000000000001; // lw 5 $fp 1
756 HD[755] = 32'b000000_00100_00101_00100_00000_000101; // add 4 4 5
757 HD[756] = 32'b100011_00010_00011_00100_00000_000000; // hdtoint 4 2 3
758 HD[757] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
759 HD[758] = 32'b000011_00010_00010_0000000000000001; // addi 2 2 1
760 HD[759] = 32'b000010_10101_00010_0000000000000001; // sw 2 $fp 1
761 HD[760] = 32'b001011_0000000000000000101101001; // jump _WhileBegin7_
762 HD[761] = 32'b000001_10101_00010_0000000000000100; // lw 2 $fp 4 / _WhileExit7_
763 HD[762] = 32'b000000_00010_10100_00000_00000_010000; // move $rv 2
764 HD[763] = 32'b010000_10110_10111_0000000000000000; // pop $ra
765 HD[764] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
766 HD[765] = 32'b010000_10110_10111_0000000000000000; // pop $ra
767 HD[766] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra

```



```

768 HD[767] = 32'b001111_10110_10111_0000000000000000; // push $ra / cria_programa
769 HD[768] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
770 HD[769] = 32'b000011_10101_10101_0000000000000011; // addi $fp $fp 7
771 HD[770] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
772 HD[771] = 32'b101000_00000000000000001001100110; // jal posicao_programa
773 HD[772] = 32'b000100_10101_10101_0000000000000011; // subi $fp $fp 7
774 HD[773] = 32'b000000_10100_00010_00000_00000_010000; // move 2 $rv
775 HD[774] = 32'b000010_10101_00010_0000000000000010; // sw 2 $fp 2
776 HD[775] = 32'b000001_10101_00010_0000000000000010; // lw 2 $fp 2
777 HD[776] = 32'b000001_00000_00011_00000000000011001; // lw 3 $global 25
778 HD[777] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
779 HD[778] = 32'b000111_00010_00000_00000000000111101; // beq 2 $zero _IfExit28_
780 HD[779] = 32'b000011_10101_10101_0000000000000011; // addi $fp $fp 7
781 HD[780] = 32'b101000_00000000000000001010010010; // jal posicao_livre_programa
782 HD[781] = 32'b000100_10101_10101_0000000000000011; // subi $fp $fp 7
783 HD[782] = 32'b000000_10100_00010_00000_00000_010000; // move 2 $rv
784 HD[783] = 32'b000010_10101_00010_0000000000000010; // sw 2 $fp 2
785 HD[784] = 32'b000001_10101_00010_0000000000000010; // lw 2 $fp 2
786 HD[785] = 32'b000001_00000_00011_00000000000011101; // lw 3 $global 29
787 HD[786] = 32'b000101_00000_00100_0000000000000001; // li 4 1
788 HD[787] = 32'b100101_00010_00011_00100_00000_000000; // regtohd 4 2 3
789 HD[788] = 32'b000001_10101_00010_0000000000000010; // lw 2 $fp 2
790 HD[789] = 32'b000001_00000_00011_00000000000011110; // lw 3 $global 30
791 HD[790] = 32'b000001_10101_00100_0000000000000000; // lw 4 $fp 0
792 HD[791] = 32'b100101_00010_00011_00100_00000_000000; // regtohd 4 2 3
793 HD[792] = 32'b000001_10101_00010_0000000000000010; // lw 2 $fp 2
794 HD[793] = 32'b000001_00000_00011_00000000000011111; // lw 3 $global 31
795 HD[794] = 32'b000001_10101_00100_0000000000000001; // lw 4 $fp 1
796 HD[795] = 32'b100101_00010_00011_00100_00000_000000; // regtohd 4 2 3
797 HD[796] = 32'b000001_00000_00010_00000000000011100; // lw 2 $global 28
798 HD[797] = 32'b000010_10101_00010_0000000000000011; // sw 2 $fp 3
799 HD[798] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
800 HD[799] = 32'b000001_00000_00011_00000000000011100; // lw 3 $global 28
801 HD[800] = 32'b000000_00010_00011_00010_00000_000101; // add 2 2 3
802 HD[801] = 32'b000010_10101_00010_0000000000000001; // sw 2 $fp 1
803 HD[802] = 32'b000101_00000_00010_0000000000000000; // li 2 0
804 HD[803] = 32'b000010_10101_00010_00000000000000110; // sw 2 $fp 6
805 HD[804] = 32'b000001_10101_00010_0000000000000011; // lw 2 $fp 3 / _WhileBegin8_
806 HD[805] = 32'b000001_10101_00011_0000000000000001; // lw 3 $fp 1
807 HD[806] = 32'b000000_00010_00011_00010_00000_010001; // slt 2 2 3
808 HD[807] = 32'b000111_00010_00000_00000000000011101; // beq 2 $zero _WhileExit8_
809 HD[808] = 32'b000001_10101_00010_00000000000000110; // lw 2 $fp 6
810 HD[809] = 32'b000101_00000_00011_0000000000000000; // li 3 0
811 HD[810] = 32'b000101_00000_00100_00000000000000111; // li 4 15
812 HD[811] = 32'b000011_10101_10101_0000000000000011; // addi $fp $fp 7
813 HD[812] = 32'b000010_10101_00100_0000000000000010; // sw 4 $fp 2
814 HD[813] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
815 HD[814] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
816 HD[815] = 32'b101000_000000000000000000001100111; // jal print_value
817 HD[816] = 32'b000100_10101_10101_0000000000000011; // subi $fp $fp 7
818 HD[817] = 32'b001001_00000_00010_0000000000000000; // in 2
819 HD[818] = 32'b000010_10101_00010_00000000000000100; // sw 2 $fp 4
820 HD[819] = 32'b001001_00000_00010_0000000000000000; // in 2
821 HD[820] = 32'b000010_10101_00010_00000000000000101; // sw 2 $fp 5
822 HD[821] = 32'b000001_10101_00010_00000000000000100; // lw 2 $fp 4
823 HD[822] = 32'b000001_10101_00011_00000000000000101; // lw 3 $fp 5
824 HD[823] = 32'b000000_00010_00011_00010_00000_000101; // add 2 2 3
825 HD[824] = 32'b000010_10101_00010_00000000000000100; // sw 2 $fp 4
826 HD[825] = 32'b000001_10101_00010_0000000000000010; // lw 2 $fp 2
827 HD[826] = 32'b000001_10101_00011_0000000000000011; // lw 3 $fp 3

```

```

828 HD[827] = 32'b000001_10101_00100_0000000000000100; // lw 4 $fp 4
829 HD[828] = 32'b100101_00010_00011_00100_00000_000000; // regtohd 4 2 3
830 HD[829] = 32'b000001_10101_00010_0000000000000011; // lw 2 $fp 3
831 HD[830] = 32'b000011_00010_00010_0000000000000001; // addi 2 2 1
832 HD[831] = 32'b000010_10101_00010_0000000000000011; // sw 2 $fp 3
833 HD[832] = 32'b000001_10101_00010_00000000000000110; // lw 2 $fp 6
834 HD[833] = 32'b000011_00010_00010_0000000000000001; // addi 2 2 1
835 HD[834] = 32'b000010_10101_00010_00000000000000110; // sw 2 $fp 6
836 HD[835] = 32'b001011_00000000000000001100100100; // jump _WhileBegin8_
837 HD[836] = 32'b000101_00000_10100_0000000000000001; // li $rv 1 / _WhileExit8_
838 HD[837] = 32'b010000_10110_10111_0000000000000000; // pop $ra
839 HD[838] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
840 HD[839] = 32'b000001_00000_00010_0000000000011011; // lw 2 $global 27 / _IfExit28_
841 HD[840] = 32'b000000_00010_10100_00000_00000_010000; // move $rv 2
842 HD[841] = 32'b010000_10110_10111_0000000000000000; // pop $ra
843 HD[842] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
844 HD[843] = 32'b010000_10110_10111_0000000000000000; // pop $ra
845 HD[844] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
846 HD[845] = 32'b001111_10110_10111_0000000000000000; // push $ra / deleta_programa
847 HD[846] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
848 HD[847] = 32'b000011_10101_10101_0000000000000010; // addi $fp $fp 2
849 HD[848] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
850 HD[849] = 32'b101000_00000000000000001001100110; // jal posicao_programa
851 HD[850] = 32'b000100_10101_10101_0000000000000010; // subi $fp $fp 2
852 HD[851] = 32'b000000_10100_00010_00000_00000_010000; // move 2 $rv
853 HD[852] = 32'b000010_10101_00010_0000000000000001; // sw 2 $fp 1
854 HD[853] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
855 HD[854] = 32'b000001_00000_00011_0000000000011001; // lw 3 $global 25
856 HD[855] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
857 HD[856] = 32'b000000_00010_00000_00010_00000_010011; // set 2 2 $zero
858 HD[857] = 32'b000111_00010_00000_0000000000001000; // beq 2 $zero _IfExit29_
859 HD[858] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
860 HD[859] = 32'b000001_00000_00011_0000000000011101; // lw 3 $global 29
861 HD[860] = 32'b000101_00000_00100_0000000000000000; // li 4 0
862 HD[861] = 32'b100101_00010_00011_00100_00000_000000; // regtohd 4 2 3
863 HD[862] = 32'b000101_00000_10100_0000000000000001; // li $rv 1
864 HD[863] = 32'b010000_10110_10111_0000000000000000; // pop $ra
865 HD[864] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
866 HD[865] = 32'b000001_00000_00010_0000000000011001; // lw 2 $global 25 / _IfExit29_
867 HD[866] = 32'b000000_00010_10100_00000_00000_010000; // move $rv 2
868 HD[867] = 32'b010000_10110_10111_0000000000000000; // pop $ra
869 HD[868] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
870 HD[869] = 32'b010000_10110_10111_0000000000000000; // pop $ra
871 HD[870] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
872 HD[871] = 32'b001111_10110_10111_0000000000000000; // push $ra / renomear_programa
873 HD[872] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
874 HD[873] = 32'b000011_10101_10101_0000000000000011; // addi $fp $fp 3
875 HD[874] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
876 HD[875] = 32'b101000_00000000000000001001100110; // jal posicao_programa
877 HD[876] = 32'b000100_10101_10101_0000000000000011; // subi $fp $fp 3
878 HD[877] = 32'b000000_10100_00010_00000_00000_010000; // move 2 $rv
879 HD[878] = 32'b000001_00000_00011_0000000000011001; // lw 3 $global 25
880 HD[879] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
881 HD[880] = 32'b000111_00010_00000_0000000000011010; // beq 2 $zero _ElseBegin30_
882 HD[881] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
883 HD[882] = 32'b000011_10101_10101_0000000000000011; // addi $fp $fp 3
884 HD[883] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
885 HD[884] = 32'b101000_00000000000000001001100110; // jal posicao_programa
886 HD[885] = 32'b000100_10101_10101_0000000000000011; // subi $fp $fp 3
887 HD[886] = 32'b000000_10100_00010_00000_00000_010000; // move 2 $rv

```

```

888 HD[887] = 32'b000010_10101_00010_0000000000000010; // sw 2 $fp 2
889 HD[888] = 32'b000001_10101_00010_0000000000000010; // lw 2 $fp 2
890 HD[889] = 32'b000001_00000_00011_0000000000011001; // lw 3 $global 25
891 HD[890] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
892 HD[891] = 32'b000000_00010_00000_00010_00000_010011; // set 2 2 $zero
893 HD[892] = 32'b000111_00010_00000_0000000000001001; // beq 2 $zero _ElseBegin31_
894 HD[893] = 32'b000001_10101_00010_0000000000000010; // lw 2 $fp 2
895 HD[894] = 32'b000001_00000_00011_0000000000011110; // lw 3 $global 30
896 HD[895] = 32'b000001_10101_00100_0000000000000001; // lw 4 $fp 1
897 HD[896] = 32'b100101_00010_00011_00100_00000_000000; // regtohd 4 2 3
898 HD[897] = 32'b000101_00000_10100_0000000000000001; // li $rv 1
899 HD[898] = 32'b010000_10110_10111_0000000000000000; // pop $ra
900 HD[899] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
901 HD[900] = 32'b001011_0000000000000000111001001; // jump _IfExit31_
902 HD[901] = 32'b000001_00000_00010_0000000000011001; // lw 2 $global 25 / _ElseBegin31_
903 HD[902] = 32'b000000_00010_10100_00000_00000_010000; // move $rv 2
904 HD[903] = 32'b010000_10110_10111_0000000000000000; // pop $ra
905 HD[904] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
906 HD[905] = 32'b001011_0000000000000000111001110; // jump _IfExit30_ / _IfExit31_
907 HD[906] = 32'b000001_00000_00010_0000000000011011; // lw 2 $global 27 / _ElseBegin30_
908 HD[907] = 32'b000000_00010_10100_00000_00000_010000; // move $rv 2
909 HD[908] = 32'b010000_10110_10111_0000000000000000; // pop $ra
910 HD[909] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
911 HD[910] = 32'b010000_10110_10111_0000000000000000; // pop $ra / _IfExit30_
912 HD[911] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
913 HD[912] = 32'b001111_10110_10111_0000000000000000; // push $ra / trilha_possui_programa
914 HD[913] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
915 HD[914] = 32'b000001_00000_00011_0000000000010111; // lw 3 $global 23
916 HD[915] = 32'b000000_00010_00011_00010_00000_010001; // slt 2 2 3
917 HD[916] = 32'b000000_00010_00000_00010_00000_010011; // set 2 2 $zero
918 HD[917] = 32'b000111_00010_00000_0000000000001110; // beq 2 $zero _IfExit32_
919 HD[918] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
920 HD[919] = 32'b000001_00000_00011_0000000000011101; // lw 3 $global 29
921 HD[920] = 32'b100100_00010_00011_00010_00000_000000; // hdtoreg 2 2 3
922 HD[921] = 32'b000001_00000_00011_0000000000011000; // lw 3 $global 24
923 HD[922] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
924 HD[923] = 32'b000000_00010_00000_00010_00000_010011; // set 2 2 $zero
925 HD[924] = 32'b000111_00010_00000_0000000000000111; // beq 2 $zero _IfExit33_
926 HD[925] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
927 HD[926] = 32'b000001_00000_00011_0000000000011110; // lw 3 $global 30
928 HD[927] = 32'b100100_00010_00011_00010_00000_000000; // hdtoreg 2 2 3
929 HD[928] = 32'b000000_00010_10100_00000_00000_010000; // move $rv 2
930 HD[929] = 32'b010000_10110_10111_0000000000000000; // pop $ra
931 HD[930] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
932 HD[931] = 32'b000001_00000_00010_0000000000011001; // lw 2 $global 25 / _IfExit33_
    _IfExit32_
933 HD[932] = 32'b000000_00010_10100_00000_00000_010000; // move $rv 2
934 HD[933] = 32'b010000_10110_10111_0000000000000000; // pop $ra
935 HD[934] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
936 HD[935] = 32'b010000_10110_10111_0000000000000000; // pop $ra
937 HD[936] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
938 HD[937] = 32'b001111_10110_10111_0000000000000000; // push $ra / existe_programa_posicao
939 HD[938] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
940 HD[939] = 32'b000001_00000_00011_0000000000010111; // lw 3 $global 23
941 HD[940] = 32'b000000_00010_00011_00010_00000_000101; // add 2 2 3
942 HD[941] = 32'b000011_10101_10101_0000000000000001; // addi $fp $fp 1
943 HD[942] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
944 HD[943] = 32'b101000_00000000000000001110010000; // jal trilha_possui_programa
945 HD[944] = 32'b000100_10101_10101_0000000000000001; // subi $fp $fp 1
946 HD[945] = 32'b000000_10100_00010_00000_00000_010000; // move 2 $rv

```

```

947 HD[946] = 32'b000000_00010_10100_00000_00000_010000; // move $rv 2
948 HD[947] = 32'b010000_10110_10111_0000000000000000; // pop $ra
949 HD[948] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
950 HD[949] = 32'b010000_10110_10111_0000000000000000; // pop $ra
951 HD[950] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
952 HD[951] = 32'b001111_10110_10111_0000000000000000; // push $ra / executa_processos
953 HD[952] = 32'b000101_00000_00010_0000000000000001; // li 2 1
954 HD[953] = 32'b000010_00000_00010_00000000000000110; // sw 2 $global 6
955 HD[954] = 32'b000001_00000_00010_00000000000001000; // lw 2 $global 8 / _WhileBegin9_
956 HD[955] = 32'b000101_00000_00011_0000000000000000; // li 3 0
957 HD[956] = 32'b000000_00010_00011_00010_00000_010010; // sgt 2 2 3
958 HD[957] = 32'b000111_00010_00000_00000000001000110; // beq 2 $zero _WhileExit9_
959 HD[958] = 32'b000011_10101_10101_0000000000000001; // addi $fp $fp 1
960 HD[959] = 32'b101000_00000000000000000001000111000; // jal fila_proximo
961 HD[960] = 32'b000100_10101_10101_0000000000000001; // subi $fp $fp 1
962 HD[961] = 32'b000000_10100_00010_00000_00000_010000; // move 2 $rv
963 HD[962] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
964 HD[963] = 32'b000101_00000_00010_00000000000000101; // li 2 5
965 HD[964] = 32'b000011_10101_10101_0000000000000001; // addi $fp $fp 1
966 HD[965] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
967 HD[966] = 32'b101000_00000000000000000000010110011; // jal print_default_msg
968 HD[967] = 32'b000100_10101_10101_0000000000000001; // subi $fp $fp 1
969 HD[968] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
970 HD[969] = 32'b000101_00000_00011_0000000000000000; // li 3 0
971 HD[970] = 32'b000101_00000_00100_00000000000001110; // li 4 14
972 HD[971] = 32'b000011_10101_10101_0000000000000001; // addi $fp $fp 1
973 HD[972] = 32'b000010_10101_00100_00000000000000010; // sw 4 $fp 2
974 HD[973] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
975 HD[974] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
976 HD[975] = 32'b101000_0000000000000000000001100111; // jal print_value
977 HD[976] = 32'b000100_10101_10101_0000000000000001; // subi $fp $fp 1
978 HD[977] = 32'b000101_00000_00010_00000000000001000; // li 2 8
979 HD[978] = 32'b000011_10101_10101_0000000000000001; // addi $fp $fp 1
980 HD[979] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
981 HD[980] = 32'b101000_00000000000000000000010110011; // jal print_default_msg
982 HD[981] = 32'b000100_10101_10101_0000000000000001; // subi $fp $fp 1
983 HD[982] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
984 HD[983] = 32'b000011_10101_10101_0000000000000001; // addi $fp $fp 1
985 HD[984] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
986 HD[985] = 32'b101000_00000000000000000001011010000; // jal carrega_programa_memoria_inst
987 HD[986] = 32'b000100_10101_10101_0000000000000001; // subi $fp $fp 1
988 HD[987] = 32'b000001_00000_00010_00000000000000100; // lw 2 $global 4
989 HD[988] = 32'b000001_00000_00011_00000000000000101; // lw 3 $global 5
990 HD[989] = 32'b000000_00010_00011_00010_00000_000101; // add 2 2 3
991 HD[990] = 32'b000000_00010_11110_00000_00000_010000; // move 30 2
992 HD[991] = 32'b000001_00000_00010_00000000000000010; // lw 2 $global 2
993 HD[992] = 32'b000001_00000_00011_00000000000000011; // lw 3 $global 3
994 HD[993] = 32'b000000_00010_00011_00010_00000_000101; // add 2 2 3
995 HD[994] = 32'b000001_00000_00011_00000000000000101; // lw 3 $global 5
996 HD[995] = 32'b000000_00010_00011_00010_00000_000101; // add 2 2 3
997 HD[996] = 32'b000000_00010_11011_00000_00000_010000; // move 27 2
998 HD[997] = 32'b000001_00000_00010_0000000000000001; // lw 2 $global 1
999 HD[998] = 32'b000000_00010_00000_00000_00000_010100; // jr 2
1000 HD[999] = 32'b00000000000000000000000000000000; // noop / _label1_
1001 HD[1000] = 32'b000101_00000_00010_00000000000000101; // li 2 5
1002 HD[1001] = 32'b000011_10101_10101_0000000000000001; // addi $fp $fp 1
1003 HD[1002] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1004 HD[1003] = 32'b101000_00000000000000000000010110011; // jal print_default_msg
1005 HD[1004] = 32'b000100_10101_10101_0000000000000001; // subi $fp $fp 1
1006 HD[1005] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0

```

```

1007 HD[1006] = 32'b000101_00000_00011_0000000000000000; // li 3 0
1008 HD[1007] = 32'b000101_00000_00100_0000000000001110; // li 4 14
1009 HD[1008] = 32'b000011_10101_10101_0000000000000001; // addi $fp $fp 1
1010 HD[1009] = 32'b000010_10101_00100_0000000000000010; // sw 4 $fp 2
1011 HD[1010] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
1012 HD[1011] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1013 HD[1012] = 32'b101000_0000000000000000001100111; // jal print_value
1014 HD[1013] = 32'b000100_10101_10101_0000000000000001; // subi $fp $fp 1
1015 HD[1014] = 32'b000101_00000_00010_0000000000001001; // li 2 9
1016 HD[1015] = 32'b000011_10101_10101_0000000000000001; // addi $fp $fp 1
1017 HD[1016] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1018 HD[1017] = 32'b101000_00000000000000000010110011; // jal print_default_msg
1019 HD[1018] = 32'b000100_10101_10101_0000000000000001; // subi $fp $fp 1
1020 HD[1019] = 32'b000101_00000_00010_0000000000000011; // li 2 3
1021 HD[1020] = 32'b000101_00000_00011_0000000000000000; // li 3 0
1022 HD[1021] = 32'b000011_10101_10101_0000000000000001; // addi $fp $fp 1
1023 HD[1022] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
1024 HD[1023] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1025 HD[1024] = 32'b101000_000000000000000000100001; // jal sleep
1026 HD[1025] = 32'b000100_10101_10101_0000000000000001; // subi $fp $fp 1
1027 HD[1026] = 32'b001011_00000000000000001110111010; // jump _WhileBegin9_
1028 HD[1027] = 32'b010000_10110_10111_0000000000000000; // pop $ra / _WhileExit9_
1029 HD[1028] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
1030 HD[1029] = 32'b001111_10110_10111_0000000000000000; // push $ra /
    executa_processos_preemptivo
1031 HD[1030] = 32'b000101_00000_00010_0000000000000000; // li 2 0
1032 HD[1031] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1033 HD[1032] = 32'b000010_10101_00010_00000000000000100; // sw 2 $fp 4
1034 HD[1033] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0 / _WhileBegin10_
1035 HD[1034] = 32'b000001_00000_00011_00000000000000111; // lw 3 $global 7
1036 HD[1035] = 32'b000000_00010_00011_00010_00000_010001; // slt 2 2 3
1037 HD[1036] = 32'b000111_00010_00000_0000000000001111; // beq 2 $zero _WhileExit10_
1038 HD[1037] = 32'b000101_00000_00010_0000000000000000; // li 2 0
1039 HD[1038] = 32'b000001_10101_00011_0000000000000000; // lw 3 $fp 0
1040 HD[1039] = 32'b000000_00011_10101_00011_00000_000101; // add 3 3 $fp
1041 HD[1040] = 32'b000010_00011_00010_00000000000000101; // sw 2 3 5
1042 HD[1041] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
1043 HD[1042] = 32'b000000_00010_00000_00010_00000_000101; // add 2 2 $global
1044 HD[1043] = 32'b000001_00010_00010_0000000000001001; // lw 2 2 9
1045 HD[1044] = 32'b000001_10101_00011_0000000000000000; // lw 3 $fp 0
1046 HD[1045] = 32'b000000_00011_10101_00011_00000_000101; // add 3 3 $fp
1047 HD[1046] = 32'b000010_00011_00010_0000000000001111; // sw 2 3 15
1048 HD[1047] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
1049 HD[1048] = 32'b000011_00010_00010_0000000000000001; // addi 2 2 1
1050 HD[1049] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1051 HD[1050] = 32'b001011_00000000000000001000001001; // jump _WhileBegin10_
1052 HD[1051] = 32'b000101_00000_00010_0000000000000010; // li 2 2 / _WhileExit10_
1053 HD[1052] = 32'b000010_00000_00010_00000000000000110; // sw 2 $global 6
1054 HD[1053] = 32'b000001_00000_00010_0000000000001000; // lw 2 $global 8 / _WhileBegin11_
1055 HD[1054] = 32'b000101_00000_00011_0000000000000000; // li 3 0
1056 HD[1055] = 32'b000000_00010_00011_00010_00000_010010; // sgt 2 2 3
1057 HD[1056] = 32'b000111_00010_00000_0000000011010011; // beq 2 $zero _WhileExit11_
1058 HD[1057] = 32'b000101_00000_00010_0000000000000000; // li 2 0
1059 HD[1058] = 32'b000010_00000_00010_0000000000001001; // sw 2 $global 19
1060 HD[1059] = 32'b000001_00000_00010_0000000000001001; // lw 2 $global 9
1061 HD[1060] = 32'b000010_10101_00010_0000000000000001; // sw 2 $fp 1
1062 HD[1061] = 32'b100110_00000000000000000000000000; // lcdclean
1063 HD[1062] = 32'b000001_10101_00010_000000000000100; // lw 2 $fp 4
1064 HD[1063] = 32'b000101_00000_00011_0000000000000001; // li 3 1
1065 HD[1064] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3

```

```

1066 HD[1065] = 32'b000111_00010_00000_00000000000111110; // beq 2 $zero _IfExit34_
1067 HD[1066] = 32'b000001_00000_00010_0000000000001000; // lw 2 $global 8
1068 HD[1067] = 32'b000101_00000_00011_0000000000000001; // li 3 1
1069 HD[1068] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
1070 HD[1069] = 32'b000000_00010_00000_00010_00000_010011; // set 2 2 $zero
1071 HD[1070] = 32'b000111_00010_00000_00000000000111001; // beq 2 $zero _IfExit35_
1072 HD[1071] = 32'b100111_00000_00000_00000_00001010010; // lcdwrite 0 82
1073 HD[1072] = 32'b100111_00001_00000_00000_00001100101; // lcdwrite 1 101
1074 HD[1073] = 32'b100111_00010_00000_00000_00001100001; // lcdwrite 2 97
1075 HD[1074] = 32'b100111_00011_00000_00000_00001101100; // lcdwrite 3 108
1076 HD[1075] = 32'b100111_00100_00000_00000_00001101001; // lcdwrite 4 105
1077 HD[1076] = 32'b100111_00101_00000_00000_00001111010; // lcdwrite 5 122
1078 HD[1077] = 32'b100111_00110_00000_00000_00001100001; // lcdwrite 6 97
1079 HD[1078] = 32'b100111_00111_00000_00000_00001101110; // lcdwrite 7 110
1080 HD[1079] = 32'b100111_01000_00000_00000_00001100100; // lcdwrite 8 100
1081 HD[1080] = 32'b100111_01001_00000_00000_00001101111; // lcdwrite 9 111
1082 HD[1081] = 32'b100111_01011_00000_00000_00001110100; // lcdwrite 11 116
1083 HD[1082] = 32'b100111_01100_00000_00000_00001110010; // lcdwrite 12 114
1084 HD[1083] = 32'b100111_01101_00000_00000_00001101111; // lcdwrite 13 111
1085 HD[1084] = 32'b100111_01110_00000_00000_00001100011; // lcdwrite 14 99
1086 HD[1085] = 32'b100111_01111_00000_00000_00001100001; // lcdwrite 15 97
1087 HD[1086] = 32'b100111_10000_00000_00000_00001100100; // lcdwrite 16 100
1088 HD[1087] = 32'b100111_10001_00000_00000_00001100101; // lcdwrite 17 101
1089 HD[1088] = 32'b100111_10011_00000_00000_00001000011; // lcdwrite 19 67
1090 HD[1089] = 32'b100111_10100_00000_00000_00001101111; // lcdwrite 20 111
1091 HD[1090] = 32'b100111_10101_00000_00000_00001101110; // lcdwrite 21 110
1092 HD[1091] = 32'b100111_10110_00000_00000_00001110100; // lcdwrite 22 116
1093 HD[1092] = 32'b100111_10111_00000_00000_00001100101; // lcdwrite 23 101
1094 HD[1093] = 32'b100111_11000_00000_00000_00001111000; // lcdwrite 24 120
1095 HD[1094] = 32'b100111_11001_00000_00000_00001110100; // lcdwrite 25 116
1096 HD[1095] = 32'b100111_11010_00000_00000_00001101111; // lcdwrite 26 111
1097 HD[1096] = 32'b000101_00000_00010_0000000000000001; // li 2 1
1098 HD[1097] = 32'b000101_00000_00011_0000000000000000; // li 3 0
1099 HD[1098] = 32'b000011_10101_10101_0000000000011001; // addi $fp $fp 25
1100 HD[1099] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
1101 HD[1100] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1102 HD[1101] = 32'b101000_0000000000000000000100001; // jal sleep
1103 HD[1102] = 32'b000100_10101_10101_0000000000011001; // subi $fp $fp 25
1104 HD[1103] = 32'b100111_11100_00000_00000_0000101110; // lcdwrite 28 46
1105 HD[1104] = 32'b000101_00000_00010_0000000000000001; // li 2 1
1106 HD[1105] = 32'b000101_00000_00011_0000000000000000; // li 3 0
1107 HD[1106] = 32'b000011_10101_10101_0000000000011001; // addi $fp $fp 25
1108 HD[1107] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
1109 HD[1108] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1110 HD[1109] = 32'b101000_0000000000000000000100001; // jal sleep
1111 HD[1110] = 32'b000100_10101_10101_0000000000011001; // subi $fp $fp 25
1112 HD[1111] = 32'b100111_11101_00000_00000_0000101110; // lcdwrite 29 46
1113 HD[1112] = 32'b000101_00000_00010_0000000000000001; // li 2 1
1114 HD[1113] = 32'b000101_00000_00011_0000000000000000; // li 3 0
1115 HD[1114] = 32'b000011_10101_10101_0000000000011001; // addi $fp $fp 25
1116 HD[1115] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
1117 HD[1116] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1118 HD[1117] = 32'b101000_0000000000000000000100001; // jal sleep
1119 HD[1118] = 32'b000100_10101_10101_0000000000011001; // subi $fp $fp 25
1120 HD[1119] = 32'b100111_11110_00000_00000_0000101110; // lcdwrite 30 46
1121 HD[1120] = 32'b000101_00000_00010_0000000000000001; // li 2 1
1122 HD[1121] = 32'b000101_00000_00011_0000000000000000; // li 3 0
1123 HD[1122] = 32'b000011_10101_10101_0000000000011001; // addi $fp $fp 25
1124 HD[1123] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
1125 HD[1124] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0

```



```

1126 HD[1125] = 32'b101000_000000000000000000000000000001; // jal sleep
1127 HD[1126] = 32'b000100_10101_10101_00000000000011001; // subi $fp $fp 25
1128 HD[1127] = 32'b000001_10101_00010_000000000000000001; // lw 2 $fp 1 / _IfExit35_ _IfExit34_
1129 HD[1128] = 32'b000011_10101_10101_00000000000011001; // addi $fp $fp 25
1130 HD[1129] = 32'b000010_10101_00010_000000000000000000; // sw 2 $fp 0
1131 HD[1130] = 32'b101000_000000000000000000000001011010000; // jal carrega_programa_memoria_inst
1132 HD[1131] = 32'b000100_10101_10101_00000000000011001; // subi $fp $fp 25
1133 HD[1132] = 32'b000011_10101_00010_00000000000001111; // addi 2 $fp 15
1134 HD[1133] = 32'b000001_00000_00011_0000000000000111; // lw 3 $global 7
1135 HD[1134] = 32'b000001_10101_00100_000000000000000001; // lw 4 $fp 1
1136 HD[1135] = 32'b000011_10101_10101_00000000000011001; // addi $fp $fp 25
1137 HD[1136] = 32'b000010_10101_00100_000000000000000010; // sw 4 $fp 2
1138 HD[1137] = 32'b000010_10101_00011_000000000000000001; // sw 3 $fp 1
1139 HD[1138] = 32'b000010_10101_00010_000000000000000000; // sw 2 $fp 0
1140 HD[1139] = 32'b101000_000000000000000000000000111101; // jal vetor_existe_valor
1141 HD[1140] = 32'b000100_10101_10101_00000000000011001; // subi $fp $fp 25
1142 HD[1141] = 32'b000000_10100_00010_00000_00000_010000; // move 2 $rv
1143 HD[1142] = 32'b000011_00010_00010_000000000000000001; // addi 2 2 1
1144 HD[1143] = 32'b000010_10101_00010_000000000000000011; // sw 2 $fp 3
1145 HD[1144] = 32'b000001_10101_00010_000000000000000011; // lw 2 $fp 3
1146 HD[1145] = 32'b000100_00010_00010_000000000000000001; // subi 2 2 1
1147 HD[1146] = 32'b000000_00010_10101_00010_00000_000101; // add 2 2 $fp
1148 HD[1147] = 32'b000001_00010_00010_0000000000000000101; // lw 2 2 5
1149 HD[1148] = 32'b000101_00000_00011_000000000000000000; // li 3 0
1150 HD[1149] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
1151 HD[1150] = 32'b000111_00010_00000_00000000000010111; // beq 2 $zero _ElseBegin36_
1152 HD[1151] = 32'b000001_10101_00010_000000000000000011; // lw 2 $fp 3
1153 HD[1152] = 32'b000001_00000_00011_0000000000000000100; // lw 3 $global 4
1154 HD[1153] = 32'b000000_00010_00011_00010_00000_000111; // mult 2 2 3
1155 HD[1154] = 32'b000001_00000_00011_0000000000000000101; // lw 3 $global 5
1156 HD[1155] = 32'b000000_00010_00011_00010_00000_000101; // add 2 2 3
1157 HD[1156] = 32'b000000_00010_11110_00000_00000_010000; // move 30 2
1158 HD[1157] = 32'b000001_10101_00010_000000000000000011; // lw 2 $fp 3
1159 HD[1158] = 32'b000001_00000_00011_000000000000000010; // lw 3 $global 2
1160 HD[1159] = 32'b000000_00010_00011_00010_00000_000111; // mult 2 2 3
1161 HD[1160] = 32'b000001_00000_00011_000000000000000011; // lw 3 $global 3
1162 HD[1161] = 32'b000000_00010_00011_00010_00000_000101; // add 2 2 3
1163 HD[1162] = 32'b000001_00000_00011_0000000000000000101; // lw 3 $global 5
1164 HD[1163] = 32'b000000_00010_00011_00010_00000_000101; // add 2 2 3
1165 HD[1164] = 32'b000000_00010_11011_00000_00000_010000; // move 27 2
1166 HD[1165] = 32'b000001_00000_00010_000000000000000001; // lw 2 $global 1
1167 HD[1166] = 32'b000010_10101_00010_000000000000000010; // sw 2 $fp 2
1168 HD[1167] = 32'b000101_00000_00010_000000000000000001; // li 2 1
1169 HD[1168] = 32'b000001_10101_00011_000000000000000011; // lw 3 $fp 3
1170 HD[1169] = 32'b000100_00011_00011_000000000000000001; // subi 3 3 1
1171 HD[1170] = 32'b000000_00011_10101_00011_00000_000101; // add 3 3 $fp
1172 HD[1171] = 32'b000010_00011_00010_0000000000000000101; // sw 2 3 5
1173 HD[1172] = 32'b001011_0000000000000000000000010010011100; // jump _IfExit36_
1174 HD[1173] = 32'b000001_10101_00010_000000000000000011; // lw 2 $fp 3 / _ElseBegin36_
1175 HD[1174] = 32'b000011_10101_10101_00000000000011001; // addi $fp $fp 25
1176 HD[1175] = 32'b000010_10101_00010_000000000000000000; // sw 2 $fp 0
1177 HD[1176] = 32'b101000_0000000000000000000000000101110010; // jal load_registradores
1178 HD[1177] = 32'b000100_10101_10101_00000000000011001; // subi $fp $fp 25
1179 HD[1178] = 32'b000000_11010_00010_00000_00000_010000; // move 2 26
1180 HD[1179] = 32'b000010_10101_00010_000000000000000010; // sw 2 $fp 2
1181 HD[1180] = 32'b000101_00000_00010_0000000000000000101; // li 2 5 / _IfExit36_
1182 HD[1181] = 32'b000011_10101_10101_00000000000011001; // addi $fp $fp 25
1183 HD[1182] = 32'b000010_10101_00010_000000000000000000; // sw 2 $fp 0
1184 HD[1183] = 32'b101000_000000000000000000000000010110011; // jal print_default_msg
1185 HD[1184] = 32'b000100_10101_10101_00000000000011001; // subi $fp $fp 25

```

```

1186 HD[1185] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
1187 HD[1186] = 32'b000101_00000_00011_0000000000000000; // li 3 0
1188 HD[1187] = 32'b000101_00000_00100_0000000000001110; // li 4 14
1189 HD[1188] = 32'b000011_10101_10101_00000000000011001; // addi $fp $fp 25
1190 HD[1189] = 32'b000010_10101_00100_0000000000000010; // sw 4 $fp 2
1191 HD[1190] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
1192 HD[1191] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1193 HD[1192] = 32'b101000_0000000000000000001100111; // jal print_value
1194 HD[1193] = 32'b000100_10101_10101_00000000000011001; // subi $fp $fp 25
1195 HD[1194] = 32'b000101_00000_00010_0000000000001000; // li 2 8
1196 HD[1195] = 32'b000011_10101_10101_00000000000011001; // addi $fp $fp 25
1197 HD[1196] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1198 HD[1197] = 32'b101000_00000000000000000010110011; // jal print_default_msg
1199 HD[1198] = 32'b000100_10101_10101_00000000000011001; // subi $fp $fp 25
1200 HD[1199] = 32'b101010_00000000000000000000000000; // intr_on
1201 HD[1200] = 32'b000001_10101_00010_0000000000000010; // lw 2 $fp 2
1202 HD[1201] = 32'b000000_00010_00000_00000_00000_010100; // jr 2
1203 HD[1202] = 32'b00000000000000000000000000000000; // noop / _label2_
1204 HD[1203] = 32'b000001_00000_00010_00000000000010011; // lw 2 $global 19
1205 HD[1204] = 32'b000101_00000_00011_0000000000000001; // li 3 1
1206 HD[1205] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
1207 HD[1206] = 32'b000111_00010_00000_00000000000010110; // beq 2 $zero _ElseBegin37_
1208 HD[1207] = 32'b000101_00000_00010_0000000000000001; // li 2 1
1209 HD[1208] = 32'b000011_10101_10101_00000000000011001; // addi $fp $fp 25
1210 HD[1209] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1211 HD[1210] = 32'b101000_000000000000000000111001000; // jal organiza_fila_pronto
1212 HD[1211] = 32'b000100_10101_10101_00000000000011001; // subi $fp $fp 25
1213 HD[1212] = 32'b000001_00000_00010_0000000000001000; // lw 2 $global 8
1214 HD[1213] = 32'b000100_00010_00010_0000000000000001; // subi 2 2 1
1215 HD[1214] = 32'b000010_00000_00010_0000000000001000; // sw 2 $global 8
1216 HD[1215] = 32'b000101_00000_00010_0000000000001001; // li 2 9
1217 HD[1216] = 32'b000011_10101_10101_00000000000011001; // addi $fp $fp 25
1218 HD[1217] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1219 HD[1218] = 32'b101000_00000000000000000010110011; // jal print_default_msg
1220 HD[1219] = 32'b000100_10101_10101_00000000000011001; // subi $fp $fp 25
1221 HD[1220] = 32'b000101_00000_00010_0000000000000010; // li 2 2
1222 HD[1221] = 32'b000101_00000_00011_0000000000000000; // li 3 0
1223 HD[1222] = 32'b000011_10101_10101_00000000000011001; // addi $fp $fp 25
1224 HD[1223] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
1225 HD[1224] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1226 HD[1225] = 32'b101000_0000000000000000000100001; // jal sleep
1227 HD[1226] = 32'b000100_10101_10101_00000000000011001; // subi $fp $fp 25
1228 HD[1227] = 32'b000101_0000000000000000001001110000; // jump _IfExit37_
1229 HD[1228] = 32'b000001_10101_00010_00000000000000011; // lw 2 $fp 3 / _ElseBegin37_
1230 HD[1229] = 32'b000011_10101_10101_00000000000011001; // addi $fp $fp 25
1231 HD[1230] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1232 HD[1231] = 32'b101000_00000000000000000011001101; // jal store_registradores
1233 HD[1232] = 32'b000100_10101_10101_00000000000011001; // subi $fp $fp 25
1234 HD[1233] = 32'b000011_10101_10101_00000000000011001; // addi $fp $fp 25
1235 HD[1234] = 32'b101000_0000000000000000001001010010; // jal fila_passa_vez
1236 HD[1235] = 32'b000100_10101_10101_00000000000011001; // subi $fp $fp 25
1237 HD[1236] = 32'b000001_00000_00010_0000000000001000; // lw 2 $global 8
1238 HD[1237] = 32'b000101_00000_00011_0000000000000001; // li 3 1
1239 HD[1238] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
1240 HD[1239] = 32'b000000_00010_00000_00010_00000_010011; // set 2 2 $zero
1241 HD[1240] = 32'b000111_00010_00000_00000000000011000; // beq 2 $zero _IfExit38_
1242 HD[1241] = 32'b100111_10000_00000_00000_00000100000; // lcdwrite 16 32
1243 HD[1242] = 32'b100111_10001_00000_00000_00001100110; // lcdwrite 17 102
1244 HD[1243] = 32'b100111_10010_00000_00000_00001101111; // lcdwrite 18 111
1245 HD[1244] = 32'b100111_10011_00000_00000_00001101001; // lcdwrite 19 105

```



```

1246 HD[1245] = 32'b100111_10100_00000_00000_00000100000; // lcdwrite 20 32
1247 HD[1246] = 32'b100111_10101_00000_00000_00001110000; // lcdwrite 21 112
1248 HD[1247] = 32'b100111_10110_00000_00000_00001110010; // lcdwrite 22 114
1249 HD[1248] = 32'b100111_10111_00000_00000_00001100101; // lcdwrite 23 101
1250 HD[1249] = 32'b100111_11000_00000_00000_00001100101; // lcdwrite 24 101
1251 HD[1250] = 32'b100111_11001_00000_00000_00001101101; // lcdwrite 25 109
1252 HD[1251] = 32'b100111_11010_00000_00000_00001110000; // lcdwrite 26 112
1253 HD[1252] = 32'b100111_11011_00000_00000_00001110100; // lcdwrite 27 116
1254 HD[1253] = 32'b100111_11100_00000_00000_00001100001; // lcdwrite 28 97
1255 HD[1254] = 32'b100111_11101_00000_00000_00001100100; // lcdwrite 29 100
1256 HD[1255] = 32'b100111_11110_00000_00000_00001101111; // lcdwrite 30 111
1257 HD[1256] = 32'b100111_11111_00000_00000_00000100000; // lcdwrite 31 32
1258 HD[1257] = 32'b000101_00000_00010_00000000000000010; // li 2 2
1259 HD[1258] = 32'b000101_00000_00011_00000000000000000; // li 3 0
1260 HD[1259] = 32'b000011_10101_10101_00000000000011001; // addi $fp $fp 25
1261 HD[1260] = 32'b000010_10101_00011_00000000000000001; // sw 3 $fp 1
1262 HD[1261] = 32'b000010_10101_00010_00000000000000000; // sw 2 $fp 0
1263 HD[1262] = 32'b101000_00000000000000000000000010001; // jal sleep
1264 HD[1263] = 32'b000100_10101_10101_00000000000011001; // subi $fp $fp 25
1265 HD[1264] = 32'b000101_00000_00010_00000000000000001; // li 2 1 / _IfExit38_ _IfExit37_
1266 HD[1265] = 32'b000010_10101_00010_000000000000000100; // sw 2 $fp 4
1267 HD[1266] = 32'b001011_00000000000000001000011101; // jump _WhileBegin11_
1268 HD[1267] = 32'b010000_10110_10111_00000000000000000; // pop $ra / _WhileExit11_
1269 HD[1268] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
1270 HD[1269] = 32'b001111_10110_10111_00000000000000000; // push $ra / seleciona_programas
1271 HD[1270] = 32'b000101_00000_00010_00000000000000000; // li 2 0
1272 HD[1271] = 32'b000010_10101_00010_00000000000000000; // sw 2 $fp 2
1273 HD[1272] = 32'b000001_00000_00010_00000000000001000; // lw 2 $global 8
1274 HD[1273] = 32'b000101_00000_00011_00000000000000000; // li 3 0
1275 HD[1274] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
1276 HD[1275] = 32'b000111_00010_00000_00000000100000000; // beq 2 $zero _IfExit39_
1277 HD[1276] = 32'b000101_00000_00010_00000000000000001; // li 2 1
1278 HD[1277] = 32'b000010_10101_00010_00000000000000000; // sw 2 $fp 0
1279 HD[1278] = 32'b000001_10101_00010_00000000000000000; // lw 2 $fp 0 / _WhileBegin12_
1280 HD[1279] = 32'b000101_00000_00011_00000000000000001; // li 3 1
1281 HD[1280] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
1282 HD[1281] = 32'b000111_00010_00000_0000000001101000; // beq 2 $zero _WhileExit12_
1283 HD[1282] = 32'b100111_00000_00000_00000_00001010011; // lcdwrite 0 83
1284 HD[1283] = 32'b100111_00001_00000_00000_00001100101; // lcdwrite 1 101
1285 HD[1284] = 32'b100111_00010_00000_00000_00001101100; // lcdwrite 2 108
1286 HD[1285] = 32'b100111_00011_00000_00000_00001100101; // lcdwrite 3 101
1287 HD[1286] = 32'b100111_00100_00000_00000_00001100011; // lcdwrite 4 99
1288 HD[1287] = 32'b100111_00101_00000_00000_00001101001; // lcdwrite 5 105
1289 HD[1288] = 32'b100111_00110_00000_00000_00001101111; // lcdwrite 6 111
1290 HD[1289] = 32'b100111_00111_00000_00000_00001101110; // lcdwrite 7 110
1291 HD[1290] = 32'b100111_01000_00000_00000_00001100101; // lcdwrite 8 101
1292 HD[1291] = 32'b100111_01001_00000_00000_00000100000; // lcdwrite 9 32
1293 HD[1292] = 32'b100111_01010_00000_00000_00001100001; // lcdwrite 10 97
1294 HD[1293] = 32'b100111_01011_00000_00000_00000100000; // lcdwrite 11 32
1295 HD[1294] = 32'b100111_01100_00000_00000_00001110001; // lcdwrite 12 113
1296 HD[1295] = 32'b100111_01101_00000_00000_00001110100; // lcdwrite 13 116
1297 HD[1296] = 32'b100111_01110_00000_00000_00001100100; // lcdwrite 14 100
1298 HD[1297] = 32'b100111_01111_00000_00000_00000101110; // lcdwrite 15 46
1299 HD[1298] = 32'b100111_10000_00000_00000_00000100000; // lcdwrite 16 32
1300 HD[1299] = 32'b100111_10001_00000_00000_00000100000; // lcdwrite 17 32
1301 HD[1300] = 32'b100111_10010_00000_00000_00001100100; // lcdwrite 18 100
1302 HD[1301] = 32'b100111_10011_00000_00000_00001100101; // lcdwrite 19 101
1303 HD[1302] = 32'b100111_10100_00000_00000_00000100000; // lcdwrite 20 32
1304 HD[1303] = 32'b100111_10101_00000_00000_00001110000; // lcdwrite 21 112
1305 HD[1304] = 32'b100111_10110_00000_00000_00001110010; // lcdwrite 22 114

```

```

1306 HD[1305] = 32'b100111_10111_00000_00000_00001101111; // lcdwrite 23 111
1307 HD[1306] = 32'b100111_11000_00000_00000_00001100111; // lcdwrite 24 103
1308 HD[1307] = 32'b100111_11001_00000_00000_00001110010; // lcdwrite 25 114
1309 HD[1308] = 32'b100111_11010_00000_00000_00001100001; // lcdwrite 26 97
1310 HD[1309] = 32'b100111_11011_00000_00000_00001101101; // lcdwrite 27 109
1311 HD[1310] = 32'b100111_11100_00000_00000_00001100001; // lcdwrite 28 97
1312 HD[1311] = 32'b100111_11101_00000_00000_00001110011; // lcdwrite 29 115
1313 HD[1312] = 32'b100111_11110_00000_00000_00000100000; // lcdwrite 30 32
1314 HD[1313] = 32'b100111_11111_00000_00000_00000100000; // lcdwrite 31 32
1315 HD[1314] = 32'b001001_00000_00010_0000000000000000; // in 2
1316 HD[1315] = 32'b000010_10101_00010_0000000000000001; // sw 2 $fp 1
1317 HD[1316] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
1318 HD[1317] = 32'b000101_00000_00011_0000000000000000; // li 3 0
1319 HD[1318] = 32'b000000_00010_00011_00010_00000_010001; // slt 2 2 3
1320 HD[1319] = 32'b000000_00010_00000_00010_00000_010011; // set 2 2 $zero
1321 HD[1320] = 32'b000111_00010_00000_00000000000011100; // beq 2 $zero _ElseBegin40_
1322 HD[1321] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
1323 HD[1322] = 32'b000101_00000_00011_00000000000001010; // li 3 10
1324 HD[1323] = 32'b000000_00010_00011_00010_00000_010010; // sgt 2 2 3
1325 HD[1324] = 32'b000000_00010_00000_00010_00000_010011; // set 2 2 $zero
1326 HD[1325] = 32'b000111_00010_00000_00000000000000100; // beq 2 $zero _ElseBegin41_
1327 HD[1326] = 32'b000101_00000_00010_0000000000000000; // li 2 0
1328 HD[1327] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1329 HD[1328] = 32'b001011_000000000000000010101000011; // jump _IfExit41_
1330 HD[1329] = 32'b000101_00000_00010_0000000000000001; // li 2 1 / _ElseBegin41_
1331 HD[1330] = 32'b000010_10101_00010_00000000000000010; // sw 2 $fp 2
1332 HD[1331] = 32'b100111_10000_00000_00000_00000100000; // lcdwrite 16 32
1333 HD[1332] = 32'b100111_10001_00000_00000_00001110000; // lcdwrite 17 112
1334 HD[1333] = 32'b100111_10010_00000_00000_00001110010; // lcdwrite 18 114
1335 HD[1334] = 32'b100111_10011_00000_00000_00001101111; // lcdwrite 19 111
1336 HD[1335] = 32'b100111_10100_00000_00000_00001100111; // lcdwrite 20 103
1337 HD[1336] = 32'b100111_10101_00000_00000_00001110010; // lcdwrite 21 114
1338 HD[1337] = 32'b100111_10110_00000_00000_00001100001; // lcdwrite 22 97
1339 HD[1338] = 32'b100111_10111_00000_00000_00001101101; // lcdwrite 23 109
1340 HD[1339] = 32'b100111_11000_00000_00000_00001100001; // lcdwrite 24 97
1341 HD[1340] = 32'b100111_11001_00000_00000_00001110011; // lcdwrite 25 115
1342 HD[1341] = 32'b100111_11010_00000_00000_00000100000; // lcdwrite 26 32
1343 HD[1342] = 32'b100111_11011_00000_00000_00000111110; // lcdwrite 27 62
1344 HD[1343] = 32'b100111_11100_00000_00000_00000100000; // lcdwrite 28 32
1345 HD[1344] = 32'b100111_11101_00000_00000_00000110001; // lcdwrite 29 49
1346 HD[1345] = 32'b100111_11110_00000_00000_00000110000; // lcdwrite 30 48
1347 HD[1346] = 32'b100111_11111_00000_00000_00000100000; // lcdwrite 31 32
1348 HD[1347] = 32'b001011_00000000000000001010101010; // jump _IfExit40_ / _IfExit41_
1349 HD[1348] = 32'b000101_00000_00010_0000000000000001; // li 2 1 / _ElseBegin40_
1350 HD[1349] = 32'b000010_10101_00010_00000000000000010; // sw 2 $fp 2
1351 HD[1350] = 32'b100111_10000_00000_00000_00000100000; // lcdwrite 16 32
1352 HD[1351] = 32'b100111_10001_00000_00000_00001110000; // lcdwrite 17 112
1353 HD[1352] = 32'b100111_10010_00000_00000_00001110010; // lcdwrite 18 114
1354 HD[1353] = 32'b100111_10011_00000_00000_00001101111; // lcdwrite 19 111
1355 HD[1354] = 32'b100111_10100_00000_00000_00001100111; // lcdwrite 20 103
1356 HD[1355] = 32'b100111_10101_00000_00000_00001110010; // lcdwrite 21 114
1357 HD[1356] = 32'b100111_10110_00000_00000_00001100001; // lcdwrite 22 97
1358 HD[1357] = 32'b100111_10111_00000_00000_00001101101; // lcdwrite 23 109
1359 HD[1358] = 32'b100111_11000_00000_00000_00001100001; // lcdwrite 24 97
1360 HD[1359] = 32'b100111_11001_00000_00000_00001110011; // lcdwrite 25 115
1361 HD[1360] = 32'b100111_11010_00000_00000_00000100000; // lcdwrite 26 32
1362 HD[1361] = 32'b100111_11011_00000_00000_00000111100; // lcdwrite 27 60
1363 HD[1362] = 32'b100111_11100_00000_00000_00000100000; // lcdwrite 28 32
1364 HD[1363] = 32'b100111_11101_00000_00000_00000110000; // lcdwrite 29 48
1365 HD[1364] = 32'b100111_11110_00000_00000_00000100000; // lcdwrite 30 32

```

```

1366 HD[1365] = 32'b100111_11111_00000_00000_00000100000; // lcdwrite 31 32
1367 HD[1366] = 32'b000001_10101_00010_0000000000000010; // lw 2 $fp 2 / _IfExit40_
1368 HD[1367] = 32'b000101_00000_00011_0000000000000001; // li 3 1
1369 HD[1368] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
1370 HD[1369] = 32'b000111_00010_00000_0000000000000111; // beq 2 $zero _IfExit42_
1371 HD[1370] = 32'b000101_00000_00010_0000000000000001; // li 2 1
1372 HD[1371] = 32'b000011_10101_10101_0000000000000100; // addi $fp $fp 4
1373 HD[1372] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1374 HD[1373] = 32'b101000_0000000000000000010110011; // jal print_default_msg
1375 HD[1374] = 32'b000100_10101_10101_0000000000000100; // subi $fp $fp 4
1376 HD[1375] = 32'b000101_00000_00010_0000000000000010; // li 2 2
1377 HD[1376] = 32'b000101_00000_00011_0000000000000000; // li 3 0
1378 HD[1377] = 32'b000011_10101_10101_0000000000000100; // addi $fp $fp 4
1379 HD[1378] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
1380 HD[1379] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1381 HD[1380] = 32'b101000_0000000000000000000100001; // jal sleep
1382 HD[1381] = 32'b000100_10101_10101_0000000000000100; // subi $fp $fp 4
1383 HD[1382] = 32'b000101_00000_00010_0000000000000000; // li 2 0
1384 HD[1383] = 32'b000010_10101_00010_0000000000000010; // sw 2 $fp 2
1385 HD[1384] = 32'b001011_0000000000000001001111110; // jump _WhileBegin12_ / _IfExit42_
1386 HD[1385] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0 / _WhileExit12_
    _WhileBegin13_
1387 HD[1386] = 32'b000001_10101_00011_0000000000000001; // lw 3 $fp 1
1388 HD[1387] = 32'b000000_00010_00011_00010_00000_010001; // slt 2 2 3
1389 HD[1388] = 32'b000111_00010_00000_0000000010001111; // beq 2 $zero _WhileExit13_
1390 HD[1389] = 32'b000101_00000_00010_0000000000000010; // li 2 2
1391 HD[1390] = 32'b000011_10101_10101_0000000000000100; // addi $fp $fp 4
1392 HD[1391] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1393 HD[1392] = 32'b101000_000000000000000000010110011; // jal print_default_msg
1394 HD[1393] = 32'b000100_10101_10101_0000000000000100; // subi $fp $fp 4
1395 HD[1394] = 32'b100111_10000_00000_00000_00001110000; // lcdwrite 16 112
1396 HD[1395] = 32'b100111_10001_00000_00000_00001100001; // lcdwrite 17 97
1397 HD[1396] = 32'b100111_10010_00000_00000_00001110010; // lcdwrite 18 114
1398 HD[1397] = 32'b100111_10011_00000_00000_00001100001; // lcdwrite 19 97
1399 HD[1398] = 32'b100111_10100_00000_00000_00000100000; // lcdwrite 20 32
1400 HD[1399] = 32'b100111_10101_00000_00000_00001110011; // lcdwrite 21 115
1401 HD[1400] = 32'b100111_10110_00000_00000_00001100101; // lcdwrite 22 101
1402 HD[1401] = 32'b100111_10111_00000_00000_00001110010; // lcdwrite 23 114
1403 HD[1402] = 32'b100111_11000_00000_00000_00000100000; // lcdwrite 24 32
1404 HD[1403] = 32'b100111_11001_00000_00000_00001100001; // lcdwrite 25 97
1405 HD[1404] = 32'b100111_11010_00000_00000_00001101100; // lcdwrite 26 108
1406 HD[1405] = 32'b100111_11011_00000_00000_00001101111; // lcdwrite 27 111
1407 HD[1406] = 32'b100111_11100_00000_00000_00001100011; // lcdwrite 28 99
1408 HD[1407] = 32'b100111_11101_00000_00000_00001100001; // lcdwrite 29 97
1409 HD[1408] = 32'b100111_11110_00000_00000_00001100100; // lcdwrite 30 100
1410 HD[1409] = 32'b100111_11111_00000_00000_00001101111; // lcdwrite 31 111
1411 HD[1410] = 32'b001001_00000_00010_0000000000000000; // in 2
1412 HD[1411] = 32'b000010_10101_00010_0000000000000011; // sw 2 $fp 3
1413 HD[1412] = 32'b000001_10101_00010_0000000000000011; // lw 2 $fp 3
1414 HD[1413] = 32'b000011_10101_10101_0000000000000100; // addi $fp $fp 4
1415 HD[1414] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1416 HD[1415] = 32'b101000_000000000000000001001100110; // jal posicao_programa
1417 HD[1416] = 32'b000100_10101_10101_0000000000000100; // subi $fp $fp 4
1418 HD[1417] = 32'b000000_10100_00010_00000_00000_010000; // move 2 $rv
1419 HD[1418] = 32'b000001_00000_00011_00000000000011001; // lw 3 $global 25
1420 HD[1419] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
1421 HD[1420] = 32'b000111_00010_00000_00000000000010100; // beq 2 $zero _ElseBegin43_
1422 HD[1421] = 32'b000101_00000_00010_0000000000000001; // li 2 1
1423 HD[1422] = 32'b000010_10101_00010_0000000000000010; // sw 2 $fp 2
1424 HD[1423] = 32'b100111_10000_00000_00000_00000100000; // lcdwrite 16 32

```

```

1425 HD[1424] = 32'b100111_10001_00000_00000_00001110000; // lcdwrite 17 112
1426 HD[1425] = 32'b100111_10010_00000_00000_00001110010; // lcdwrite 18 114
1427 HD[1426] = 32'b100111_10011_00000_00000_00001101111; // lcdwrite 19 111
1428 HD[1427] = 32'b100111_10100_00000_00000_00001100111; // lcdwrite 20 103
1429 HD[1428] = 32'b100111_10101_00000_00000_00000101110; // lcdwrite 21 46
1430 HD[1429] = 32'b100111_10110_00000_00000_00000100000; // lcdwrite 22 32
1431 HD[1430] = 32'b100111_10111_00000_00000_00001101001; // lcdwrite 23 105
1432 HD[1431] = 32'b100111_11000_00000_00000_00001101110; // lcdwrite 24 110
1433 HD[1432] = 32'b100111_11001_00000_00000_00001110110; // lcdwrite 25 118
1434 HD[1433] = 32'b100111_11010_00000_00000_00001100001; // lcdwrite 26 97
1435 HD[1434] = 32'b100111_11011_00000_00000_00001101100; // lcdwrite 27 108
1436 HD[1435] = 32'b100111_11100_00000_00000_00001101001; // lcdwrite 28 105
1437 HD[1436] = 32'b100111_11101_00000_00000_00001100100; // lcdwrite 29 100
1438 HD[1437] = 32'b100111_11110_00000_00000_00001101111; // lcdwrite 30 111
1439 HD[1438] = 32'b100111_11111_00000_00000_00000100000; // lcdwrite 31 32
1440 HD[1439] = 32'b001011_000000000000000010111101000; // jump _IfExit43_
1441 HD[1440] = 32'b000001_10101_00010_00000000000000011; // lw 2 $fp 3 / _ElseBegin43_
1442 HD[1441] = 32'b000011_10101_10101_000000000000000100; // addi $fp $fp 4
1443 HD[1442] = 32'b000010_10101_00010_00000000000000000; // sw 2 $fp 0
1444 HD[1443] = 32'b101000_00000000000000000000111101000; // jal fila_existe_processo
1445 HD[1444] = 32'b000100_10101_10101_000000000000000100; // subi $fp $fp 4
1446 HD[1445] = 32'b000000_10100_00010_00000_00000_010000; // move 2 $rv
1447 HD[1446] = 32'b000001_00000_00011_00000000000010110; // lw 3 $global 22
1448 HD[1447] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
1449 HD[1448] = 32'b000111_00010_00000_00000000000101110; // beq 2 $zero _ElseBegin44_
1450 HD[1449] = 32'b000001_10101_00010_00000000000000011; // lw 2 $fp 3
1451 HD[1450] = 32'b000011_10101_10101_000000000000000100; // addi $fp $fp 4
1452 HD[1451] = 32'b000010_10101_00010_00000000000000000; // sw 2 $fp 0
1453 HD[1452] = 32'b101000_000000000000000000001000011111; // jal insere_processo_fila_pronto
1454 HD[1453] = 32'b000100_10101_10101_000000000000000100; // subi $fp $fp 4
1455 HD[1454] = 32'b000001_10101_00010_00000000000000000; // lw 2 $fp 0
1456 HD[1455] = 32'b000011_00010_00010_00000000000000001; // addi 2 2 1
1457 HD[1456] = 32'b000010_10101_00010_00000000000000000; // sw 2 $fp 0
1458 HD[1457] = 32'b100110_00000000000000000000000000000; // lcdclean
1459 HD[1458] = 32'b000101_00000_00010_000000000000000101; // li 2 5
1460 HD[1459] = 32'b000011_10101_10101_000000000000000100; // addi $fp $fp 4
1461 HD[1460] = 32'b000010_10101_00010_00000000000000000; // sw 2 $fp 0
1462 HD[1461] = 32'b101000_000000000000000000000010110011; // jal print_default_msg
1463 HD[1462] = 32'b000100_10101_10101_000000000000000100; // subi $fp $fp 4
1464 HD[1463] = 32'b000001_10101_00010_00000000000000011; // lw 2 $fp 3
1465 HD[1464] = 32'b000101_00000_00011_00000000000000000; // li 3 0
1466 HD[1465] = 32'b000101_00000_00100_0000000000001110; // li 4 14
1467 HD[1466] = 32'b000011_10101_10101_000000000000000100; // addi $fp $fp 4
1468 HD[1467] = 32'b000010_10101_00010_00000000000000010; // sw 4 $fp 2
1469 HD[1468] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
1470 HD[1469] = 32'b000010_10101_00010_00000000000000000; // sw 2 $fp 0
1471 HD[1470] = 32'b101000_00000000000000000000001100111; // jal print_value
1472 HD[1471] = 32'b000100_10101_10101_000000000000000100; // subi $fp $fp 4
1473 HD[1472] = 32'b100111_10000_00000_00000_00001101001; // lcdwrite 16 105
1474 HD[1473] = 32'b100111_10001_00000_00000_00001101110; // lcdwrite 17 110
1475 HD[1474] = 32'b100111_10010_00000_00000_00001110011; // lcdwrite 18 115
1476 HD[1475] = 32'b100111_10011_00000_00000_00001100101; // lcdwrite 19 101
1477 HD[1476] = 32'b100111_10100_00000_00000_00001110010; // lcdwrite 20 114
1478 HD[1477] = 32'b100111_10101_00000_00000_00001101001; // lcdwrite 21 105
1479 HD[1478] = 32'b100111_10110_00000_00000_00001100100; // lcdwrite 22 100
1480 HD[1479] = 32'b100111_10111_00000_00000_00001101111; // lcdwrite 23 111
1481 HD[1480] = 32'b100111_11001_00000_00000_00001101110; // lcdwrite 25 110
1482 HD[1481] = 32'b100111_11010_00000_00000_00001100001; // lcdwrite 26 97
1483 HD[1482] = 32'b100111_11100_00000_00000_00001100110; // lcdwrite 28 102
1484 HD[1483] = 32'b100111_11101_00000_00000_00001101001; // lcdwrite 29 105

```

```

1485 HD[1484] = 32'b100111_11110_00000_00000_00001101100; // lcdwrite 30 108
1486 HD[1485] = 32'b100111_11111_00000_00000_00001100001; // lcdwrite 31 97
1487 HD[1486] = 32'b000101_00000_00010_0000000000000010; // li 2 2
1488 HD[1487] = 32'b000101_00000_00011_0000000000000000; // li 3 0
1489 HD[1488] = 32'b000011_10101_10101_0000000000000100; // addi $fp $fp 4
1490 HD[1489] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
1491 HD[1490] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1492 HD[1491] = 32'b101000_0000000000000000000100001; // jal sleep
1493 HD[1492] = 32'b000100_10101_10101_0000000000000100; // subi $fp $fp 4
1494 HD[1493] = 32'b001011_0000000000000001011101000; // jump _IfExit44_
1495 HD[1494] = 32'b000101_00000_00010_0000000000000001; // li 2 1 / _ElseBegin44_
1496 HD[1495] = 32'b000010_10101_00010_0000000000000010; // sw 2 $fp 2
1497 HD[1496] = 32'b100111_10000_00000_00000_00001110000; // lcdwrite 16 112
1498 HD[1497] = 32'b100111_10001_00000_00000_00001110010; // lcdwrite 17 114
1499 HD[1498] = 32'b100111_10010_00000_00000_00001101111; // lcdwrite 18 111
1500 HD[1499] = 32'b100111_10011_00000_00000_00001100011; // lcdwrite 19 99
1501 HD[1500] = 32'b100111_10100_00000_00000_00000101110; // lcdwrite 20 46
1502 HD[1501] = 32'b100111_10101_00000_00000_00000100000; // lcdwrite 21 32
1503 HD[1502] = 32'b100111_10110_00000_00000_00001101010; // lcdwrite 22 106
1504 HD[1503] = 32'b100111_10111_00000_00000_00001100001; // lcdwrite 23 97
1505 HD[1504] = 32'b100111_11000_00000_00000_00000100000; // lcdwrite 24 32
1506 HD[1505] = 32'b100111_11001_00000_00000_00001100001; // lcdwrite 25 97
1507 HD[1506] = 32'b100111_11010_00000_00000_00001101100; // lcdwrite 26 108
1508 HD[1507] = 32'b100111_11011_00000_00000_00001101111; // lcdwrite 27 111
1509 HD[1508] = 32'b100111_11100_00000_00000_00001100011; // lcdwrite 28 99
1510 HD[1509] = 32'b100111_11101_00000_00000_00001100001; // lcdwrite 29 97
1511 HD[1510] = 32'b100111_11110_00000_00000_00001100100; // lcdwrite 30 100
1512 HD[1511] = 32'b100111_11111_00000_00000_00001101111; // lcdwrite 31 111
1513 HD[1512] = 32'b000001_10101_00010_0000000000000010; // lw 2 $fp 2 / _IfExit44_ _IfExit43_
1514 HD[1513] = 32'b000101_00000_00011_0000000000000001; // li 3 1
1515 HD[1514] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
1516 HD[1515] = 32'b000111_00010_00000_0000000000001111; // beq 2 $zero _IfExit45_
1517 HD[1516] = 32'b000101_00000_00010_0000000000000001; // li 2 1
1518 HD[1517] = 32'b000011_10101_10101_0000000000000100; // addi $fp $fp 4
1519 HD[1518] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1520 HD[1519] = 32'b101000_0000000000000000010110011; // jal print_default_msg
1521 HD[1520] = 32'b000100_10101_10101_0000000000000100; // subi $fp $fp 4
1522 HD[1521] = 32'b000101_00000_00010_0000000000000010; // li 2 2
1523 HD[1522] = 32'b000101_00000_00011_0000000000000000; // li 3 0
1524 HD[1523] = 32'b000011_10101_10101_0000000000000100; // addi $fp $fp 4
1525 HD[1524] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
1526 HD[1525] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1527 HD[1526] = 32'b101000_00000000000000000000010001; // jal sleep
1528 HD[1527] = 32'b001000_10101_10101_0000000000000100; // subi $fp $fp 4
1529 HD[1528] = 32'b000101_00000_00010_0000000000000000; // li 2 0
1530 HD[1529] = 32'b000010_10101_00010_0000000000000010; // sw 2 $fp 2
1531 HD[1530] = 32'b001011_00000000000000010101101001; // jump _WhileBegin13_ / _IfExit45_
1532 HD[1531] = 32'b010000_10110_10111_0000000000000000; // pop $ra / _WhileExit13_ _IfExit39_
1533 HD[1532] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
1534 HD[1533] = 32'b001111_10110_10111_0000000000000000; // push $ra / menu_opt0_executar_prog
1535 HD[1534] = 32'b100111_00000_00000_00000_00000100000; // lcdwrite 0 32
1536 HD[1535] = 32'b100111_00001_00000_00000_00000100000; // lcdwrite 1 32
1537 HD[1536] = 32'b100111_00010_00000_00000_00001000001; // lcdwrite 2 65
1538 HD[1537] = 32'b100111_00011_00000_00000_00001110100; // lcdwrite 3 116
1539 HD[1538] = 32'b100111_00100_00000_00000_00001101001; // lcdwrite 4 105
1540 HD[1539] = 32'b100111_00101_00000_00000_00001110110; // lcdwrite 5 118
1541 HD[1540] = 32'b100111_00110_00000_00000_00001100001; // lcdwrite 6 97
1542 HD[1541] = 32'b100111_00111_00000_00000_00001110010; // lcdwrite 7 114
1543 HD[1542] = 32'b100111_01000_00000_00000_00000100000; // lcdwrite 8 32
1544 HD[1543] = 32'b100111_01001_00000_00000_00000100000; // lcdwrite 9 32

```

```

1545 HD[1544] = 32'b100111_01010_00000_00000_00001101101; // lcdwrite 10 109
1546 HD[1545] = 32'b100111_01011_00000_00000_00001101111; // lcdwrite 11 111
1547 HD[1546] = 32'b100111_01100_00000_00000_00001100100; // lcdwrite 12 100
1548 HD[1547] = 32'b100111_01101_00000_00000_00001101111; // lcdwrite 13 111
1549 HD[1548] = 32'b100111_01110_00000_00000_00000100000; // lcdwrite 14 32
1550 HD[1549] = 32'b100111_01111_00000_00000_00000100000; // lcdwrite 15 32
1551 HD[1550] = 32'b100111_10000_00000_00000_00000100000; // lcdwrite 16 32
1552 HD[1551] = 32'b100111_10001_00000_00000_00001110000; // lcdwrite 17 112
1553 HD[1552] = 32'b100111_10010_00000_00000_00001110010; // lcdwrite 18 114
1554 HD[1553] = 32'b100111_10011_00000_00000_00001100101; // lcdwrite 19 101
1555 HD[1554] = 32'b100111_10100_00000_00000_00001100101; // lcdwrite 20 101
1556 HD[1555] = 32'b100111_10101_00000_00000_00001101101; // lcdwrite 21 109
1557 HD[1556] = 32'b100111_10110_00000_00000_00001110000; // lcdwrite 22 112
1558 HD[1557] = 32'b100111_10111_00000_00000_00001110100; // lcdwrite 23 116
1559 HD[1558] = 32'b100111_11000_00000_00000_00001101001; // lcdwrite 24 105
1560 HD[1559] = 32'b100111_11001_00000_00000_00001110110; // lcdwrite 25 118
1561 HD[1560] = 32'b100111_11010_00000_00000_00001101111; // lcdwrite 26 111
1562 HD[1561] = 32'b100111_11011_00000_00000_00000111111; // lcdwrite 27 63
1563 HD[1562] = 32'b100111_11100_00000_00000_00000100000; // lcdwrite 28 32
1564 HD[1563] = 32'b100111_11101_00000_00000_00001111001; // lcdwrite 29 121
1565 HD[1564] = 32'b100111_11110_00000_00000_00000111110; // lcdwrite 30 62
1566 HD[1565] = 32'b100111_11111_00000_00000_00000110000; // lcdwrite 31 48
1567 HD[1566] = 32'b001001_00000_00010_0000000000000000; // in 2
1568 HD[1567] = 32'b000010_10101_00010_0000000000000001; // sw 2 $fp 1
1569 HD[1568] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1 / _WhileBegin14_
1570 HD[1569] = 32'b000101_00000_00011_0000000000000000; // li 3 0
1571 HD[1570] = 32'b000000_00010_00011_00010_00000_010001; // slt 2 2 3
1572 HD[1571] = 32'b000111_00010_00000_00000000000000100; // beq 2 $zero _WhileExit14_
1573 HD[1572] = 32'b001001_00000_00010_0000000000000000; // in 2
1574 HD[1573] = 32'b000010_10101_00010_0000000000000001; // sw 2 $fp 1
1575 HD[1574] = 32'b001011_00000000000000011000100000; // jump _WhileBegin14_
1576 HD[1575] = 32'b000011_10101_10101_00000000000000010; // addi $fp $fp 2 / _WhileExit14_
1577 HD[1576] = 32'b101000_00000000000000010011110101; // jal seleciona_programas
1578 HD[1577] = 32'b000100_10101_10101_00000000000000010; // subi $fp $fp 2
1579 HD[1578] = 32'b100110_0000000000000000000000000000; // lcdclean
1580 HD[1579] = 32'b100111_00001_00000_00000_00001001001; // lcdwrite 1 73
1581 HD[1580] = 32'b100111_00010_00000_00000_00001101110; // lcdwrite 2 110
1582 HD[1581] = 32'b100111_00011_00000_00000_00001101001; // lcdwrite 3 105
1583 HD[1582] = 32'b100111_00100_00000_00000_00001100011; // lcdwrite 4 99
1584 HD[1583] = 32'b100111_00101_00000_00000_00001101001; // lcdwrite 5 105
1585 HD[1584] = 32'b100111_00110_00000_00000_00001100001; // lcdwrite 6 97
1586 HD[1585] = 32'b100111_00111_00000_00000_00001101110; // lcdwrite 7 110
1587 HD[1586] = 32'b100111_01000_00000_00000_00001100100; // lcdwrite 8 100
1588 HD[1587] = 32'b100111_01001_00000_00000_00001101111; // lcdwrite 9 111
1589 HD[1588] = 32'b100111_01011_00000_00000_00001100101; // lcdwrite 11 101
1590 HD[1589] = 32'b100111_01100_00000_00000_00001111000; // lcdwrite 12 120
1591 HD[1590] = 32'b100111_01101_00000_00000_00001100101; // lcdwrite 13 101
1592 HD[1591] = 32'b100111_01110_00000_00000_00001100011; // lcdwrite 14 99
1593 HD[1592] = 32'b100111_10001_00000_00000_00000111110; // lcdwrite 17 62
1594 HD[1593] = 32'b100111_10010_00000_00000_00000111110; // lcdwrite 18 62
1595 HD[1594] = 32'b000101_00000_00010_0000000000000001; // li 2 1
1596 HD[1595] = 32'b000101_00000_00011_0000000000000000; // li 3 0
1597 HD[1596] = 32'b000011_10101_10101_00000000000000010; // addi $fp $fp 2
1598 HD[1597] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
1599 HD[1598] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1600 HD[1599] = 32'b101000_00000000000000000000000100001; // jal sleep
1601 HD[1600] = 32'b000100_10101_10101_00000000000000010; // subi $fp $fp 2
1602 HD[1601] = 32'b100111_10111_00000_00000_00000111110; // lcdwrite 23 62
1603 HD[1602] = 32'b100111_11000_00000_00000_00000111110; // lcdwrite 24 62
1604 HD[1603] = 32'b000101_00000_00010_0000000000000001; // li 2 1

```



```

1605 HD[1604] = 32'b000101_00000_00011_0000000000000000; // li 3 0
1606 HD[1605] = 32'b000011_10101_10101_0000000000000000; // addi $fp $fp 2
1607 HD[1606] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
1608 HD[1607] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1609 HD[1608] = 32'b101000_0000000000000000000100001; // jal sleep
1610 HD[1609] = 32'b000100_10101_10101_0000000000000000; // subi $fp $fp 2
1611 HD[1610] = 32'b100111_11101_00000_00000_00000111110; // lcdwrite 29 62
1612 HD[1611] = 32'b100111_11110_00000_00000_00000111110; // lcdwrite 30 62
1613 HD[1612] = 32'b000101_00000_00010_0000000000000001; // li 2 1
1614 HD[1613] = 32'b000101_00000_00011_0000000000000000; // li 3 0
1615 HD[1614] = 32'b000011_10101_10101_0000000000000000; // addi $fp $fp 2
1616 HD[1615] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
1617 HD[1616] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1618 HD[1617] = 32'b101000_0000000000000000000100001; // jal sleep
1619 HD[1618] = 32'b000100_10101_10101_0000000000000000; // subi $fp $fp 2
1620 HD[1619] = 32'b000001_00000_00010_0000000000001000; // lw 2 $global 8
1621 HD[1620] = 32'b000101_00000_00011_0000000000000001; // li 3 1
1622 HD[1621] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
1623 HD[1622] = 32'b000111_00010_00000_0000000000000101; // beq 2 $zero _ElseBegin46_
1624 HD[1623] = 32'b000011_10101_10101_0000000000000000; // addi $fp $fp 2
1625 HD[1624] = 32'b101000_00000000000000000110110111; // jal executa_processos
1626 HD[1625] = 32'b000100_10101_10101_0000000000000000; // subi $fp $fp 2
1627 HD[1626] = 32'b001011_0000000000000000011001100110; // jump _IfExit46_
1628 HD[1627] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1 / _ElseBegin46_
1629 HD[1628] = 32'b000101_00000_00011_0000000000000000; // li 3 0
1630 HD[1629] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
1631 HD[1630] = 32'b000111_00010_00000_0000000000000101; // beq 2 $zero _ElseBegin47_
1632 HD[1631] = 32'b000011_10101_10101_0000000000000000; // addi $fp $fp 2
1633 HD[1632] = 32'b101000_00000000000000000110110111; // jal executa_processos
1634 HD[1633] = 32'b000100_10101_10101_0000000000000000; // subi $fp $fp 2
1635 HD[1634] = 32'b001011_0000000000000000011001100110; // jump _IfExit47_
1636 HD[1635] = 32'b000011_10101_10101_0000000000000000; // addi $fp $fp 2 / _ElseBegin47_
1637 HD[1636] = 32'b101000_0000000000000000010000000101; // jal executa_processos_preemptivo
1638 HD[1637] = 32'b000100_10101_10101_0000000000000000; // subi $fp $fp 2
1639 HD[1638] = 32'b100111_00000_00000_00000_00000100000; // lcdwrite 0 32 / _IfExit47_
    _IfExit46_
1640 HD[1639] = 32'b100111_00001_00000_00000_00000101101; // lcdwrite 1 45
1641 HD[1640] = 32'b100111_00010_00000_00000_00000101101; // lcdwrite 2 45
1642 HD[1641] = 32'b100111_00011_00000_00000_00001010000; // lcdwrite 3 80
1643 HD[1642] = 32'b100111_00100_00000_00000_00001110010; // lcdwrite 4 114
1644 HD[1643] = 32'b100111_00101_00000_00000_00001101111; // lcdwrite 5 111
1645 HD[1644] = 32'b100111_00110_00000_00000_00001100011; // lcdwrite 6 99
1646 HD[1645] = 32'b100111_00111_00000_00000_00001100101; // lcdwrite 7 101
1647 HD[1646] = 32'b100111_01000_00000_00000_00001110011; // lcdwrite 8 115
1648 HD[1647] = 32'b100111_01001_00000_00000_00001110011; // lcdwrite 9 115
1649 HD[1648] = 32'b100111_01010_00000_00000_00001101111; // lcdwrite 10 111
1650 HD[1649] = 32'b100111_01011_00000_00000_00001110011; // lcdwrite 11 115
1651 HD[1650] = 32'b100111_01100_00000_00000_00000101101; // lcdwrite 12 45
1652 HD[1651] = 32'b100111_01101_00000_00000_00000101101; // lcdwrite 13 45
1653 HD[1652] = 32'b100111_01110_00000_00000_00000101101; // lcdwrite 14 45
1654 HD[1653] = 32'b100111_01111_00000_00000_00000100000; // lcdwrite 15 32
1655 HD[1654] = 32'b100111_10000_00000_00000_00000100000; // lcdwrite 16 32
1656 HD[1655] = 32'b100111_10001_00000_00000_00000101101; // lcdwrite 17 45
1657 HD[1656] = 32'b100111_10010_00000_00000_00000101101; // lcdwrite 18 45
1658 HD[1657] = 32'b100111_10011_00000_00000_00001000110; // lcdwrite 19 70
1659 HD[1658] = 32'b100111_10100_00000_00000_00001101001; // lcdwrite 20 105
1660 HD[1659] = 32'b100111_10101_00000_00000_00001101110; // lcdwrite 21 110
1661 HD[1660] = 32'b100111_10110_00000_00000_00001100001; // lcdwrite 22 97
1662 HD[1661] = 32'b100111_10111_00000_00000_00001101100; // lcdwrite 23 108
1663 HD[1662] = 32'b100111_11000_00000_00000_00001101001; // lcdwrite 24 105

```

```

1664 HD[1663] = 32'b100111_11001_00000_00000_00001111010; // lcdwrite 25 122
1665 HD[1664] = 32'b100111_11010_00000_00000_00001100001; // lcdwrite 26 97
1666 HD[1665] = 32'b100111_11011_00000_00000_00001100100; // lcdwrite 27 100
1667 HD[1666] = 32'b100111_11100_00000_00000_00001101111; // lcdwrite 28 111
1668 HD[1667] = 32'b100111_11101_00000_00000_00001110011; // lcdwrite 29 115
1669 HD[1668] = 32'b100111_11110_00000_00000_00000101101; // lcdwrite 30 45
1670 HD[1669] = 32'b100111_11111_00000_00000_00000100000; // lcdwrite 31 32
1671 HD[1670] = 32'b000101_00000_00010_0000000000000000; // li 2 0
1672 HD[1671] = 32'b001010_00010_00000_0000000000000000; // out 2
1673 HD[1672] = 32'b010000_10110_10111_0000000000000000; // pop $ra
1674 HD[1673] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
1675 HD[1674] = 32'b001111_10110_10111_0000000000000000; // push $ra / menu_opt1_escrever_prog
1676 HD[1675] = 32'b000101_00000_00010_0000000000000000; // li 2 0
1677 HD[1676] = 32'b000010_10101_00010_0000000000000001; // sw 2 $fp 3
1678 HD[1677] = 32'b100111_00000_00000_00000_00001000100; // lcdwrite 0 68
1679 HD[1678] = 32'b100111_00001_00000_00000_00001100101; // lcdwrite 1 101
1680 HD[1679] = 32'b100111_00010_00000_00000_00001110100; // lcdwrite 2 116
1681 HD[1680] = 32'b100111_00011_00000_00000_00001100101; // lcdwrite 3 101
1682 HD[1681] = 32'b100111_00100_00000_00000_00001110010; // lcdwrite 4 114
1683 HD[1682] = 32'b100111_00101_00000_00000_00001101101; // lcdwrite 5 109
1684 HD[1683] = 32'b100111_00110_00000_00000_00001101001; // lcdwrite 6 105
1685 HD[1684] = 32'b100111_00111_00000_00000_00001101110; // lcdwrite 7 110
1686 HD[1685] = 32'b100111_01000_00000_00000_00001100101; // lcdwrite 8 101
1687 HD[1686] = 32'b100111_01001_00000_00000_00000100000; // lcdwrite 9 32
1688 HD[1687] = 32'b100111_01010_00000_00000_00001110101; // lcdwrite 10 117
1689 HD[1688] = 32'b100111_01011_00000_00000_00001101101; // lcdwrite 11 109
1690 HD[1689] = 32'b100111_01100_00000_00000_00000100000; // lcdwrite 12 32
1691 HD[1690] = 32'b100111_01101_00000_00000_00001001001; // lcdwrite 13 73
1692 HD[1691] = 32'b100111_01110_00000_00000_00001000100; // lcdwrite 14 68
1693 HD[1692] = 32'b100111_01111_00000_00000_00000100000; // lcdwrite 15 32
1694 HD[1693] = 32'b100111_10000_00000_00000_00001110000; // lcdwrite 16 112
1695 HD[1694] = 32'b100111_10001_00000_00000_00001100001; // lcdwrite 17 97
1696 HD[1695] = 32'b100111_10010_00000_00000_00001110010; // lcdwrite 18 114
1697 HD[1696] = 32'b100111_10011_00000_00000_00001100001; // lcdwrite 19 97
1698 HD[1697] = 32'b100111_10100_00000_00000_00000100000; // lcdwrite 20 32
1699 HD[1698] = 32'b100111_10101_00000_00000_00001101111; // lcdwrite 21 111
1700 HD[1699] = 32'b100111_10110_00000_00000_00000100000; // lcdwrite 22 32
1701 HD[1700] = 32'b100111_10111_00000_00000_00001110000; // lcdwrite 23 112
1702 HD[1701] = 32'b100111_11000_00000_00000_00001110010; // lcdwrite 24 114
1703 HD[1702] = 32'b100111_11001_00000_00000_00001101111; // lcdwrite 25 111
1704 HD[1703] = 32'b100111_11010_00000_00000_00001100111; // lcdwrite 26 103
1705 HD[1704] = 32'b100111_11011_00000_00000_00001110010; // lcdwrite 27 114
1706 HD[1705] = 32'b100111_11100_00000_00000_00001100001; // lcdwrite 28 97
1707 HD[1706] = 32'b100111_11101_00000_00000_00001101101; // lcdwrite 29 109
1708 HD[1707] = 32'b100111_11110_00000_00000_00001100001; // lcdwrite 30 97
1709 HD[1708] = 32'b100111_11111_00000_00000_00000100000; // lcdwrite 31 32
1710 HD[1709] = 32'b001001_00000_00010_0000000000000000; // in 2
1711 HD[1710] = 32'b000101_00000_00011_0000001111101000; // li 3 1000
1712 HD[1711] = 32'b000011_10101_10101_0000000000000100; // addi $fp $fp 4
1713 HD[1712] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
1714 HD[1713] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1715 HD[1714] = 32'b101000_00000000000000000001011010; // jal mod
1716 HD[1715] = 32'b000100_10101_10101_0000000000000100; // subi $fp $fp 4
1717 HD[1716] = 32'b000000_10100_00010_00000_00000_010000; // move 2 $rv
1718 HD[1717] = 32'b000010_10101_00010_0000000000000001; // sw 2 $fp 1
1719 HD[1718] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
1720 HD[1719] = 32'b000101_00000_00011_0000000000000000; // li 3 0
1721 HD[1720] = 32'b000000_00010_00011_00010_00000_010010; // sgt 2 2 3
1722 HD[1721] = 32'b000000_00010_00000_00010_00000_010011; // set 2 2 $zero
1723 HD[1722] = 32'b000111_00010_00000_0000000000001001; // beq 2 $zero _ElseBegin48_

```



```

1724 HD[1723] = 32'b000101_00000_00010_0000000000000110; // li 2 6
1725 HD[1724] = 32'b000011_10101_10101_0000000000000100; // addi $fp $fp 4
1726 HD[1725] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1727 HD[1726] = 32'b101000_0000000000000000010110011; // jal print_default_msg
1728 HD[1727] = 32'b000100_10101_10101_0000000000000100; // subi $fp $fp 4
1729 HD[1728] = 32'b000101_00000_00010_0000000000000001; // li 2 1
1730 HD[1729] = 32'b000010_10101_00010_0000000000000011; // sw 2 $fp 3
1731 HD[1730] = 32'b001011_00000000000000011101010011; // jump _IfExit48_
1732 HD[1731] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1 / _ElseBegin48_
1733 HD[1732] = 32'b000011_10101_10101_0000000000000100; // addi $fp $fp 4
1734 HD[1733] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1735 HD[1734] = 32'b101000_000000000000000001001100110; // jal posicao_programa
1736 HD[1735] = 32'b000100_10101_10101_0000000000000100; // subi $fp $fp 4
1737 HD[1736] = 32'b000000_10100_00010_00000_00000_010000; // move 2 $rv
1738 HD[1737] = 32'b000001_00000_00011_0000000000011001; // lw 3 $global 25
1739 HD[1738] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
1740 HD[1739] = 32'b000000_00010_00000_00010_00000_010011; // set 2 2 $zero
1741 HD[1740] = 32'b000111_00010_00000_0000000000010101; // beq 2 $zero _ElseBegin49_
1742 HD[1741] = 32'b000101_00000_00010_0000000000000101; // li 2 5
1743 HD[1742] = 32'b000011_10101_10101_0000000000000100; // addi $fp $fp 4
1744 HD[1743] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1745 HD[1744] = 32'b101000_0000000000000000010110011; // jal print_default_msg
1746 HD[1745] = 32'b000100_10101_10101_0000000000000100; // subi $fp $fp 4
1747 HD[1746] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
1748 HD[1747] = 32'b000101_00000_00011_0000000000000000; // li 3 0
1749 HD[1748] = 32'b000101_00000_00100_0000000000000110; // li 4 14
1750 HD[1749] = 32'b000011_10101_10101_0000000000000100; // addi $fp $fp 4
1751 HD[1750] = 32'b000010_10101_00100_0000000000000010; // sw 4 $fp 2
1752 HD[1751] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
1753 HD[1752] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1754 HD[1753] = 32'b101000_00000000000000000001100111; // jal print_value
1755 HD[1754] = 32'b000100_10101_10101_0000000000000100; // subi $fp $fp 4
1756 HD[1755] = 32'b000101_00000_00010_0000000000000111; // li 2 7
1757 HD[1756] = 32'b000011_10101_10101_0000000000000100; // addi $fp $fp 4
1758 HD[1757] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1759 HD[1758] = 32'b101000_0000000000000000010110011; // jal print_default_msg
1760 HD[1759] = 32'b000100_10101_10101_0000000000000100; // subi $fp $fp 4
1761 HD[1760] = 32'b001011_00000000000000011101010011; // jump _IfExit49_
1762 HD[1761] = 32'b100110_00000000000000000000000000; // lcdclean / _ElseBegin49_
1763 HD[1762] = 32'b100111_00000_00000_00000_00001000100; // lcdwrite 0 68
1764 HD[1763] = 32'b100111_00001_00000_00000_00001100101; // lcdwrite 1 101
1765 HD[1764] = 32'b100111_00010_00000_00000_00001110100; // lcdwrite 2 116
1766 HD[1765] = 32'b100111_00011_00000_00000_00001100101; // lcdwrite 3 101
1767 HD[1766] = 32'b100111_00100_00000_00000_00001110010; // lcdwrite 4 114
1768 HD[1767] = 32'b100111_00101_00000_00000_00001101101; // lcdwrite 5 109
1769 HD[1768] = 32'b100111_00110_00000_00000_00001101001; // lcdwrite 6 105
1770 HD[1769] = 32'b100111_00111_00000_00000_00001101110; // lcdwrite 7 110
1771 HD[1770] = 32'b100111_01000_00000_00000_00001100101; // lcdwrite 8 101
1772 HD[1771] = 32'b100111_01010_00000_00000_00001101111; // lcdwrite 10 111
1773 HD[1772] = 32'b100111_01100_00000_00000_00001010100; // lcdwrite 12 84
1774 HD[1773] = 32'b100111_01101_00000_00000_00001100001; // lcdwrite 13 97
1775 HD[1774] = 32'b100111_01110_00000_00000_00001101101; // lcdwrite 14 109
1776 HD[1775] = 32'b100111_10010_00000_00000_00001100100; // lcdwrite 18 100
1777 HD[1776] = 32'b100111_10011_00000_00000_00001101111; // lcdwrite 19 111
1778 HD[1777] = 32'b100111_10101_00000_00000_00001110000; // lcdwrite 21 112
1779 HD[1778] = 32'b100111_10110_00000_00000_00001110010; // lcdwrite 22 114
1780 HD[1779] = 32'b100111_10111_00000_00000_00001101111; // lcdwrite 23 111
1781 HD[1780] = 32'b100111_11000_00000_00000_00001100111; // lcdwrite 24 103
1782 HD[1781] = 32'b100111_11001_00000_00000_00001110010; // lcdwrite 25 114
1783 HD[1782] = 32'b100111_11010_00000_00000_00001100001; // lcdwrite 26 97

```

```

1784 HD[1783] = 32'b100111_11011_00000_00000_00001101101; // lcdwrite 27 109
1785 HD[1784] = 32'b100111_11100_00000_00000_00001100001; // lcdwrite 28 97
1786 HD[1785] = 32'b001001_00000_00010_0000000000000000; // in 2
1787 HD[1786] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 2
1788 HD[1787] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 2
1789 HD[1788] = 32'b000101_00000_00011_0000000000000000; // li 3 0
1790 HD[1789] = 32'b000000_00010_00011_00010_00000_010010; // sgt 2 2 3
1791 HD[1790] = 32'b000000_00010_00000_00010_00000_010011; // set 2 2 $zero
1792 HD[1791] = 32'b000111_00010_00000_00000000000010100; // beq 2 $zero _ElseBegin50_
1793 HD[1792] = 32'b100111_10000_00000_00000_00001010100; // lcdwrite 16 84
1794 HD[1793] = 32'b100111_10001_00000_00000_00001100001; // lcdwrite 17 97
1795 HD[1794] = 32'b100111_10010_00000_00000_00001101101; // lcdwrite 18 109
1796 HD[1795] = 32'b100111_10011_00000_00000_00000100000; // lcdwrite 19 32
1797 HD[1796] = 32'b100111_10100_00000_00000_00001100100; // lcdwrite 20 100
1798 HD[1797] = 32'b100111_10101_00000_00000_00001100101; // lcdwrite 21 101
1799 HD[1798] = 32'b100111_10110_00000_00000_00001110110; // lcdwrite 22 118
1800 HD[1799] = 32'b100111_10111_00000_00000_00001100101; // lcdwrite 23 101
1801 HD[1800] = 32'b100111_11000_00000_00000_00000100000; // lcdwrite 24 32
1802 HD[1801] = 32'b100111_11001_00000_00000_00001110011; // lcdwrite 25 115
1803 HD[1802] = 32'b100111_11010_00000_00000_00001100101; // lcdwrite 26 101
1804 HD[1803] = 32'b100111_11011_00000_00000_00001110010; // lcdwrite 27 114
1805 HD[1804] = 32'b100111_11100_00000_00000_00000100000; // lcdwrite 28 32
1806 HD[1805] = 32'b100111_11101_00000_00000_00000111110; // lcdwrite 29 62
1807 HD[1806] = 32'b100111_11110_00000_00000_00000100000; // lcdwrite 30 32
1808 HD[1807] = 32'b100111_11111_00000_00000_00000110000; // lcdwrite 31 48
1809 HD[1808] = 32'b000101_00000_00010_00000000000000001; // li 2 1
1810 HD[1809] = 32'b000010_10101_00010_000000000000000011; // sw 2 $fp 3
1811 HD[1810] = 32'b001011_000000000000000011101010011; // jump _IfExit50_
1812 HD[1811] = 32'b100110_0000000000000000000000000000; // lcdclean / _ElseBegin50_
1813 HD[1812] = 32'b100111_00000_00000_00000_00001001001; // lcdwrite 0 73
1814 HD[1813] = 32'b100111_00001_00000_00000_00001101110; // lcdwrite 1 110
1815 HD[1814] = 32'b100111_00010_00000_00000_00001110011; // lcdwrite 2 115
1816 HD[1815] = 32'b100111_00011_00000_00000_00001101001; // lcdwrite 3 105
1817 HD[1816] = 32'b100111_00100_00000_00000_00001110010; // lcdwrite 4 114
1818 HD[1817] = 32'b100111_00101_00000_00000_00001100001; // lcdwrite 5 97
1819 HD[1818] = 32'b100111_00110_00000_00000_00000111010; // lcdwrite 6 58
1820 HD[1819] = 32'b100111_01000_00000_00000_00001101001; // lcdwrite 8 105
1821 HD[1820] = 32'b100111_01001_00000_00000_00001101110; // lcdwrite 9 110
1822 HD[1821] = 32'b100111_01010_00000_00000_00001110011; // lcdwrite 10 115
1823 HD[1822] = 32'b100111_01011_00000_00000_00001110100; // lcdwrite 11 116
1824 HD[1823] = 32'b100111_10000_00000_00000_00001100100; // lcdwrite 16 100
1825 HD[1824] = 32'b100111_10001_00000_00000_00001101111; // lcdwrite 17 111
1826 HD[1825] = 32'b100111_10011_00000_00000_00001010000; // lcdwrite 19 80
1827 HD[1826] = 32'b100111_10100_00000_00000_00001110010; // lcdwrite 20 114
1828 HD[1827] = 32'b100111_10101_00000_00000_00001101111; // lcdwrite 21 111
1829 HD[1828] = 32'b100111_10110_00000_00000_00001100111; // lcdwrite 22 103
1830 HD[1829] = 32'b100111_10111_00000_00000_00001110010; // lcdwrite 23 114
1831 HD[1830] = 32'b100111_11000_00000_00000_00001100001; // lcdwrite 24 97
1832 HD[1831] = 32'b100111_11001_00000_00000_00001101101; // lcdwrite 25 109
1833 HD[1832] = 32'b100111_11010_00000_00000_00001100001; // lcdwrite 26 97
1834 HD[1833] = 32'b000001_10101_00010_00000000000000001; // lw 2 $fp 1
1835 HD[1834] = 32'b000101_00000_00011_00000000000000001; // li 3 1
1836 HD[1835] = 32'b000101_00000_00100_0000000000001110; // li 4 14
1837 HD[1836] = 32'b000011_10101_10101_00000000000000100; // addi $fp $fp 4
1838 HD[1837] = 32'b000010_10101_00100_0000000000000010; // sw 4 $fp 2
1839 HD[1838] = 32'b000010_10101_00011_00000000000000001; // sw 3 $fp 1
1840 HD[1839] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1841 HD[1840] = 32'b101000_00000000000000000001100111; // jal print_value
1842 HD[1841] = 32'b000100_10101_10101_00000000000000100; // subi $fp $fp 4
1843 HD[1842] = 32'b000001_10101_00010_00000000000000001; // lw 2 $fp 1

```

```

1844 HD[1843] = 32'b000001_10101_00011_0000000000000010; // lw 3 $fp 2
1845 HD[1844] = 32'b000011_10101_10101_0000000000000100; // addi $fp $fp 4
1846 HD[1845] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
1847 HD[1846] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1848 HD[1847] = 32'b101000_0000000000000000101111111; // jal cria_programa
1849 HD[1848] = 32'b000100_10101_10101_0000000000000100; // subi $fp $fp 4
1850 HD[1849] = 32'b000000_10100_00010_00000_00000_010000; // move 2 $rv
1851 HD[1850] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1852 HD[1851] = 32'b100110_000000000000000000000000; // lcdclean
1853 HD[1852] = 32'b000101_00000_00010_0000000000000101; // li 2 5
1854 HD[1853] = 32'b000011_10101_10101_0000000000000100; // addi $fp $fp 4
1855 HD[1854] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1856 HD[1855] = 32'b101000_00000000000000000010110011; // jal print_default_msg
1857 HD[1856] = 32'b000100_10101_10101_0000000000000100; // subi $fp $fp 4
1858 HD[1857] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
1859 HD[1858] = 32'b000101_00000_00011_0000000000000000; // li 3 0
1860 HD[1859] = 32'b000101_00000_00100_0000000000001110; // li 4 14
1861 HD[1860] = 32'b000011_10101_10101_0000000000000100; // addi $fp $fp 4
1862 HD[1861] = 32'b000010_10101_00100_0000000000000010; // sw 4 $fp 2
1863 HD[1862] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
1864 HD[1863] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1865 HD[1864] = 32'b101000_000000000000000000110011; // jal print_value
1866 HD[1865] = 32'b000100_10101_10101_0000000000000100; // subi $fp $fp 4
1867 HD[1866] = 32'b100111_10011_00000_00000_00001100110; // lcdwrite 19 102
1868 HD[1867] = 32'b100111_10100_00000_00000_00001101111; // lcdwrite 20 111
1869 HD[1868] = 32'b100111_10101_00000_00000_00001101001; // lcdwrite 21 105
1870 HD[1869] = 32'b100111_10111_00000_00000_00001100011; // lcdwrite 23 99
1871 HD[1870] = 32'b100111_11000_00000_00000_00001110010; // lcdwrite 24 114
1872 HD[1871] = 32'b100111_11001_00000_00000_00001101001; // lcdwrite 25 105
1873 HD[1872] = 32'b100111_11010_00000_00000_00001100001; // lcdwrite 26 97
1874 HD[1873] = 32'b100111_11011_00000_00000_00001100100; // lcdwrite 27 100
1875 HD[1874] = 32'b100111_11100_00000_00000_00001101111; // lcdwrite 28 111
1876 HD[1875] = 32'b000001_10101_00010_0000000000000011; // lw 2 $fp 3 / _IfExit50_ _IfExit49_
    _IfExit48_
1877 HD[1876] = 32'b000101_00000_00011_0000000000000001; // li 3 1
1878 HD[1877] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
1879 HD[1878] = 32'b000111_00010_00000_0000000000001111; // beq 2 $zero _IfExit51_
1880 HD[1879] = 32'b000101_00000_00010_0000000000000001; // li 2 1
1881 HD[1880] = 32'b000011_10101_10101_0000000000000100; // addi $fp $fp 4
1882 HD[1881] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1883 HD[1882] = 32'b101000_00000000000000000010110011; // jal print_default_msg
1884 HD[1883] = 32'b000100_10101_10101_0000000000000100; // subi $fp $fp 4
1885 HD[1884] = 32'b000101_00000_00010_0000000000000010; // li 2 4
1886 HD[1885] = 32'b000101_00000_00011_0000000000000000; // li 3 0
1887 HD[1886] = 32'b000011_10101_10101_0000000000000100; // addi $fp $fp 4
1888 HD[1887] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
1889 HD[1888] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1890 HD[1889] = 32'b101000_0000000000000000000100001; // jal sleep
1891 HD[1890] = 32'b000100_10101_10101_0000000000000100; // subi $fp $fp 4
1892 HD[1891] = 32'b010000_10110_10111_0000000000000000; // pop $ra
1893 HD[1892] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
1894 HD[1893] = 32'b000101_00000_00010_0000000000000000; // li 2 0 / _IfExit51_
1895 HD[1894] = 32'b001010_00010_00000_0000000000000000; // out 2
1896 HD[1895] = 32'b010000_10110_10111_0000000000000000; // pop $ra
1897 HD[1896] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
1898 HD[1897] = 32'b001111_10110_10111_0000000000000000; // push $ra / menu_opt2_renomear_prog
1899 HD[1898] = 32'b000101_00000_00010_0000000000000000; // li 2 0
1900 HD[1899] = 32'b000010_10101_00010_0000000000000011; // sw 2 $fp 3
1901 HD[1900] = 32'b000101_00000_00010_0000000000000010; // li 2 2
1902 HD[1901] = 32'b000011_10101_10101_0000000000000100; // addi $fp $fp 4

```

```

1903 HD[1902] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1904 HD[1903] = 32'b101000_00000000000000000010110011; // jal print_default_msg
1905 HD[1904] = 32'b000100_10101_10101_00000000000000100; // subi $fp $fp 4
1906 HD[1905] = 32'b100111_10000_00000_00000_00000100000; // lcdwrite 16 32
1907 HD[1906] = 32'b100111_10001_00000_00000_00001100001; // lcdwrite 17 97
1908 HD[1907] = 32'b100111_10010_00000_00000_00000100000; // lcdwrite 18 32
1909 HD[1908] = 32'b100111_10011_00000_00000_00001110011; // lcdwrite 19 115
1910 HD[1909] = 32'b100111_10100_00000_00000_00001100101; // lcdwrite 20 101
1911 HD[1910] = 32'b100111_10101_00000_00000_00001110010; // lcdwrite 21 114
1912 HD[1911] = 32'b100111_10110_00000_00000_00000100000; // lcdwrite 22 32
1913 HD[1912] = 32'b100111_10111_00000_00000_00001110010; // lcdwrite 23 114
1914 HD[1913] = 32'b100111_11000_00000_00000_00001100101; // lcdwrite 24 101
1915 HD[1914] = 32'b100111_11001_00000_00000_00001101110; // lcdwrite 25 110
1916 HD[1915] = 32'b100111_11010_00000_00000_00001101111; // lcdwrite 26 111
1917 HD[1916] = 32'b100111_11011_00000_00000_00001101101; // lcdwrite 27 109
1918 HD[1917] = 32'b100111_11100_00000_00000_00001100101; // lcdwrite 28 101
1919 HD[1918] = 32'b100111_11101_00000_00000_00001100001; // lcdwrite 29 97
1920 HD[1919] = 32'b100111_11110_00000_00000_00001100100; // lcdwrite 30 100
1921 HD[1920] = 32'b100111_11111_00000_00000_00001101111; // lcdwrite 31 111
1922 HD[1921] = 32'b001001_00000_00010_0000000000000000; // in 2
1923 HD[1922] = 32'b000101_00000_00011_0000001111101000; // li 3 1000
1924 HD[1923] = 32'b000011_10101_10101_00000000000000100; // addi $fp $fp 4
1925 HD[1924] = 32'b000010_10101_00011_00000000000000001; // sw 3 $fp 1
1926 HD[1925] = 32'b000010_10101_00010_00000000000000000; // sw 2 $fp 0
1927 HD[1926] = 32'b101000_000000000000000000001011010; // jal mod
1928 HD[1927] = 32'b000100_10101_10101_00000000000000100; // subi $fp $fp 4
1929 HD[1928] = 32'b000000_10100_00010_00000_00000_010000; // move 2 $rv
1930 HD[1929] = 32'b000010_10101_00010_00000000000000001; // sw 2 $fp 1
1931 HD[1930] = 32'b000001_10101_00010_00000000000000001; // lw 2 $fp 1
1932 HD[1931] = 32'b000101_00000_00011_00000000000000000; // li 3 0
1933 HD[1932] = 32'b000000_00010_00011_00010_00000_010010; // sgt 2 2 3
1934 HD[1933] = 32'b000000_00010_00000_00010_00000_010011; // set 2 2 $zero
1935 HD[1934] = 32'b000111_00010_00000_00000000000000100; // beq 2 $zero _ElseBegin52_
1936 HD[1935] = 32'b000101_00000_00010_00000000000000001; // li 2 1
1937 HD[1936] = 32'b000010_10101_00010_000000000000000011; // sw 2 $fp 3
1938 HD[1937] = 32'b001011_000000000000000010000101011; // jump _IfExit52_
1939 HD[1938] = 32'b000001_10101_00010_00000000000000001; // lw 2 $fp 1 / _ElseBegin52_
1940 HD[1939] = 32'b000011_10101_10101_00000000000000100; // addi $fp $fp 4
1941 HD[1940] = 32'b000010_10101_00010_00000000000000000; // sw 2 $fp 0
1942 HD[1941] = 32'b101000_00000000000000000001001100110; // jal posicao_programa
1943 HD[1942] = 32'b000100_10101_10101_00000000000000100; // subi $fp $fp 4
1944 HD[1943] = 32'b000000_10100_00010_00000_00000_010000; // move 2 $rv
1945 HD[1944] = 32'b000001_00000_00011_00000000000011001; // lw 3 $global 25
1946 HD[1945] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
1947 HD[1946] = 32'b000111_00010_00000_00000000000010101; // beq 2 $zero _ElseBegin53_
1948 HD[1947] = 32'b000101_00000_00010_0000000000000011; // li 2 3
1949 HD[1948] = 32'b000011_10101_10101_00000000000000100; // addi $fp $fp 4
1950 HD[1949] = 32'b000010_10101_00010_00000000000000000; // sw 2 $fp 0
1951 HD[1950] = 32'b101000_0000000000000000000010110011; // jal print_default_msg
1952 HD[1951] = 32'b000100_10101_10101_00000000000000100; // subi $fp $fp 4
1953 HD[1952] = 32'b000001_10101_00010_00000000000000001; // lw 2 $fp 1
1954 HD[1953] = 32'b000101_00000_00011_00000000000000000; // li 3 0
1955 HD[1954] = 32'b000101_00000_00100_0000000000001110; // li 4 14
1956 HD[1955] = 32'b000011_10101_10101_00000000000000100; // addi $fp $fp 4
1957 HD[1956] = 32'b000010_10101_00100_0000000000000010; // sw 4 $fp 2
1958 HD[1957] = 32'b000010_10101_00011_00000000000000001; // sw 3 $fp 1
1959 HD[1958] = 32'b000010_10101_00010_00000000000000000; // sw 2 $fp 0
1960 HD[1959] = 32'b101000_000000000000000000001100111; // jal print_value
1961 HD[1960] = 32'b000100_10101_10101_00000000000000100; // subi $fp $fp 4
1962 HD[1961] = 32'b000101_00000_00010_00000000000000100; // li 2 4

```

```

1963 HD[1962] = 32'b000011_10101_10101_0000000000000100; // addi $fp $fp 4
1964 HD[1963] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
1965 HD[1964] = 32'b101000_00000000000000000010110011; // jal print_default_msg
1966 HD[1965] = 32'b000100_10101_10101_0000000000000100; // subi $fp $fp 4
1967 HD[1966] = 32'b001011_000000000000000100000101011; // jump _IfExit53_
1968 HD[1967] = 32'b100111_00000_00000_00000_00000100000; // lcdwrite 0 32 / _ElseBegin53_
1969 HD[1968] = 32'b100111_00001_00000_00000_00001000100; // lcdwrite 1 68
1970 HD[1969] = 32'b100111_00010_00000_00000_00001100101; // lcdwrite 2 101
1971 HD[1970] = 32'b100111_00011_00000_00000_00001100110; // lcdwrite 3 102
1972 HD[1971] = 32'b100111_00100_00000_00000_00001101001; // lcdwrite 4 105
1973 HD[1972] = 32'b100111_00101_00000_00000_00001101110; // lcdwrite 5 110
1974 HD[1973] = 32'b100111_00110_00000_00000_00001100001; // lcdwrite 6 97
1975 HD[1974] = 32'b100111_00111_00000_00000_00000100000; // lcdwrite 7 32
1976 HD[1975] = 32'b100111_01000_00000_00000_00001110101; // lcdwrite 8 117
1977 HD[1976] = 32'b100111_01001_00000_00000_00001101101; // lcdwrite 9 109
1978 HD[1977] = 32'b100111_01010_00000_00000_00000100000; // lcdwrite 10 32
1979 HD[1978] = 32'b100111_01011_00000_00000_00001101110; // lcdwrite 11 110
1980 HD[1979] = 32'b100111_01100_00000_00000_00001101111; // lcdwrite 12 111
1981 HD[1980] = 32'b100111_01101_00000_00000_00001110110; // lcdwrite 13 118
1982 HD[1981] = 32'b100111_01110_00000_00000_00001101111; // lcdwrite 14 111
1983 HD[1982] = 32'b100111_01111_00000_00000_00000100000; // lcdwrite 15 32
1984 HD[1983] = 32'b100111_10000_00000_00000_00000100000; // lcdwrite 16 32
1985 HD[1984] = 32'b100111_10001_00000_00000_00001001001; // lcdwrite 17 73
1986 HD[1985] = 32'b100111_10010_00000_00000_00001100100; // lcdwrite 18 100
1987 HD[1986] = 32'b100111_10011_00000_00000_00001100101; // lcdwrite 19 101
1988 HD[1987] = 32'b100111_10100_00000_00000_00001101110; // lcdwrite 20 110
1989 HD[1988] = 32'b100111_10101_00000_00000_00001110100; // lcdwrite 21 116
1990 HD[1989] = 32'b100111_10110_00000_00000_00001101001; // lcdwrite 22 105
1991 HD[1990] = 32'b100111_10111_00000_00000_00001100110; // lcdwrite 23 102
1992 HD[1991] = 32'b100111_11000_00000_00000_00001101001; // lcdwrite 24 105
1993 HD[1992] = 32'b100111_11001_00000_00000_00001100011; // lcdwrite 25 99
1994 HD[1993] = 32'b100111_11010_00000_00000_00001100001; // lcdwrite 26 97
1995 HD[1994] = 32'b100111_11011_00000_00000_00001100100; // lcdwrite 27 100
1996 HD[1995] = 32'b100111_11100_00000_00000_00001101111; // lcdwrite 28 111
1997 HD[1996] = 32'b100111_11101_00000_00000_00001110010; // lcdwrite 29 114
1998 HD[1997] = 32'b100111_11110_00000_00000_00000100000; // lcdwrite 30 32
1999 HD[1998] = 32'b100111_11111_00000_00000_00000100000; // lcdwrite 31 32
2000 HD[1999] = 32'b001001_00000_00010_0000000000000000; // in 2
2001 HD[2000] = 32'b000101_00000_00011_0000001111101000; // li 3 1000
2002 HD[2001] = 32'b000011_10101_10101_0000000000000100; // addi $fp $fp 4
2003 HD[2002] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
2004 HD[2003] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2005 HD[2004] = 32'b101000_00000000000000000001011010; // jal mod
2006 HD[2005] = 32'b000100_10101_10101_0000000000000100; // subi $fp $fp 4
2007 HD[2006] = 32'b000000_10100_00010_00000_00000_010000; // move 2 $rv
2008 HD[2007] = 32'b000010_10101_00010_0000000000000010; // sw 2 $fp 2
2009 HD[2008] = 32'b000001_10101_00010_0000000000000010; // lw 2 $fp 2
2010 HD[2009] = 32'b000101_00000_00011_0000000000000000; // li 3 0
2011 HD[2010] = 32'b000000_00010_00011_00010_00000_010010; // sgt 2 2 3
2012 HD[2011] = 32'b000000_00010_00000_00010_00000_010011; // set 2 2 $zero
2013 HD[2012] = 32'b000111_00010_00000_0000000000000100; // beq 2 $zero _ElseBegin54_
2014 HD[2013] = 32'b000101_00000_00010_0000000000000001; // li 2 1
2015 HD[2014] = 32'b000010_10101_00010_0000000000000011; // sw 2 $fp 3
2016 HD[2015] = 32'b001011_000000000000000100000101011; // jump _IfExit54_
2017 HD[2016] = 32'b000001_10101_00010_0000000000000010; // lw 2 $fp 2 / _ElseBegin54_
2018 HD[2017] = 32'b000011_10101_10101_0000000000000100; // addi $fp $fp 4
2019 HD[2018] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2020 HD[2019] = 32'b101000_0000000000000000001001100110; // jal posicao_programa
2021 HD[2020] = 32'b000100_10101_10101_0000000000000100; // subi $fp $fp 4
2022 HD[2021] = 32'b000000_10100_00010_00000_00000_010000; // move 2 $rv

```

```

2023 HD[2022] = 32'b000001_00000_00011_0000000000011001; // lw 3 $global 25
2024 HD[2023] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
2025 HD[2024] = 32'b000111_00010_00000_00000000000110000; // beq 2 $zero _ElseBegin55_
2026 HD[2025] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
2027 HD[2026] = 32'b000001_10101_00011_00000000000000010; // lw 3 $fp 2
2028 HD[2027] = 32'b000011_10101_10101_00000000000000100; // addi $fp $fp 4
2029 HD[2028] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
2030 HD[2029] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2031 HD[2030] = 32'b101000_000000000000000001101100111; // jal renomear_programa
2032 HD[2031] = 32'b000100_10101_10101_00000000000000100; // subi $fp $fp 4
2033 HD[2032] = 32'b000101_00000_00010_00000000000000101; // li 2 5
2034 HD[2033] = 32'b000011_10101_10101_00000000000000100; // addi $fp $fp 4
2035 HD[2034] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2036 HD[2035] = 32'b101000_0000000000000000010110011; // jal print_default_msg
2037 HD[2036] = 32'b000100_10101_10101_00000000000000100; // subi $fp $fp 4
2038 HD[2037] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
2039 HD[2038] = 32'b000101_00000_00011_0000000000000000; // li 3 0
2040 HD[2039] = 32'b000101_00000_00100_0000000000001110; // li 4 14
2041 HD[2040] = 32'b000011_10101_10101_00000000000000100; // addi $fp $fp 4
2042 HD[2041] = 32'b000010_10101_00100_0000000000000010; // sw 4 $fp 2
2043 HD[2042] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
2044 HD[2043] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2045 HD[2044] = 32'b101000_0000000000000000001100111; // jal print_value
2046 HD[2045] = 32'b000100_10101_10101_00000000000000100; // subi $fp $fp 4
2047 HD[2046] = 32'b100111_10000_00000_00000_00001110010; // lcdwrite 16 114
2048 HD[2047] = 32'b100111_10001_00000_00000_00001100101; // lcdwrite 17 101
2049 HD[2048] = 32'b100111_10010_00000_00000_00001101110; // lcdwrite 18 110
2050 HD[2049] = 32'b100111_10011_00000_00000_00001101111; // lcdwrite 19 111
2051 HD[2050] = 32'b100111_10100_00000_00000_00001101101; // lcdwrite 20 109
2052 HD[2051] = 32'b100111_10101_00000_00000_00001100101; // lcdwrite 21 101
2053 HD[2052] = 32'b100111_10110_00000_00000_00001100001; // lcdwrite 22 97
2054 HD[2053] = 32'b100111_10111_00000_00000_00001100100; // lcdwrite 23 100
2055 HD[2054] = 32'b100111_11000_00000_00000_00001101111; // lcdwrite 24 111
2056 HD[2055] = 32'b100111_11001_00000_00000_00000100000; // lcdwrite 25 32
2057 HD[2056] = 32'b100111_11010_00000_00000_00001110000; // lcdwrite 26 112
2058 HD[2057] = 32'b100111_11011_00000_00000_00000101111; // lcdwrite 27 47
2059 HD[2058] = 32'b100111_11100_00000_00000_00000100000; // lcdwrite 28 32
2060 HD[2059] = 32'b100111_11101_00000_00000_00000100000; // lcdwrite 29 32
2061 HD[2060] = 32'b100111_11110_00000_00000_00000100000; // lcdwrite 30 32
2062 HD[2061] = 32'b100111_11111_00000_00000_00000100000; // lcdwrite 31 32
2063 HD[2062] = 32'b000001_10101_00010_0000000000000010; // lw 2 $fp 2
2064 HD[2063] = 32'b000101_00000_00011_0000000000000001; // li 3 1
2065 HD[2064] = 32'b000101_00000_00100_0000000000001110; // li 4 14
2066 HD[2065] = 32'b000011_10101_10101_00000000000000100; // addi $fp $fp 4
2067 HD[2066] = 32'b000010_10101_00100_0000000000000010; // sw 4 $fp 2
2068 HD[2067] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
2069 HD[2068] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2070 HD[2069] = 32'b101000_0000000000000000001100111; // jal print_value
2071 HD[2070] = 32'b000100_10101_10101_00000000000000100; // subi $fp $fp 4
2072 HD[2071] = 32'b001011_000000000000000100000101011; // jump _IfExit55_
2073 HD[2072] = 32'b000101_00000_00010_00000000000000101; // li 2 5 / _ElseBegin55_
2074 HD[2073] = 32'b000011_10101_10101_00000000000000100; // addi $fp $fp 4
2075 HD[2074] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2076 HD[2075] = 32'b101000_0000000000000000010110011; // jal print_default_msg
2077 HD[2076] = 32'b000100_10101_10101_00000000000000100; // subi $fp $fp 4
2078 HD[2077] = 32'b000001_10101_00010_0000000000000010; // lw 2 $fp 2
2079 HD[2078] = 32'b000101_00000_00011_0000000000000000; // li 3 0
2080 HD[2079] = 32'b000101_00000_00100_0000000000001110; // li 4 14
2081 HD[2080] = 32'b000011_10101_10101_00000000000000100; // addi $fp $fp 4
2082 HD[2081] = 32'b000010_10101_00100_0000000000000010; // sw 4 $fp 2

```



```

2083 HD[2082] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
2084 HD[2083] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2085 HD[2084] = 32'b101000_000000000000000000110011; // jal print_value
2086 HD[2085] = 32'b000100_10101_10101_0000000000000100; // subi $fp $fp 4
2087 HD[2086] = 32'b000101_00000_00010_0000000000000111; // li 2 7
2088 HD[2087] = 32'b000011_10101_10101_0000000000000100; // addi $fp $fp 4
2089 HD[2088] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2090 HD[2089] = 32'b101000_00000000000000000010110011; // jal print_default_msg
2091 HD[2090] = 32'b000100_10101_10101_0000000000000100; // subi $fp $fp 4
2092 HD[2091] = 32'b000001_10101_00010_0000000000000011; // lw 2 $fp 3 / _IfExit55_ _IfExit54_
    _IfExit53_ _IfExit52_
2093 HD[2092] = 32'b000101_00000_00011_0000000000000001; // li 3 1
2094 HD[2093] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
2095 HD[2094] = 32'b000111_00010_00000_00000000000010100; // beq 2 $zero _IfExit56_
2096 HD[2095] = 32'b000101_00000_00010_0000000000000001; // li 2 1
2097 HD[2096] = 32'b000011_10101_10101_0000000000000100; // addi $fp $fp 4
2098 HD[2097] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2099 HD[2098] = 32'b101000_00000000000000000010110011; // jal print_default_msg
2100 HD[2099] = 32'b000100_10101_10101_0000000000000100; // subi $fp $fp 4
2101 HD[2100] = 32'b000101_00000_00010_0000000000000110; // li 2 6
2102 HD[2101] = 32'b000011_10101_10101_0000000000000100; // addi $fp $fp 4
2103 HD[2102] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2104 HD[2103] = 32'b101000_00000000000000000010110011; // jal print_default_msg
2105 HD[2104] = 32'b000100_10101_10101_0000000000000100; // subi $fp $fp 4
2106 HD[2105] = 32'b000101_00000_00010_0000000000000100; // li 2 4
2107 HD[2106] = 32'b000101_00000_00011_0000000000000000; // li 3 0
2108 HD[2107] = 32'b000011_10101_10101_0000000000000100; // addi $fp $fp 4
2109 HD[2108] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
2110 HD[2109] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2111 HD[2110] = 32'b101000_0000000000000000000100001; // jal sleep
2112 HD[2111] = 32'b000100_10101_10101_0000000000000100; // subi $fp $fp 4
2113 HD[2112] = 32'b010000_10110_10111_0000000000000000; // pop $ra
2114 HD[2113] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
2115 HD[2114] = 32'b000101_00000_00010_0000000000000000; // li 2 0 / _IfExit56_
2116 HD[2115] = 32'b001010_00010_00000_0000000000000000; // out 2
2117 HD[2116] = 32'b010000_10110_10111_0000000000000000; // pop $ra
2118 HD[2117] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
2119 HD[2118] = 32'b001111_10110_10111_0000000000000000; // push $ra / menu_opt3_deletar_prog
2120 HD[2119] = 32'b000101_00000_00010_0000000000000010; // li 2 2
2121 HD[2120] = 32'b000011_10101_10101_0000000000000010; // addi $fp $fp 2
2122 HD[2121] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2123 HD[2122] = 32'b101000_00000000000000000010110011; // jal print_default_msg
2124 HD[2123] = 32'b000100_10101_10101_0000000000000010; // subi $fp $fp 2
2125 HD[2124] = 32'b100111_10000_00000_00000_00000100000; // lcdwrite 16 32
2126 HD[2125] = 32'b100111_10001_00000_00000_00001100001; // lcdwrite 17 97
2127 HD[2126] = 32'b100111_10010_00000_00000_00000100000; // lcdwrite 18 32
2128 HD[2127] = 32'b100111_10011_00000_00000_00001110011; // lcdwrite 19 115
2129 HD[2128] = 32'b100111_10100_00000_00000_00001100101; // lcdwrite 20 101
2130 HD[2129] = 32'b100111_10101_00000_00000_00001110010; // lcdwrite 21 114
2131 HD[2130] = 32'b100111_10110_00000_00000_00000100000; // lcdwrite 22 32
2132 HD[2131] = 32'b100111_10111_00000_00000_00001100100; // lcdwrite 23 100
2133 HD[2132] = 32'b100111_11000_00000_00000_00001100101; // lcdwrite 24 101
2134 HD[2133] = 32'b100111_11001_00000_00000_00001101100; // lcdwrite 25 108
2135 HD[2134] = 32'b100111_11010_00000_00000_00001100101; // lcdwrite 26 101
2136 HD[2135] = 32'b100111_11011_00000_00000_00001110100; // lcdwrite 27 116
2137 HD[2136] = 32'b100111_11100_00000_00000_00001100001; // lcdwrite 28 97
2138 HD[2137] = 32'b100111_11101_00000_00000_00001100100; // lcdwrite 29 100
2139 HD[2138] = 32'b100111_11110_00000_00000_00001101111; // lcdwrite 30 111
2140 HD[2139] = 32'b100111_11111_00000_00000_00000100000; // lcdwrite 31 32
2141 HD[2140] = 32'b001001_00000_00010_0000000000000000; // in 2

```

```

2142 HD[2141] = 32'b000101_00000_00011_0000001111101000; // li 3 1000
2143 HD[2142] = 32'b000011_10101_10101_0000000000000010; // addi $fp $fp 2
2144 HD[2143] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
2145 HD[2144] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2146 HD[2145] = 32'b101000_0000000000000000001011010; // jal mod
2147 HD[2146] = 32'b000100_10101_10101_0000000000000010; // subi $fp $fp 2
2148 HD[2147] = 32'b000000_10100_00010_00000_00000_010000; // move 2 $rv
2149 HD[2148] = 32'b000010_10101_00010_0000000000000001; // sw 2 $fp 1
2150 HD[2149] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
2151 HD[2150] = 32'b000011_10101_10101_0000000000000010; // addi $fp $fp 2
2152 HD[2151] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2153 HD[2152] = 32'b101000_00000000000000001101001101; // jal deleta_programa
2154 HD[2153] = 32'b000100_10101_10101_0000000000000010; // subi $fp $fp 2
2155 HD[2154] = 32'b000000_10100_00010_00000_00000_010000; // move 2 $rv
2156 HD[2155] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2157 HD[2156] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
2158 HD[2157] = 32'b000101_00000_00011_0000000000000000; // li 3 0
2159 HD[2158] = 32'b000000_00010_00011_00010_00000_010010; // sgt 2 2 3
2160 HD[2159] = 32'b000000_00010_00000_00010_00000_010011; // set 2 2 $zero
2161 HD[2160] = 32'b000111_00010_00000_0000000000010101; // beq 2 $zero _ElseBegin57_
2162 HD[2161] = 32'b000101_00000_00010_0000000000000001; // li 2 1
2163 HD[2162] = 32'b000011_10101_10101_0000000000000010; // addi $fp $fp 2
2164 HD[2163] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2165 HD[2164] = 32'b101000_00000000000000000010110011; // jal print_default_msg
2166 HD[2165] = 32'b000100_10101_10101_0000000000000010; // subi $fp $fp 2
2167 HD[2166] = 32'b000101_00000_00010_0000000000000110; // li 2 6
2168 HD[2167] = 32'b000011_10101_10101_0000000000000010; // addi $fp $fp 2
2169 HD[2168] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2170 HD[2169] = 32'b101000_00000000000000000010110011; // jal print_default_msg
2171 HD[2170] = 32'b000100_10101_10101_0000000000000010; // subi $fp $fp 2
2172 HD[2171] = 32'b000101_00000_00010_0000000000000100; // li 2 4
2173 HD[2172] = 32'b000101_00000_00011_0000000000000000; // li 3 0
2174 HD[2173] = 32'b000011_10101_10101_0000000000000010; // addi $fp $fp 2
2175 HD[2174] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
2176 HD[2175] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2177 HD[2176] = 32'b101000_0000000000000000000100001; // jal sleep
2178 HD[2177] = 32'b000100_10101_10101_0000000000000010; // subi $fp $fp 2
2179 HD[2178] = 32'b010000_10110_10111_0000000000000000; // pop $ra
2180 HD[2179] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
2181 HD[2180] = 32'b001011_00000000000000100010111011; // jump _IfExit57_
2182 HD[2181] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0 / _ElseBegin57_
2183 HD[2182] = 32'b000001_00000_00011_0000000000011001; // lw 3 $global 25
2184 HD[2183] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
2185 HD[2184] = 32'b000011_00010_00000_0000000000010101; // beq 2 $zero _ElseBegin58_
2186 HD[2185] = 32'b000101_00000_00010_000000000000011; // li 2 3
2187 HD[2186] = 32'b000011_10101_10101_0000000000000010; // addi $fp $fp 2
2188 HD[2187] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2189 HD[2188] = 32'b101000_00000000000000000010110011; // jal print_default_msg
2190 HD[2189] = 32'b000100_10101_10101_0000000000000010; // subi $fp $fp 2
2191 HD[2190] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
2192 HD[2191] = 32'b000101_00000_00011_0000000000000000; // li 3 0
2193 HD[2192] = 32'b000101_00000_00100_0000000000001110; // li 4 14
2194 HD[2193] = 32'b000011_10101_10101_0000000000000010; // addi $fp $fp 2
2195 HD[2194] = 32'b000010_10101_00100_0000000000000010; // sw 4 $fp 2
2196 HD[2195] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
2197 HD[2196] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2198 HD[2197] = 32'b101000_000000000000000000110011; // jal print_value
2199 HD[2198] = 32'b000100_10101_10101_0000000000000010; // subi $fp $fp 2
2200 HD[2199] = 32'b000101_00000_00010_0000000000000100; // li 2 4
2201 HD[2200] = 32'b000011_10101_10101_0000000000000010; // addi $fp $fp 2

```



```

2202 HD[2201] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2203 HD[2202] = 32'b101000_00000000000000000010110011; // jal print_default_msg
2204 HD[2203] = 32'b000100_10101_10101_0000000000000000; // subi $fp $fp 2
2205 HD[2204] = 32'b001011_000000000000000100010111011; // jump _IfExit58_
2206 HD[2205] = 32'b000101_00000_00010_00000000000000101; // li 2 5 / _ElseBegin58_
2207 HD[2206] = 32'b000011_10101_10101_0000000000000000; // addi $fp $fp 2
2208 HD[2207] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2209 HD[2208] = 32'b101000_00000000000000000010110011; // jal print_default_msg
2210 HD[2209] = 32'b000100_10101_10101_0000000000000000; // subi $fp $fp 2
2211 HD[2210] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 1
2212 HD[2211] = 32'b000101_00000_00011_0000000000000000; // li 3 0
2213 HD[2212] = 32'b000101_00000_00100_0000000000001110; // li 4 14
2214 HD[2213] = 32'b000011_10101_10101_0000000000000000; // addi $fp $fp 2
2215 HD[2214] = 32'b000010_10101_00100_0000000000000000; // sw 4 $fp 2
2216 HD[2215] = 32'b000010_10101_00011_0000000000000000; // sw 3 $fp 1
2217 HD[2216] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2218 HD[2217] = 32'b101000_000000000000000000110011; // jal print_value
2219 HD[2218] = 32'b000100_10101_10101_0000000000000000; // subi $fp $fp 2
2220 HD[2219] = 32'b100111_10000_00000_00000_00000100000; // lcdwrite 16 32
2221 HD[2220] = 32'b100111_10001_00000_00000_00000100000; // lcdwrite 17 32
2222 HD[2221] = 32'b100111_10010_00000_00000_00001100110; // lcdwrite 18 102
2223 HD[2222] = 32'b100111_10011_00000_00000_00001101111; // lcdwrite 19 111
2224 HD[2223] = 32'b100111_10100_00000_00000_00001101001; // lcdwrite 20 105
2225 HD[2224] = 32'b100111_10101_00000_00000_00000100000; // lcdwrite 21 32
2226 HD[2225] = 32'b100111_10110_00000_00000_00001100100; // lcdwrite 22 100
2227 HD[2226] = 32'b100111_10111_00000_00000_00001100101; // lcdwrite 23 101
2228 HD[2227] = 32'b100111_11000_00000_00000_00001101100; // lcdwrite 24 108
2229 HD[2228] = 32'b100111_11001_00000_00000_00001100101; // lcdwrite 25 101
2230 HD[2229] = 32'b100111_11010_00000_00000_00001110100; // lcdwrite 26 116
2231 HD[2230] = 32'b100111_11011_00000_00000_00001100001; // lcdwrite 27 97
2232 HD[2231] = 32'b100111_11100_00000_00000_00001100100; // lcdwrite 28 100
2233 HD[2232] = 32'b100111_11101_00000_00000_00001101111; // lcdwrite 29 111
2234 HD[2233] = 32'b100111_11110_00000_00000_00000100000; // lcdwrite 30 32
2235 HD[2234] = 32'b100111_11111_00000_00000_00000100000; // lcdwrite 31 32
2236 HD[2235] = 32'b000101_00000_00010_0000000000000000; // li 2 0 / _IfExit58_ _IfExit57_
2237 HD[2236] = 32'b001010_00010_00000_0000000000000000; // out 2
2238 HD[2237] = 32'b010000_10110_10111_0000000000000000; // pop $ra
2239 HD[2238] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
2240 HD[2239] = 32'b001111_10110_10111_0000000000000000; // push $ra / menu_opt4_listar_prog
2241 HD[2240] = 32'b000101_00000_00010_0000000000000000; // li 2 0
2242 HD[2241] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2243 HD[2242] = 32'b000010_10101_00010_00000000000010001; // sw 2 $fp 17
2244 HD[2243] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0 / _WhileBegin15_
2245 HD[2244] = 32'b000001_00000_00011_0000000000000000; // lw 3 $global 0
2246 HD[2245] = 32'b000000_00010_00011_00010_00000_010001; // slt 2 2 3
2247 HD[2246] = 32'b000111_00010_00000_00000000000010111; // beq 2 $zero _WhileExit15_
2248 HD[2247] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
2249 HD[2248] = 32'b000011_10101_10101_00000000000010010; // addi $fp $fp 18
2250 HD[2249] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2251 HD[2250] = 32'b101000_0000000000000000001110101001; // jal existe_programa_posicao
2252 HD[2251] = 32'b000100_10101_10101_00000000000010010; // subi $fp $fp 18
2253 HD[2252] = 32'b000000_10100_00010_00000_00000_010000; // move 2 $rv
2254 HD[2253] = 32'b000001_10101_00011_0000000000000000; // lw 3 $fp 0
2255 HD[2254] = 32'b000000_00011_10101_00011_00000_000101; // add 3 3 $fp
2256 HD[2255] = 32'b000010_00011_00010_0000000000000000; // sw 2 3 1
2257 HD[2256] = 32'b000010_10101_00010_00000000000010000; // sw 2 $fp 16
2258 HD[2257] = 32'b000001_10101_00010_00000000000010000; // lw 2 $fp 16
2259 HD[2258] = 32'b000001_00000_00011_00000000000011001; // lw 3 $global 25
2260 HD[2259] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
2261 HD[2260] = 32'b000000_00010_00000_00010_00000_010011; // set 2 2 $zero

```

```

2262 HD[2261] = 32'b000111_00010_00000_0000000000000100; // beq 2 $zero _IfExit59_
2263 HD[2262] = 32'b000001_10101_00010_0000000000010001; // lw 2 $fp 17
2264 HD[2263] = 32'b000011_00010_00010_0000000000000001; // addi 2 2 1
2265 HD[2264] = 32'b000010_10101_00010_0000000000010001; // sw 2 $fp 17
2266 HD[2265] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0 / _IfExit59_
2267 HD[2266] = 32'b000011_00010_00010_0000000000000001; // addi 2 2 1
2268 HD[2267] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2269 HD[2268] = 32'b001011_000000000000000100011000011; // jump _WhileBegin15_
2270 HD[2269] = 32'b100110_000000000000000000000000; // lcdclean / _WhileExit15_
2271 HD[2270] = 32'b100111_00001_00000_00000_00001010001; // lcdwrite 1 81
2272 HD[2271] = 32'b100111_00010_00000_00000_00001110100; // lcdwrite 2 116
2273 HD[2272] = 32'b100111_00011_00000_00000_00001100100; // lcdwrite 3 100
2274 HD[2273] = 32'b100111_00101_00000_00000_00001010000; // lcdwrite 5 80
2275 HD[2274] = 32'b100111_00110_00000_00000_00001110010; // lcdwrite 6 114
2276 HD[2275] = 32'b100111_00111_00000_00000_00001101111; // lcdwrite 7 111
2277 HD[2276] = 32'b100111_01000_00000_00000_00001100111; // lcdwrite 8 103
2278 HD[2277] = 32'b100111_01001_00000_00000_00001110011; // lcdwrite 9 115
2279 HD[2278] = 32'b100111_01010_00000_00000_00000111010; // lcdwrite 10 58
2280 HD[2279] = 32'b000001_10101_00010_0000000000010001; // lw 2 $fp 17
2281 HD[2280] = 32'b000101_00000_00011_0000000000000000; // li 3 0
2282 HD[2281] = 32'b000101_00000_00100_0000000000001110; // li 4 14
2283 HD[2282] = 32'b000011_10101_10101_0000000000010010; // addi $fp $fp 18
2284 HD[2283] = 32'b000010_10101_00100_0000000000000010; // sw 4 $fp 2
2285 HD[2284] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
2286 HD[2285] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2287 HD[2286] = 32'b101000_00000000000000000001100111; // jal print_value
2288 HD[2287] = 32'b000100_10101_10101_0000000000010010; // subi $fp $fp 18
2289 HD[2288] = 32'b100111_10011_00000_00000_00001101111; // lcdwrite 19 111
2290 HD[2289] = 32'b100111_10101_00000_00000_00001010000; // lcdwrite 21 80
2291 HD[2290] = 32'b100111_10110_00000_00000_00001110010; // lcdwrite 22 114
2292 HD[2291] = 32'b100111_10111_00000_00000_00001101111; // lcdwrite 23 111
2293 HD[2292] = 32'b100111_11000_00000_00000_00001100111; // lcdwrite 24 103
2294 HD[2293] = 32'b100111_11001_00000_00000_00000111010; // lcdwrite 25 58
2295 HD[2294] = 32'b000101_00000_00010_0000000000000000; // li 2 0
2296 HD[2295] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2297 HD[2296] = 32'b000010_10101_00010_0000000000010001; // sw 2 $fp 17
2298 HD[2297] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0 / _WhileBegin16_
2299 HD[2298] = 32'b000001_00000_00011_0000000000000000; // lw 3 $global 0
2300 HD[2299] = 32'b000000_00010_00011_00010_00000_010001; // slt 2 2 3
2301 HD[2300] = 32'b000111_00010_00000_00000000000100101; // beq 2 $zero _WhileExit16_
2302 HD[2301] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
2303 HD[2302] = 32'b000000_00010_10101_00010_00000_000101; // add 2 2 $fp
2304 HD[2303] = 32'b000001_00010_00010_0000000000000001; // lw 2 2 1
2305 HD[2304] = 32'b000010_10101_00010_0000000000010000; // sw 2 $fp 16
2306 HD[2305] = 32'b000001_10101_00010_0000000000010000; // lw 2 $fp 16
2307 HD[2306] = 32'b000001_00000_00011_0000000000011001; // lw 3 $global 25
2308 HD[2307] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
2309 HD[2308] = 32'b000000_00010_00000_00010_00000_010011; // set 2 2 $zero
2310 HD[2309] = 32'b000111_00010_00000_0000000000011000; // beq 2 $zero _IfExit60_
2311 HD[2310] = 32'b000001_10101_00010_0000000000010001; // lw 2 $fp 17
2312 HD[2311] = 32'b000011_00010_00010_0000000000000001; // addi 2 2 1
2313 HD[2312] = 32'b000010_10101_00010_0000000000010001; // sw 2 $fp 17
2314 HD[2313] = 32'b000001_10101_00010_0000000000010001; // lw 2 $fp 17
2315 HD[2314] = 32'b000101_00000_00011_0000000000000001; // li 3 1
2316 HD[2315] = 32'b000101_00000_00100_0000000000000010; // li 4 2
2317 HD[2316] = 32'b000011_10101_10101_0000000000010010; // addi $fp $fp 18
2318 HD[2317] = 32'b000010_10101_00100_0000000000000010; // sw 4 $fp 2
2319 HD[2318] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
2320 HD[2319] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2321 HD[2320] = 32'b101000_00000000000000000001100111; // jal print_value

```

```

2322 HD[2321] = 32'b000100_10101_10101_00000000000010010; // subi $fp $fp 18
2323 HD[2322] = 32'b000001_10101_00010_00000000000010000; // lw 2 $fp 16
2324 HD[2323] = 32'b000101_00000_00011_00000000000000001; // li 3 1
2325 HD[2324] = 32'b000101_00000_00100_0000000000001110; // li 4 14
2326 HD[2325] = 32'b000011_10101_10101_00000000000010010; // addi $fp $fp 18
2327 HD[2326] = 32'b000010_10101_00100_00000000000000010; // sw 4 $fp 2
2328 HD[2327] = 32'b000010_10101_00011_00000000000000001; // sw 3 $fp 1
2329 HD[2328] = 32'b000010_10101_00010_00000000000000000; // sw 2 $fp 0
2330 HD[2329] = 32'b101000_00000000000000000001100111; // jal print_value
2331 HD[2330] = 32'b000100_10101_10101_00000000000010010; // subi $fp $fp 18
2332 HD[2331] = 32'b000101_00000_00010_00000000000000000; // li 2 0
2333 HD[2332] = 32'b001010_00010_00000_00000000000000000; // out 2
2334 HD[2333] = 32'b000001_10101_00010_00000000000000000; // lw 2 $fp 0 / _IfExit60_
2335 HD[2334] = 32'b000011_00010_00010_00000000000000001; // addi 2 2 1
2336 HD[2335] = 32'b000010_10101_00010_00000000000000000; // sw 2 $fp 0
2337 HD[2336] = 32'b001011_000000000000000100011111001; // jump _WhileBegin16_
2338 HD[2337] = 32'b010000_10110_10111_00000000000000000; // pop $ra / _WhileExit16_
2339 HD[2338] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
2340 HD[2339] = 32'b001111_10110_10111_00000000000000000; // push $ra / menu_opt5_reiniciar
2341 HD[2340] = 32'b100110_000000000000000000000000000; // lcdclean
2342 HD[2341] = 32'b100111_00010_00000_00000_00001010010; // lcdwrite 2 82
2343 HD[2342] = 32'b100111_00011_00000_00000_00001100101; // lcdwrite 3 101
2344 HD[2343] = 32'b100111_00100_00000_00000_00001101001; // lcdwrite 4 105
2345 HD[2344] = 32'b100111_00101_00000_00000_00001101110; // lcdwrite 5 110
2346 HD[2345] = 32'b100111_00110_00000_00000_00001101001; // lcdwrite 6 105
2347 HD[2346] = 32'b100111_00111_00000_00000_00001100011; // lcdwrite 7 99
2348 HD[2347] = 32'b100111_01000_00000_00000_00001101001; // lcdwrite 8 105
2349 HD[2348] = 32'b100111_01001_00000_00000_00001100001; // lcdwrite 9 97
2350 HD[2349] = 32'b100111_01010_00000_00000_00001101110; // lcdwrite 10 110
2351 HD[2350] = 32'b100111_01011_00000_00000_00001100100; // lcdwrite 11 100
2352 HD[2351] = 32'b100111_01100_00000_00000_00001101111; // lcdwrite 12 111
2353 HD[2352] = 32'b000101_00000_00010_00000000000000001; // li 2 1
2354 HD[2353] = 32'b000101_00000_00011_00000000000000000; // li 3 0
2355 HD[2354] = 32'b000010_10101_00011_00000000000000001; // sw 3 $fp 1
2356 HD[2355] = 32'b000010_10101_00010_00000000000000000; // sw 2 $fp 0
2357 HD[2356] = 32'b101000_0000000000000000000100001; // jal sleep
2358 HD[2357] = 32'b100111_10000_00000_00000_00000111110; // lcdwrite 16 62
2359 HD[2358] = 32'b100111_11111_00000_00000_00000111100; // lcdwrite 31 60
2360 HD[2359] = 32'b000101_00000_00010_00000000000000001; // li 2 1
2361 HD[2360] = 32'b000101_00000_00011_00000000000000000; // li 3 0
2362 HD[2361] = 32'b000010_10101_00011_00000000000000001; // sw 3 $fp 1
2363 HD[2362] = 32'b000010_10101_00010_00000000000000000; // sw 2 $fp 0
2364 HD[2363] = 32'b101000_0000000000000000000100001; // jal sleep
2365 HD[2364] = 32'b100111_10010_00000_00000_00000111110; // lcdwrite 18 62
2366 HD[2365] = 32'b100111_11101_00000_00000_00000111100; // lcdwrite 29 60
2367 HD[2366] = 32'b000101_00000_00010_00000000000000001; // li 2 1
2368 HD[2367] = 32'b000101_00000_00011_00000000000000000; // li 3 0
2369 HD[2368] = 32'b000010_10101_00011_00000000000000001; // sw 3 $fp 1
2370 HD[2369] = 32'b000010_10101_00010_00000000000000000; // sw 2 $fp 0
2371 HD[2370] = 32'b101000_0000000000000000000100001; // jal sleep
2372 HD[2371] = 32'b100111_10100_00000_00000_00000111110; // lcdwrite 20 62
2373 HD[2372] = 32'b100111_11011_00000_00000_00000111100; // lcdwrite 27 60
2374 HD[2373] = 32'b000101_00000_00010_00000000000000001; // li 2 1
2375 HD[2374] = 32'b000101_00000_00011_00000000000000000; // li 3 0
2376 HD[2375] = 32'b000010_10101_00011_00000000000000001; // sw 3 $fp 1
2377 HD[2376] = 32'b000010_10101_00010_00000000000000000; // sw 2 $fp 0
2378 HD[2377] = 32'b101000_0000000000000000000100001; // jal sleep
2379 HD[2378] = 32'b100111_10110_00000_00000_00000111110; // lcdwrite 22 62
2380 HD[2379] = 32'b100111_11001_00000_00000_00000111100; // lcdwrite 25 60
2381 HD[2380] = 32'b000101_00000_00010_00000000000000001; // li 2 1

```

```
2382 HD[2381] = 32'b000101_00000_00011_0000000000000000; // li 3 0
2383 HD[2382] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
2384 HD[2383] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2385 HD[2384] = 32'b101000_0000000000000000000100001; // jal sleep
2386 HD[2385] = 32'b100110_000000000000000000000000; // lcdclean
2387 HD[2386] = 32'b000101_00000_00010_0000000000000001; // li 2 1
2388 HD[2387] = 32'b000101_00000_00011_0000000000000000; // li 3 0
2389 HD[2388] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
2390 HD[2389] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2391 HD[2390] = 32'b101000_0000000000000000000100001; // jal sleep
2392 HD[2391] = 32'b100001_000000000000000000000000; // upbios
2393 HD[2392] = 32'b010000_10110_10111_0000000000000000; // pop $ra
2394 HD[2393] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
2395 HD[2394] = 32'b001111_10110_10111_0000000000000000; // push $ra / menu_opt6_desligar
2396 HD[2395] = 32'b100110_000000000000000000000000; // lcdclean
2397 HD[2396] = 32'b100111_00011_00000_00000_00001000100; // lcdwrite 3 68
2398 HD[2397] = 32'b100111_00100_00000_00000_00001100101; // lcdwrite 4 101
2399 HD[2398] = 32'b100111_00101_00000_00000_00001110011; // lcdwrite 5 115
2400 HD[2399] = 32'b100111_00110_00000_00000_00001101100; // lcdwrite 6 108
2401 HD[2400] = 32'b100111_00111_00000_00000_00001101001; // lcdwrite 7 105
2402 HD[2401] = 32'b100111_01000_00000_00000_00001100111; // lcdwrite 8 103
2403 HD[2402] = 32'b100111_01001_00000_00000_00001100001; // lcdwrite 9 97
2404 HD[2403] = 32'b100111_01010_00000_00000_00001101110; // lcdwrite 10 110
2405 HD[2404] = 32'b100111_01011_00000_00000_00001100100; // lcdwrite 11 100
2406 HD[2405] = 32'b100111_01100_00000_00000_00001101111; // lcdwrite 12 111
2407 HD[2406] = 32'b000101_00000_00010_0000000000000001; // li 2 1
2408 HD[2407] = 32'b000101_00000_00011_0000000000000000; // li 3 0
2409 HD[2408] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
2410 HD[2409] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2411 HD[2410] = 32'b101000_0000000000000000000100001; // jal sleep
2412 HD[2411] = 32'b100111_10000_00000_00000_00000111110; // lcdwrite 16 62
2413 HD[2412] = 32'b100111_11111_00000_00000_00000111100; // lcdwrite 31 60
2414 HD[2413] = 32'b000101_00000_00010_0000000000000001; // li 2 1
2415 HD[2414] = 32'b000101_00000_00011_0000000000000000; // li 3 0
2416 HD[2415] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
2417 HD[2416] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2418 HD[2417] = 32'b101000_0000000000000000000100001; // jal sleep
2419 HD[2418] = 32'b100111_10010_00000_00000_00000111110; // lcdwrite 18 62
2420 HD[2419] = 32'b100111_11101_00000_00000_00000111100; // lcdwrite 29 60
2421 HD[2420] = 32'b000101_00000_00010_0000000000000001; // li 2 1
2422 HD[2421] = 32'b000101_00000_00011_0000000000000000; // li 3 0
2423 HD[2422] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
2424 HD[2423] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2425 HD[2424] = 32'b101000_0000000000000000000100001; // jal sleep
2426 HD[2425] = 32'b100111_10100_00000_00000_00000111110; // lcdwrite 20 62
2427 HD[2426] = 32'b100111_11011_00000_00000_00000111100; // lcdwrite 27 60
2428 HD[2427] = 32'b000101_00000_00010_0000000000000001; // li 2 1
2429 HD[2428] = 32'b000101_00000_00011_0000000000000000; // li 3 0
2430 HD[2429] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
2431 HD[2430] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2432 HD[2431] = 32'b101000_0000000000000000000100001; // jal sleep
2433 HD[2432] = 32'b100111_10110_00000_00000_00000111110; // lcdwrite 22 62
2434 HD[2433] = 32'b100111_11001_00000_00000_00000111100; // lcdwrite 25 60
2435 HD[2434] = 32'b000101_00000_00010_0000000000000001; // li 2 1
2436 HD[2435] = 32'b000101_00000_00011_0000000000000000; // li 3 0
2437 HD[2436] = 32'b000010_10101_00011_0000000000000001; // sw 3 $fp 1
2438 HD[2437] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2439 HD[2438] = 32'b101000_0000000000000000000100001; // jal sleep
2440 HD[2439] = 32'b100110_000000000000000000000000; // lcdclean
2441 HD[2440] = 32'b010000_10110_10111_0000000000000000; // pop $ra
```

```

2442 HD[2441] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
2443 HD[2442] = 32'b001111_10110_10111_0000000000000000; // push $ra / menu_opt7_help
2444 HD[2443] = 32'b100111_00000_00000_00000_00000100000; // lcdwrite 0 32
2445 HD[2444] = 32'b100111_00001_00000_00000_00001001001; // lcdwrite 1 73
2446 HD[2445] = 32'b100111_00010_00000_00000_00001101110; // lcdwrite 2 110
2447 HD[2446] = 32'b100111_00011_00000_00000_00001110011; // lcdwrite 3 115
2448 HD[2447] = 32'b100111_00100_00000_00000_00001101001; // lcdwrite 4 105
2449 HD[2448] = 32'b100111_00101_00000_00000_00001110010; // lcdwrite 5 114
2450 HD[2449] = 32'b100111_00110_00000_00000_00001100001; // lcdwrite 6 97
2451 HD[2450] = 32'b100111_00111_00000_00000_00000100000; // lcdwrite 7 32
2452 HD[2451] = 32'b100111_01000_00000_00000_00000101000; // lcdwrite 8 40
2453 HD[2452] = 32'b100111_01001_00000_00000_00000110001; // lcdwrite 9 49
2454 HD[2453] = 32'b100111_01010_00000_00000_00000101001; // lcdwrite 10 41
2455 HD[2454] = 32'b100111_01011_00000_00000_00000100000; // lcdwrite 11 32
2456 HD[2455] = 32'b100111_01100_00000_00000_00001110000; // lcdwrite 12 112
2457 HD[2456] = 32'b100111_01101_00000_00000_00001100001; // lcdwrite 13 97
2458 HD[2457] = 32'b100111_01110_00000_00000_00001110010; // lcdwrite 14 114
2459 HD[2458] = 32'b100111_01111_00000_00000_00001100001; // lcdwrite 15 97
2460 HD[2459] = 32'b100111_10000_00000_00000_00000100000; // lcdwrite 16 32
2461 HD[2460] = 32'b100111_10001_00000_00000_00001101001; // lcdwrite 17 105
2462 HD[2461] = 32'b100111_10010_00000_00000_00001110010; // lcdwrite 18 114
2463 HD[2462] = 32'b100111_10011_00000_00000_00000100000; // lcdwrite 19 32
2464 HD[2463] = 32'b100111_10100_00000_00000_00001110000; // lcdwrite 20 112
2465 HD[2464] = 32'b100111_10101_00000_00000_00001100001; // lcdwrite 21 97
2466 HD[2465] = 32'b100111_10110_00000_00000_00001110010; // lcdwrite 22 114
2467 HD[2466] = 32'b100111_10111_00000_00000_00001100001; // lcdwrite 23 97
2468 HD[2467] = 32'b100111_11000_00000_00000_00000100000; // lcdwrite 24 32
2469 HD[2468] = 32'b100111_11001_00000_00000_00001101111; // lcdwrite 25 111
2470 HD[2469] = 32'b100111_11010_00000_00000_00000100000; // lcdwrite 26 32
2471 HD[2470] = 32'b100111_11011_00000_00000_00001110000; // lcdwrite 27 112
2472 HD[2471] = 32'b100111_11100_00000_00000_00001110010; // lcdwrite 28 114
2473 HD[2472] = 32'b100111_11101_00000_00000_00001101111; // lcdwrite 29 111
2474 HD[2473] = 32'b100111_11110_00000_00000_00001111000; // lcdwrite 30 120
2475 HD[2474] = 32'b100111_11111_00000_00000_00000101110; // lcdwrite 31 46
2476 HD[2475] = 32'b001001_00000_00010_0000000000000000; // in 2
2477 HD[2476] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2478 HD[2477] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0 / _WhileBegin17_
2479 HD[2478] = 32'b000101_00000_00011_0000000000000001; // li 3 1
2480 HD[2479] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
2481 HD[2480] = 32'b000000_00010_00000_00010_00000_010011; // set 2 2 $zero
2482 HD[2481] = 32'b000111_00010_00000_00000000000000100; // beq 2 $zero _WhileExit17_
2483 HD[2482] = 32'b001001_00000_00010_0000000000000000; // in 2
2484 HD[2483] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2485 HD[2484] = 32'b000101_0000000000000000100110101101; // jump _WhileBegin17_
2486 HD[2485] = 32'b100111_01001_00000_00000_00000110010; // lcdwrite 9 50 / _WhileExit17_
2487 HD[2486] = 32'b100111_11011_00000_00000_00001100001; // lcdwrite 27 97
2488 HD[2487] = 32'b100111_11100_00000_00000_00001101110; // lcdwrite 28 110
2489 HD[2488] = 32'b100111_11101_00000_00000_00001110100; // lcdwrite 29 116
2490 HD[2489] = 32'b100111_11110_00000_00000_00000101110; // lcdwrite 30 46
2491 HD[2490] = 32'b100111_11111_00000_00000_00000100000; // lcdwrite 31 32
2492 HD[2491] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0 / _WhileBegin18_
2493 HD[2492] = 32'b000101_00000_00011_00000000000000010; // li 3 2
2494 HD[2493] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
2495 HD[2494] = 32'b000000_00010_00000_00010_00000_010011; // set 2 2 $zero
2496 HD[2495] = 32'b000111_00010_00000_00000000000000100; // beq 2 $zero _WhileExit18_
2497 HD[2496] = 32'b001001_00000_00010_0000000000000000; // in 2
2498 HD[2497] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2499 HD[2498] = 32'b001011_0000000000000000100110111011; // jump _WhileBegin18_
2500 HD[2499] = 32'b100111_01001_00000_00000_00000110011; // lcdwrite 9 51 / _WhileExit18_
2501 HD[2500] = 32'b100111_10000_00000_00000_00000100000; // lcdwrite 16 32

```

```

2502 HD[2501] = 32'b100111_10001_00000_00000_00001100011; // lcdwrite 17 99
2503 HD[2502] = 32'b100111_10010_00000_00000_00001101111; // lcdwrite 18 111
2504 HD[2503] = 32'b100111_10011_00000_00000_00001101110; // lcdwrite 19 110
2505 HD[2504] = 32'b100111_10100_00000_00000_00001100110; // lcdwrite 20 102
2506 HD[2505] = 32'b100111_10101_00000_00000_00001101001; // lcdwrite 21 105
2507 HD[2506] = 32'b100111_10110_00000_00000_00001110010; // lcdwrite 22 114
2508 HD[2507] = 32'b100111_10111_00000_00000_00001101101; // lcdwrite 23 109
2509 HD[2508] = 32'b100111_11000_00000_00000_00001100001; // lcdwrite 24 97
2510 HD[2509] = 32'b100111_11001_00000_00000_00001110010; // lcdwrite 25 114
2511 HD[2510] = 32'b100111_11010_00000_00000_00000100000; // lcdwrite 26 32
2512 HD[2511] = 32'b100111_11011_00000_00000_00001101111; // lcdwrite 27 111
2513 HD[2512] = 32'b100111_11100_00000_00000_00001110000; // lcdwrite 28 112
2514 HD[2513] = 32'b100111_11101_00000_00000_00001100011; // lcdwrite 29 99
2515 HD[2514] = 32'b100111_11110_00000_00000_00001100001; // lcdwrite 30 97
2516 HD[2515] = 32'b100111_11111_00000_00000_00001101111; // lcdwrite 31 111
2517 HD[2516] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0 / _WhileBegin19_
2518 HD[2517] = 32'b000101_00000_00011_0000000000000001; // li 3 3
2519 HD[2518] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
2520 HD[2519] = 32'b000000_00010_00000_00010_00000_010011; // set 2 2 $zero
2521 HD[2520] = 32'b000111_00010_00000_0000000000000100; // beq 2 $zero _WhileExit19_
2522 HD[2521] = 32'b001001_00000_00010_0000000000000000; // in 2
2523 HD[2522] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2524 HD[2523] = 32'b001011_000000000000000100111010100; // jump _WhileBegin19_
2525 HD[2524] = 32'b010000_10110_10111_0000000000000000; // pop $ra / _WhileExit19_
2526 HD[2525] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
2527 HD[2526] = 32'b001111_10110_10111_0000000000000000; // push $ra / print_menu
2528 HD[2527] = 32'b100111_00000_00000_00000_00000100000; // lcdwrite 0 32
2529 HD[2528] = 32'b100111_00001_00000_00000_00000100000; // lcdwrite 1 32
2530 HD[2529] = 32'b100111_00010_00000_00000_00001101001; // lcdwrite 2 105
2531 HD[2530] = 32'b100111_00011_00000_00000_00001101100; // lcdwrite 3 108
2532 HD[2531] = 32'b100111_00100_00000_00000_00001101111; // lcdwrite 4 111
2533 HD[2532] = 32'b100111_00101_00000_00000_00001101111; // lcdwrite 5 111
2534 HD[2533] = 32'b100111_00110_00000_00000_00001110000; // lcdwrite 6 112
2535 HD[2534] = 32'b100111_00111_00000_00000_00000110011; // lcdwrite 7 51
2536 HD[2535] = 32'b100111_01000_00000_00000_00000110010; // lcdwrite 8 50
2537 HD[2536] = 32'b100111_01001_00000_00000_00000100000; // lcdwrite 9 32
2538 HD[2537] = 32'b100111_01010_00000_00000_00001001101; // lcdwrite 10 77
2539 HD[2538] = 32'b100111_01011_00000_00000_00001000101; // lcdwrite 11 69
2540 HD[2539] = 32'b100111_01100_00000_00000_00001001110; // lcdwrite 12 78
2541 HD[2540] = 32'b100111_01101_00000_00000_00001010101; // lcdwrite 13 85
2542 HD[2541] = 32'b100111_01110_00000_00000_00000100000; // lcdwrite 14 32
2543 HD[2542] = 32'b100111_01111_00000_00000_00000100000; // lcdwrite 15 32
2544 HD[2543] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
2545 HD[2544] = 32'b000101_00000_00011_0000000000000000; // li 3 0
2546 HD[2545] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
2547 HD[2546] = 32'b000111_00010_00000_0000000000010010; // beq 2 $zero _ElseBegin61_
2548 HD[2547] = 32'b100111_10000_00000_00000_00000100000; // lcdwrite 16 32
2549 HD[2548] = 32'b100111_10001_00000_00000_00001000101; // lcdwrite 17 69
2550 HD[2549] = 32'b100111_10010_00000_00000_00001111000; // lcdwrite 18 120
2551 HD[2550] = 32'b100111_10011_00000_00000_00001100101; // lcdwrite 19 101
2552 HD[2551] = 32'b100111_10100_00000_00000_00001100011; // lcdwrite 20 99
2553 HD[2552] = 32'b100111_10101_00000_00000_00001110101; // lcdwrite 21 117
2554 HD[2553] = 32'b100111_10110_00000_00000_00001110100; // lcdwrite 22 116
2555 HD[2554] = 32'b100111_10111_00000_00000_00001100001; // lcdwrite 23 97
2556 HD[2555] = 32'b100111_11000_00000_00000_00001110010; // lcdwrite 24 114
2557 HD[2556] = 32'b100111_11001_00000_00000_00000100000; // lcdwrite 25 32
2558 HD[2557] = 32'b100111_11010_00000_00000_00001010000; // lcdwrite 26 80
2559 HD[2558] = 32'b100111_11011_00000_00000_00001110010; // lcdwrite 27 114
2560 HD[2559] = 32'b100111_11100_00000_00000_00001101111; // lcdwrite 28 111
2561 HD[2560] = 32'b100111_11101_00000_00000_00001100111; // lcdwrite 29 103

```



```

2562 HD[2561] = 32'b100111_11110_00000_00000_00000101110; // lcdwrite 30 46
2563 HD[2562] = 32'b100111_11111_00000_00000_00000100000; // lcdwrite 31 32
2564 HD[2563] = 32'b001011_000000000000000101010010010; // jump _IfExit61_
2565 HD[2564] = 32'b000001_10101_00010_00000000000000000; // lw 2 $fp 0 / _ElseBegin61_
2566 HD[2565] = 32'b000101_00000_00011_00000000000000001; // li 3 1
2567 HD[2566] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
2568 HD[2567] = 32'b000111_00010_00000_00000000000010010; // beq 2 $zero _ElseBegin62_
2569 HD[2568] = 32'b100111_10000_00000_00000_00000100000; // lcdwrite 16 32
2570 HD[2569] = 32'b100111_10001_00000_00000_00001000101; // lcdwrite 17 69
2571 HD[2570] = 32'b100111_10010_00000_00000_00001110011; // lcdwrite 18 115
2572 HD[2571] = 32'b100111_10011_00000_00000_00001100011; // lcdwrite 19 99
2573 HD[2572] = 32'b100111_10100_00000_00000_00001110010; // lcdwrite 20 114
2574 HD[2573] = 32'b100111_10101_00000_00000_00001100101; // lcdwrite 21 101
2575 HD[2574] = 32'b100111_10110_00000_00000_00001110110; // lcdwrite 22 118
2576 HD[2575] = 32'b100111_10111_00000_00000_00001100101; // lcdwrite 23 101
2577 HD[2576] = 32'b100111_11000_00000_00000_00001110010; // lcdwrite 24 114
2578 HD[2577] = 32'b100111_11001_00000_00000_00000100000; // lcdwrite 25 32
2579 HD[2578] = 32'b100111_11010_00000_00000_00001010000; // lcdwrite 26 80
2580 HD[2579] = 32'b100111_11011_00000_00000_00001110010; // lcdwrite 27 114
2581 HD[2580] = 32'b100111_11100_00000_00000_00001101111; // lcdwrite 28 111
2582 HD[2581] = 32'b100111_11101_00000_00000_00001100111; // lcdwrite 29 103
2583 HD[2582] = 32'b100111_11110_00000_00000_00000101110; // lcdwrite 30 46
2584 HD[2583] = 32'b100111_11111_00000_00000_00000100000; // lcdwrite 31 32
2585 HD[2584] = 32'b001011_000000000000000101010010010; // jump _IfExit62_
2586 HD[2585] = 32'b000001_10101_00010_00000000000000000; // lw 2 $fp 0 / _ElseBegin62_
2587 HD[2586] = 32'b000101_00000_00011_00000000000000001; // li 3 2
2588 HD[2587] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
2589 HD[2588] = 32'b000111_00010_00000_00000000000010010; // beq 2 $zero _ElseBegin63_
2590 HD[2589] = 32'b100111_10000_00000_00000_00000100000; // lcdwrite 16 32
2591 HD[2590] = 32'b100111_10001_00000_00000_00001010010; // lcdwrite 17 82
2592 HD[2591] = 32'b100111_10010_00000_00000_00001100101; // lcdwrite 18 101
2593 HD[2592] = 32'b100111_10011_00000_00000_00001101110; // lcdwrite 19 110
2594 HD[2593] = 32'b100111_10100_00000_00000_00001101111; // lcdwrite 20 111
2595 HD[2594] = 32'b100111_10101_00000_00000_00001101101; // lcdwrite 21 109
2596 HD[2595] = 32'b100111_10110_00000_00000_00001100101; // lcdwrite 22 101
2597 HD[2596] = 32'b100111_10111_00000_00000_00001100001; // lcdwrite 23 97
2598 HD[2597] = 32'b100111_11000_00000_00000_00001110010; // lcdwrite 24 114
2599 HD[2598] = 32'b100111_11001_00000_00000_00000100000; // lcdwrite 25 32
2600 HD[2599] = 32'b100111_11010_00000_00000_00001010000; // lcdwrite 26 80
2601 HD[2600] = 32'b100111_11011_00000_00000_00001110010; // lcdwrite 27 114
2602 HD[2601] = 32'b100111_11100_00000_00000_00001101111; // lcdwrite 28 111
2603 HD[2602] = 32'b100111_11101_00000_00000_00001100111; // lcdwrite 29 103
2604 HD[2603] = 32'b100111_11110_00000_00000_00000101110; // lcdwrite 30 46
2605 HD[2604] = 32'b100111_11111_00000_00000_00000100000; // lcdwrite 31 32
2606 HD[2605] = 32'b001011_000000000000000101010010010; // jump _IfExit63_
2607 HD[2606] = 32'b000001_10101_00010_00000000000000000; // lw 2 $fp 0 / _ElseBegin63_
2608 HD[2607] = 32'b000101_00000_00011_00000000000000001; // li 3 3
2609 HD[2608] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
2610 HD[2609] = 32'b000111_00010_00000_00000000000010010; // beq 2 $zero _ElseBegin64_
2611 HD[2610] = 32'b100111_10000_00000_00000_00000100000; // lcdwrite 16 32
2612 HD[2611] = 32'b100111_10001_00000_00000_00001000100; // lcdwrite 17 68
2613 HD[2612] = 32'b100111_10010_00000_00000_00001100101; // lcdwrite 18 101
2614 HD[2613] = 32'b100111_10011_00000_00000_00001101100; // lcdwrite 19 108
2615 HD[2614] = 32'b100111_10100_00000_00000_00001100101; // lcdwrite 20 101
2616 HD[2615] = 32'b100111_10101_00000_00000_00001110100; // lcdwrite 21 116
2617 HD[2616] = 32'b100111_10110_00000_00000_00001100001; // lcdwrite 22 97
2618 HD[2617] = 32'b100111_10111_00000_00000_00001110010; // lcdwrite 23 114
2619 HD[2618] = 32'b100111_11000_00000_00000_00000100000; // lcdwrite 24 32
2620 HD[2619] = 32'b100111_11001_00000_00000_00000100000; // lcdwrite 25 32
2621 HD[2620] = 32'b100111_11010_00000_00000_00001010000; // lcdwrite 26 80

```

```

2622 HD[2621] = 32'b100111_11011_00000_00000_00001110010; // lcdwrite 27 114
2623 HD[2622] = 32'b100111_11100_00000_00000_00001101111; // lcdwrite 28 111
2624 HD[2623] = 32'b100111_11101_00000_00000_00001100111; // lcdwrite 29 103
2625 HD[2624] = 32'b100111_11110_00000_00000_00000101110; // lcdwrite 30 46
2626 HD[2625] = 32'b100111_11111_00000_00000_00000100000; // lcdwrite 31 32
2627 HD[2626] = 32'b001011_000000000000000101010010010; // jump _IfExit64_
2628 HD[2627] = 32'b000001_10101_00010_00000000000000000; // lw 2 $fp 0 / _ElseBegin64_
2629 HD[2628] = 32'b000101_00000_00011_00000000000000100; // li 3 4
2630 HD[2629] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
2631 HD[2630] = 32'b000111_00010_00000_00000000000010010; // beq 2 $zero _ElseBegin65_
2632 HD[2631] = 32'b100111_10000_00000_00000_00000100000; // lcdwrite 16 32
2633 HD[2632] = 32'b100111_10001_00000_00000_00001001100; // lcdwrite 17 76
2634 HD[2633] = 32'b100111_10010_00000_00000_00001101001; // lcdwrite 18 105
2635 HD[2634] = 32'b100111_10011_00000_00000_00001110011; // lcdwrite 19 115
2636 HD[2635] = 32'b100111_10100_00000_00000_00001110100; // lcdwrite 20 116
2637 HD[2636] = 32'b100111_10101_00000_00000_00001100001; // lcdwrite 21 97
2638 HD[2637] = 32'b100111_10110_00000_00000_00001110010; // lcdwrite 22 114
2639 HD[2638] = 32'b100111_10111_00000_00000_00000100000; // lcdwrite 23 32
2640 HD[2639] = 32'b100111_11000_00000_00000_00000100000; // lcdwrite 24 32
2641 HD[2640] = 32'b100111_11001_00000_00000_00000100000; // lcdwrite 25 32
2642 HD[2641] = 32'b100111_11010_00000_00000_00001010000; // lcdwrite 26 80
2643 HD[2642] = 32'b100111_11011_00000_00000_00001110010; // lcdwrite 27 114
2644 HD[2643] = 32'b100111_11100_00000_00000_00001101111; // lcdwrite 28 111
2645 HD[2644] = 32'b100111_11101_00000_00000_00001100111; // lcdwrite 29 103
2646 HD[2645] = 32'b100111_11110_00000_00000_00000101110; // lcdwrite 30 46
2647 HD[2646] = 32'b100111_11111_00000_00000_00000100000; // lcdwrite 31 32
2648 HD[2647] = 32'b001011_000000000000000101010010010; // jump _IfExit65_
2649 HD[2648] = 32'b000001_10101_00010_00000000000000000; // lw 2 $fp 0 / _ElseBegin65_
2650 HD[2649] = 32'b000101_00000_00011_00000000000000101; // li 3 5
2651 HD[2650] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
2652 HD[2651] = 32'b000111_00010_00000_00000000000010010; // beq 2 $zero _ElseBegin66_
2653 HD[2652] = 32'b100111_10000_00000_00000_00000100000; // lcdwrite 16 32
2654 HD[2653] = 32'b100111_10001_00000_00000_00001010010; // lcdwrite 17 82
2655 HD[2654] = 32'b100111_10010_00000_00000_00001100101; // lcdwrite 18 101
2656 HD[2655] = 32'b100111_10011_00000_00000_00001101001; // lcdwrite 19 105
2657 HD[2656] = 32'b100111_10100_00000_00000_00001101110; // lcdwrite 20 110
2658 HD[2657] = 32'b100111_10101_00000_00000_00001101001; // lcdwrite 21 105
2659 HD[2658] = 32'b100111_10110_00000_00000_00001100011; // lcdwrite 22 99
2660 HD[2659] = 32'b100111_10111_00000_00000_00001101001; // lcdwrite 23 105
2661 HD[2660] = 32'b100111_11000_00000_00000_00001100001; // lcdwrite 24 97
2662 HD[2661] = 32'b100111_11001_00000_00000_00001110010; // lcdwrite 25 114
2663 HD[2662] = 32'b100111_11010_00000_00000_00000100000; // lcdwrite 26 32
2664 HD[2663] = 32'b100111_11011_00000_00000_00001010011; // lcdwrite 27 83
2665 HD[2664] = 32'b100111_11100_00000_00000_00000101110; // lcdwrite 28 46
2666 HD[2665] = 32'b100111_11101_00000_00000_00001001111; // lcdwrite 29 79
2667 HD[2666] = 32'b100111_11110_00000_00000_00000101110; // lcdwrite 30 46
2668 HD[2667] = 32'b100111_11111_00000_00000_00000100000; // lcdwrite 31 32
2669 HD[2668] = 32'b001011_000000000000000101010010010; // jump _IfExit66_
2670 HD[2669] = 32'b000001_10101_00010_00000000000000000; // lw 2 $fp 0 / _ElseBegin66_
2671 HD[2670] = 32'b000101_00000_00011_00000000000000110; // li 3 6
2672 HD[2671] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
2673 HD[2672] = 32'b000111_00010_00000_00000000000010010; // beq 2 $zero _ElseBegin67_
2674 HD[2673] = 32'b100111_10000_00000_00000_00000100000; // lcdwrite 16 32
2675 HD[2674] = 32'b100111_10001_00000_00000_00001000100; // lcdwrite 17 68
2676 HD[2675] = 32'b100111_10010_00000_00000_00001100101; // lcdwrite 18 101
2677 HD[2676] = 32'b100111_10011_00000_00000_00001110011; // lcdwrite 19 115
2678 HD[2677] = 32'b100111_10100_00000_00000_00001101100; // lcdwrite 20 108
2679 HD[2678] = 32'b100111_10101_00000_00000_00001101001; // lcdwrite 21 105
2680 HD[2679] = 32'b100111_10110_00000_00000_00001100111; // lcdwrite 22 103
2681 HD[2680] = 32'b100111_10111_00000_00000_00001100001; // lcdwrite 23 97

```



```

2682 HD[2681] = 32'b100111_11000_00000_00000_00001110010; // lcdwrite 24 114
2683 HD[2682] = 32'b100111_11001_00000_00000_00000100000; // lcdwrite 25 32
2684 HD[2683] = 32'b100111_11010_00000_00000_00000100000; // lcdwrite 26 32
2685 HD[2684] = 32'b100111_11011_00000_00000_00001010011; // lcdwrite 27 83
2686 HD[2685] = 32'b100111_11100_00000_00000_00000101110; // lcdwrite 28 46
2687 HD[2686] = 32'b100111_11101_00000_00000_00001001111; // lcdwrite 29 79
2688 HD[2687] = 32'b100111_11110_00000_00000_00000101110; // lcdwrite 30 46
2689 HD[2688] = 32'b100111_11111_00000_00000_00000100000; // lcdwrite 31 32
2690 HD[2689] = 32'b001011_000000000000000101010010010; // jump _IfExit67_
2691 HD[2690] = 32'b100111_10000_00000_00000_00000100000; // lcdwrite 16 32 / _ElseBegin67_
2692 HD[2691] = 32'b100111_10001_00000_00000_00000101101; // lcdwrite 17 45
2693 HD[2692] = 32'b100111_10010_00000_00000_00000101101; // lcdwrite 18 45
2694 HD[2693] = 32'b100111_10011_00000_00000_00001000010; // lcdwrite 19 66
2695 HD[2694] = 32'b100111_10100_00000_00000_00001100001; // lcdwrite 20 97
2696 HD[2695] = 32'b100111_10101_00000_00000_00001110011; // lcdwrite 21 115
2697 HD[2696] = 32'b100111_10110_00000_00000_00001101000; // lcdwrite 22 104
2698 HD[2697] = 32'b100111_10111_00000_00000_00000100000; // lcdwrite 23 32
2699 HD[2698] = 32'b100111_11000_00000_00000_00000100000; // lcdwrite 24 32
2700 HD[2699] = 32'b100111_11001_00000_00000_00001001000; // lcdwrite 25 72
2701 HD[2700] = 32'b100111_11010_00000_00000_00001100101; // lcdwrite 26 101
2702 HD[2701] = 32'b100111_11011_00000_00000_00001101100; // lcdwrite 27 108
2703 HD[2702] = 32'b100111_11100_00000_00000_00001110000; // lcdwrite 28 112
2704 HD[2703] = 32'b100111_11101_00000_00000_00000101101; // lcdwrite 29 45
2705 HD[2704] = 32'b100111_11110_00000_00000_00000101101; // lcdwrite 30 45
2706 HD[2705] = 32'b100111_11111_00000_00000_00000100000; // lcdwrite 31 32
2707 HD[2706] = 32'b010000_10110_10111_0000000000000000; // pop $ra / _IfExit67_ _IfExit66_
    _IfExit65_ _IfExit64_ _IfExit63_ _IfExit62_ _IfExit61_
2708 HD[2707] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
2709 HD[2708] = 32'b001111_10110_10111_0000000000000000; // push $ra / tela_prox
2710 HD[2709] = 32'b000101_00000_00010_0000000000000000; // li 2 0
2711 HD[2710] = 32'b000010_10101_00010_00000000000000001; // sw 2 $fp 1
2712 HD[2711] = 32'b000101_00000_00010_00000000000000011; // li 2 7
2713 HD[2712] = 32'b000010_10101_00010_00000000000000010; // sw 2 $fp 2
2714 HD[2713] = 32'b000001_10101_00010_00000000000000000; // lw 2 $fp 0
2715 HD[2714] = 32'b000001_10101_00011_00000000000000010; // lw 3 $fp 2
2716 HD[2715] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
2717 HD[2716] = 32'b000111_00010_00000_00000000000000110; // beq 2 $zero _ElseBegin68_
2718 HD[2717] = 32'b000001_10101_00010_00000000000000001; // lw 2 $fp 1
2719 HD[2718] = 32'b000000_00010_10100_00000_00000_010000; // move $rv 2
2720 HD[2719] = 32'b010000_10110_10111_0000000000000000; // pop $ra
2721 HD[2720] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
2722 HD[2721] = 32'b001011_000000000000000101010100111; // jump _IfExit68_
2723 HD[2722] = 32'b000001_10101_00010_00000000000000000; // lw 2 $fp 0 / _ElseBegin68_
2724 HD[2723] = 32'b000011_00010_00010_00000000000000001; // addi 2 2 1
2725 HD[2724] = 32'b000000_00010_10100_00000_00000_010000; // move $rv 2
2726 HD[2725] = 32'b010000_10110_10111_0000000000000000; // pop $ra
2727 HD[2726] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
2728 HD[2727] = 32'b010000_10110_10111_0000000000000000; // pop $ra / _IfExit68_
2729 HD[2728] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
2730 HD[2729] = 32'b001111_10110_10111_0000000000000000; // push $ra / tela_ant
2731 HD[2730] = 32'b000101_00000_00010_00000000000000000; // li 2 0
2732 HD[2731] = 32'b000010_10101_00010_00000000000000001; // sw 2 $fp 1
2733 HD[2732] = 32'b000101_00000_00010_00000000000000011; // li 2 7
2734 HD[2733] = 32'b000010_10101_00010_00000000000000010; // sw 2 $fp 2
2735 HD[2734] = 32'b000001_10101_00010_00000000000000000; // lw 2 $fp 0
2736 HD[2735] = 32'b000001_10101_00011_00000000000000001; // lw 3 $fp 1
2737 HD[2736] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
2738 HD[2737] = 32'b000111_00010_00000_00000000000000110; // beq 2 $zero _ElseBegin69_
2739 HD[2738] = 32'b000001_10101_00010_00000000000000010; // lw 2 $fp 2
2740 HD[2739] = 32'b000000_00010_10100_00000_00000_010000; // move $rv 2

```

```

2741 HD[2740] = 32'b010000_10110_10111_0000000000000000; // pop $ra
2742 HD[2741] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
2743 HD[2742] = 32'b001011_00000000000000010101011100; // jump _IfExit69_
2744 HD[2743] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0 / _ElseBegin69_
2745 HD[2744] = 32'b000100_00010_00010_0000000000000001; // subi 2 2 1
2746 HD[2745] = 32'b000000_00010_10100_00000_00000_010000; // move $rv 2
2747 HD[2746] = 32'b010000_10110_10111_0000000000000000; // pop $ra
2748 HD[2747] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
2749 HD[2748] = 32'b010000_10110_10111_0000000000000000; // pop $ra / _IfExit69_
2750 HD[2749] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
2751 HD[2750] = 32'b001111_10110_10111_0000000000000000; // push $ra / bash_OS
2752 HD[2751] = 32'b000101_00000_00010_0000000000000000; // li 2 0
2753 HD[2752] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2754 HD[2753] = 32'b000101_00000_00010_0000000000000001; // li 2 1 / _WhileBegin20_
2755 HD[2754] = 32'b000111_00010_00000_0000000001100010; // beq 2 $zero _WhileExit20_
2756 HD[2755] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
2757 HD[2756] = 32'b000011_10101_10101_0000000000000010; // addi $fp $fp 2
2758 HD[2757] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2759 HD[2758] = 32'b101000_00000000000000010011101110; // jal print_menu
2760 HD[2759] = 32'b000100_10101_10101_0000000000000010; // subi $fp $fp 2
2761 HD[2760] = 32'b001001_00000_00010_0000000000000000; // in 2
2762 HD[2761] = 32'b000010_10101_00010_0000000000000001; // sw 2 $fp 1
2763 HD[2762] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1
2764 HD[2763] = 32'b000101_00000_00011_0000000000000001; // li 3 1
2765 HD[2764] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
2766 HD[2765] = 32'b000111_00010_00000_0000000000001001; // beq 2 $zero _ElseBegin70_
2767 HD[2766] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
2768 HD[2767] = 32'b000011_10101_10101_0000000000000010; // addi $fp $fp 2
2769 HD[2768] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2770 HD[2769] = 32'b101000_000000000000000101010010100; // jal tela_prox
2771 HD[2770] = 32'b000100_10101_10101_00000000000000010; // subi $fp $fp 2
2772 HD[2771] = 32'b000000_10100_00010_00000_00000_010000; // move 2 $rv
2773 HD[2772] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2774 HD[2773] = 32'b001011_000000000000000101100100011; // jump _IfExit70_
2775 HD[2774] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1 / _ElseBegin70_
2776 HD[2775] = 32'b000101_00000_00011_0000000000000010; // li 3 2
2777 HD[2776] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
2778 HD[2777] = 32'b000111_00010_00000_0000000000001001; // beq 2 $zero _ElseBegin71_
2779 HD[2778] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
2780 HD[2779] = 32'b000011_10101_10101_0000000000000010; // addi $fp $fp 2
2781 HD[2780] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2782 HD[2781] = 32'b101000_000000000000000101010101001; // jal tela_ant
2783 HD[2782] = 32'b000100_10101_10101_0000000000000010; // subi $fp $fp 2
2784 HD[2783] = 32'b000000_10100_00010_00000_00000_010000; // move 2 $rv
2785 HD[2784] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2786 HD[2785] = 32'b001011_000000000000000101100100011; // jump _IfExit71_
2787 HD[2786] = 32'b000001_10101_00010_0000000000000001; // lw 2 $fp 1 / _ElseBegin71_
2788 HD[2787] = 32'b000101_00000_00011_0000000000000011; // li 3 3
2789 HD[2788] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
2790 HD[2789] = 32'b000111_00010_00000_0000000000111110; // beq 2 $zero _IfExit72_
2791 HD[2790] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0
2792 HD[2791] = 32'b000101_00000_00011_0000000000000000; // li 3 0
2793 HD[2792] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
2794 HD[2793] = 32'b000111_00010_00000_000000000000101; // beq 2 $zero _ElseBegin73_
2795 HD[2794] = 32'b000011_10101_10101_0000000000000010; // addi $fp $fp 2
2796 HD[2795] = 32'b101000_00000000000000010111111101; // jal menu_opt0_executar_prog
2797 HD[2796] = 32'b000100_10101_10101_0000000000000010; // subi $fp $fp 2
2798 HD[2797] = 32'b001011_000000000000000101100100011; // jump _IfExit73_
2799 HD[2798] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0 / _ElseBegin73_
2800 HD[2799] = 32'b000101_00000_00011_0000000000000001; // li 3 1

```

```

2801 HD[2800] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
2802 HD[2801] = 32'b000111_00010_00000_00000000000000101; // beq 2 $zero _ElseBegin74_
2803 HD[2802] = 32'b000011_10101_10101_0000000000000010; // addi $fp $fp 2
2804 HD[2803] = 32'b101000_00000000000000011010001010; // jal menu_opt1_escrever_prog
2805 HD[2804] = 32'b000100_10101_10101_0000000000000010; // subi $fp $fp 2
2806 HD[2805] = 32'b001011_000000000000000101100100011; // jump _IfExit74_
2807 HD[2806] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0 / _ElseBegin74_
2808 HD[2807] = 32'b000101_00000_00011_0000000000000010; // li 3 2
2809 HD[2808] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
2810 HD[2809] = 32'b000111_00010_00000_00000000000000101; // beq 2 $zero _ElseBegin75_
2811 HD[2810] = 32'b000011_10101_10101_0000000000000010; // addi $fp $fp 2
2812 HD[2811] = 32'b101000_0000000000000001101101001; // jal menu_opt2_renomear_prog
2813 HD[2812] = 32'b000100_10101_10101_0000000000000010; // subi $fp $fp 2
2814 HD[2813] = 32'b001011_000000000000000101100100011; // jump _IfExit75_
2815 HD[2814] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0 / _ElseBegin75_
2816 HD[2815] = 32'b000101_00000_00011_0000000000000011; // li 3 3
2817 HD[2816] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
2818 HD[2817] = 32'b000111_00010_00000_00000000000000101; // beq 2 $zero _ElseBegin76_
2819 HD[2818] = 32'b000011_10101_10101_0000000000000010; // addi $fp $fp 2
2820 HD[2819] = 32'b101000_000000000000000100001000110; // jal menu_opt3_deletar_prog
2821 HD[2820] = 32'b000100_10101_10101_0000000000000010; // subi $fp $fp 2
2822 HD[2821] = 32'b001011_000000000000000101100100011; // jump _IfExit76_
2823 HD[2822] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0 / _ElseBegin76_
2824 HD[2823] = 32'b000101_00000_00011_0000000000000100; // li 3 4
2825 HD[2824] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
2826 HD[2825] = 32'b000111_00010_00000_00000000000000101; // beq 2 $zero _ElseBegin77_
2827 HD[2826] = 32'b000011_10101_10101_0000000000000010; // addi $fp $fp 2
2828 HD[2827] = 32'b101000_000000000000000100010111111; // jal menu_opt4_listar_prog
2829 HD[2828] = 32'b000100_10101_10101_0000000000000010; // subi $fp $fp 2
2830 HD[2829] = 32'b001011_000000000000000101100100011; // jump _IfExit77_
2831 HD[2830] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0 / _ElseBegin77_
2832 HD[2831] = 32'b000101_00000_00011_00000000000000101; // li 3 5
2833 HD[2832] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
2834 HD[2833] = 32'b000111_00010_00000_00000000000000101; // beq 2 $zero _ElseBegin78_
2835 HD[2834] = 32'b000011_10101_10101_0000000000000010; // addi $fp $fp 2
2836 HD[2835] = 32'b101000_000000000000000100100100011; // jal menu_opt5_reiniciar
2837 HD[2836] = 32'b000100_10101_10101_0000000000000010; // subi $fp $fp 2
2838 HD[2837] = 32'b001011_000000000000000101100100011; // jump _IfExit78_
2839 HD[2838] = 32'b000001_10101_00010_0000000000000000; // lw 2 $fp 0 / _ElseBegin78_
2840 HD[2839] = 32'b000101_00000_00011_00000000000000110; // li 3 6
2841 HD[2840] = 32'b000000_00010_00011_00010_00000_010011; // set 2 2 3
2842 HD[2841] = 32'b000111_00010_00000_00000000000000111; // beq 2 $zero _ElseBegin79_
2843 HD[2842] = 32'b000011_10101_10101_0000000000000010; // addi $fp $fp 2
2844 HD[2843] = 32'b101000_000000000000000100101011010; // jal menu_opt6_desligar
2845 HD[2844] = 32'b000100_10101_10101_0000000000000010; // subi $fp $fp 2
2846 HD[2845] = 32'b010000_10110_10111_0000000000000000; // pop $ra
2847 HD[2846] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
2848 HD[2847] = 32'b001011_000000000000000101100100011; // jump _IfExit79_
2849 HD[2848] = 32'b000011_10101_10101_0000000000000010; // addi $fp $fp 2 / _ElseBegin79_
2850 HD[2849] = 32'b101000_000000000000000100110001010; // jal menu_opt7_help
2851 HD[2850] = 32'b000100_10101_10101_0000000000000010; // subi $fp $fp 2
2852 HD[2851] = 32'b001011_000000000000000101011000001; // jump _WhileBegin20_ / _IfExit79_
    _IfExit78_ _IfExit77_ _IfExit76_ _IfExit75_ _IfExit74_ _IfExit73_ _IfExit72_
    _IfExit71_ _IfExit70_
2853 HD[2852] = 32'b010000_10110_10111_0000000000000000; // pop $ra / _WhileExit20_
2854 HD[2853] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
2855 HD[2854] = 32'b001111_10110_10111_0000000000000000; // push $ra / init_global
2856 HD[2855] = 32'b000101_00000_00010_0000000000000000; // li 2 0
2857 HD[2856] = 32'b000010_10101_00010_0000000000000000; // sw 2 $fp 0
2858 HD[2857] = 32'b000101_00000_00010_00000000000001111; // li 2 15

```

```

2859 HD[2858] = 32'b000010_00000_00010_0000000000000000; // sw 2 $global 0
2860 HD[2859] = 32'b0000101_00000_00010_0000101110111000; // li 2 3000
2861 HD[2860] = 32'b000010_00000_00010_0000000000000001; // sw 2 $global 1
2862 HD[2861] = 32'b0000101_00000_00010_000000001000000000; // li 2 256
2863 HD[2862] = 32'b000010_00000_00010_00000000000000010; // sw 2 $global 2
2864 HD[2863] = 32'b0000101_00000_00010_000001000000000000; // li 2 2048
2865 HD[2864] = 32'b000010_00000_00010_000000000000000011; // sw 2 $global 3
2866 HD[2865] = 32'b0000101_00000_00010_000000001000000000; // li 2 256
2867 HD[2866] = 32'b000010_00000_00010_000000000000000100; // sw 2 $global 4
2868 HD[2867] = 32'b0000101_00000_00010_00000000000100000; // li 2 32
2869 HD[2868] = 32'b000010_00000_00010_0000000000000000101; // sw 2 $global 5
2870 HD[2869] = 32'b0000101_00000_00010_000000000000000000; // li 2 0
2871 HD[2870] = 32'b000010_00000_00010_0000000000000000110; // sw 2 $global 6
2872 HD[2871] = 32'b0000101_00000_00010_00000000000000001010; // li 2 10
2873 HD[2872] = 32'b000010_00000_00010_0000000000000000111; // sw 2 $global 7
2874 HD[2873] = 32'b0000101_00000_00010_000000000000000000; // li 2 0
2875 HD[2874] = 32'b000010_00000_00010_00000000000000001000; // sw 2 $global 8
2876 HD[2875] = 32'b000001_10101_00010_000000000000000000; // lw 2 $fp 0 / _WhileBegin21_
2877 HD[2876] = 32'b0000101_00000_00011_000000000000001010; // li 3 10
2878 HD[2877] = 32'b000000_00010_00011_00010_00000_010001; // slt 2 2 3
2879 HD[2878] = 32'b000111_00010_00000_000000000000001001; // beq 2 $zero _WhileExit21_
2880 HD[2879] = 32'b0000101_00000_00010_000000000000000000; // li 2 0
2881 HD[2880] = 32'b000001_10101_00011_000000000000000000; // lw 3 $fp 0
2882 HD[2881] = 32'b000000_00011_00000_00011_00000_000101; // add 3 3 $global
2883 HD[2882] = 32'b000010_00011_00010_000000000000001001; // sw 2 3 9
2884 HD[2883] = 32'b000001_10101_00010_000000000000000000; // lw 2 $fp 0
2885 HD[2884] = 32'b000011_00010_00010_000000000000000001; // addi 2 2 1
2886 HD[2885] = 32'b000010_10101_00010_000000000000000000; // sw 2 $fp 0
2887 HD[2886] = 32'b0001011_0000000000000000101100111011; // jump _WhileBegin21_
2888 HD[2887] = 32'b0000101_00000_00010_000000000000000000; // li 2 0 / _WhileExit21_
2889 HD[2888] = 32'b000010_00000_00010_0000000000000010011; // sw 2 $global 19
2890 HD[2889] = 32'b0000101_00000_00010_000000000000000000; // li 2 0
2891 HD[2890] = 32'b000010_00000_00010_0000000000000010100; // sw 2 $global 20
2892 HD[2891] = 32'b0000101_00000_00010_000000000000000000; // li 2 0
2893 HD[2892] = 32'b000100_00010_00010_000000000000000001; // subi 2 2 1
2894 HD[2893] = 32'b000010_00000_00010_0000000000000010101; // sw 2 $global 21
2895 HD[2894] = 32'b0000101_00000_00010_000000000000000000; // li 2 0
2896 HD[2895] = 32'b000100_00010_00010_000000000000000001; // subi 2 2 1
2897 HD[2896] = 32'b000010_00000_00010_0000000000000010110; // sw 2 $global 22
2898 HD[2897] = 32'b0000101_00000_00010_000000000000000011; // li 2 3
2899 HD[2898] = 32'b000010_00000_00010_0000000000000010111; // sw 2 $global 23
2900 HD[2899] = 32'b0000101_00000_00010_000000000000000000; // li 2 0
2901 HD[2900] = 32'b000010_00000_00010_0000000000000011000; // sw 2 $global 24
2902 HD[2901] = 32'b0000101_00000_00010_000000000000000000; // li 2 0
2903 HD[2902] = 32'b000100_00010_00010_000000000000000001; // subi 2 2 1
2904 HD[2903] = 32'b000010_00000_00010_0000000000000011001; // sw 2 $global 25
2905 HD[2904] = 32'b0000101_00000_00010_000000000000000000; // li 2 0
2906 HD[2905] = 32'b000100_00010_00010_0000000000000000010; // subi 2 2 2
2907 HD[2906] = 32'b000010_00000_00010_0000000000000011010; // sw 2 $global 26
2908 HD[2907] = 32'b0000101_00000_00010_000000000000000000; // li 2 0
2909 HD[2908] = 32'b000100_00010_00010_0000000000000000011; // subi 2 2 3
2910 HD[2909] = 32'b000010_00000_00010_0000000000000011011; // sw 2 $global 27
2911 HD[2910] = 32'b0000101_00000_00010_0000000000000000011; // li 2 3
2912 HD[2911] = 32'b000010_00000_00010_0000000000000011100; // sw 2 $global 28
2913 HD[2912] = 32'b0000101_00000_00010_000000000000000000; // li 2 0
2914 HD[2913] = 32'b000010_00000_00010_0000000000000011101; // sw 2 $global 29
2915 HD[2914] = 32'b0000101_00000_00010_000000000000000001; // li 2 1
2916 HD[2915] = 32'b000010_00000_00010_0000000000000011110; // sw 2 $global 30
2917 HD[2916] = 32'b0000101_00000_00010_0000000000000000010; // li 2 2
2918 HD[2917] = 32'b000010_00000_00010_0000000000000011111; // sw 2 $global 31

```

```
2919 HD[2918] = 32'b000101_00000_00010_0000000000000011; // li 2 3
2920 HD[2919] = 32'b000010_00000_00010_0000000000100000; // sw 2 $global 32
2921 HD[2920] = 32'b010000_10110_10111_0000000000000000; // pop $ra
2922 HD[2921] = 32'b000000_10111_00000_00000_00000_010100; // jr $ra
2923 HD[2922] = 32'b101000_000000000000000101100100110; // jal init_global / main
2924 HD[2923] = 32'b101000_000000000000000101010111110; // jal bash_OS
2925 HD[2924] = 32'b11111111111111111111111111111111; // halt / _halt_
```

Algoritmo B.1 – Sistema Operacional.